

CUDA Kernel 面试背题笔记

本书整理自 LeetCUDA 项目，系统汇编了 CUDA 面试中最高频的 GPU Kernel 实现，涵盖从 Warp/Block 原语、Elementwise、Softmax、GEMM、FlashAttention 等面试核心知识体系。

每个 Kernel 配备详细的中文面试要点注释 (WHY + HOW)，并以递进式优化路线 (naive → tiling → vectorize → tensor core → warp specialized) 展示 GPU 性能调优思维。

30 个 Kernel · 9 个 Phase · 全中文注释

项目主页: <https://github.com/xlite-dev/LeetCUDA>

更新日期: 2026 年 7 月 20 日

```

1 // =====
2 // notes-v2.cu — CUDA Kernel 面试刷题笔记
3 // =====
4 //
5 // 整理自 LeetCUDA 项目 (https://github.com/xlite-dev/LeetCUDA) , 涵盖:
6 //   - 面试高频 CUDA kernel 的完整实现 (~30 个 kernel)
7 //   - 每类 kernel 附带详细的面试要点注释 (WHY + HOW)
8 //   - 优化技术的递进式讲解 (naive → tiling → vectorize → tensor core → ws)
9 //   - BLAS 语义: N=col-major(Normal), T=row-major(Transposed)
10 //
11 // 10 个 Phase 覆盖:
12 //   Phase 0 — 面试框架速查 (GPU 架构 / Memory Hierarchy / Roofline / 优化清单)
13 //   Phase 1 — 基础原语: Warp Reduce / Block Reduce / Dot Product (含 broadcast 增强版)
14 //   Phase 2 — Elementwise: ReLU / Elementwise Add / Histogram (基础 + float4 向量化 + atomic)
15 //   Phase 3 — Softmax: naive → safe → online + RMS/Layer Norm
16 //   Phase 4 — RoPE: 旋转位置编码 (Llama 风格 theta=10000)
17 //   Phase 5 — Mat Transpose: 基础版 + BCF merge_write 最佳版 (Bank Conflict专题)
18 //   Phase 6 — GEMV: SGEMV K32/K128/K16 (warp-per-row)
19 //   Phase 7 — GEMM *: SGEMM → HGEMM → MMA m16n8k16(TN布局) → WGMMMA m64n128k16
20 //   Phase 8 — FlashAttention-2split_q (FA-2, 含 online softmax + P@V 寄存器复用)
21 //
22 // =====
23 // Phase 0: 面试框架速查 (纯注释, 面试开场必备的基础知识)
24 // =====
25
26 // ---- GPU 架构速查 ----
27 //
28 // SM (Streaming Multiprocessor) 内部结构:
29 //   - Warp Scheduler x4: 每 SM 4 个 warp scheduler, 每个每周期可发射 1 条指令
30 //   - Register File: 每 SM 65536 × 32-bit (4 bytes) = 256KB
31 //   - Shared Memory / L1: 可配置, 最大 shared memory ~228KB (Hopper)
32 //   - Tensor Cores: Hopper 每 SM 4 个; Blackwell 数量随型号/定义不同, 建议以官方 ISV guide 为准
33 //   - Warp = 32 threads: 最小调度单元, SIMT 执行模型
34 //
35 // Memory Hierarchy 带宽数量级 (H100 参考):
36 //   HBM3: ~3.35 TB/s (理论), 实际 ~2.5-3.0 TB/s
37 //   L2 Cache: ~12 TB/s (50MB, 跨 SM 共享)
38 //   L1/SMEM: ~19 TB/s (每 SM ~228KB)
39 //   Register: ~0 延迟, ~100+ TB/s 等效带宽
40 //
41 // 关键瓶颈判断:
42 //   Memory-bound: AI (Arithmetic Intensity) < 机器 FLOPS/带宽 比值
43 //   Compute-bound: AI 足够大, 受限于计算吞吐
44 //   Latency-bound: 线程不够多, 无法隐藏内存延迟
45 //
46 // Occupancy 公式:
47 //   occupancy = active_warps / max_warps_per_SM
48 //   受三类资源分别取下限: 每线程寄存器数 → threads/SM; 每 block shared memory
49 //   → blocks/SM; block 大小 → blocks/SM
50 //
51 // ---- 常见优化手段速查清单 ----
52 //
53 // 1. Coalesced Memory Access (合并访问)
54 //   - 同一 warp 的线程访问连续的 128B 对齐地址 → 1 次内存事务
55 //   - 否则产生多次事务 (最坏 32 次)
56 //
57 // 2. Tiling (分块)
58 //   - 将数据从 HBM 分块加载到 shared memory 复用, 减少 HBM 访问
59 //   - GEMM: Block Tile (BM×BN) + K Tile (BK)
60 //
61 // 3. Vectorized Memory Access (向量化)
62 //   - 使用 float4/half2 等向量类型, 减少 load/store 指令数

```

```

63 // - float4 = 128-bit, 单条指令加载 16 bytes
64 //
65 // 4. Thread Tile (寄存器分块)
66 // - 每个线程计算多个输出元素 (TM×TN), 提高计算密度
67 // - 减少线程总数, 降低同步开销
68 //
69 // 5. Bank Conflict Avoidance
70 // - Shared memory 有 32 banks × 4 bytes
71 // - 同 warp 多线程访问同一 bank 的不同地址 → bank conflict → 串行化
72 // - 解决方案: PAD (在每行末尾加 1 个元素打破对齐)
73 //
74 // 6. Pipeline / Double Buffering (流水线)
75 // - cp.async 异步拷贝下一批数据时同时做当前批的计算
76 // - Stage 数 = 2/3/4, 权衡 shared memory 占用和延迟隐藏
77 //
78 // 7. Tensor Core (MMA / WGMMa)
79 // - MMA m16n8k16 (Ampere): warp 级指令, 单 warp 完成 16×8×16 的矩阵乘
80 // - WGMMa m64n128k16 (Hopper): warpgroup 级指令 (128 threads), 异步执行
81 //
82 // 8. Warp Specialization (Hopper+)
83 // - Producer warpgroup 做 TMA 数据搬运, Consumer warpgroup 做计算
84 // - 通过 cuda::barrier 同步, 完全解耦数据搬运和计算
85 //
86 // 9. TMA (Tensor Memory Accelerator, Hopper+)
87 // - 硬件 DMA 引擎, 支持 1D~5D 寻址, 低寄存器开销
88 // - 配合 cp.async.bulk 实现异步数据搬运
89
90 // ---- Roofline 分析公式 ----
91 //
92 // AI (Arithmetic Intensity) = FLOPs / Bytes_transferred
93 //
94 // GEMM (M=N=K=4096):
95 // FLOPs = 2 × M × N × K = 2 × 40963 ≈ 137 GFLOPs
96 // Bytes = (M×K + K×N + M×N) × sizeof(float) ≈ 200 MB
97 // AI ≈ 137G / 200M ≈ 685 FLOPs/Byte → compute-bound (远超 H100 ridge point:
98 // FP16 TC ≈ 295:1, FP32 ≈ 20:1)
99 //
100 // GEMV (M=4096, K=4096):
101 // Bytes = (M×K + K + M) × sizeof(float) ≈ 67 MB
102 // AI ≈ 33M / 67M ≈ 0.5 FLOPs/Byte → severely memory-bound
103 //
104 // Softmax (N=4096): AI ≈ (5×N) / (2×N×4) = 5/8 ≈ 0.625 FLOPs/Byte → memory-bound
105
106 // =====
107 // Phase 1: 头文件 + 宏定义 + 基础原语 (Warp Reduce / Block Reduce)
108 // =====
109 // 面试要点:
110 // - warp_reduce: 用 __shfl_xor_sync 做蝶形归约, O(logN) 步, 无需 shared
111 //   memory
112 // - block_reduce: 两级归约 (warp → shared memory → warp0 broadcast),
113 //   注意最后必须 broadcast 回所有线程 (__shfl_sync), 否则只有 warp0 知道结果
114 // - 为什么不用 __shfl_down_sync? xor 模式所有线程做相同工作量, 更均衡
115
116 #include <algorithm>
117 #include <cstring>
118 #include <cuda_fp16.h>
119 #include <cuda_runtime.h>
120 #include <float.h>
121 #include <stdio.h>
122 #include <stdlib.h>
123 #include <math.h>
124 #include <cublas_v2.h>
125 #include <cuda.h>

```

```

126 #include <cuda/barrier>
127 #include <cuda/ptx>
128
129 #if defined(NOTES_V2_ENABLE_CUDNN)
130 #pragma nv_diag_suppress 128
131 #include <cuda/cudnn_frontend.h>
132 #pragma nv_diag_default 128
133 namespace fe = cudnn_frontend;
134 #endif
135
136 #if defined(NOTES_V2_ENABLE_TMA_MMA_WS) && CUDART_VERSION < 13000
137 #error "NOTES_V2_ENABLE_TMA_MMA_WS requires CUDA Toolkit 13.0 or newer"
138 #endif
139
140 #define INT4(value) (reinterpret_cast<int4 *>(&(value)))[0])
141 #define FLOAT4(value) (reinterpret_cast<float4 *>(&(value)))[0])
142 #define HALF2(value) (reinterpret_cast<half2 *>(&(value)))[0])
143
144 static constexpr int kWarpSize = 32;
145
146 // =====
147 // Phase 1a: Warp Reduce (warp 内归约, 纯寄存器操作, 无需 shared memory)
148 // =====
149
150 // Warp Reduce Sum — generic (used by both FP32 and FP16 contexts)
151 // 使用 __shfl_xor_sync 做蝶形归约 (butterfly reduction)
152 // 复杂度 O(logN), N=32 时只需 5 步
153 // 模板参数: T=数据类型, kWarpWidth=segment width (默认 32)
154 // kWarpWidth 会作为 __shfl_xor_sync 的第 4 个实参 width, 限制 shuffle 在同一 segment 内
155 // 当 kWarpWidth < 32 (如 FA 中 kWarpWidth=4) 时, 只有同 segment 的 lane 参与通信
156 // 蝶形归约示意 (以 warpSize=8 为例, 实际 warpSize=32 有 5 次迭代):
157 //
158 // 初始: 每个 lane 持有自己的值 v0..v7
159 // lane: 0 1 2 3 4 5 6 7
160 // val: v0 v1 v2 v3 v4 v5 v6 v7
161 //
162 // mask=4 (第1次迭代, lane i 与 lane i^4 交换并累加):
163 //
164 //      ┌──────────┐
165 // 对: (0,4) (1,5) (2,6) (3,7)
166 //
167 // lane: 0 1 2 3 4 5 6 7
168 // val: v0+v4 v1+v5 v2+v6 v3+v7 v4+v0 v5+v1 v6+v2 v7+v3
169 //
170 // mask=2 (第2次迭代, lane i 与 lane i^2 交换并累加):
171 //
172 //      ┌───┐ ┌───┐ ┌───┐ ┌───┐
173 // 对: (0,2) (1,3) (4,6) (5,7)
174 //      ─── 前4个一组 ───      ─── 后4个一组 ───
175 //
176 // lane: 0 1 2 3 4 5 6 7 (每 lane 持有前一轮2个值的和再加本轮配对)
177 // val: Σ{0,2,4,6} Σ{1,3,5,7} Σ{0,2,4,6} Σ{1,3,5,7} Σ{0,2,4,6} Σ{1,3,5,7} Σ{0,2,4,6} Σ{1,3,5,7}
178 //      = v0+v2+v4+v6 ... (逐步归约)
179 //
180 // mask=1 (第3次迭代, lane i 与 lane i^1 交换并累加):
181 //
182 //      ┌─┐ ┌─┐ ┌─┐ ┌─┐ ┌─┐ ┌─┐ ┌─┐ ┌─┐
183 // 对: (0,1) (2,3) (4,5) (6,7)
184 //
185 // lane: 0 1 2 3 4 5 6 7
186 // val: Σall Σall Σall Σall Σall Σall Σall Σall ← 所有 lane 拥有全归约结果!
187 //
188 // mask=0: 循环终止, 归约完成。
189 //
190 // 关键性质:
191 // - XOR 配对是对称的: lane i 的配对对象是 lane i^mask, 而 (i^mask)^mask = i

```

```

189 // - 每轮每个 lane 只和恰好 1 个其他 lane 通信 (一对一, 无冲突)
190 // - 每轮信息传递距离减半: 16→8→4→2→1 (距离减半, 信息翻倍)
191 // - 0(log2 N) 步完成, 无需 shared memory, 纯寄存器操作
192 // - __shfl_xor_sync 第四个参数 kWarpWidth 限制 segment width:
193 // 当 kWarpWidth=4 时, 只有同 segment(4个一组)内的 lane 参与 shuffle
194 template <const int kWarpWidth = kWarpSize, typename T = float>
195 __device__ __forceinline__ T warp_reduce_sum(T val) {
196 #pragma unroll
197     for (int mask = kWarpWidth >> 1; mask >= 1; mask >>= 1) {
198         val += __shfl_xor_sync(0xffffffff, val, mask, kWarpWidth);
199     }
200     return val;
201 }
202
203 // Warp Reduce Max — generic
204 template <const int kWarpWidth = kWarpSize, typename T = float>
205 __device__ __forceinline__ T warp_reduce_max(T val) {
206 #pragma unroll
207     for (int mask = kWarpWidth >> 1; mask >= 1; mask >>= 1) {
208         val = max(val, __shfl_xor_sync(0xffffffff, val, mask, kWarpWidth));
209     }
210     return val;
211 }
212
213 // =====
214 // Phase 1b: Block Reduce (block 内归约, 两级: warp → shared memory → warp reduce)
215 // =====
216
217 // Block Reduce Sum — FP32 (增强版, 带 broadcast)
218 // 两级归约流程:
219 // 1. 每个 warp 内做 warp_reduce_sum → 得到每 warp 的一个值
220 // 2. warp leader (lane=0) 写入 shared memory
221 // 3. syncthreads 后, lane 0~kNumWarps-1 读取 shared memory
222 // 4. 所有 warp 内再做一次 warp_reduce<kNumWarps> → 得到最终结果
223 // 5. __shfl_sync broadcast 到所有线程 (关键! 否则每个warp只有lane<kNumWarps 知道结果)
224 template <const int kNumThreads = 256>
225 __device__ float block_reduce_sum(float val) {
226     constexpr int kNumWarps = (kNumThreads + kWarpSize - 1) / kWarpSize;
227     int warp = threadIdx.x / kWarpSize;
228     int lane = threadIdx.x % kWarpSize;
229     __shared__ float shared[kNumWarps];
230
231     float value = warp_reduce_sum<kWarpSize>(val);
232     if (lane == 0)
233         shared[warp] = value;
234     __syncthreads();
235     value = (lane < kNumWarps) ? shared[lane] : 0.0f;
236     value = warp_reduce_sum<kNumWarps>(value);
237     // 关键: broadcast 结果到所有线程, 后续用 result
238     // 做除法等操作时所有线程都能拿到
239     value = __shfl_sync(0xffffffff, value, 0, 32);
240     return value;
241 }
242
243 // Block Reduce Max — FP32 (增强版, 带 broadcast)
244 template <const int kNumThreads = 256>
245 __device__ float block_reduce_max(float val) {
246     constexpr int kNumWarps = (kNumThreads + kWarpSize - 1) / kWarpSize;
247     int warp = threadIdx.x / kWarpSize;
248     int lane = threadIdx.x % kWarpSize;
249     __shared__ float shared[kNumWarps];
250
251     float value = warp_reduce_max<kWarpSize>(val);

```

```

252     if (lane == 0)
253         shared[warp] = value;
254     __syncthreads();
255     value = (lane < kNumWarps) ? shared[lane] : -FLT_MAX;
256     value = warp_reduce_max<kNumWarps>(value);
257     value = __shfl_sync(0xffffffff, value, 0, 32);
258     return value;
259 }
260
261 // ---- Block Reduce Sum All: y = sum(a[0..N-1]) ----
262 // 多 block 各自做 warp->smem->warp0 reduce, 然后 atomicAdd 到全局 y
263 // 跨 block 求和的常见模式, 适合 N 较大时使用;
264 // Grid: ((N + 255) / 256, 1, 1)
265 // Block: (256, 1, 1)
266 // source: LeetCUDA/kernels/reduce/block_all_reduce.cu
267 template <const int kNumThreads = 256>
268 __global__ void block_reduce_all(float *a, float *y, int N) {
269     int tid = threadIdx.x;
270     int idx = blockIdx.x * kNumThreads + tid;
271     constexpr int kNumWarps = (kNumThreads + kWarpSize - 1) / kWarpSize;
272     __shared__ float shared[kNumWarps];
273
274     float val = (idx < N) ? a[idx] : 0.0f;
275     int warp = tid / kWarpSize;
276     int lane = tid % kWarpSize;
277
278     val = warp_reduce_sum<kWarpSize>(val);
279     if (lane == 0)
280         shared[warp] = val;
281     __syncthreads();
282
283     val = (lane < kNumWarps) ? shared[lane] : 0.0f;
284     if (warp == 0)
285         val = warp_reduce_sum<kNumWarps>(val);
286     if (tid == 0) // tid == 0, not lane 0. 只有 block 内的一个线程负责写回全局结果, 避免重复累加
287         atomicAdd(y, val);
288 }
289
290 // ---- Dot Product: y = sum(a[i] * b[i]) ----
291 // 核心模式: elementwise 乘法 -> block reduce -> atomicAdd 全局累加
292 // Grid: ((N + 255) / 256, 1, 1)
293 // Block: (256, 1, 1)
294 // source: LeetCUDA/kernels/dot-product/dot_product.cu
295 template <const int kNumThreads = 256>
296 __global__ void dot(float *a, float *b, float *y, int N) {
297     int tid = threadIdx.x;
298     int idx = blockIdx.x * kNumThreads + tid;
299     constexpr int kNumWarps = (kNumThreads + kWarpSize - 1) / kWarpSize;
300     __shared__ float shared[kNumWarps];
301
302     float prod = (idx < N) ? a[idx] * b[idx] : 0.0f;
303     int warp = tid / kWarpSize;
304     int lane = tid % kWarpSize;
305
306     prod = warp_reduce_sum<kWarpSize>(prod);
307     if (lane == 0)
308         shared[warp] = prod;
309     __syncthreads();
310
311     prod = (lane < kNumWarps) ? shared[lane] : 0.0f;
312     if (warp == 0) // 只需要 warp 0 的线程继续 reduce 即可
313         prod = warp_reduce_sum<kNumWarps>(prod);
314     if (tid == 0) // tid == 0, not lane 0. 只有 block 内的一个线程负责写回全局结果, 避免重复累加

```

```

315     atomicAdd(y, prod);
316 }
317
318 // Dot Product + float4
319 // Grid: ((N + 255) / 256, 1, 1), 每个block处理256元素, float4向量化后每个线程处理4元素
320 // Block: (64, 1, 1), 256/4=64
321 // 注意: 该版本默认输入地址满足 float4 对齐; 最适合 N 按 4 对齐的场景
322 // source: LeetCUDA/kernels/dot-product/dot_product.cu
323 template <const int kNumThreads = 256 / 4>
324 __global__ void dot_vec4(float *a, float *b, float *y, int N) {
325     int tid = threadIdx.x;
326     int idx = (blockIdx.x * kNumThreads + tid) * 4;
327     constexpr int kNumWarps = (kNumThreads + kWarpSize - 1) / kWarpSize;
328     __shared__ float shared[kNumWarps];
329
330     float4 reg_a = FLOAT4(a[idx]);
331     float4 reg_b = FLOAT4(b[idx]);
332     float prod = (idx < N) ? (reg_a.x * reg_b.x + reg_a.y * reg_b.y +
333                             reg_a.z * reg_b.z + reg_a.w * reg_b.w)
334                       : 0.0f;
335     int warp = tid / kWarpSize;
336     int lane = tid % kWarpSize;
337
338     prod = warp_reduce_sum<kWarpSize>(prod);
339     if (lane == 0)
340         shared[warp] = prod;
341     __syncthreads();
342
343     prod = (lane < kNumWarps) ? shared[lane] : 0.0f;
344     if (warp == 0)
345         prod = warp_reduce_sum<kNumWarps>(prod);
346     if (tid == 0)
347         atomicAdd(y, prod);
348 }
349
350
351 // =====
352 // Phase 2: Elementwise Ops (逐元素操作, 演示 coalesced access + vectorize)
353 // =====
354 // 面试要点:
355 //   - 逐元素操作是最简单的 kernel, 核心考点是 memory coalescing
356 //   - float4 向量化可将内存事务数减为 1/4, 大幅提升 bandwidth utilization
357 //   - grid/block 维度设计: grid(N/threads), block(threads), 一维即可
358
359 // ---- ReLU: y = max(0, x) ----
360 // Grid: ((N + 255) / 256, 1, 1)
361 // Block: (256, 1, 1)
362 // source: LeetCUDA/kernels/relu/relu.cu
363 __global__ void relu(float *x, float *y, int N) {
364     int idx = blockIdx.x * blockDim.x + threadIdx.x;
365     if (idx < N)
366         y[idx] = fmaxf(0.0f, x[idx]);
367 }
368
369 // ReLU + float4 向量化: 每个线程处理 4 个元素, 减少 75% 的 load/store 指令
370 // block(64)x4(float4)=256 元素/block, 与基础版吞吐相同
371 // Grid: ((N + 255) / 256, 1, 1)
372 // Block: (64, 1, 1)
373 // 注意: 该版本默认地址满足 float4 对齐; 最适合 N 按 4 对齐的场景
374 // source: LeetCUDA/kernels/relu/relu.cu
375 __global__ void relu_vec4(float *x, float *y, int N) {
376     int idx = (blockIdx.x * blockDim.x + threadIdx.x) * 4;
377     if (idx < N) {

```

```

378     float4 reg_x = FLOAT4(x[idx]); // 单条 128-bit load
379     float4 reg_y;
380     reg_y.x = fmaxf(0.0f, reg_x.x);
381     reg_y.y = fmaxf(0.0f, reg_x.y);
382     reg_y.z = fmaxf(0.0f, reg_x.z);
383     reg_y.w = fmaxf(0.0f, reg_x.w);
384     FLOAT4(y[idx]) = reg_y; // 单条 128-bit store
385 }
386 }
387
388 // ---- Elementwise Add: c = a + b ----
389 // Grid: ((N + 255) / 256, 1, 1)
390 // Block: (256, 1, 1)
391 // source: LeetCUDA/kernels/elementwise/elementwise.cu
392 __global__ void elementwise_add(float *a, float *b, float *c, int N) {
393     int idx = blockIdx.x * blockDim.x + threadIdx.x;
394     if (idx < N)
395         c[idx] = a[idx] + b[idx];
396 }
397
398 // Elementwise Add + float4 向量化
399 // Grid: ((N + 255) / 256, 1, 1)
400 // Block: (64, 1, 1), block(64)*4(float4)=256 元素/block
401 // 注意: 主路径要求 4 元素对齐; 尾部不足 4 个元素时回退到标量处理
402 // source: LeetCUDA/kernels/elementwise/elementwise.cu
403 __global__ void elementwise_add_vec4(float *a, float *b, float *c, int N) {
404     int idx = 4 * (blockIdx.x * blockDim.x + threadIdx.x);
405     if ((idx + 3) < N) {
406         float4 reg_a = FLOAT4(a[idx]);
407         float4 reg_b = FLOAT4(b[idx]);
408         float4 reg_c;
409         reg_c.x = reg_a.x + reg_b.x;
410         reg_c.y = reg_a.y + reg_b.y;
411         reg_c.z = reg_a.z + reg_b.z;
412         reg_c.w = reg_a.w + reg_b.w;
413         FLOAT4(c[idx]) = reg_c;
414     } else if (idx < N) {
415         for (int i = 0; (idx + i) < N; i++) {
416             c[idx + i] = a[idx + i] + b[idx + i];
417         }
418     }
419 }
420
421 // ---- Histogram: y[a[i]]++ ----
422 // 演示 atomicAdd 的用法: 多个线程可能同时更新同一个 bin
423 // Grid: ((N + 255) / 256, 1, 1)
424 // Block: (256, 1, 1)
425 // source: LeetCUDA/kernels/histogram/histogram.cu
426 __global__ void histogram(int *a, int *y, int N) {
427     int idx = blockIdx.x * blockDim.x + threadIdx.x;
428     if (idx < N)
429         atomicAdd(&(y[a[idx]]), 1);
430 }
431
432 // ---- Merge Attn States: 合并 Split-KV Attention 的部分结果 ----
433 // 实现 FlashInfer 论文 (arXiv 2501.01005) Section 2.2 的 attention 合并逻辑。
434 //
435 // 面试要点:
436 //   - 应用场景: Split-KV attention 将序列沿 K 维分段计算 attention,
437 //     每段产出部分结果 (O_i, LSE_i), 本 kernel 将两段按指数权重加权合并。
438 //   - 数学公式 (5 步推导):
439 //     Step 1 — max 归一化 (数值稳定, 与 safe softmax 同理):
440 //         L_max = max(LSE_1, LSE_2)

```



```

441 // Step 2 — 还原指数权重 (un-normalized) :
442 //      w_1 = exp(LSE_1 - L_max), w_2 = exp(LSE_2 - L_max)
443 // Step 3 — 归一化为混合比例:
444 //      alpha = w_1 / (w_1 + w_2), beta = w_2 / (w_1 + w_2)
445 //      满足 alpha + beta = 1
446 // Step 4 — 逐元素加权合并:
447 //      O = alpha * O_1 + beta * O_2
448 // - LSE 布局 [num_heads, num_tokens] (与 FlashAttention-2惯例一致,
449 //   lse[head_idx][token_idx])
450 // - Output 布局 [num_tokens, num_heads, head_size] (token 维在最外,
451 //   展平为 [T0H0, T0H1, ..., T1H0, ...])
452 // -  $-\infty$  LSE  $\rightarrow$   $-\infty$ : 空 attention 段 (causal mask 等导致全部 score
453 //   为  $-\infty$ ) 的 LSE 可能为  $+\infty$ , 替换后  $\exp(-\infty - L_{\max}) = 0$ ,
454 //   该段权重退化为 0
455 // - 128-bit 向量化: uint4 = 4  $\times$  float, 每线程处理 4 个输出元素,
456 //   pack load  $\rightarrow$  FMA  $\rightarrow$  pack store 一条流水线
457 // - AI  $\approx$  3 FLOP / 20 bytes  $\approx$  0.15 FLOPS/Byte  $\rightarrow$  severely memory-bound
458 //
459 // 线程到数据的 3D 映射 (以 num_tokens=2, num_heads=2, head_size=8 为例):
460 // kPackSize = 16 / sizeof(float) = 4 (每线程 4 个 float = 128-bit)
461 // threads_per_head = head_size / 4 = 2
462 // total_threads = num_tokens * num_heads * threads_per_head = 8
463 //
464 // global_idx:      0      1      2      3      4      5      6      7
465 // token_head_idx:  0      0      1      1      2      2      3      3
466 //   (第几个 (token,head) 对, 行优先)
467 // pack_idx:        0      1      0      1      0      1      0      1
468 //   (该 (token,head) 对内第几个 pack)
469 // token_idx:        0      0      0      0      1      1      1      1
470 //   (token_head_idx / num_heads, token 变化慢)
471 // head_idx:         0      0      1      1      0      0      1      1
472 //   (token_head_idx % num_heads, head 变化快)
473 // pack_offset:      0      4      0      4      0      4      0      4
474 //   (pack_idx * 4, 该 pack 在 head_size 中的起始元素偏移)
475 // head_offset:      0      0      8      8      16     16     24     24
476 //   (token_idx * num_heads * head_size + head_idx * head_size,
477 //   该 (token,head) 对在 output 展平数组中的起始偏移)
478 //
479 // Grid: ((total_threads + 127) / 128, 1, 1)
480 // Block: (128, 1, 1)
481 // source: LeetCUDA/kernels/openai-triton/merge-attn-states/cuda_merge_attn_states.cu
482 __global__ void merge_attn_states(
483     float *output,           // [num_tokens, num_heads, head_size]
484     const float *prefix_output, // [num_tokens, num_heads, head_size]
485     const float *prefix_lse,   // [num_heads, num_tokens]
486     const float *suffix_output, // [num_tokens, num_heads, head_size]
487     const float *suffix_lse,   // [num_heads, num_tokens]
488     int num_tokens, int num_heads, int head_size) {
489
490     constexpr int kNumThreads = 128;
491     constexpr int kPackSize = 16 / sizeof(float); // 4 floats = 128-bit
492     using pack_t = uint4;
493
494     // 每个 (token, head) 对需要 threads_per_head 个线程覆盖 head_size 个元素
495     const int threads_per_head = head_size / kPackSize;
496     // 实际有效总线程数, 超过这个数的线程会被忽略, block是按照kNumThreads=128
497     // 来分配的, 那么最后一个block的线程数可能会超过实际需要的线程数
498     const int total_threads = num_tokens * num_heads * threads_per_head;
499
500     const int global_idx = blockIdx.x * kNumThreads + threadIdx.x;
501     if (global_idx >= total_threads)
502         return;
503

```

```

504 // global_idx → (token_head_idx, pack_idx) → (token_idx, head_idx, pack_offset)
505 // token_head_idx: 第几个 (token, head) 对, 行优先展平
506 const int token_head_idx = global_idx / threads_per_head;
507 // pack_idx: 该 (token, head) 对内第几个 pack, 0 ~ threads_per_head-1
508 const int pack_idx = global_idx % threads_per_head;
509
510 // token_head_idx 分解为 (token_idx, head_idx)
511 const int token_idx = token_head_idx / num_heads; // token 变化慢 (外维)
512 const int head_idx = token_head_idx % num_heads; // head 变化快 (内维)
513
514 // pack_offset: 该 pack 覆盖的元素在 head_size 中的起始偏移 (0, 4, 8, ...)
515 const int pack_offset = pack_idx * kPackSize;
516 // head_offset: 该 (token, head) 对在 output 展平一维数组中的起始偏移
517 const int head_offset = token_idx * num_heads * head_size + head_idx * head_size;
518
519 // 定位到当前 (token, head) 的输出段起始
520 const float *prefix_head = prefix_output + head_offset;
521 const float *suffix_head = suffix_output + head_offset;
522 float *output_head = output + head_offset;
523
524 // LSE 布局 [num_heads, num_tokens]: lse[head_idx][token_idx]
525 float p_lse = prefix_lse[head_idx * num_tokens + token_idx];
526 float s_lse = suffix_lse[head_idx * num_tokens + token_idx];
527
528 // inf → -inf: 空 attention 段 LSE 可能为 +inf, 替换后权重退化为 0
529 p_lse = isinf(p_lse) ? -INFINITY : p_lse;
530 s_lse = isinf(s_lse) ? -INFINITY : s_lse;
531
532 // Step 1: max 归一化 (与 safe softmax 同理, 防 exp 溢出)
533 const float max_lse = fmaxf(p_lse, s_lse);
534 p_lse -= max_lse;
535 s_lse -= max_lse;
536
537 // Step 2-3: 指数还原 → 混合比例 alpha, beta
538 const float p_se = expf(p_lse);
539 const float s_se = expf(s_lse);
540 const float out_se = p_se + s_se;
541 const float p_scale = p_se / out_se; // alpha = w_1 / (w_1 + w_2)
542 const float s_scale = s_se / out_se; // beta = w_2 / (w_1 + w_2)
543
544 // Step 4: 逐元素加权合并 0 = alpha * 0_1 + beta * 0_2
545 // 128-bit 向量化: uint4 load → per-element FMA → uint4 store
546 if (pack_offset < head_size) {
547     pack_t p_pack =
548         reinterpret_cast<const pack_t *>(prefix_head)[pack_idx];
549     pack_t s_pack =
550         reinterpret_cast<const pack_t *>(suffix_head)[pack_idx];
551     pack_t o_pack;
552
553     #pragma unroll
554     for (int i = 0; i < kPackSize; ++i) {
555         const float p_v = reinterpret_cast<const float *>(&p_pack)[i];
556         const float s_v = reinterpret_cast<const float *>(&s_pack)[i];
557         const float o_v = p_v * p_scale + (s_v * s_scale); // FMA
558         reinterpret_cast<float *>(&o_pack)[i] = o_v;
559     }
560
561     reinterpret_cast<pack_t *>(output_head)[pack_idx] = o_pack;
562 }
563 }
564
565 // =====
566 // Phase 3: Reduce 类 Ops — Softmax / RMS Norm / Layer Norm

```

```

567 // =====
568 // 面试要点:
569 //   - Softmax 三种实现递进: naive (溢出) → safe (2-pass, max 减法) →
570 //   online (1-pass, 增量更新)
571 //   - Online Softmax 是 FlashAttention-2的数学基础
572 //   - RMS Norm vs Layer Norm: RMS 只需 1 次 reduce, Layer Norm 需要 2 次 (mean
573 //   + variance)
574 //   - Per-token 设计: 一个 block 处理一个 token, 无需跨 block 同步
575
576 // =====
577 // Phase 3a: Softmax — 三级递进 (面试核心考点)
578 // =====
579 // 面试常问: 「Softmax 有哪些实现方式? 各有什么优缺点? 」
580 // 回答线索: naive(溢出) → safe → online
581
582 // ---- Level 1: 基础 Softmax (per-token, 无 max 减法, 数值不稳定) ----
583 // grid(S*h/h, h), block(h), 一个 block 处理一个 token
584 // 问题: x 值很大时 exp(x) 溢出为 inf
585 template <const int kNumThreads = 256>
586 // Grid: (S, 1, 1), S=batch*seq_len, DISPATCH_SOFTMAX_F32_PER_TOKEN_KERNEL
587 // Block: (H, 1, 1), 由外层 dispatch 选择 H=32/64/128/256/512/1024, 一个 block 处理一个 token
588 // source: LeetCUDA/kernels/softmax/softmax.cu
589 __global__ void softmax_per_token(float *x, float *y, int N) {
590     const int tid = threadIdx.x;
591     const int idx = blockIdx.x * blockDim.x + tid;
592
593     float exp_val = (idx < N) ? expf(x[idx]) : 0.0f;
594     float exp_sum = block_reduce_sum<kNumThreads>(exp_val);
595     if (idx < N)
596         y[idx] = exp_val / exp_sum;
597 }
598
599 // ---- Level 2: Safe Softmax (2-pass: 先 max 再 exp, 数值稳定) ----
600 // 面试重点: 为什么 Softmax 需要 Safe?
601 //   - expf 溢出阈值约 88.7: exp(88)≈1.65e38 (接近 float32 上限 3.4e38), exp(89)≈4.5e38 已溢出为 inf
602 //   - 减去 max 后: exp(x - max) ≤ exp(0) = 1.0, 永不超过 1
603 //   - 数学等价性: softmax(x) = softmax(x - c) 对任意常数 c 成立
604 //   - 代价: 2 次 block reduce (先 max, 再 sum), 但仍 O(N/B) 高效
605 // Grid: (S, 1, 1)
606 // Block: (H, 1, 1), 由外层 dispatch 选择 H=32/64/128/256/512/1024
607 // source: LeetCUDA/kernels/softmax/softmax.cu
608 template <const int kNumThreads = 256>
609 __global__ void safe_softmax_per_token(float *x, float *y, int N) {
610     const int tid = threadIdx.x;
611     const int idx = blockIdx.x * blockDim.x + tid;
612
613     // Pass 1: block reduce max — 找最大值
614     float val = (idx < N) ? x[idx] : -FLT_MAX;
615     float max_val = block_reduce_max<kNumThreads>(val);
616
617     // Pass 2: exp(x - max) → block reduce sum
618     float exp_val = (idx < N) ? expf(x[idx] - max_val) : 0.0f;
619     float exp_sum = block_reduce_sum<kNumThreads>(exp_val);
620
621     if (idx < N)
622         y[idx] = exp_val / exp_sum;
623 }
624
625 // ---- Level 3: Online Safe Softmax (FlashAttention-2的数学基础) ----
626 // 面试重点:
627 // 两种使用场景 (公式不同, 但结果等价) :
628 //   1) 单元素增量更新 (online update, 处理新元素 x_i):
629 //       m_new = max(m_old, x_i)

```

```

630 //      d_new = d_old * exp(m_old - m_new) + exp(x_i - m_new)
631 //  2) 二元合并 (binary merge, 合并两个部分累加器, warp/block reduce 使用):
632 //      m = max(m1, m2) safe softmax用的max值
633 //      d = d1*exp(m1-m) + d2*exp(m2-m) softmax用的分母值
634 //      当 m1≥m2 时退化为 d1 + d2*exp(m2-m1) (d_bigger + d_smaller*exp(m_smaller - m_bigger))
635 //  算法来源: "Online normalizer calculation for softmax" (arXiv:1805.02867)
636 //  Warp Reduce for Online Softmax — binary merge of two partial (m,d) accumulators.
637 //
638 //  不同于单元元素增量更新公式 (m_new = max(m_old, x_i); d_new = d_old*exp(m_old-m_new) + exp(x_i-m_new)) ,
639 //  warp reduce 中每次合并的是两个**已各自归一化到各自 max 的部分累加器**:
640 //  (m1,d1): max 和  $\sum \exp(x_j - m1)$ , 覆盖集合 S1
641 //  (m2,d2): max 和  $\sum \exp(x_k - m2)$ , 覆盖集合 S2
642 //
643 //  合并公式 (对称, 不依赖"新/旧"概念) :
644 //  m = max(m1, m2)
645 //  d = d1*exp(m1-m) + d2*exp(m2-m) ← m1≥m2 时退化为 d1 + d2*exp(m2-m1)
646 //
647 //  该操作满足结合律 → XOR butterfly 任意归约顺序均保证全局正确结果。
648 struct __align__(8) MD {
649     float m; // running max
650     float d; // running denominator (sum of exp(x - max))
651 };
652
653 template <const int kWarpWidth = kWarpSize>
654 __device__ __forceinline__ MD warp_reduce_md(MD md1) {
655     #pragma unroll
656     for (int mask = kWarpWidth >> 1; mask >= 1; mask >>= 1) {
657         MD md2;
658         md2.m = __shfl_xor_sync(0xffffffff, md1.m, mask, kWarpWidth);
659         md2.d = __shfl_xor_sync(0xffffffff, md1.d, mask, kWarpWidth);
660
661         MD b_m = (md1.m > md2.m) ? md1 : md2; // max
662         MD s_m = (md1.m > md2.m) ? md2 : md1;
663
664         // b_m.d 无需 rescale (其基准 max 已是全局 max) ,
665         // s_m.d 需 rescale: exp(s_m.m - b_m.m)
666         md1.d = b_m.d + s_m.d * __expf(s_m.m - b_m.m);
667         md1.m = b_m.m;
668     }
669     return md1;
670 }
671
672 // 注意: 这里默认一个 block 处理一个 token; 边界线程的 d=0 只参与归约, 不会写回 y
673 // Grid: (S, 1, 1)
674 // Block: (H, 1, 1), 由外层 dispatch 选择 H=32/64/128/256/512/1024
675 // source: LeetCUDA/kernels/softmax/softmax.cu
676 template <const int kNumThreads = 256>
677 __global__ void online_safe_softmax_per_token(const float *x, float *y, int N) {
678     int tid = threadIdx.x;
679     int idx = blockIdx.x * kNumThreads + threadIdx.x;
680     const int kNumWarps = (kNumThreads + kWarpSize - 1) / kWarpSize;
681     __shared__ MD shared[kNumWarps];
682
683     float val = (idx < N) ? x[idx] : -FLT_MAX;
684     int warp = tid / kWarpSize;
685     int lane = tid % kWarpSize;
686
687     // 初始化: 每个线程持有一个 (max, denom) 对
688     MD md;
689     md.m = idx < N ? val : -FLT_MAX;
690     md.d = idx < N ? 1.0f : 0.0f;
691
692     // 第一级规约: warp_reduce_md 在归约中自动更新 m 和 d

```

```

693 md = warp_reduce_md<kWarpSize>(md);
694
695 if (lane == 0)
696     shared[warp] = md;
697 __syncthreads();
698
699 // 第二级归约: 每个 warp 结果再做一次 warp_reduce_md (复用 block_reduce 模式)
700 md = lane < kNumWarps ? shared[lane] : MD{-FLT_MAX, 0.0f};
701 md = warp_reduce_md<kWarpSize>(md); // 用kWarpSize确保每个lane都能拿到最终结果
702
703 // 用全局 max 和 denom 做最终 softmax
704 float d_inv = __fdividef(1.0f, md.d);
705 // 边界线程即使看到 d=0 的填充值, 也不会走到写回路径
706 if (idx < N) {
707     y[idx] = __expf(val - md.m) * d_inv;
708 }
709 }
710
711 // =====
712 // Phase 3b: RMS Normalization (1-pass reduce)
713 // =====
714 // 面试要点:
715 //   - RMS Norm:  $y = (x / \text{rms}(x)) * g$ ,  $1/\text{rms}(x) = \text{rsqrt}(\text{mean}(x^2))$ 
716 //   - 只需 1 次 block reduce (sum of squares), 比 Layer Norm 少 1 次同步
717 //   - Llama 系列使用 RMS Norm
718 //   - grid(N, K/K), block(K): 一行一个 block
719 // Grid: (N, 1, 1), N=batch*seq_len, 每行一个 block
720 // Block: (128, 1, 1), kNumThreads=K=128 (K>128 时调整模板参数)
721 // source: LeetCUDA/kernels/rms-norm/rms_norm.cu
722 template <const int kNumThreads = 128>
723 __global__ void rms_norm(float *x, float *y, float g, int N, int K) {
724     int tid = threadIdx.x;
725     int idx = blockIdx.x * blockDim.x + threadIdx.x;
726     const float epsilon = 1e-5f;
727
728     __shared__ float s_variance;
729     float value = (idx < N * K) ? x[idx] : 0.0f;
730     float variance = value * value;
731     variance = block_reduce_sum<kNumThreads>(variance);
732     if (tid == 0)
733         s_variance = rsqrtf(variance / (float)K + epsilon); // 1/rms(x)
734     __syncthreads();
735     if (idx < N * K)
736         y[idx] = (value * s_variance) * g;
737 }
738
739 // RMS Norm + float4
740 // Grid: (N, 1, 1)
741 // Block: (32, 1, 1), 128/4=32; 对应一行 K 元素按 4 个一组交给 32 个线程处理
742 // 注意: 该版本默认 K 按 4 对齐, 且输入/输出地址满足 float4 对齐
743 // source: LeetCUDA/kernels/rms-norm/rms_norm.cu
744 template <const int kNumThreads = 128 / 4>
745 __global__ void rms_norm_vec4(float *x, float *y, float g, int N, int K) {
746     int tid = threadIdx.x;
747     int idx = (blockIdx.x * blockDim.x + threadIdx.x) * 4;
748     const float epsilon = 1e-5f;
749
750     __shared__ float s_variance;
751     float4 reg_x = FLOAT4(x[idx]);
752     float variance = (idx < N * K) ? (reg_x.x * reg_x.x + reg_x.y * reg_x.y +
753                                     reg_x.z * reg_x.z + reg_x.w * reg_x.w)
754                                     : 0.0f;
755     variance = block_reduce_sum<kNumThreads>(variance);

```

```

756     if (tid == 0)
757         s_variance = rsqrtf(variance / (float)K + epsilon);
758     __syncthreads();
759     float4 reg_y;
760     reg_y.x = reg_x.x * s_variance * g;
761     reg_y.y = reg_x.y * s_variance * g;
762     reg_y.z = reg_x.z * s_variance * g;
763     reg_y.w = reg_x.w * s_variance * g;
764     if (idx < N * K)
765         FLOAT4(y[idx]) = reg_y;
766 }
767
768 // =====
769 // Phase 3c: Layer Normalization (2-pass reduce)
770 // =====
771 // 面试要点:
772 //   - Layer Norm:  $y = ((x - \text{mean}) / \text{std}) * g + b$ ,  $\text{std} = \sqrt{\text{variance}}$ ,  $\text{variance} = \text{mean}((x - \text{mean})^2)$ 
773 //   - 需要 2 次 block reduce: 先 mean (sum/K), 再 variance (sum((x-mean)2)/K)
774 //   - 两次 __syncthreads 必须到位, 否则 s_mean 未对所有线程可见就计算 variance
775 // Grid: (N, 1, 1), 一行一个 block
776 // Block: (128, 1, 1), kNumThreads=K=128
777 // source: LeetCUDA/kernels/layer-norm/layer_norm.cu
778 template <const int kNumThreads = 128>
779 __global__ void layer_norm(float *x, float *y, float g, float b, int N, int K) {
780     int tid = threadIdx.x;
781     int idx = blockIdx.x * blockDim.x + threadIdx.x;
782     const float epsilon = 1e-5f;
783
784     __shared__ float s_mean;
785     __shared__ float s_variance;
786     float value = (idx < N * K) ? x[idx] : 0.0f;
787
788     // Pass 1: compute mean
789     float sum = block_reduce_sum<kNumThreads>(value);
790     if (tid == 0)
791         s_mean = sum / (float)K;
792     __syncthreads(); // 必须等待 s_mean 对所有线程可见
793
794     // Pass 2: compute variance = (x - mean)2
795     float variance = (value - s_mean) * (value - s_mean);
796     variance = block_reduce_sum<kNumThreads>(variance);
797     if (tid == 0)
798         s_variance = rsqrtf(variance / (float)K + epsilon); // 1/std
799     __syncthreads(); // 必须等待 s_variance 对所有线程可见
800
801     if (idx < N * K)
802         y[idx] = ((value - s_mean) * s_variance) * g + b;
803 }
804
805 // Layer Norm + float4
806 // Grid: (N, 1, 1)
807 // Block: (32, 1, 1), 128/4=32; 对应一行 K 元素按 4 个一组交给 32 个线程处理
808 // 注意: 该版本默认 K 按 4 对齐, 且输入/输出地址满足 float4 对齐
809 // source: LeetCUDA/kernels/layer-norm/layer_norm.cu
810 template <const int kNumThreads = 128 / 4>
811 __global__ void layer_norm_vec4(float *x, float *y, float g, float b, int N, int K) {
812     int tid = threadIdx.x;
813     int idx = (blockIdx.x * blockDim.x + threadIdx.x) * 4;
814     const float epsilon = 1e-5f;
815
816     __shared__ float s_mean;
817     __shared__ float s_variance;
818     float4 reg_x = FLOAT4(x[idx]);

```

```

819 float value = (idx < N * K) ? (reg_x.x + reg_x.y + reg_x.z + reg_x.w) : 0.0f;
820
821 float sum = block_reduce_sum<kNumThreads>(value);
822 if (tid == 0)
823     s_mean = sum / (float)K;
824 __syncthreads();
825
826 float4 reg_x_hat;
827 reg_x_hat.x = reg_x.x - s_mean;
828 reg_x_hat.y = reg_x.y - s_mean;
829 reg_x_hat.z = reg_x.z - s_mean;
830 reg_x_hat.w = reg_x.w - s_mean;
831 float variance = reg_x_hat.x * reg_x_hat.x + reg_x_hat.y * reg_x_hat.y +
832                 reg_x_hat.z * reg_x_hat.z + reg_x_hat.w * reg_x_hat.w;
833 variance = block_reduce_sum<kNumThreads>(variance);
834 if (tid == 0)
835     s_variance = rsqrtf(variance / (float)K + epsilon);
836 __syncthreads();
837
838 float4 reg_y;
839 reg_y.x = reg_x_hat.x * s_variance * g + b;
840 reg_y.y = reg_x_hat.y * s_variance * g + b;
841 reg_y.z = reg_x_hat.z * s_variance * g + b;
842 reg_y.w = reg_x_hat.w * s_variance * g + b;
843 if (idx < N * K)
844     FLOAT4(y[idx]) = reg_y;
845 }
846
847 // =====
848 // Phase 4: RoPE — 旋转位置编码 (Rotary Position Embedding)
849 // =====
850 // 面试要点:
851 //   - RoPE 数学公式: 对每对相邻维度做 2D 旋转
852 //     [x1']   [cos(θ)  -sin(θ)] [x1]
853 //     [x2'] = [sin(θ)   cos(θ)] [x2]
854 //   -  $\theta_i = 1 / (\text{theta}^{(2i/d)})$ , theta=10000.0f (Llama 风格)
855 //   - token_pos = idx / N: token 在序列中的位置
856 //   - token_idx = idx % N: token 内的维度对索引
857 //   - 输入 [seq_len, hidden_size], 输出同形状
858
859 // Grid: ((seq_len * N + 255) / 256, 1, 1)
860 // Block: (256, 1, 1)
861 // source: LeetCUDA/kernels/rope/rope.cu
862 __global__ void rope(float *x, float *out, int seq_len, int N) {
863     int idx = blockIdx.x * blockDim.x + threadIdx.x;
864     float x1 = x[idx * 2];
865     float x2 = x[idx * 2 + 1];
866     int token_pos = idx / N; // 序列位置
867     int token_idx = idx % N; // 维度对索引
868
869     // 频率计算:  $\theta_i = 1 / (10000^{(2i/d)})$ 
870     float exp_v = 1.0f / powf(10000.0f, 2 * token_idx / (N * 2.0f));
871     float sin_v = sinf(token_pos * exp_v);
872     float cos_v = cosf(token_pos * exp_v);
873
874     // 2D 旋转
875     float out1 = x1 * cos_v - x2 * sin_v;
876     float out2 = x1 * sin_v + x2 * cos_v;
877     out[idx * 2] = out1;
878     out[idx * 2 + 1] = out2;
879 }
880
881 // =====

```



```

882 // Phase 5: Mat Transpose — 矩阵转置 (Bank Conflict 专题)
883 // =====
884 // 面试要点 (Bank Conflict 专题) :
885 //   - Shared memory 有 32 个 bank, 每个 bank 4 bytes (32-bit)
886 //   - 同一 warp 的多个线程访问同一 bank 的不同地址 → bank conflict
887 //   - n-way bank conflict: n 个线程冲突 → 访问串行化为 n 次
888 //   - 解决方案: PAD (在每行末尾加 1 个元素, 打破地址对齐)
889 //
890 // 转置的四步演进:
891 //   naive: 非合并写入 (列优先写) → 每个 warp 产生 32 次内存事务
892 //   shared: 写入 smem (行优先) → 从 smem 读取 (列优先) → 合并写入 gmem
893 //   BCF: smem 布局 [kWarpSize_S*4][kWarpSize_S+PAD] = [64][17], PAD=1 加在第二维消除 bank conflict
894 //   merge_write: 进一步将 4 次 separate store 合并为 1 次 float4 store
895 // 注: 本文件仅实现 Level 1(naive) 与 Level 4(BCF+merge_write), Level 2/3 省略
896
897 // 每个线程处理 1 个元素, block(16,16)
898 // 读: x 按行优先访问, warp 的 32 线程访问 32 个连续地址 → 合并读取 ✓
899 // 写: y 按列优先写入, 32 线程跨 row 行分散 → 非合并写入 x (32 次内存事务)
900 // Grid: ((col + 15) / 16, (row + 15) / 16, 1), 每线程 1 元素
901 // Block: (16, 16, 1)
902 // source: LeetCUDA/kernels/mat-transpose/mat_transpose.cu
903 __global__ void mat_transpose(float *x, float *y, const int row, const int col) {
904     const int c = blockIdx.x * blockDim.x + threadIdx.x; // col in x
905     const int r = blockIdx.y * blockDim.y + threadIdx.y; // row in x
906     if (r < row && c < col) {
907         // x[r][c] → y[c][r]; x stride=col, y (transposed) stride=row
908         y[c * row + r] = x[r * col + c];
909     }
910 }
911
912 // Grid: ((col + 15) / 16, (row + 63) / 64, 1), 每线程 4 元素(float4)
913 // Block: (16, 16, 1)
914 // 注意: 该版本默认按 float4 打包写回; 最适合 row 能按 4 对齐的场景
915 // source: LeetCUDA/kernels/mat-transpose/mat_transpose.cu
916 __global__ void mat_transpose_padded(
917     float *x, float *y, const int row, const int col) {
918     const int tx = threadIdx.x;
919     const int ty = threadIdx.y;
920
921     constexpr int TILE = 16;
922     constexpr int PAD = 1;
923     // Bank conflict fix: 每行多 1 个元素, 打破 32-bank 对齐
924     __shared__ float tile[TILE * 4][TILE + PAD]; // 64x16
925
926     // x 空间坐标; 每线程覆盖 4 行 (actual row = x_r * 4)
927     const int x_c = blockIdx.x * TILE + tx;
928     const int x_r = blockIdx.y * TILE + ty;
929
930     if (x_r * 4 < row && x_c < col) {
931         // Step 1: 4 rows × 1 col → smem (row-major);
932 #pragma unroll
933         for (int i = 0; i < 4; ++i) {
934             tile[ty * 4 + i][tx] = x[(x_r * 4 + i) * col + x_c];
935         }
936         __syncthreads();
937
938         // Step 2: Transposed read from smem
939         float4 reg_trans;
940         reg_trans.x = tile[tx * 4 + 0][ty];
941         reg_trans.y = tile[tx * 4 + 1][ty];
942         reg_trans.z = tile[tx * 4 + 2][ty];
943         reg_trans.w = tile[tx * 4 + 3][ty];
944     }

```



```

945 // y 空间坐标: y_r = x 的列, y_c = x 的行
946 const int y_r = blockIdx.x * TILE + ty; // = x col
947 const int y_c = (blockIdx.y * TILE + tx) * 4; // = x row
948
949 // Coalesced write: y[y_r][y_c .. y_c+3]
950 FLOAT4(y[y_r * row + y_c]) = reg_trans;
951 }
952 }
953
954 // =====
955 // Phase 6: GEMV — 矩阵向量乘 (M or N = 1, 纯 memory-bound 算子, warp-per-row 策略)
956 // =====
957 // 面试要点:
958 // - GEMV 是典型的 memory-bound 算子:  $AI \approx O(1)$ , 瓶颈在内存带宽
959 // - 核心策略: warp-per-row (每个 warp 负责矩阵的一行)
960 // - 不同 K 值对应不同分块策略:
961 //   K=32 倍数: 一个 warp 的 32 线程恰好覆盖 K 维
962 //   K=128 倍数: 每个线程用 float4 处理 4 元素, warp 覆盖 128
963 //   K=16 < 32: 一个 warp 用不满, 用 kRowPerWarp=2 让每个 warp 处理 2 行
964 //
965 // a: MxK, x: Kx1, y: Mx1, 计算:  $y = a * x$ ; N = 1
966
967 // ---- SGEMV K32: 基础 warp-per-row ----
968 // 设计: block(32, 4), blockDim.x=kWarpSize=32 (K 需为 32 倍数时一轮覆盖, 否则内层循环 kNumWarps 次)
969 // grid(M/4), 每个 warp 负责一行
970 // K 为 32 的倍数时, warp 的 32 个线程恰好覆盖 K 维
971 // Grid: ((M + 3) / 4, 1, 1), 每 block 处理 4 行
972 // Block: (32, 4, 1), 每行 1 warp
973 // 注意: 该版本最适合 K 按 32 对齐; 当 K 更小时通常切到 K16 这类专用分支
974 // source: LeetCUDA/kernels/sgemv/sgemv.cu
975 __global__ void sgemv_k32(float *a, float *x, float *y, int M, int K) {
976     int tx = threadIdx.x; // 0~31
977     int ty = threadIdx.y; // 0~3
978     int lane = tx % kWarpSize; // 0~31
979     int m = blockIdx.x * blockDim.y + ty; // 全局行号
980     if (m < M) {
981         float sum = 0.0f;
982         // 沿 K 维的迭代数 = ceil(K/32), 每个 warp 要累加完整的K, 那么
983         // 每个thread就要负责累加NUM_ITERS个元素, NUM_ITERS = ceil(K/32)
984         const int NUM_ITERS = (K + kWarpSize - 1) / kWarpSize;
985 #pragma unroll
986         for (int w = 0; w < NUM_ITERS; ++w) {
987             // 假设K是32的整倍数, m * K 本行的起始地址, x: Kx1
988             int k = w * kWarpSize + lane;
989             sum += a[m * K + k] * x[k];
990         }
991         sum = warp_reduce_sum<kWarpSize>(sum);
992         // 每个 warp 处理一行, lane 0 写回结果
993         if (lane == 0)
994             y[m] = sum;
995     }
996 }
997
998 // ---- SGEMV K128: float4 向量化 ----
999 // 每个线程处理 4 个元素(float4), 一个 warp 覆盖 128 个元素
1000 // Grid: ((M + 3) / 4, 1, 1)
1001 // Block: (32, 4, 1)
1002 // 注意: 该版本最适合 K 按 128 对齐, 且 x/a 的地址满足 float4 对齐
1003 // source: LeetCUDA/kernels/sgemv/sgemv.cu
1004 __global__ void sgemv_k128(float *a, float *x, float *y, int M, int K) {
1005     int tx = threadIdx.x; // 0~31
1006     int ty = threadIdx.y; // 0~3
1007     int lane = tx % kWarpSize; // 0~31

```

```

1008 int m = blockDim.y * blockIdx.x + ty;
1009
1010 if (m < M) {
1011     float sum = 0.0f;
1012     // 沿 K 维的迭代数 = ceil(K/128), 每个 warp 每轮用 float4 覆盖 128 个 K 元素
1013     const int NUM_ITERS = ((K + kWarpSize - 1) / kWarpSize) + 4 - 1) / 4;
1014 #pragma unroll
1015     for (int w = 0; w < NUM_ITERS; ++w) {
1016         int k = (w * kWarpSize + lane) * 4;
1017         float4 reg_x = FLOAT4(x[k]);
1018         float4 reg_a = FLOAT4(a[m * K + k]);
1019         sum += (reg_a.x * reg_x.x + reg_a.y * reg_x.y + reg_a.z * reg_x.z +
1020             reg_a.w * reg_x.w);
1021     }
1022     sum = warp_reduce_sum<kWarpSize>(sum);
1023     if (lane == 0)
1024         y[m] = sum;
1025 }
1026 }
1027
1028 // ---- SGEMV K16: K < WarpSize, kRowPerWarp=2 ----
1029 // 面试亮点: K=16 < 32, 一个 warp 可以处理多行
1030 // kRowPerWarp=2, kNumLanePerRow=16, 前 16 个 lane 处理 row0, 后 16 个 lane 处理 row1
1031 // Grid: ((M + 7) / 8, 1, 1), NUM_ROWS=8
1032 // Block: (32, 4, 1)
1033 // 注意: 这一版是面向 K=16 的专用写法; kRowPerWarp=2 时一个 warp 同时处理 2 行
1034 // source: LeetCUDA/kernels/sgemv/sgemv.cu
1035 template <const int kRowPerWarp = 2>
1036 __global__ void sgemv_k16(float *A, float *x, float *y, int M, int K) {
1037     constexpr int kNumLanePerRow = (kWarpSize + kRowPerWarp - 1) / kRowPerWarp; // 16
1038     int tx = threadIdx.x;
1039     int ty = threadIdx.y;
1040     int lane = tx % kWarpSize;
1041     int k = lane % kNumLanePerRow; // row0: 0~15, t0~t15; row1: 0~15, t16~t31
1042     int m = (blockDim.y * blockIdx.x + ty) * kRowPerWarp + lane / kNumLanePerRow;
1043     if (m < M) {
1044         float sum = A[m * K + k] * x[k];
1045         // 按照kNumLanePerRow=16, 分2组各自做 warp reduce sum, k==0的lane写回结果
1046         sum = warp_reduce_sum<kNumLanePerRow>(sum);
1047         // 注意: 判断条件是 k == 0, 不是 lane == 0!
1048         if (k == 0)
1049             y[m] = sum;
1050     }
1051 }
1052
1053 // =====
1054 // Phase 7: GEMM — 矩阵矩阵乘 (GPU 最重要的算子, 面试核心考点)
1055 // =====
1056 // 面试要点 (GEMM 优化五层金字塔):
1057 //   Level 1 — Tiling (分块 + shared memory): 将数据从 HBM 搬到 SMEM 复用
1058 //   Level 2 — Thread Tile (寄存器分块): 每个线程计算 TM×TN
1059 //   个元素, 提高计算密度 Level 3 — Vectorize (向量化访存): float4/half2, 减少
1060 //   load/store 指令数 Level 4 — Tensor Core (MMA
1061 //   m16n8k16): 硬件矩阵乘单元, warp 级指令 Level 5 — Warp Specialization +
1062 //   TMA (WGMMMA m64n128k16): Hopper 异步执行
1063 //
1064 // 计算密度递进:
1065 //   Level 1: AI ≈ B_K / (2×sizeof) ≈ 32/8 = 4 → 仍是 memory-bound
1066 //   Level 2: AI ≈ TM×TN×B_K / (2×sizeof) ≈ 8×8×8/8 = 64 → compute-bound
1067 //   Level 4: Tensor Core 提供硬件加速的 256 FMA/cycle/warp → 大幅提升吞吐
1068
1069 // =====
1070 // Phase 7a: SGEMM (非 Tensor Core 路径)

```

```

1071 // =====
1072
1073 // ---- Level 1: SGEMM — Block Tile 32x32 + K Tile 32 ----
1074 // 最基础的 tiling 实现, 演示 shared memory 的核心用法
1075 // C = A x B, C[M, N] = A[M, K] x B[K, N]
1076 // BM=BN=32, BK=32, block(32, 32), 一个线程计算 c 的一个元素
1077 // Grid: ((N + 31) / 32, (M + 31) / 32, 1)
1078 // Block: (32, 32, 1), 1024 线程
1079 // source: LeetCUDA/kernels/sgemm/sgemm.cu
1080 __global__ void sgemm(float *a, float *b, float *c, int M, int N, int K) {
1081     constexpr int BM = 32; // vec 版: 32x4 = 128
1082     constexpr int BN = 32; // vec 版: 32x4 = 128
1083     constexpr int BK = 32;
1084     __shared__ float s_a[BM][BK], s_b[BK][BN]; // 32x32x4=4KB smem, float = 4 bytes
1085
1086     int bx = blockIdx.x;
1087     int by = blockIdx.y;
1088     int tx = threadIdx.x;
1089     int tid = threadIdx.y * blockDim.x + tx;
1090
1091     // 线程到 smem 的映射: 32x32 线程, 每个线程加载 a 和 b 各 1 个元素
1092     // 技巧: 一般来说 “/” 表示线程不是连续排布的, “%” 表示线程是连续排布的
1093     // 因此, 在需要考虑连续访问的维度使用 “%”, 比如, 连续的线程访问列方向连续的元素
1094     // A[M, K], M的stride=K, K的stride=1 → 线程连续访问 K 维度 → 用 %,
1095     // 线程不连续访问 M 维度 → 用 /;
1096     int load_smem_a_m = tid / 32; // row 0~31 由 32 线程加载;
1097     int load_smem_a_k = tid % 32; // col 0~31 由 32 线程加载;
1098     int load_smem_b_k = tid / 32; // row 0~31 由 32 线程加载;
1099     int load_smem_b_n = tid % 32; // col 0~31 由 32 线程加载;
1100     int load_gmem_a_m = by * BM + load_smem_a_m; // gmem row;
1101     int load_gmem_b_n = bx * BN + load_smem_b_n; // gmem col;
1102
1103     float sum = 0.f; // 遍历完整的K, slice K;
1104     // 这里不用pragma unroll, 因为K不是编译器常量, 编译器无法展开循环
1105     for (int bk = 0; bk < (K + BK - 1) / BK; ++bk) {
1106         int load_gmem_a_k = bk * BK + load_smem_a_k; // A [M, K]
1107         int load_gmem_a_addr = load_gmem_a_m * K + load_gmem_a_k;
1108         s_a[load_smem_a_m][load_smem_a_k] = a[load_gmem_a_addr];
1109         int load_gmem_b_k = bk * BK + load_smem_b_k; // B [K, N]
1110         int load_gmem_b_addr = load_gmem_b_k * N + load_gmem_b_n;
1111         s_b[load_smem_b_k][load_smem_b_n] = b[load_gmem_b_addr];
1112         __syncthreads(); // 确保整个 smem tile 加载完毕
1113
1114     #pragma unroll
1115     for (int k = 0; k < BK; ++k) {
1116         int comp_smem_a_m = load_smem_a_m; // vec 版: 0~127, 0, 1, 2, ... (连续)
1117         int comp_smem_b_n = load_smem_b_n; // vec 版: 0~127, 0, 4, 8, ... (间隔)
1118         sum += s_a[comp_smem_a_m][k] * s_b[k][comp_smem_b_n];
1119     }
1120     __syncthreads(); // 确保 smem 不会在下一轮加载时被覆盖
1121 }
1122 int store_gmem_c_m = load_gmem_a_m; // vec 版: 0~127, 0, 1, 2, ... (连续) [128x128]
1123 int store_gmem_c_n = load_gmem_b_n; // vec 版: 0~127, 0, 4, 8, ... (间隔)
1124 int store_gmem_c_addr = store_gmem_c_m * N + store_gmem_c_n;
1125 c[store_gmem_c_addr] = sum; // C [M, N] = A[M, K] x B[K, N]
1126 }
1127
1128 // ---- Level 1+: SGEMM Vec4 — Block Tile 128x128 + K Tile 32 + Thread Tile 4x4 ----
1129 // 在 Level 1 基础上引入两层优化:
1130 // 1) float4 向量化加载: A/B 各用 1 条 128-bit load 取代 4 条 32-bit load
1131 // 2) Thread Tile 4x4: 每线程计算 16 个 C 元素, 提升计算/访存比 (AI 从
1132 // BK/2≈16 提升到 TM*TN*BK/2≈256), 减少线程总数带来的同步开销
1133 // C = A x B, C[M, N] = A[M, K] x B[K, N], A/B 均 row-major

```

```

1134 // BM=BN=128, BK=32, block(32, 32)=1024 线程, 每线程负责 4×4=16 个 C 元素
1135 // 1024 × 16 = 16384 = 128 × 128 ✓
1136 //
1137 // 线程到 4×4 tile 的映射 (与加载映射解耦, 独立计算更清晰):
1138 // m_tile = tid / 32 (0~31), 每 tile 4 行 → 行 [m_tile*4, m_tile*4+3], 覆盖 0~127
1139 // n_tile = tid % 32 (0~31), 每 tile 4 列 → 列 [n_tile*4, n_tile*4+3], 覆盖 0~127
1140 //
1141 // 加载映射 (每线程 4 个元素, float4):
1142 // A[128][32]: a_m = tid/8 (8 线程/行), a_k = (tid%8)*4 (4 列/线程) → 8×4=32 列 ✓
1143 // B[32][128]: b_k = tid/32 (32 线程/行), b_n = (tid%32)*4 (4 列/线程) → 32×4=128 列 ✓
1144 // row-major 下 A[m][k..k+3] 与 B[k][n..n+3] 均连续 → float4 load 合法
1145 //
1146 // △ Bank Conflict 提示 (面试加分点):
1147 // s_b[32][128] 上 warp 内 32 线程按 stride=4 访问 (tid%32 决定列 0,4,8,...,124)
1148 // → 每 4 个线程落同一 bank 不同地址 → 4-way bank conflict。生产代码可用
1149 // s_b[BK][BN+1] PAD 打散, 这里保持最简布局便于讲解。
1150 //
1151 // Grid: ((N + 127) / 128, (M + 127) / 128, 1)
1152 // Block: (32, 32, 1), 1024 线程
1153 // 假设: M/N 为 128 的倍数, K 为 32 的倍数 (与 Level 1 naive 版一致的边界约定)
1154 // source: LeetCUDA/kernels/sgemm/sgemm.cu (vec4 variant)
1155 __global__ void sgemm_vec4(float *a, float *b, float *c, int M, int N, int K) {
1156     constexpr int BM = 128;
1157     constexpr int BN = 128;
1158     constexpr int BK = 32;
1159     __shared__ float s_a[BM][BK]; // 128*32*4 = 16KB, float = 4 bytes
1160     __shared__ float s_b[BK][BN]; // 32*128*4 = 16KB
1161
1162     int bx = blockIdx.x;
1163     int by = blockIdx.y;
1164     int tx = threadIdx.x;
1165     int tid = threadIdx.y * blockDim.x + tx; // 0~1023
1166
1167     // 加载 A: 每线程加载 s_a[a_m][a_k..a_k+3] 共 4 个元素
1168     int load_smem_a_m = tid / 8; // 0~127, 8 线程/行
1169     int load_smem_a_k = (tid % 8) * 4; // 0,4,...,28
1170     // 加载 B: 每线程加载 s_b[b_k][b_n..b_n+3] 共 4 个元素
1171     int load_smem_b_k = tid / 32; // 0~31, 32 线程/行
1172     int load_smem_b_n = (tid % 32) * 4; // 0,4,...,124
1173
1174     int load_gmem_a_m = by * BM + load_smem_a_m;
1175     int load_gmem_b_n = bx * BN + load_smem_b_n;
1176
1177     // 4×4 Thread Tile 基址 (独立于加载映射), 这里compute索引的计算逻辑要和
1178     // load索引的计算逻辑分开, load/compute是可以独立索引的, 理解这点很重要。
1179     // 目标C Tile为[BM,BN]=[128x128], 有32x32线程, 则每个线程处理4x4 tile
1180     // 那么, 就可以不重不漏地覆盖[32x4,32x4]=[128x128]的大小
1181     int comp_smem_a_m_base = (tid / 32) * 4; // 0,4,8,...,124
1182     int comp_smem_b_n_base = (tid % 32) * 4; // 0,4,8,...,124
1183
1184     float sum[4][4] = {0.f};
1185     for (int bk = 0; bk < (K + BK - 1) / BK; ++bk) {
1186         int load_gmem_a_k = bk * BK + load_smem_a_k; // A [M, K]
1187         int load_gmem_a_addr = load_gmem_a_m * K + load_gmem_a_k;
1188         FLOAT4(s_a[load_smem_a_m][load_smem_a_k]) = FLOAT4(a[load_gmem_a_addr]); // s_a [BM,BK]
1189         int load_gmem_b_k = bk * BK + load_smem_b_k; // B [K, N]
1190         int load_gmem_b_addr = load_gmem_b_k * N + load_gmem_b_n;
1191         FLOAT4(s_b[load_smem_b_k][load_smem_b_n]) = FLOAT4(b[load_gmem_b_addr]); // s_b [BK,BN]
1192         __syncthreads();
1193     }
1194     #pragma unroll
1195     for (int k = 0; k < BK; ++k) {
1196         // 每次迭代加载 4 个 A 元素 + 4 个 B 元素, 再做 4×4=16 次 FMA

```

```

1197     float a_vals[4] = {s_a[comp_smem_a_m_base + 0][k],
1198                       s_a[comp_smem_a_m_base + 1][k],
1199                       s_a[comp_smem_a_m_base + 2][k],
1200                       s_a[comp_smem_a_m_base + 3][k]};
1201     float b_vals[4] = {s_b[k][comp_smem_b_n_base + 0],
1202                       s_b[k][comp_smem_b_n_base + 1],
1203                       s_b[k][comp_smem_b_n_base + 2],
1204                       s_b[k][comp_smem_b_n_base + 3]};
1205 #pragma unroll
1206     for (int i = 0; i < 4; ++i) {
1207 #pragma unroll
1208         for (int j = 0; j < 4; ++j) {
1209             sum[i][j] += a_vals[i] * b_vals[j];
1210         }
1211     }
1212 }
1213 __syncthreads();
1214 }
1215
1216 // 存储 4x4: 每行 4 个元素连续 → 可用 float4 store (要求 N 为 4 的倍数以保证对齐)
1217 int store_gmem_c_m = by * BM + comp_smem_a_m_base;
1218 int store_gmem_c_n = bx * BN + comp_smem_b_n_base;
1219 #pragma unroll
1220 for (int i = 0; i < 4; ++i) {
1221     int store_gmem_c_addr = (store_gmem_c_m + i) * N + store_gmem_c_n;
1222     float4 reg_c;
1223     reg_c.x = sum[i][0];
1224     reg_c.y = sum[i][1];
1225     reg_c.z = sum[i][2];
1226     reg_c.w = sum[i][3];
1227     FLOAT4(c[store_gmem_c_addr]) = reg_c;
1228 }
1229 }
1230
1231 // =====
1232 // Phase 7b: HGEMM — Tensor Core 路径 (MMA m16n8k16 + WGMMA m64n128k16)
1233 // =====
1234 // 面试要点:
1235 // - MMA (Matrix Multiply-Accumulate): Ampere+ 的 Tensor Core 指令
1236 // - m16n8k16 含义: M=16, N=8, K=16 的矩阵乘, 结果 [16x8] = [16x16]x[16x8]
1237 // - ldmatrix: 从 shared memory 加载 16x16 的矩阵片段到寄存器 (4 条 32-bit
1238 //   寄存器)
1239 // - Multistage Pipeline: s2/s3/s4 个 stage, 用 cp.async 异步加载下一批数据
1240 // - Block Swizzle: 在 grid 维度做 swizzle, 改善 L2 cache locality
1241 //
1242 // ★ TN 布局详解 (面试高频考点):
1243 // TN 命名约定: 来自 BLAS 的 op(A) × op(B) 语义
1244 //   BLAS 源自 Fortran, 默认列优先 (column-major) 存储
1245 //   N = Normal (列优先, BLAS 原生格式)
1246 //   T = Transposed (行优先, 相对 BLAS 来说是"转置过的")
1247 //   第一个字母 → A 的 op, 第二个字母 → B 的 op
1248 //   所以 TN 表示: A 是行优先 (相对 BLAS=Transposed), B 是列优先 (相对
1249 //   BLAS=Normal) 即: C = op(A) × op(B) = A^T × B? 不对! 在 row-major
1250 //   视角下: TN = A row-major [MxK], B^T row-major [NxK] (等价于 B col-major [KxN]) 在 cuBLAS
1251 //   调用中: cublasGemmEx(..., CUBLAS_OP_T, CUBLAS_OP_N, ...)
1252 //   T on A: BLAS 把 row-major 的 A 视为 A^T, 传 T 表示"转置回去"
1253 //   N on B: B 已经是 BLAS 原生的 col-major, 无需转置
1254 //
1255 // 记忆口诀: TN = A行(T) B列(N), 第一字母 A 第二字母 B
1256 //   T = row-major (行优先, 对 BLAS 来说是 transposed)
1257 //   N = col-major (列优先, BLAS native = normal)
1258 //
1259 // LeetCUDA _nn 布局对比 (A/B 均 row-major 自然存储): C[MxN] = A[MxK] × B[KxN]

```

```

1260 // - A: row-major [M, K], B: row-major [K, N]
1261 // - 按 N=col-major/T=row-major 约定, 二者 BLAS 视角均为 T → cuBLAS 等效 (T,T)
1262 // - 问题: ldmatrix 默认加载 col-major, B 是 row-major 需要 .trans
1263 //
1264 // TN 布局: C[M×N] = A[M×K] × B[K×N], B 以 B^T=[N×K] row-major 存储 (A 行优先, B 列优先)
1265 // - A [M×K]: row-major → 全局索引 A[m*K + k], smem s_a[BM][BK]
1266 // - B^T [N×K]: row-major → 全局索引 B[n*K+k] 即访问原 B 元素 (k,n) (△ 内维连续的是 K)
1267 // - 优势: B^T 已是 row-major, ldmatrix 无需 .trans, 天然匹配 MMA row.col
1268 // MMA 指令: mma.sync.aligned.m16n8k16.row.col
1269 // - row.col: A 输入 row-major, B 输入 col-major → 与 TN 布局天然匹配
1270 //
1271 // WGMMA (Warp Group MMA, Hopper+):
1272 // - warpgroup 级指令 (128 threads = 4 warps), 异步执行
1273 // - m64n128k16: 一次处理 64×128×16 的矩阵乘 (总 tile 量 131072, 是 MMA m16n8k16 的 64 倍)
1274 // - Warp Specialization: Producer(128 threads) 做 TMA 搬运, Consumer(128
1275 // threads) 做计算
1276 // - TMA: 硬件 DMA, ~零寄存器开销, 支持 1D~5D 寻址
1277 //
1278 // =====
1279 // Phase 7b-1: MMA PTX 宏定义
1280 // =====
1281 //
1282 // ---- gmem → smem: cp.async ----
1283 // cp.async.commit_group / wait_group / wait_all 语义 (PTX ISA §9.7.9.25.3) :
1284 // - commit_group: 将此前所有未提交的 cp.async 归入一个新的 async-group (per-thread) 。
1285 // - wait_group N: 阻塞直到最多 N 个 async-group 尚未完成 (即 pending ≤ N) 。
1286 // N=0 → 等所有 group 完成。与 wgmma.wait_group 语义一致。
1287 // - wait_all: 等价于 commit_group + wait_group 0。
1288 #define CP_ASYNC_COMMIT_GROUP() asm volatile("cp.async.commit_group;\n" ::)
1289 #define CP_ASYNC_WAIT_ALL() asm volatile("cp.async.wait_all;\n" ::)
1290 #define CP_ASYNC_WAIT_GROUP(n) \
1291     asm volatile("cp.async.wait_group %0;\n" :: "n"(n))
1292 //
1293 // 注意: cg 只支持 16 bytes, ca 支持 4/8/16 bytes
1294 #define CP_ASYNC_CG(dst, src, bytes) \
1295     asm volatile( \
1296         "cp.async.cg.shared.global.L2::128B [%0], [%1], %2;\n" :: "r"(dst), \
1297         "l"(src), "n"(bytes))
1298 //
1299 // ---- ldmatrix: smem → register (Tensor Core 专用) ----
1300 // ldmatrix.sync.aligned.xN.m8n8.shared.b16
1301 // 每次加载 8×8 的 half 矩阵片段到 1/2/4 条 32-bit 寄存器
1302 // aligned: 要求 128-bit 对齐, trans: 转置加载
1303 #define LDMATRIX_X4(R0, R1, R2, R3, addr) \
1304     asm volatile( \
1305         "ldmatrix.sync.aligned.x4.m8n8.shared.b16 {%0, %1, %2, %3}, [%4];\n" \
1306         : "=r"(R0), "=r"(R1), "=r"(R2), "=r"(R3) \
1307         : "r"(addr))
1308 //
1309 #define LDMATRIX_X2(R0, R1, addr) \
1310     asm volatile("ldmatrix.sync.aligned.x2.m8n8.shared.b16 {%0, %1}, [%2];\n" \
1311         : "=r"(R0), "=r"(R1) \
1312         : "r"(addr))
1313 //
1314 // ldmatrix.x2.trans: 转置加载 (用于 NN 布局中需要 col-major B 矩阵的场景)
1315 // FA 中 V[Bc,d] 为 row-major, 但 P@V 的 MMA 需要 col-major 的 B → 使用 trans
1316 #define LDMATRIX_X2_T(R0, R1, addr) \
1317     asm volatile( \
1318         "ldmatrix.sync.aligned.x2.trans.m8n8.shared.b16 {%0, %1}, [%2];\n" \
1319         : "=r"(R0), "=r"(R1) \
1320         : "r"(addr))
1321 //
1322 // ---- mma.sync.aligned.m16n8k16.row.col.f16.f16.f16.f16 ----

```



```

1323 // m16n8k16: M=16, N=8, K=16 (Ampere Tensor Core 的基本 tile)
1324 // row.col: A 是 row-major, B 是 col-major
1325 // f16.f16.f16.f16: A/B 是 f16, C/D 是 f16 (f32 累加版本用 f32.f16.f16.f32)
1326 // 2 个输出寄存器 (RD0, RD1), 4 个 A 寄存器 + 2 个 B 寄存器
1327 // C 矩阵大小 16×8=128 元素 = 128 个 half; 32 线程分担, 每线程 4 half = 2 个 uint32
1328 #define HMMA16816(RD0, RD1, RA0, RA1, RA2, RA3, RB0, RB1, RC0, RC1) \
1329     asm volatile( \
1330         "mma.sync.aligned.m16n8k16.row.col.f16.f16.f16.f16 {%0, %1}, {%2, %3, " \
1331         "%4, %5}, {%6, %7}, {%8, %9};\n" \
1332         : "=r"(RD0), "=r"(RD1) \
1333         : "r"(RA0), "r"(RA1), "r"(RA2), "r"(RA3), "r"(RB0), "r"(RB1), "r"(RC0), \
1334         "r"(RC1))
1335
1336 // ---- MMA 辅助函数 ----
1337 // div_ceil: 整数除法向上取整
1338 #define HOST_DEVICE_INLINE __device__ __host__ inline
1339 HOST_DEVICE_INLINE int div_ceil(int a, int b) {
1340     return (a % b != 0) ? (a / b + 1) : (a / b);
1341 }
1342
1343 // =====
1344 // Phase 7b-2: HGEMM MMA — m16n8k16 + multistage pipeline + TN 布局 (统一循环版)
1345 // =====
1346 // 面试重点 — Tile Hierarchy:
1347 //   MMA Atom:          m16n8k16 (1 条 MMA 指令处理的最小 tile)
1348 //   MMA Tile (more warps): 2×4=8 个 MMA atom → [2×16, 8×4]=[32,32]
1349 //   VAL Tile (more values): 4×4=16 expand → [32×4, 32×4]=[128,128]
1350 //   实际: kMmaTileM=2, kMmaTileN=4, kValTileM=4, kValTileN=4
1351 //           → BM=16×2×4=128, BN=8×4×4=128, Warps=2×4=8, Threads=8×32=256
1352 //
1353 // ★ TN 布局 (T=A 行优先, N=B 列优先):
1354 //   - A[M][K]: row-major → 全局索引 A[m*K + k], smem s_a[BM][BK]
1355 //   - B[K][N]: col-major (等价于 B^T[N][K] row-major) → 全局索引 B[n*K+k]
1356 //   - ldmatrix A: 用 x4 (非转置), A row-major 原生匹配
1357 //   - ldmatrix B: 用 x2 (非转置), B^T row-major 逐行加载 = B 的列, 天然匹配 MMA row.col
1358 //   - MMA 指令: mma.sync.aligned.m16n8k16.row.col → 天然匹配 TN 布局
1359 //
1360 // ★ 统一循环设计 (k 从 0 开始, 消除尾端重复代码):
1361 //   - k 从 0 开始: 每次迭代 k 直接对应 tile k, sel = k % kStages (零偏移)
1362 //   - 加载语义为"预取未来": 迭代 k 加载 tile (k+kStages-1) 供后续使用
1363 //   - cp.async 条件化: 仅当 k+kStages-1 < NUM_K_TILES 时加载
1364 //   - WAIT_GROUP 自适应: 满载期用 kStages-2, 尾部排空用 0
1365 //
1366 // ★ Block Swizzle: 把 C 的逻辑 tile 布局改写为紧凑的二维发射窗口, 增加 A/B tile 的 L2 reuse 机会。
1367 //   C tile 坐标为 (by, bx), by 沿 M 方向, bx 沿 N 方向; 一个 bx 读取同一块 B^T[bx*BN:(bx+1)*BN, :]
1368 //   (TN 布局中 B^T[N][K] 为 row-major), 一个 by 读取同一块 A[by*BM:(by+1)*BM, :]。
1369 //   下图只画 4 个逻辑上相邻的 CTA; SM0~SM3 仅表示可能的发射序列, 不是硬件 SM 绑定或调度保证。
1370 //
1371 //   No Swizzle: grid = (tiles_n, tiles_m, 1), blockIdx.x = bx。
1372 //   C-Matrix (同一 by 行; CTA 先在很宽的 N 方向范围内展开):
1373 //
1374 //       N / bx → 0          1          2          3          ...
1375 //       M ↓
1376 //       by = 0  |  SM0   |  SM1   |  SM2   |  SM3   |  ...
1377 //               |  A 0,B0 |  A 0,B1 |  A 0,B2 |  A 0,B3 |
1378 //               +-----+
1379 //
1380 //   同一 by 的 CTA 复用 A[by], 但分别读取 B^T 0、B^T 1 等不同 B tile。只有 scheduler
1381 //   推进到下一条 by 行、再次遇到相同 bx 时, 才有机会复用 B; 当 tiles_n 很大时, 这段时间内
1382 //   L2 更容易被其他 B tile 挤占。
1383 //
1384 //   Thread Block Swizzle: 先把 N 切成 S 个连续窗口; 一个 z slice 只含 grid_x 个 bx。
1385 //   下图用 grid_x=2 展示一个窄窗口, scheduler 更早进入下一条 by 行:

```

```

1386 //
1387 //      N / local x → 0      1
1388 //      M ↓
1389 //      by = 0      

|     |     |
|-----|-----|
| SM0 | SM1 |
| SM2 | SM3 |

 → A 0; B^T 0, B^T 1
1390 //
1391 //      by = 1      

|     |     |
|-----|-----|
| SM2 | SM3 |
| SM0 | SM1 |

 → A 1; B^T 0, B^T 1
1392 //
1393 //
1394 // 对于这个 2x2 窗口：同一行横向复用 A (SM0/SM1、SM2/SM3)，同一列纵向复用 B
1395 // (SM0/SM2、SM1/SM3)。真实 grid_x 不必等于 2，但缩小它会缩短再次访问相同 bx 的距离。
1396 // 此优化只提高 scheduler 在相近时间执行这些 CTA、命中 L2 的机会，不保证 CTA 的 z 顺序、
1397 // 缓存驻留或 L2 hit。
1398 //
1399 // Launch (kBlockSwizzle=false, 当前默认路径)：
1400 //   grid = (ceil(N / BN), ceil(M / BM), 1), blockIdx.x 直接就是 N-tile 编号 bx。
1401 // Launch (kBlockSwizzle=true, 需由 launch 端显式构造 3D grid)：
1402 //   tiles_n = ceil(N / BN), S = ceil(N / 2048)
1403 //   grid = (ceil(tiles_n / S), ceil(M / BM), S)
1404 //   bx = blockIdx.z * grid.x + blockIdx.x, col_start = bx * BN。
1405 // 例如 N=8192、BN=128: tiles_n=64, S=4, grid.x=16; z=0/1/2/3 分别覆盖
1406 // bx=0..15/16..31/32..47/48..63, 即 columns [0,2048)/[2048,4096)/[4096,6144)/[6144,8192)。
1407 // grid.x*grid.z 可能产生 bx>=tiles_n 的冗余 CTA。当前 kernel 仍要求 M/N/K 分别按 BM/BN/BK
1408 // 对齐; ceil 只保证逻辑 tile 编号不遗漏, 并不使部分尾 tile 变为安全。
1409 // Block: (256, 1, 1), 8 warps
1410 // source: LeetCUDA/kernels/hgemm/mma/basic/hgemm_mma_stage_tn.cu
1411 // =====
1412 template <const int kMmaM = 16,           // MMA atom M dim (m16n8k16)
1413           const int kMmaN = 8,           // MMA atom N dim
1414           const int kMmaK = 16,          // MMA atom K dim, also BK tile = kMmaK
1415           const int kMmaTileM = 2,       // warps along M, 2 → warp tile M = 32
1416           const int kMmaTileN = 4,       // warps along N, 4 → warp tile N = 32
1417           const int kValTileM = 4,       // value-repeat along M, BM = 16*2*4 = 128
1418           const int kValTileN = 4,       // value-repeat along N, BN = 8*4*4 = 128
1419           const int kStages = 3,         // cp.async pipeline depth
1420           const int kBlockSwizzle = 0>   // 1 enables 3D grid swizzle for L2 locality
1421 __global__ void __launch_bounds__(256)
1422   hgemm_mma_stages_tn(half *A, half *B, half *C, int M, int N, int K) {
1423   static_assert(kBlockSwizzle == 0 || kBlockSwizzle == 1, "kBlockSwizzle must be 0 or 1");
1424   static_assert(kStages >= 2, "kStages must be >= 2 for cp.async pipeline");
1425   // Block Swizzle: 0 时 bx=blockIdx.x; 1 时将 (blockIdx.z, blockIdx.x)
1426   // 线性化为原始 N-tile 编号 bx=z*gridDim.x+x。by 始终是 M-tile 编号。
1427   const int bx = ((int)kBlockSwizzle) * blockIdx.z * gridDim.x + blockIdx.x;
1428   const int by = blockIdx.y;
1429   constexpr int BM = kMmaM * kMmaTileM * kValTileM; // 16*2*4=128
1430   constexpr int BN = kMmaN * kMmaTileN * kValTileN; // 8*4*4=128
1431   constexpr int BK = kMmaK; // 16
1432
1433   // Dynamic shared memory: kStages 个 stage 的 A 和 B
1434   // TN 布局: s_a[BM][BK]=[128][16](A row-major), s_b[BN][BK]=[128][16](B^T
1435   // row-major, 即 B col-major [K×N] 在 smem 中按 B^T[N×K] 存储)
1436   // 原始实现会按配置决定是否给 A/B 的 K 维加 PAD; 尤其 B 在 TN 布局下常见会额外
1437   // 加 B_PAD 来打散 bank 映射, 避免按列访问时出现明显 bank conflict。这里先保留最简 PAD=0 版本。
1438   extern __shared__ half smem[];
1439   half *s_a = smem;
1440   half *s_b = smem + kStages * BM * BK; // A 和 B 连续存放
1441   constexpr int s_a_stage_offset = BM * BK; // 128*16
1442   constexpr int s_b_stage_offset = BN * BK; // 128*16 △ BN(128)×BK(16) = B^T row-major
1443   const int tid = threadIdx.y * blockDim.x + threadIdx.x;
1444   const int warp_id = tid / kWarpSize; // 0~7
1445   const int lane_id = tid % kWarpSize; // 0~31
1446   // warp_m变化快(0->0,1->1), warp_n变化慢([0,1]->0,[2,3]->1,...), 因此,
1447   // 这种2x4的MMA(Warp) layout是按照col major的顺序来排列MMA0-MMA7的
1448   //      N direction (warp_n)

```



```

1449 //          0      1      2      3
1450 // M direction (warp_m) 0 MMA0   MMA2   MMA4   MMA6
1451 //          1 MMA1   MMA3   MMA5   MMA7
1452 // MMA的排布方式不是唯一的，现在这样排是因为结合MMA/VAL Tile之后，刚好能覆盖整个
1453 // C Tile[128,128]。只要调整MMA/VAL Tile，这里的排布方式也可以跟着调整。要注意的
1454 // 是，MMA0-7逻辑上是可以认为是并行执行的，各自的计算结果累计加到对应的C Tile位置上。
1455 const int warp_m = warp_id % kMmaTileM; // 0,1 (M 方向 2 个 warp) kMmaTileM = 2
1456 const int warp_n = warp_id / kMmaTileM; // 0,1,2,3 (N 方向 4 个 warp)
1457
1458 // 线程到 global memory 的映射 (用于加载 A 和 B) 共 256 个线程
1459 // TN 布局关键: A[m*K+k] 是 row-major, B^T[n*K+k] 是 row-major (内维连续的是 K)
1460 int load_smem_a_m = tid / 2; // 0~127
1461 int load_smem_a_k = (tid % 2 == 0) ? 0 : 8; // 0, 8
1462 int load_smem_b_n = tid / 2; // 0~127 → B^T 的 N 方向 (row-major 的行)
1463 int load_smem_b_k = (tid % 2 == 0) ? 0 : 8; // 0, 8 → B^T 的 K 方向 (row-major 的列)
1464 int load_gmem_a_m = by * BM + load_smem_a_m; // C/A 全局行号 = M 方向的 tile 起始 + 线程偏移
1465 int load_gmem_b_n = bx * BN + load_smem_b_n; // C/B 全局列号 = N 方向的 tile 起始 + 线程偏移
1466 if (load_gmem_a_m >= M || load_gmem_b_n >= N)
1467     return;
1468
1469 // 8个Warps(MMAs)的排布是M方向2个，N方向4个，则MMA Atom一次性能处理的tile大小是[2*16,4*8]=[32,32]。
1470 // CUDA中每个block能放的线程数是有上限的 (warp数量有限)，一般为4或者8个warps。为了能处理更大的C tile，
1471 // 需要把MMA Atom的tile再M方向和N方向各自重复4次，得到[128,128]的C block tile，这就是Value Tile的作用。
1472 // MMA Tile对应到cutlass cute中的TiledMMA的概念，Value Tile对应到cutlass cute中的PermuteMNK的概念。
1473 uint32_t RC[kValTileM][kValTileN][2] = {0}; // 初始化为 0
1474
1475 // CVTA: 一次转换 smem 基地址，避免每次 cp.async 都做转换
1476 uint32_t smem_a_base_ptr = __cvta_generic_to_shared(s_a);
1477 uint32_t smem_b_base_ptr = __cvta_generic_to_shared(s_b);
1478
1479 // 预加载前 (kStages-1) 个 stage
1480 // TN 布局: A 的 gmem 索引用 m*K+k(row-major), B^T 用 n*K+k(row-major, 即 B col-major [K×N])
1481 #pragma unroll
1482 for (int k = 0; k < (kStages - 1); ++k) {
1483     int load_gmem_a_k = k * BK + load_smem_a_k;
1484     int load_gmem_a_addr = load_gmem_a_m * K + load_gmem_a_k; // A: [m][k]
1485     uint32_t load_smem_a_ptr = (smem_a_base_ptr +
1486         (k * s_a_stage_offset + load_smem_a_m * BK + load_smem_a_k) * sizeof(half)
1487     );
1488     CP_ASYNC_CG(load_smem_a_ptr, &A[load_gmem_a_addr], 16);
1489
1490     int load_gmem_b_k = k * BK + load_smem_b_k;
1491     // B^T: [n][k] row-major (即 B[k][n] col-major) △
1492     int load_gmem_b_addr = load_gmem_b_n * K + load_gmem_b_k;
1493     uint32_t load_smem_b_ptr = (smem_b_base_ptr +
1494         (k * s_b_stage_offset + load_smem_b_n * BK + load_smem_b_k) * sizeof(half)
1495     );
1496     CP_ASYNC_CG(load_smem_b_ptr, &B[load_gmem_b_addr], 16);
1497
1498     CP_ASYNC_COMMIT_GROUP();
1499 }
1500
1501 CP_ASYNC_WAIT_GROUP(kStages - 2); // 允许有 kStages-2 个group未完成
1502 __syncthreads();
1503
1504 // 统一循环: k 从 0 开始，每次迭代负责 tile k (加载 + 计算合并为单循环)
1505 const int NUM_K_TILES = div_ceil(K, BK);
1506 // 此处不用pragma unroll，因为 K 不是编译期常量，因此NUM_K_TILES 也不是编译期常量
1507 for (int k = 0; k < NUM_K_TILES; ++k) {
1508     int smem_sel = k % kStages; // 计算 tile k 的 stage
1509     int smem_sel_next = (k + kStages - 1) % kStages; // 预加载目标 stage
1510
1511     // 条件加载: 预加载 tile (k+kStages-1) 供将来使用。因为在prefetch loop中已经

```

```

1512 // load了(kStages-1) 个 stage, 因此这里是从 tile (k+kStages-1) 开始预加载
1513 // TN 布局: A 的 gmem 地址用 m*K+k (row-major), B^T 的 gmem 地址用 n*K+k
1514 // (row-major, 内维连续的是 K)
1515 if (k + kStages - 1 < NUM_K_TILES) {
1516     int load_gmem_a_k = (k + kStages - 1) * BK + load_smem_a_k;
1517     int load_gmem_a_addr = load_gmem_a_m * K + load_gmem_a_k; // A: row-major [m][k]
1518     int load_gmem_b_k = (k + kStages - 1) * BK + load_smem_b_k;
1519     int load_gmem_b_addr = load_gmem_b_n * K + load_gmem_b_k; // B^T: row-major [n][k]
1520
1521     uint32_t load_smem_a_ptr =
1522         (smem_a_base_ptr + (smem_sel_next * s_a_stage_offset +
1523             load_smem_a_m * BK + load_smem_a_k) *
1524             sizeof(half));
1525     CP_ASYNC.CG(load_smem_a_ptr, &A[load_gmem_a_addr], 16);
1526
1527     uint32_t load_smem_b_ptr =
1528         (smem_b_base_ptr + (smem_sel_next * s_b_stage_offset +
1529             load_smem_b_n * BK + load_smem_b_k) *
1530             sizeof(half));
1531     CP_ASYNC.CG(load_smem_b_ptr, &B[load_gmem_b_addr], 16);
1532     CP_ASYNC.COMMIT_GROUP();
1533 }
1534
1535 // ldmatrix: 从 smem_sel 加载 A 和 B 到寄存器
1536 // TN 布局关键: A 用 x4 (非转置), 因为 A 是 row-major; B 用 x2 (非转置), smem 中
1537 // B^T 为 row-major, 逐行加载即得 B 的列, 天然匹配 col-major B
1538 uint32_t RA[kValTileM][4]; // [4][4], M方向4次repeat, A regs = 4 uint32_t per MMA
1539 uint32_t RB[kValTileN][2]; // [4][2], N方向4次repeat, B regs = 2 uint32_t per MMA
1540
1541 // ldmatrix.x4: 加载 A 的 m16k16 片段 (row-major A, 非转置)
1542 #pragma unroll
1543 for (int i = 0; i < kValTileM; ++i) {
1544     // {0,1} * (16 * 4) + i * 16 = {0,64} + {0,16,32,48} = {0,16,32,48,64,80,96,112}
1545     // 其中 warp_m {0,1}; warp_m_0 offsets {0,16,32,48}; warp_m_1 offsets {64,80,96,112}
1546     int warp_smem_a_m = warp_m * (kMmaM * kValTileM) + i * kMmaM;
1547     // {0,16,32,...,112} + {0~15} = {0~127}, 按照col-major的顺序访问A的4个8x8 matrix (16x16)
1548     // ldmatrix.{...}.x4.{...} 需要warp内32个线程都参与, 每个线程提供一个有效的, 不重叠的addr
1549     int lane_smem_a_m = warp_smem_a_m + lane_id % 16; // t{0...15}=0~15, t{16...31}=0~15
1550     int lane_smem_a_k = (lane_id / 16) * 8; // 0, 8, t{0...15}=0, t{16...31}=8
1551     uint32_t lane_smem_a_ptr =
1552         (smem_a_base_ptr +
1553             (smem_sel * s_a_stage_offset + lane_smem_a_m * BK + lane_smem_a_k) *
1554             sizeof(half));
1555     LDMATRIX_X4(RA[i][0], RA[i][1], RA[i][2], RA[i][3], lane_smem_a_ptr);
1556 }
1557
1558 // ldmatrix.x2: 加载 B 的 k16n8 片段 (非转置)
1559 // 为什么不用 .trans? 因为 smem 中存的是 B^T row-major [N][K],
1560 // ldmatrix 逐行加载 B^T 的行 = B 的列, 天然给出 col-major B fragment → 直接匹配 MMA row.col
1561 #pragma unroll
1562 for (int j = 0; j < kValTileN; ++j) {
1563     // {0,...,3} * (8 * 4) + j * 8 = {0,32,64,96} + {0,8,16,24} = {0,8,...,120}
1564     // warp_n_0 offsets {0,8,16,24}; warp_n_1 offsets {32,40,48,56}
1565     // warp_n_2 offsets {64,72,80,88}; warp_n_3 offsets {96,104,112,120}
1566     int warp_smem_b_n = warp_n * (kMmaN * kValTileN) + j * kMmaN;
1567     // {0,8,...,120} + {0~7} = {0~127}, 按照row-major的顺序访问B^T的2个8x8 matrix (8x16)
1568     // ldmatrix.{...}.x2.{...} 需要warp内前16个线程参与, 后16个线程传的addr会被忽略
1569     int lane_smem_b_n = warp_smem_b_n + lane_id % 8; // t{0...7}=0~7, t{8...15}=0~7
1570     int lane_smem_b_k = ((lane_id / 8) % 2) * 8; // t{0...7}=0, t{8...15}=8
1571     uint32_t lane_smem_b_ptr =
1572         (smem_b_base_ptr +
1573             (smem_sel * s_b_stage_offset + lane_smem_b_n * BK + lane_smem_b_k) *
1574             sizeof(half));

```

```

1575     LDMATRIX_X2(RB[j][0], RB[j][1], lane_smem_b_ptr);
1576 }
1577
1578 // MMA compute: 每个Warp(MMA) 在M方向重复kValTileM(4)次, 在N方向重复kValTileN(4)次
1579 #pragma unroll
1580 for (int i = 0; i < kValTileM; ++i) {
1581     #pragma unroll
1582     for (int j = 0; j < kValTileN; ++j) {
1583         MMA16816(RC[i][j][0], RC[i][j][1], // C fragment
1584                 RA[i][0], RA[i][1], RA[i][2], RA[i][3], // A fragment
1585                 RB[j][0], RB[j][1], // B fragment
1586                 RC[i][j][0], RC[i][j][1]);
1587     }
1588 }
1589
1590 // 自适应等待: 流水线满载期用 kStages-2, 尾部排空用 0
1591 if (k + kStages - 1 < NUM_K_TILES) {
1592     CP_ASYNC_WAIT_GROUP(kStages - 2);
1593 } else { // 对于尾部的 k, 等待所有剩余的 cp.async 完成
1594     CP_ASYNC_WAIT_GROUP(0);
1595 }
1596 __syncthreads();
1597 }
1598
1599 // Epilogue: 寄存器 → global memory (通过 warp shuffle + 128-bit store)
1600 {
1601     for (int i = 0; i < kValTileM; ++i) {
1602         // RC[kValTileM][kValTileN][2] 中 RC[...] [...] [2] 中保存了2个 uint32
1603         // 寄存器, 每个uint32 寄存器表示两个临近的fp16值: {c0,c1}, 然后RC[...] [...] [0]
1604         // 和RC[...] [...] [1]代表的是按照col-major排布的2个8x8子矩阵上 (不同物理行, 跨8x8)
1605         // 同一个位置上的元素, 也就是, 实际代表了2个不同的行的元素, 因此要分开RC0, RC1;
1606         // RC0表示第一行, 8个half, 可以用4个uint32寄存器来装; 同理, RC1表示第二行。
1607         uint32_t RC0[kValTileN][4]; // 32 bits x 4 = 128 bits = 8 half
1608         uint32_t RC1[kValTileN][4]; // 32 bits x 4 = 128 bits = 8 half
1609         // =====
1610         // MMA m16n8k16 C fragment — a single 16x8 matrix (registers per warp):
1611         // Thread t holds 4 half values → RC[0] for rows 0-7, RC[1] for rows 8-15.
1612         // Mapping: row = t/4, col-pair = t%4 (each uint32 = 2 half values).
1613         //
1614         //      cols: c0-1    c2-3    c4-5    c6-7
1615         //  r0: T0{c0,c1}  T1{c2,c3}  T2{c4,c5}  T3{c6,c7}  ↵
1616         //  r1: T4{c0,c1}  T5{c2,c3}  T6{c4,c5}  T7{c6,c7}  |
1617         //  r2: T8{c0,c1}  T9{c2,c3}  T10{c4,c5} T11{c6,c7} | RC[0]
1618         //  ...          |
1619         //  r7: T28{c0,c1} T29{c2,c3} T30{c4,c5} T31{c6,c7} ↵
1620         //  r8: T0{c0,c1}  T1{c2,c3}  T2{c4,c5}  T3{c6,c7}  ↵
1621         //  r9: T4{c0,c1}  T5{c2,c3}  T6{c4,c5}  T7{c6,c7}  |
1622         // r10: T8{c0,c1}  T9{c2,c3}  T10{c4,c5} T11{c6,c7} | RC[1]
1623         //  ...          |
1624         // r15: T28{c0,c1} T29{c2,c3} T30{c4,c5} T31{c6,c7} ↵
1625         //
1626         // Within a 4-lane group (e.g. T0-T3 for row 0):
1627         //   lane+0 holds {c0,c1}, lane+1 holds {c2,c3},
1628         //   lane+2 holds {c4,c5}, lane+3 holds {c6,c7}.
1629         // shfl 从 lane+1/+2/+3 各收 2 个 half 到 lane+0, 4 次 shuffle 凑齐
1630         // 一行 8 个 half = {c0,c1,c2,c3,c4,c5,c6,c7}, 再由 lane%4==0 128-bit store.
1631         // =====
1632     #pragma unroll
1633     for (int j = 0; j < kValTileN; ++j) {
1634         RC0[j][0] = RC[i][j][0];
1635         RC1[j][0] = RC[i][j][1];
1636         RC0[j][1] = __shfl_sync(0xffffffff, RC[i][j][0], lane_id + 1);
1637         RC0[j][2] = __shfl_sync(0xffffffff, RC[i][j][0], lane_id + 2);

```

```

1638     RC0[j][3] = __shfl_sync(0xffffffff, RC[i][j][0], lane_id + 3);
1639     RC1[j][1] = __shfl_sync(0xffffffff, RC[i][j][1], lane_id + 1);
1640     RC1[j][2] = __shfl_sync(0xffffffff, RC[i][j][1], lane_id + 2);
1641     RC1[j][3] = __shfl_sync(0xffffffff, RC[i][j][1], lane_id + 3);
1642 }
1643 // 每 4 个 lane 中只有 lane 0 做 128-bit store
1644 // lane_id / 4 → 行号映射 (每 4 个连续 lane 负责上8x8矩阵和下8x8矩阵各一行) :
1645 //
1646 // | lane_id | lane_id/4 | RC[0] 的行 | RC[1] 的行 |
1647 // |-----|-----|-----|-----|
1648 // | 0~3    | 0        | row 0     | row 8     |
1649 // | 4~7    | 1        | row 1     | row 9     |
1650 // | 8~11   | 2        | row 2     | row 10    |
1651 // | 12~15  | 3        | row 3     | row 11    |
1652 // | 16~19  | 4        | row 4     | row 12    |
1653 // | 20~23  | 5        | row 5     | row 13    |
1654 // | 24~27  | 6        | row 6     | row 14    |
1655 // | 28~31  | 7        | row 7     | row 15    |
1656 //
1657 if (lane_id % 4 == 0) {
1658     // {0,1} * (16 * 4) + i * 16 = {0,64} + {0,16,32,48} = {0,16,32,48,64,80,96,112}
1659     int store_warp_smem_c_m = warp_m * (kMmaM * kValTileM) + i * kMmaM; // smem row
1660     // 这里用 lane_id / 4 → {0,1,2,3,4,5,6,7}, 对应 RC0/RC1 (+8) 的行号, 表示2个8x8矩阵的行号
1661     int store_lane_gmem_c_m = by * BM + store_warp_smem_c_m + lane_id / 4; // gmem row
1662 #pragma unroll
1663     for (int j = 0; j < kValTileN; ++j) {
1664         // {0,...,3} * (8 * 4) + j * 8 = {0,32,64,96} + {0,8,16,24} = {0,8,...,120}
1665         int store_warp_smem_c_n = warp_n * (kMmaN * kValTileN) + j * kMmaN; // smem col
1666         int store_lane_gmem_c_n = bx * BN + store_warp_smem_c_n; // gmem col
1667         int store_gmem_c_addr_0 = store_lane_gmem_c_m * N + store_lane_gmem_c_n; // 1-th 8x8 matrix
1668         int store_gmem_c_addr_1 = (store_lane_gmem_c_m + 8) * N + store_lane_gmem_c_n; // 2-th 8x8 matrix
1669         // 128-bit store: 一次写入 8 个 half
1670         *reinterpret_cast<float4*>(&C[store_gmem_c_addr_0]) =
1671             *reinterpret_cast<float4*>(&RC0[j][0]);
1672         *reinterpret_cast<float4*>(&C[store_gmem_c_addr_1]) =
1673             *reinterpret_cast<float4*>(&RC1[j][0]);
1674     }
1675 }
1676 }
1677 }
1678 }
1679
1680 // =====
1681 // Phase 7b-3: HGEMM MMA Swizzle — m16n8k16 + multistage pipeline + TN 布局
1682 // + XOR swizzle 消除 smem bank conflict (统一循环版)
1683 // =====
1684 // 面试要点 (Swizzle 原理 — smem Bank Conflict 消除) :
1685 // - 问题: ldmatrix 以 8x8 片段 (m8n8) 从 smem 加载 16x16 的矩阵。同一 warp 中
1686 //   不同 lane 访问的地址在 bank 上产生冲突 → 串行化 (most common: 2/4-way)。
1687 // - XOR swizzle 方案: 对列地址做 `((j>>3) ^ (i>>2)) % 2 << 3` 的 XOR 置换,
1688 //   将连续 4 行的 bank 映射打散。每 4 行为一组的 pattern 交替:
1689 //   rows 0~3: 列 0→地址偏移 0, 列 8→地址偏移 8
1690 //   rows 4~7: 列 0→地址偏移 8, 列 8→地址偏移 0 (XOR 翻转)
1691 //   rows 8~11: repeat rows 0~3 pattern
1692 //   rows 12~15: repeat rows 4~7 pattern
1693 // - 效果: 原来 n-way 的 bank conflict 降低到 1-way (fully conflict-free)。
1694 // - 优势: 无需 smem PAD (不浪费空间), 原理上完全消除特定 pattern 的 bank conflict。
1695 // - 局限性: 要求 kColStride ≤ 16 (即 BK ≤ 16), kStep ∈ {4,8}。
1696 //   对 MMA m16n8k16 的 BK=16 而言刚好满足。
1697 //
1698 // 参考: LeetCUDA/kernels/hgemm/mma/swizzle/hgemm_mma_stage_tn_swizzle.cu
1699 // 参考: https://zhuanlan.zhihu.com/p/4746910252
1700

```

```

1701 // ---- Swizzle 辅助函数 ----
1702
1703 // i: row index; j: col index.
1704 // Returns the chunk-level (8 fp16 = 16 B per chunk) column swizzle, i.e.
1705 // 0 / 8 / 16 / ... that the caller adds to the chunk-aligned base column.
1706 // The within-chunk offset (`j & 7`) is preserved by the caller.
1707 //
1708 // kColStride matches the equivalent CuTe ``Swizzle<B, M, S>`` swizzle
1709 // hardware mode used by TMA bulk-tensor descriptors:
1710 // * kColStride = 16 fp16 = 32 B/row -> Swizzle<1, 4, 3> = SWIZZLE_32B
1711 // * kColStride = 32 fp16 = 64 B/row -> Swizzle<2, 4, 3> = SWIZZLE_64B
1712 // * kColStride = 64 fp16 = 128 B/row -> Swizzle<3, 4, 3> = SWIZZLE_128B
1713 // The XOR mask is derived from the byte-address bit positions used by the
1714 // underlying CuTe swizzle pattern: bits {4..(4+B-1)} XOR bits
1715 // {7..(7+B-1)} of the absolute byte address. Translating to (i, j) with
1716 // row stride ``kColStride * 2`` bytes:
1717 // * 32B (B=1) : (j>>3) ^ (i>>2) [chunk in {0, 1}]
1718 // * 64B (B=2) : chunk(2b) ^ {(i>>1)&1, (i>>2)&1} [{0..3}]
1719 // * 128B (B=3) : chunk(3b) ^ {i&1, (i>>1)&1, (i>>2)&1} [{0..7}]
1720 // source: LeetCUDA/ffpa-attn/cuffpa/swizzle.cuh
1721 template <const int kColStride = 16, const int kStep = 8>
1722 static __device__ __forceinline__ int permuted(int i, int j) {
1723     static_assert(kColStride == 16 || kColStride == 32 || kColStride == 64 ||
1724         kColStride == 8,
1725         "kColStride must be one of {8, 16, 32, 64} (matches "
1726         "SWIZZLE_32B/64B/128B + the "
1727         "kStep=4 narrow legacy case).");
1728     static_assert(kStep == 4 || kStep == 8, "kStep must be 8 or 4.");
1729     static_assert(kColStride % kStep == 0,
1730         "kColStride must be multiple of kStep.");
1731     if constexpr (kStep == 4) {
1732         static_assert(kColStride <= 16, "kStep=4 only supports (kColStride <= 16).");
1733         return (((j >> 2) ^ (i >> 2)) % (kColStride >> 2)) << 2;
1734     } else if constexpr (kColStride == 16) {
1735         return (((j >> 3) ^ (i >> 2)) & 1) << 3; // SWIZZLE_32B
1736     } else if constexpr (kColStride == 32) {
1737         const int chunk = (j >> 3) & 3;
1738         const int xor_mask = ((i >> 1) & 1) | (((i >> 2) & 1) << 1);
1739         return (chunk ^ xor_mask) << 3; // SWIZZLE_64B
1740     } else {
1741         // kColStride == 64
1742         const int chunk = (j >> 3) & 7;
1743         const int xor_mask =
1744             (i & 1) | (((i >> 1) & 1) << 1) | (((i >> 2) & 1) << 2);
1745         return (chunk ^ xor_mask) << 3; // SWIZZLE_128B
1746     }
1747 }
1748 // Manually SMEM swizzling for bank conflict free.
1749 // -----
1750 // [INFO] Assert smem store layout col_stride <= 16, prefer 16. |
1751 // [INFO] For logical_col_stride > 16, we have to permute the |
1752 // [INFO] smem store layout using col major ZigZag method: |
1753 // [INFO] e.g., --> Q smem logical layout [Br][64]. |
1754 // [INFO] --> col major ZigZag permuted --> |
1755 // [INFO] --> Q smem store layout [4][Br][16]. |
1756 // -----
1757 // -----swizzle layout-----
1758 // -----logical col 0~64, step 8-----
1759 // -----smem col 0~16, step 8-----
1760 // -----
1761 // |bank |b 0~3 |b 4~7 |b 0~3 |b 4~7 |b 0~3 |b 4~7 |b 0~3 |b 4~7 |
1762 // |row 0 | 0 | 8 | 0 | 8 | 0 | 8 | 0 | 8 |

```

```

1764 // |bank |b 8~11|b12~15|b 8~11|b12~15|b 8~11|b12~15|b 8~11|b12~15|
1765 // |row 1 | 0 | 8 | 0 | 8 | 0 | 8 | 0 | 8 |
1766 // |bank |b16~19|b20~23|b16~19|b20~23|b16~19|b20~23|b16~19|b20~23|
1767 // |row 2 | 0 | 8 | 0 | 8 | 0 | 8 | 0 | 8 |
1768 // |bank |b24~27|b28~31|b24~27|b28~31|b24~27|b28~31|b24~27|b28~31|
1769 // |row 3 | 0 | 8 | 0 | 8 | 0 | 8 | 0 | 8 |
1770 // -----
1771 // |bank |b 0~3 |b 4~7 |b 0~3 |b 4~7 |b 0~3 |b 4~7 |b 0~3 |b 4~7 |
1772 // |row 4 | 8 | 0 | 8 | 0 | 8 | 0 | 8 | 0 |
1773 // |bank |b 8~11|b12~15|b 8~11|b12~15|b 8~11|b12~15|b 8~11|b12~15|
1774 // |row 5 | 8 | 0 | 8 | 0 | 8 | 0 | 8 | 0 |
1775 // |bank |b16~19|b20~23|b16~19|b20~23|b16~19|b20~23|b16~19|b20~23|
1776 // |row 6 | 8 | 0 | 8 | 0 | 8 | 0 | 8 | 0 |
1777 // |bank |b24~27|b28~31|b24~27|b28~31|b24~27|b28~31|b24~27|b28~31|
1778 // |row 7 | 8 | 0 | 8 | 0 | 8 | 0 | 8 | 0 |
1779 // -----
1780 // |bank |b 0~3 |b 4~7 |b 0~3 |b 4~7 |b 0~3 |b 4~7 |b 0~3 |b 4~7 |
1781 // |row 8 | 0 | 8 | 0 | 8 | 0 | 8 | 0 | 8 |
1782 // |bank |b 8~11|b12~15|b 8~11|b12~15|b 8~11|b12~15|b 8~11|b12~15|
1783 // |row 9 | 0 | 8 | 0 | 8 | 0 | 8 | 0 | 8 |
1784 // |bank |b16~19|b20~23|b16~19|b20~23|b16~19|b20~23|b16~19|b20~23|
1785 // |row 10| 0 | 8 | 0 | 8 | 0 | 8 | 0 | 8 |
1786 // |bank |b24~27|b28~31|b24~27|b28~31|b24~27|b28~31|b24~27|b28~31|
1787 // |row 11| 0 | 8 | 0 | 8 | 0 | 8 | 0 | 8 |
1788 // -----
1789 // |bank |b 0~3 |b 4~7 |b 0~3 |b 4~7 |b 0~3 |b 4~7 |b 0~3 |b 4~7 |
1790 // |row 12| 8 | 0 | 8 | 0 | 8 | 0 | 8 | 0 |
1791 // |bank |b 8~11|b12~15|b 8~11|b12~15|b 8~11|b12~15|b 8~11|b12~15|
1792 // |row 13| 8 | 0 | 8 | 0 | 8 | 0 | 8 | 0 |
1793 // |bank |b16~19|b20~23|b16~19|b20~23|b16~19|b20~23|b16~19|b20~23|
1794 // |row 14| 8 | 0 | 8 | 0 | 8 | 0 | 8 | 0 |
1795 // |bank |b24~27|b28~31|b24~27|b28~31|b24~27|b28~31|b24~27|b28~31|
1796 // |row 15| 8 | 0 | 8 | 0 | 8 | 0 | 8 | 0 |
1797 // -----
1798 template <const int kColStride = 16>
1799 static __device__ __forceinline__ int swizzle(int i, int j) {
1800     return permuted<kColStride, 8>(i, j);
1801 }
1802
1803 // =====
1804 // Phase 7b-3: HGEMM MMA — m16n8k16 + multistage pipeline + TN 布局 + XOR swizzle
1805 // + Register Double Buffering (generic kValTileK, default BK=64)
1806 // =====
1807 // 在 Phase 7b-2 的 smem XOR swizzle 基础上增加寄存器双缓冲:
1808 // - kValTileK: 每个 BK tile 包含 kValTileK 个 kMmaK slice (BK = kMmaK * kValTileK)
1809 // - 统一 BK tile: 所有 k_step slice 连续存放在同一个 BK 宽的 smem tile 中
1810 // - RA[2][kValTileM][4] / RB[2][kValTileN][2]: 双份寄存器乒乓切换 (与 kValTileK 无关)
1811 // - ldmatrix 与 MMA 计算重叠: 加载 k_step+1 的同时用另一组寄存器做 k_step 计算
1812 // - 支持任意 kValTileK >= 2; 默认 kValTileK=4 (BK=64), kStages=2 推荐 64KB smem
1813 //
1814 // ★ smem 布局:
1815 // s_a: [stage0][stage1]...[stage(kStages-1)], 每个 stage = BM × BK = 128 × BK
1816 // s_b: 紧接 s_a 之后
1817 // 每个 stage 内 k_step slice 列偏移 = k_step * kMmaK
1818 //
1819 // ★ 寄存器双缓冲时间线 (每 k 迭代内):
1820 // Step 1: G→S cp.async 预取 stage(k+kStages-1) 全部 k_step (条件)
1821 // Step 2: MMA k_step=0 (用 reg[load], 已在上轮 post-wait 中 ldmatrix)
1822 // Step 3: for k_step=1..kValTileK-1: ldmatrix(列偏移 k_step*kMmaK)→reg[store], flip, MMA→reg[load]
1823 // Step 4: adaptive wait + __syncthreads
1824 // Step 5: S→R ldmatrix 预加载 stage(k+1) k_step=0 → reg[store] (条件)
1825 //
1826 // 参考: LeetCUDA/kernels/hgemm/mma/swizzle/hgemm_mma_stage_tn_swizzle_x2.cu

```



```

1827 // Grid: ((N+127)/128/S, (M+127)/128, S), S=(N+2047)/2048, 3D block swizzle
1828 // Block: (256, 1, 1), 8 warps
1829 template <const int kMmaM = 16,           // MMA atom M dim (m16n8k16)
1830           const int kMmaN = 8,           // MMA atom N dim
1831           const int kMmaK = 16,          // MMA atom K dim
1832           const int kMmaTileM = 2,       // warps along M, 2 → warp tile M = 32
1833           const int kMmaTileN = 4,       // warps along N, 4 → warp tile N = 32
1834           const int kValTileM = 4,       // value-repeat along M, BM = 16*2*4 = 128
1835           const int kValTileN = 4,       // value-repeat along N, BN = 8*4*4 = 128
1836           const int kValTileK = 4,       // MMA_K slices per BK tile, BK = kMmaK * kValTileK
1837           const int kStages = 2,         // cp.async pipeline depth (2 for kValTileK>=4 to fit smem)
1838           const int kBlockSwizzle = 0>   // 1 enables 3D grid swizzle for L2 locality
1839 __global__ void __launch_bounds__(256)
1840   hgemm_mma_stages_tn_swizzle(half *A, half *B, half *C, int M, int N, int K) {
1841   static_assert(kValTileK >= 2, "Register double buffering requires kValTileK >= 2");
1842   static_assert(kBlockSwizzle == 0 || kBlockSwizzle == 1, "kBlockSwizzle must be 0 or 1");
1843   static_assert(kStages >= 2, "kStages must be >= 2 for cp.async pipeline");
1844   // Block Swizzle: 在 grid x 维度做 swizzle, 改善 L2 cache 局部性
1845   const int bx = ((int)kBlockSwizzle) * blockIdx.z * gridDim.x + blockIdx.x;
1846   const int by = blockIdx.y;
1847   constexpr int BM = kMmaM * kMmaTileM * kValTileM; // 16*2*4=128
1848   constexpr int BN = kMmaN * kMmaTileN * kValTileN; // 8*4*4=128
1849   constexpr int BK = kMmaK * kValTileK;             // 16 * kValTileK
1850
1851   // kStages stages, each with BM×BK for A, BN×BK for B
1852   // smem: kValTileK 个 kMmaK slice 连续存放在同一个 BK 宽 tile 中
1853   extern __shared__ half smem[];
1854   half *s_a = smem;
1855   half *s_b = smem + kStages * BM * BK;
1856   constexpr int s_a_stage_offset = BM * BK;
1857   constexpr int s_b_stage_offset = BN * BK;
1858
1859   const int tid = threadIdx.y * blockDim.x + threadIdx.x;
1860   const int warp_id = tid / kWarpSize;
1861   const int lane_id = tid % kWarpSize;
1862   const int warp_m = warp_id % kMmaTileM; // 0,1 (M 方向 2 个 warp) kMmaTileM = 2
1863   const int warp_n = warp_id / kMmaTileM; // 0,1,2,3 (N 方向 4 个 warp)
1864
1865   // 线程到 global memory 的映射 (用于加载 A 和 B) 共 256 个线程
1866   // TN 布局关键: A[m*K+k] 是 row-major, B^T[n*K+k] 是 row-major (内维连续的是 K)
1867   // 注意: smem_a_k 和 smem_b_k 依然使用 0/8, 在后续的 kValTileK 循环中
1868   // 会加上 k_step*kMmaK, 最终覆盖全部 BK 列
1869   int load_smem_a_m = tid / 2;           // 0~127
1870   int load_smem_a_k = (tid % 2 == 0) ? 0 : 8; // 0, 8
1871   int load_smem_b_n = tid / 2;           // 0~127 → B^T 的 N 方向 (row-major 的行)
1872   int load_smem_b_k = (tid % 2 == 0) ? 0 : 8; // 0, 8 → B^T 的 K 方向 (row-major 的列)
1873   int load_gmem_a_m = by * BM + load_smem_a_m; // C/A 全局行号 = M 方向的 tile 起始 + 线程偏移
1874   int load_gmem_b_n = bx * BN + load_smem_b_n; // C/B 全局列号 = N 方向的 tile 起始 + 线程偏移
1875   if (load_gmem_a_m >= M || load_gmem_b_n >= N)
1876     return;
1877
1878   // 8个Warps(MMAs)的排布是M方向2个, N方向4个, 则MMA Atom一次性能处理的tile大小是[2*16,4*8]=[32,32].
1879   // CUDA中每个block能放的线程数是有上限的 (warp数量有限), 一般为4或者8个warps。为了能处理更大的C tile,
1880   // 需要把MMA Atom的tile再M方向和N方向各自重复4次, 得到[128,128]的C block tile, 这就是Value Tile的作用。
1881   // MMA Tile对应到cutlass cute中的TiledMMA的概念, Value Tile对应到cutlass cute中的PermuteMNMK的概念。
1882   uint32_t RC[kValTileM][kValTileN][2] = {0}; // 初始化为 0
1883
1884   uint32_t smem_a_base_ptr = __cvta_generic_to_shared(s_a);
1885   uint32_t smem_b_base_ptr = __cvta_generic_to_shared(s_b);
1886
1887   // Prefetch (kStages-1) stages: load all k_step slices for each early stage
1888   #pragma unroll
1889   for (int k = 0; k < (kStages - 1); ++k) {

```



```

1890 #pragma unroll
1891 for (int k_step = 0; k_step < kValTileK; ++k_step) {
1892     int load_gmem_a_k = k * BK + (k_step * kMmaK) + load_smem_a_k;
1893     int load_gmem_a_addr = load_gmem_a_m * K + load_gmem_a_k;
1894     int load_gmem_b_k = k * BK + (k_step * kMmaK) + load_smem_b_k;
1895     int load_gmem_b_addr = load_gmem_b_n * K + load_gmem_b_k;
1896
1897     uint32_t load_smem_a_ptr =
1898         (smem_a_base_ptr +
1899          (k * s_a_stage_offset + load_smem_a_m * BK +
1900           k_step * kMmaK +
1901            swizzle<kMmaK>(load_smem_a_m, load_smem_a_k)) *
1902           sizeof(half));
1903     CP_ASYNC.CG(load_smem_a_ptr, &A[load_gmem_a_addr], 16);
1904
1905     uint32_t load_smem_b_ptr =
1906         (smem_b_base_ptr +
1907          (k * s_b_stage_offset + load_smem_b_n * BK +
1908           k_step * kMmaK +
1909            swizzle<kMmaK>(load_smem_b_n, load_smem_b_k)) *
1910           sizeof(half));
1911     CP_ASYNC.CG(load_smem_b_ptr, &B[load_gmem_b_addr], 16);
1912 }
1913 CP_ASYNC.COMMIT_GROUP();
1914 }
1915
1916 CP_ASYNC.WAIT_GROUP(kStages - 2);
1917 __syncthreads();
1918
1919 // Register double buffers: RA[0/1] for k_step=0/1 A data, RB[0/1] for B data
1920 uint32_t RA[2][kValTileM][4];
1921 uint32_t RB[2][kValTileN][2];
1922 int reg_st_idx = 0; // write target for ldmatrix
1923 int reg_ld_idx = 1; // read source for MMA
1924
1925 // Initial ldmatrix: load stage 0, S -> R, k_step=0 (0~15列) -> reg[0]
1926 // 此时, reg_st_idx = 0, 对0位置的寄存器buffer做初始化。
1927 #pragma unroll
1928 for (int i = 0; i < kValTileM; ++i) {
1929     int warp_smem_a_m = warp_m * (kMmaM * kValTileM) + i * kMmaM;
1930     int lane_smem_a_m = warp_smem_a_m + lane_id % 16;
1931     int lane_smem_a_k = (lane_id / 16) * 8;
1932     uint32_t lane_smem_a_ptr =
1933         (smem_a_base_ptr +
1934          (0 * s_a_stage_offset + lane_smem_a_m * BK +
1935           swizzle<kMmaK>(lane_smem_a_m, lane_smem_a_k)) *
1936           sizeof(half));
1937     LDMATRIX_X4(RA[reg_st_idx][i][0], RA[reg_st_idx][i][1],
1938                 RA[reg_st_idx][i][2], RA[reg_st_idx][i][3],
1939                 lane_smem_a_ptr);
1940 }
1941 #pragma unroll
1942 for (int j = 0; j < kValTileN; ++j) {
1943     int warp_smem_b_n = warp_n * (kMmaN * kValTileN) + j * kMmaN;
1944     int lane_smem_b_n = warp_smem_b_n + lane_id % 8;
1945     int lane_smem_b_k = ((lane_id / 8) % 2) * 8;
1946     uint32_t lane_smem_b_ptr =
1947         (smem_b_base_ptr +
1948          (0 * s_b_stage_offset + lane_smem_b_n * BK +
1949           swizzle<kMmaK>(lane_smem_b_n, lane_smem_b_k)) *
1950           sizeof(half));
1951     LDMATRIX_X2(RB[reg_st_idx][j][0], RB[reg_st_idx][j][1],
1952                 lane_smem_b_ptr);

```

```

1953 }
1954
1955 // 统一循环: k 从 0 开始, 条件 G→S / S→R / wait 处理所有边界情况
1956 // BK = kMmaK * kValTileK, 每个 k 迭代覆盖 BK 个 K 元素
1957 const int NUM_K_TILES = div_ceil(K, BK);
1958 for (int k = 0; k < NUM_K_TILES; ++k) {
1959     int smem_sel = k % kStages;
1960     int smem_sel_next = (k + kStages - 1) % kStages;
1961
1962     // G→S: 条件预取 stage(k+kStages-1) 全部 k_step slice
1963     if (k + kStages - 1 < NUM_K_TILES) {
1964 #pragma unroll
1965         for (int k_step = 0; k_step < kValTileK; ++k_step) {
1966             int load_gmem_a_k =
1967                 (k + kStages - 1) * BK + (k_step * kMmaK) + load_smem_a_k;
1968             int load_gmem_a_addr = load_gmem_a_m * K + load_gmem_a_k;
1969             int load_gmem_b_k =
1970                 (k + kStages - 1) * BK + (k_step * kMmaK) + load_smem_b_k;
1971             int load_gmem_b_addr = load_gmem_b_n * K + load_gmem_b_k;
1972
1973             uint32_t load_smem_a_ptr =
1974                 (smem_a_base_ptr +
1975                  (smem_sel_next * s_a_stage_offset + load_smem_a_m * BK +
1976                   k_step * kMmaK +
1977                   swizzle<kMmaK>(load_smem_a_m, load_smem_a_k)) *
1978                  sizeof(half));
1979             CP_ASYNC_CG(load_smem_a_ptr, &A[load_gmem_a_addr], 16);
1980
1981             uint32_t load_smem_b_ptr =
1982                 (smem_b_base_ptr +
1983                  (smem_sel_next * s_b_stage_offset + load_smem_b_n * BK +
1984                   k_step * kMmaK +
1985                   swizzle<kMmaK>(load_smem_b_n, load_smem_b_k)) *
1986                  sizeof(half));
1987             CP_ASYNC_CG(load_smem_b_ptr, &B[load_gmem_b_addr], 16);
1988         }
1989         CP_ASYNC_COMMIT_GROUP();
1990     }
1991
1992     // 内层 k_step 循环: flip → ldmatrix(k_step+1) → MMA, 乒乓交替
1993     // 初始: st=0, ld=1, reg[0]=preload k_step=0; reg[1]=未初始化
1994     // 每轮: flip→ldmatrix next k_step→reg[st], MMA reg[ld] (k_step+1 的 ldmatrix 条件化)
1995
1996 #pragma unroll
1997     for (int k_step = 0; k_step < kValTileK; ++k_step) {
1998         reg_st_idx ^= 1; // 0 → 1; 1 → 0
1999         reg_ld_idx ^= 1; // 1 → 0; 0 → 1
2000
2001         if (k_step + 1 < kValTileK) {
2002             int smem_k_offset = (k_step + 1) * kMmaK;
2003 #pragma unroll
2004             for (int i = 0; i < kValTileM; ++i) {
2005                 int warp_smem_a_m = warp_m * (kMmaM * kValTileM) + i * kMmaM;
2006                 int lane_smem_a_m = warp_smem_a_m + lane_id % 16;
2007                 int lane_smem_a_k = (lane_id / 16) * 8;
2008                 uint32_t lane_smem_a_ptr =
2009                     (smem_a_base_ptr +
2010                      (smem_sel * s_a_stage_offset + lane_smem_a_m * BK +
2011                       smem_k_offset +
2012                       swizzle<kMmaK>(lane_smem_a_m, lane_smem_a_k)) *
2013                      sizeof(half));
2014                 LDMATRIX_X4(RA[reg_st_idx][i][0], RA[reg_st_idx][i][1],
2015                             RA[reg_st_idx][i][2], RA[reg_st_idx][i][3],

```

```

2016         lane_smem_a_ptr);
2017     }
2018     #pragma unroll
2019     for (int j = 0; j < kValTileN; ++j) {
2020         int warp_smem_b_n = warp_n * (kMmaN * kValTileN) + j * kMmaN;
2021         int lane_smem_b_n = warp_smem_b_n + lane_id % 8;
2022         int lane_smem_b_k = ((lane_id / 8) % 2) * 8;
2023         uint32_t lane_smem_b_ptr =
2024             (smem_b_base_ptr +
2025              (smem_sel * s_b_stage_offset + lane_smem_b_n * BK +
2026               smem_k_offset +
2027               swizzle<kMmaK>(lane_smem_b_n, lane_smem_b_k)) *
2028               sizeof(half));
2029         LDMATRIX_X2(RB[reg_st_idx][j][0], RB[reg_st_idx][j][1],
2030                     lane_smem_b_ptr);
2031     }
2032 }
2033
2034 #pragma unroll
2035 for (int i = 0; i < kValTileM; ++i) {
2036     #pragma unroll
2037     for (int j = 0; j < kValTileN; ++j) {
2038         HMMMA16816(RC[i][j][0], RC[i][j][1],
2039                    RA[reg_ld_idx][i][0], RA[reg_ld_idx][i][1],
2040                    RA[reg_ld_idx][i][2], RA[reg_ld_idx][i][3],
2041                    RB[reg_ld_idx][j][0], RB[reg_ld_idx][j][1],
2042                    RC[i][j][0], RC[i][j][1]);
2043     }
2044 }
2045 }
2046
2047 // 自适应等待: 满载期用 kStages-2, 尾部排空用 0
2048 if (k + kStages - 1 < NUM_K_TILES) {
2049     CP_ASYNC_WAIT_GROUP(kStages - 2);
2050 } else {
2051     CP_ASYNC_WAIT_GROUP(0);
2052 }
2053 __syncthreads();
2054
2055 // 从下一阶段加载 k_step=0 数据, 供下一轮 k 迭代使用, 此时 reg_st_idx=0
2056 if (k + 1 < NUM_K_TILES) {
2057     smem_sel = (k + 1) % kStages; // old smem_sel + 1
2058     #pragma unroll
2059     for (int i = 0; i < kValTileM; ++i) {
2060         int warp_smem_a_m = warp_m * (kMmaM * kValTileM) + i * kMmaM;
2061         int lane_smem_a_m = warp_smem_a_m + lane_id % 16;
2062         int lane_smem_a_k = (lane_id / 16) * 8;
2063         uint32_t lane_smem_a_ptr =
2064             (smem_a_base_ptr +
2065              (smem_sel * s_a_stage_offset + lane_smem_a_m * BK +
2066               swizzle<kMmaK>(lane_smem_a_m, lane_smem_a_k)) *
2067               sizeof(half));
2068         LDMATRIX_X4(RA[reg_st_idx][i][0], RA[reg_st_idx][i][1],
2069                     RA[reg_st_idx][i][2], RA[reg_st_idx][i][3],
2070                     lane_smem_a_ptr);
2071     }
2072     #pragma unroll
2073     for (int j = 0; j < kValTileN; ++j) {
2074         int warp_smem_b_n = warp_n * (kMmaN * kValTileN) + j * kMmaN;
2075         int lane_smem_b_n = warp_smem_b_n + lane_id % 8;
2076         int lane_smem_b_k = ((lane_id / 8) % 2) * 8;
2077         uint32_t lane_smem_b_ptr =
2078             (smem_b_base_ptr +

```

```

2079         (smem_sel * s_b_stage_offset + lane_smem_b_n * BK +
2080         swizzle<kMmaK>(lane_smem_b_n, lane_smem_b_k)) *
2081         sizeof(half));
2082     LDMATRIX_X2(RB[reg_st_idx][j][0], RB[reg_st_idx][j][1],
2083     lane_smem_b_ptr);
2084 }
2085 }
2086 }
2087
2088 // Epilogue: 复用 RA[2][4][4] 寄存器做 warp shuffle → 128-bit collective store
2089 // 做RC的warp shuffle正好需要[2][4][4]的寄存器空间, RA[2][4][4]正好可以复用
2090 for (int i = 0; i < kValTileM; ++i) {
2091 #pragma unroll
2092     for (int j = 0; j < kValTileN; ++j) {
2093         RA[0][j][0] = RC[i][j][0];
2094         RA[1][j][0] = RC[i][j][1];
2095         RA[0][j][1] = __shfl_sync(0xffffffff, RC[i][j][0], lane_id + 1);
2096         RA[0][j][2] = __shfl_sync(0xffffffff, RC[i][j][0], lane_id + 2);
2097         RA[0][j][3] = __shfl_sync(0xffffffff, RC[i][j][0], lane_id + 3);
2098         RA[1][j][1] = __shfl_sync(0xffffffff, RC[i][j][1], lane_id + 1);
2099         RA[1][j][2] = __shfl_sync(0xffffffff, RC[i][j][1], lane_id + 2);
2100         RA[1][j][3] = __shfl_sync(0xffffffff, RC[i][j][1], lane_id + 3);
2101     }
2102     if (lane_id % 4 == 0) {
2103         int store_warp_smem_c_m = warp_m * (kMmaM * kValTileM) + i * kMmaM;
2104         int store_lane_gmem_c_m = by * BM + store_warp_smem_c_m + lane_id / 4;
2105 #pragma unroll
2106         for (int j = 0; j < kValTileN; ++j) {
2107             int store_warp_smem_c_n = warp_n * (kMmaN * kValTileN) + j * kMmaN;
2108             int store_lane_gmem_c_n = bx * BN + store_warp_smem_c_n;
2109             int store_gmem_c_addr_0 =
2110                 store_lane_gmem_c_m * N + store_lane_gmem_c_n;
2111             int store_gmem_c_addr_1 =
2112                 (store_lane_gmem_c_m + 8) * N + store_lane_gmem_c_n;
2113             *reinterpret_cast<float4*>(&C[store_gmem_c_addr_0]) =
2114                 *reinterpret_cast<float4*>(&RA[0][j][0]);
2115             *reinterpret_cast<float4*>(&C[store_gmem_c_addr_1]) =
2116                 *reinterpret_cast<float4*>(&RA[1][j][0]);
2117         }
2118     }
2119 }
2120 }
2121
2122 // =====
2123 // Phase 7c: HGEMM CuTe — CUTLASS CuTe DSL 实现 (SM80+, Tensor Core 全自动调度)
2124 // =====
2125 // 面试要点 (CuTe vs 手写 MMA PTX) :
2126 // - CuTe (CUTLASS Templates) 是 NVIDIA CUTLASS 3.x 的核心 DSL, 提供编译期
2127 //   Tensor 抽象, 自动推导 MMA、Copy、Swizzle 的线程-数据映射
2128 // - 与手写 MMA PTX (Phase 7b) 的对比:
2129 //   - 手写 MMA: 手动计算 smem 地址、手动 ldmatrix/mma PTX、手动 swizzle、手动 epilogue shuffle
2130 //   - CuTe: 声明 TiledMMA / TiledCopy / SmemLayout, 编译器自动推导线程-数据映射,
2131 //     cute::copy / cute::gemm 自动展开为高效指令序列
2132 // - 核心抽象 ("CuTe 五要素") :
2133 //   1. Tensor: 全局/共享/寄存器数据的逻辑视图 = (ptr, shape, stride)
2134 //   2. TiledMMA: 描述 MMA 指令 + warp/warpgroup 排布 + value tile 的 compile-time type
2135 //   3. TiledCopy: 描述数据搬运 (G→S, S→R, R→S, S→G) 的线程-元素映射
2136 //   4. Layout + Swizzle: 描述 smem 的数据排布和 bank conflict 消除
2137 //   5. Pipeline: cp.async + multistage, 由 cute::copy 自动管理
2138 //
2139 // CuTe kernel 的参数推导链 (launch wrapper 负责实例化, kernel 只接收实例化后的类型) :
2140 //   MMA Atom (SM80_16x8x16_F16F16F16F16_TN)
2141 //   → TiledMMA (EURepeat=MxNxK, ValTile=MxNxK)

```

```

2142 //      → TiledCopy (G2S/S2R/R2S/S2G, 每种有 ThrLayout + ValLayout)
2143 //      → SmemLayout (Atom + Swizzle + tile_to_shape + Stage)
2144 //
2145 // 调参口诀 (面试速记):
2146 //      BM/BN/BK 越大 → smem 越大 → occupancy 越低, 但 K 循环更少 → 指令开销更低
2147 //      Swizzle<B,M,S> 的核心不是改变逻辑 Tensor 的 shape, 而是把一维 offset
2148 //      重新解释为一个二维逻辑空间, 再把二维坐标映射回 bank-conflict-free 的物理 offset:
2149 //      1. 连续的 2^M 个元素组成二维空间中的一个基本元素; (元素宽度)
2150 //      2. 连续的 2^S 个基本元素组成一行; (列数)
2151 //      3. 二维空间包含 2^B 行; (行数)
2152 //      4. 对二维坐标做列置换: icol' = irow ^ icol;
2153 //      5. 保留基本元素内部的低 M 位, 再将置换后的二维坐标编码回 offset。
2154 //      因此 M 描述基本元素宽度, S 描述二维空间的列数, B 描述二维空间的行数。
2155 //      CuTe 的位级实现等价于:
2156 //      offset' = offset ^ ((offset & YYY) >> S),
2157 //      YYY = ((1 << B) - 1) << (M + S)。
2158 //      对 Swizzle<3,3,3>: 每个基本元素有 2^3=8 个值, 每行有 2^3=8 个基本元素,
2159 //      二维空间有 2^3=8 行; 元素地址位 [6:8] XOR 到 [3:5], 低 3 位保持不变。
2160 //      一个完整 swizzle 周期覆盖 2^(M+S+B)=2^9=512 个元素 (FP16 下为 1024B)。
2161 //      kStage: 2=最低延迟, 3/4=更好隐藏延迟 → 权衡 smem 占用
2162 //      EURepeat: 在 M/N/K 方向重复 MMA atom 的执行单元布局, 决定 TiledMMA 的线程排布
2163 //      与逻辑 tile 组织; MMA atom 越多, 需要的线程数就越多
2164 //
2165 // ★ 与手写 MMA Swizzle (Phase 7b-3) 的本质区别:
2166 // - 手写 swizzle: 在 smem 地址计算时手动 XOR 列索引
2167 // - CuTe Swizzle: SmemLayout 声明式指定地址位的 XOR pattern, cute::copy 自动应用
2168 // - CuTe 优势: 类型安全, 编译器可在编译期推导映射; 布局变更通常只需修改类型定义
2169 //
2170 // ★ 本实现的 Tile 配置:
2171 // - gmem/smem tile: BM=128, BN=256, BK=32, kStage=2; 这是 A/B tile 的目标 shape。
2172 // - MMA atom: SM80_16x8x16_F16F16F16F16_TN; EURepeat=2x2x1, MMA_P_T=32x32x16。
2173 // - TiledMMA 的逻辑 MMA tile 是 32x32x16, G2S/S2R copy 再把它映射到更大的 BMxBN tile;
2174 //   BN=256 不是由“8x2x16”直接推导出的 MMA tile, 而是本 wrapper 选择的 B tile 尺寸。
2175 // - Smem Swizzle<3,3,3>: 512-element address period 的 XOR 重排, 针对 ldmatrix
2176 //   的访问模式降低 bank conflict; 512 是 swizzle 周期, 不等于 A atom 的逻辑元素数。
2177 // - Threads: size(MMA{}) = 128 (4 warps × 2x2 EU repeat)。
2178 // - Dynamic smem: Stage=2 时 A[128x32] + B[256x32] 共 49,152 bytes (约 48 KiB)。
2179 //
2180 // 参考: LeetCUDA/kernels/hgemm/cutlass/hgemm_mma_stage_tn_cute.cu
2181 // 参考: https://zhuanlan.zhihu.com/p/671419093 (CuTe Swizzle 详解)
2182 // =====
2183
2184 #if defined(NOTES_V2_ENABLE_CUTE)
2185
2186 #include <cute/tensor.hpp>
2187
2188 // =====
2189 // Phase 7c-1: CuTe HGEMM Kernel — TN 布局 + Smem Swizzle + Multistage Pipeline
2190 // =====
2191 // TN 布局: C[MxN] = A[MxK] × B^T[NxK]
2192 // - A[M][K] row-major, B^T[N][K] row-major (等价 B col-major [KxN])
2193 // - CuTe 中通过 make_tensor(make_gmem_ptr(ptr), shape, stride) 描述
2194 //
2195 // Pipeline 流程 (stage 是 K tile 的循环缓冲槽, 不是一条完整计算链):
2196 // 1. PREFETCH: 预加载 kStage-1 个 K tile (G→S via cp.async), 填满 stage。
2197 // 2. 主循环 over K tiles:
2198 //   a. 从当前 stage 做 S→R: cute::copy(...) → ldmatrix;
2199 //   b. 对当前 K slice 做 MMA: cute::gemm(...) → mma.sync;
2200 //   c. 在处理当前 tile 的首个 MMA slice 时, 异步提交下一个 tile 的 G→S;
2201 //   d. 当前 tile 的最后一个 K slice 完成后等待并切换 smem stage。
2202 // 3. Epilogue: R→S (UniversalCopy 写入 C scratchpad) → S→G (128-bit store)。
2203 //
2204 // 面试对比 — 手写 MMA vs CuTe 代码量:

```

```

2205 // 手写: ldmatrix PTX 地址计算 + swizzle XOR + mma PTX + epilogue shuffle (~200行)
2206 // CuTe: partition + copy + gemm (~80行), 其余由 launch wrapper 的类型推导完成
2207 template <typename T,                                // element type (half)
2208         int BM, int BN, int BK,                      // block tile: BM=128, BN=256, BK=32
2209         int kStage,                                  // cp.async pipeline depth (2)
2210         typename TiledMMA,                          // MMA: SM80_16x8x16_F16F16F16_TN, EURepeat=2x2x1
2211         typename G2SCopyA,                          // G→S A: SM80_CP_ASYNC_CACHEGLOBAL<uint128_t>
2212         typename G2SCopyB,                          // G→S B: same as G2SCopyA
2213         typename SmemLayoutA,                       // smem A: Swizzle<3,3,3> + 8×BK atom → (BM,BK,kStage)
2214         typename SmemLayoutB,                       // smem B: same atom → (BN,BK,kStage)
2215         typename SmemLayoutC,                       // C scratchpad: Swizzle<3,3,3> + 32×32 atom, pipe=4
2216         typename S2RCopyAtomA,                      // S→R A: SM75_U32x4_LDSM_N (ldmatrix.x4)
2217         typename S2RCopyAtomB,                      // S→R B: same as S2RCopyAtomA
2218         typename R2SCopyAtomC,                      // R→S C: UniversalCopy<int> (32-bit store)
2219         typename S2GCopyAtomC,                      // S→G C: UniversalCopy<uint128_t> (128-bit wide store)
2220         typename S2GCopyC,                          // S→G TiledCopy: ThrLayout{32,4} × ValLayout{1,8}
2221         const int BlockSwizzle = 0>                 // 1 enables 3D grid swizzle for L2 locality
2222 __global__ void hgemm_mma_stages_tn_cute(T *Aptr, T *Bptr, T *Dptr, int m,
2223                                         int n, int k) {
2224     using namespace cute;
2225     static_assert(BlockSwizzle == 0 || BlockSwizzle == 1, "BlockSwizzle must be 0 or 1");
2226     static_assert(kStage >= 2, "kStage must be >= 2 for cp.async pipeline");
2227
2228     // 动态 shared memory: KStage 个 A/B tile; C epilogue 复用当前 A stage 的空间。
2229     extern __shared__ T shm_data[];
2230     T *Ashm = shm_data;
2231     T *Bshm = shm_data + cute::cosize(SmemLayoutA{});
2232
2233     int idx = threadIdx.x;
2234     // BlockSwizzle: 0/1 控制是否启用 thread block swizzle, 改善 L2 cache 局部性
2235     int ix = ((int)BlockSwizzle) * blockIdx.z * gridDim.x + blockIdx.x;
2236     int iy = blockIdx.y; // M 方向 block 索引
2237     // 当前 kernel 只有 CTA 级边界判断, 没有 tile 内的逐元素 predication; 调用方需保证
2238     // M % BM == 0、N % BN == 0、K % BK == 0, 才能避免边界 tile 的越界访问或未写回。
2239     if (iy * BM >= m || ix * BN >= n) return;
2240
2241     // CuTe Tensor 抽象: (ptr, shape, stride) 三元组描述数据布局
2242     // make_stride(leading_dim, Int<1>{}) → row-major: 同行相邻元素步长=1, 跨行步长=leading_dim
2243     Tensor A = make_tensor(make_gmem_ptr(Aptr), make_shape(m, k),
2244                             make_stride(k, Int<1>{}));
2245     Tensor B = make_tensor(make_gmem_ptr(Bptr), make_shape(n, k),
2246                             make_stride(k, Int<1>{}));
2247     Tensor D = make_tensor(make_gmem_ptr(Dptr), make_shape(m, n),
2248                             make_stride(n, Int<1>{}));
2249
2250     // local_tile: 按 tiler 切出当前 CTA 的逻辑 tile; 返回 Tensor 仍保留未切出的 remainder mode。
2251     // make_coord(iy, _): 固定 M/N 方向的 CTA 坐标, _ 保留 K 方向的 tile remainder:
2252     // gA: (BM, BK, num_tile_k), gB: (BN, BK, num_tile_k), gD: (BM, BN)。
2253     Tensor gA = local_tile(A, make_tile(Int<BM>{}, Int<BK>{}), make_coord(iy, _));
2254     Tensor gB = local_tile(B, make_tile(Int<BN>{}, Int<BK>{}), make_coord(ix, _));
2255     Tensor gD = local_tile(D, make_tile(Int<BM>{}, Int<BN>{}), make_coord(iy, ix));
2256
2257     // Shared memory Tensor: 按 SmemLayout 解读 smem 区域
2258     auto sA = make_tensor(make_smem_ptr(Ashm), SmemLayoutA{}); // (BM, BK, kStage)
2259     auto sB = make_tensor(make_smem_ptr(Bshm), SmemLayoutB{}); // (BN, BK, kStage)
2260
2261     // TiledMMA partition: 将 MMA 的 A/B/C tile 映射到各线程的寄存器 fragment
2262     TiledMMA tiled_mma;
2263     auto thr_mma = tiled_mma.get_slice(idx);
2264     auto tCrA = thr_mma.partition_fragment_A(gA(_, _, 0)); // (MMA, MMA_M, MMA_K)
2265     auto tCrB = thr_mma.partition_fragment_B(gB(_, _, 0)); // (MMA, MMA_N, MMA_K)
2266     auto tCrD = thr_mma.partition_fragment_C(gD);           // (MMA, MMA_M, MMA_N)
2267     clear(tCrD); // 累加器清零

```



```

2268 // G2S TiledCopy: 描述 global → shared memory 的数据搬运 (128-bit cp.async)。
2269 G2SCopyA g2s_tiled_copy_a;
2270
2271 auto g2s_thr_copy_a = g2s_tiled_copy_a.get_slice(idx);
2272 auto tAgA_copy = g2s_thr_copy_a.partition_S(gA); // (CPY, CPY_M, CPY_K, num_tile_k)
2273 auto tAsA_copy = g2s_thr_copy_a.partition_D(sA); // (CPY, CPY_M, CPY_K, kStage)
2274
2275 G2SCopyB g2s_tiled_copy_b;
2276 auto g2s_thr_copy_b = g2s_tiled_copy_b.get_slice(idx);
2277 auto tBgB_copy = g2s_thr_copy_b.partition_S(gB); // (CPY, CPY_N, CPY_K, num_tile_k)
2278 auto tBsB_copy = g2s_thr_copy_b.partition_D(sB);
2279
2280 // S2R TiledCopy: 描述 shared → register 的数据搬运 (使用 ldmatrix)
2281 // make_tiled_copy_A/B: 根据 TiledMMA 自动推导 S2R copy 的线程-数据映射
2282 auto s2r_tiled_copy_a = make_tiled_copy_A(S2RCopyAtomA{}, tiled_mma);
2283 auto s2r_thr_copy_a = s2r_tiled_copy_a.get_slice(idx);
2284 auto tAsA = s2r_thr_copy_a.partition_S(sA); // (CPY, CPY_M, CPY_K, kStage)
2285 auto tCrA_view = s2r_thr_copy_a.retile_D(tCrA); // (CPY, CPY_M, CPY_K) — 与 rA 寄存器布局对齐
2286
2287 auto s2r_tiled_copy_b = make_tiled_copy_B(S2RCopyAtomB{}, tiled_mma);
2288 auto s2r_thr_copy_b = s2r_tiled_copy_b.get_slice(idx);
2289 auto tBsB = s2r_thr_copy_b.partition_S(sB);
2290 auto tCrB_view = s2r_thr_copy_b.retile_D(tCrB);
2291
2292 // PREFETCH: 预加载前 (kStage - 1) 个 K tile, 填满循环缓冲; 每次 copy 提交一个异步 G→S 操作。
2293 int itile_to_read = 0, ismem_read = 0, ismem_write = 0;
2294 #pragma unroll
2295 for (int istage = 0; istage < kStage - 1; ++istage) {
2296     cute::copy(g2s_tiled_copy_a, tAgA_copy(_, _, _, istage),
2297               tAsA_copy(_, _, _, istage));
2298     cute::copy(g2s_tiled_copy_b, tBgB_copy(_, _, _, istage),
2299               tBsB_copy(_, _, _, istage));
2300     cp_async_fence();
2301     ++itile_to_read;
2302     ++ismem_write;
2303 }
2304 cp_async_wait<kStage - 2>();
2305 __syncthreads();
2306
2307 // K 维第一层循环 — 按 BK 大小将 K 切分为 num_k_tiles 个 tile。
2308 // 外层 k_tile 遍历这些 BK-tile, 内层 k_step 遍历 tile 内的 MMA_K slice。
2309 // 当前实现使用整除, 要求 K % BK == 0; 没有为尾部 K tile 做 predication/padding。
2310 int num_k_tiles = k / BK;
2311 // 循环前预加载首轮数据: 将第一个 K tile 的第 0 个 K slice 从 smem 加载到寄存器。
2312 cute::copy(s2r_tiled_copy_a, tAsA(_, _, 0, ismem_read), tCrA_view(_, _, 0));
2313 cute::copy(s2r_tiled_copy_b, tBsB(_, _, 0, ismem_read), tCrB_view(_, _, 0));
2314
2315 #pragma unroll 1
2316 for (int k_tile = 0; k_tile < num_k_tiles; ++k_tile) {
2317     // num_k_steps: BK tile 内 K 方向的 MMA 迭代次数, 取自 fragment 的第三 mode (K-mode)。
2318     //
2319     // 为什么只显式展开 K? MMA_M 和 MMA_N 去哪了?
2320     // tCrA=(MMA, MMA_M, MMA_K), tCrB=(MMA, MMA_N, MMA_K), tCrD=(MMA, MMA_M, MMA_N)
2321     // - K 方向: 每个 k_step 需要从 smem 加载**不同的** A/B 数据 (ldmatrix),
2322     // 所以 K 循环必须显式写, 每次迭代做 S→R copy + cute::gemm。
2323     // - M/N 方向: 同一个 k_step 内, 所有 M、N 位置的 MMA atom 共享
2324     // 同一批寄存器数据。cute::gemm 根据 tiled_mma 的类型信息 (EURepeat=2×2)
2325     // 在编译器自动展开为覆盖全部 (MMA_M, MMA_N) 对的 mma.sync 指令序列,
2326     // 无需手动写 M/N 循环。——这等价于手写 MMA 中的:
2327     //     for (i=0; i<kValTileM; ++i)
2328     //         for (j=0; j<kValTileN; ++j)
2329     //             HMMA16816(RC[i][j][0], RC[i][j][1], ...);
2330     //

```



```

2331 // CUTLASS 推导链 (mma_atom.hpp) :
2332 //   make_tiled_mma 将 MMA_P_T::<2> (即 kMmaPK) 存入 PermutationMnK
2333 //   → TiledMMA::permutation_mnk<2>() 返回 kMmaPK
2334 //   → thrfrg_A 对 A(M,K) 按 (permutation_mnk<0>(), permutation_mnk<2>())
2335 //   做 logical_divide, 即按 (kMmaPM, kMmaPK) tile 化 K 维:
2336 //       K 被分为 BK/kMmaPK 个 perm-K slice
2337 //       随后按 AtomShape_MNK<2> (MMA_K_atom) 做 zipped_divide
2338 //       每个 perm-K 含 kMmaEURepeatK 个 atom-K slice
2339 //   → partition_A 选当前线程后, K-mode size = BK / kMmaPK = 32 / 16 = 2
2340 // 本配置: kMmaPK=16 (MMA_K_atom=16 × kMmaEURepeatK=1), BK=32 → num_k_steps=2
2341 int num_k_steps = size<2>(tCrA);
2342
2343 #pragma unroll
2344 for (int k_step = 0; k_step < num_k_steps; ++k_step) {
2345     int k_step_next = (k_step + 1) % num_k_steps;
2346
2347     if (k_step == num_k_steps - 1) {
2348         // 等待下一个 K tile 的 smem 数据就绪, 然后切换到下一个 stage。
2349         cp_async_wait<kStage - 2>();
2350         __syncthreads();
2351         ismem_read = (ismem_read + 1) % kStage;
2352     }
2353
2354     // S→R: 加载下一组 A/B fragment 到寄存器。
2355     //
2356     // cute::copy 原生支持多 mode——理论上可以一次加载全部 K slice:
2357     //   cute::copy(s2r_tiled_copy_a, tAsA(_, _, _, ismem_read), tCrA_view);
2358     // 但这里刻意逐 k_step_next 加载, 目的是做**寄存器双缓冲**::
2359     //   当前 k_step 做 MMA 计算时, ldmatrix 预加载 k_step_next 的数据,
2360     //   让 S→R 延迟与 MMA 计算重叠。
2361     // 如果一次性加载全部 K slice, S→R 和 MMA 就会彻底串行。
2362     cute::copy(s2r_tiled_copy_a, tAsA(_, _, k_step_next, ismem_read),
2363               tCrA_view(_, _, k_step_next));
2364     cute::copy(s2r_tiled_copy_b, tBsB(_, _, k_step_next, ismem_read),
2365               tCrB_view(_, _, k_step_next));
2366
2367     if (k_step == 0) {
2368         // G→S: 提交下一个 K tile (整个 BK) 的异步拷贝。
2369         if (itile_to_read < num_k_tiles) {
2370             cute::copy(g2s_tiled_copy_a, tAgA_copy(_, _, _, itile_to_read),
2371                       tAsA_copy(_, _, _, ismem_write));
2372             cute::copy(g2s_tiled_copy_b, tBgB_copy(_, _, _, itile_to_read),
2373                       tBsB_copy(_, _, _, ismem_write));
2374             ++itile_to_read;
2375             ismem_write = (ismem_write + 1) % kStage;
2376         }
2377         cp_async_fence();
2378     }
2379
2380     // MMA: cute::gemm 内部根据 tiled_mma 的类型信息, 自动展开为覆盖
2381     // 全部 (MMA_M, MMA_N) 对的 mma.sync 指令序列, 累加到 tCrD。
2382     cute::gemm(tiled_mma, tCrD, tCrA(_, _, k_step), tCrB(_, _, k_step), tCrD);
2383 }
2384 }
2385
2386 // Epilogue: D 寄存器 → shared memory → global memory。
2387 // 使用当前 A stage 作为 C scratchpad, 避免为 epilogue 单独分配完整的 C shared memory。
2388 // 这里是“复用 storage”, 不是把 A Tensor 转换成 C Tensor:
2389 //   sA 的逻辑 shape 是 (BM, BK, kStage)=(128,32,2), 而 sA(_,_,ismem_read)
2390 //   只选出其中一个 A-stage 的 storage 起点; sC 随后以完全独立的 SmemLayoutC
2391 //   解释这同一段地址。当前 wrapper 的固定配置下:
2392 //       一个 A stage: 128 × 32 = 4096 个 half;
2393 //       C scratchpad: 32 × 32 × 4 = 4096 个 half。

```

```

2394 // 两者容量刚好相等, 但 logical shape 和 layout 不是同一个: A 的 layout atom
2395 // 是 8x32, C 的 layout atom 是 32x32, 二者都含 Swizzle<3,3,3>, 却不能把
2396 // C 数据再按 SmemLayoutA 读取。launch wrapper 的 static_assert 用当前配置
2397 // 检查 C scratchpad 能放进一个 A pipe; 此处只别名该单个 stage, 不是整块双缓冲 sA。
2398 //
2399 // mainloop 已结束, 不会再通过 sA 的 A/B load view 消费这个 pipe 的旧 A 数据。
2400 // 从这一行开始, 对这块地址的所有访问都通过 SmemLayoutC 派生的 sC 完成: R2S
2401 // 用它写 C, S2G 用它读 C。因此 C 的 swizzle/address mapping 在写和读两侧一致。
2402 auto sC = make_tensor(sA(_, _, ismem_read).data(), SmemLayoutC{});
2403
2404 // R2S TiledCopy: 寄存器 → shared memory (使用 UniversalCopy 做类型转换)
2405 // R2SCopyAtomC = Copy_Atom<UniversalCopy<int>, T>; 以 32-bit 粒度搬运 T 数据
2406 auto r2s_tiled_copy_c = make_tiled_copy_C(R2SCopyAtomC{}, tiled_mma);
2407 auto r2s_thr_copy_c = r2s_tiled_copy_c.get_slice(idx);
2408 // tCrD 是 TiledMMA 产生的 accumulator fragment, 逻辑 shape 为
2409 // (MMA, MMA_M, MMA_N)。它的寄存器元素已经就位, 但该 layout 是为 MMA
2410 // accumulator 服务的, 并不直接按 R2S copy atom 的 source-value 顺序表达。
2411 //
2412 // retile_S 中的 S 指“这个 TiledCopy 的 source side”, 不是 shared memory。
2413 // 它基于 r2s_tiled_copy_c 的 reference layout 创建一个共享 tCrD.data() 的
2414 // zero-copy layout view, 将逻辑坐标表示为 (CPY, CPY_M, CPY_N)。这一步不读取
2415 // 或写入寄存器/共享内存, 不进行 dtype conversion, 也不交换 lane 间数据; 真正的
2416 // R2S 数据搬运发生在下面的 cute::copy(r2s_tiled_copy_c, ..., tCsC_r2s(...))。
2417 // 可将它与主循环的 s2r_thr_copy_a.retire_D(tCrA) 对照: 二者都是为了让已有
2418 // fragment 的 per-thread value ordering 匹配某个 TiledCopy 的 source/destination
2419 // 角色, 而不是另一次 memory transfer。
2420 // tCrD: (MMA, MMA_M, MMA_N) -> retire -> tCrC_r2s: (CPY, CPY_M, CPY_N)
2421 auto tCrC_r2s = r2s_thr_copy_c.retire_S(tCrD); // (CPY, CPY_M, CPY_N)
2422 // get_slice(idx) 已固定 CTA 内当前 logical copy thread; partition_D 再按
2423 // R2S TiledCopy 的 destination mapping 切分 sC。推导链:
2424 // MMA_P_T = Tile<kMmaPM, kMmaPN, kMmaPK> = Tile<32, 32, 16>
2425 // → TiledMMA::tile_size<0>(mma) = 32, tile_size<1>(mma) = 32
2426 // → make_tiled_copy_C 的 tiler = (32, 32)
2427 // → SmemLayoutC 的逻辑 M×N = (kMmaPM, kMmaPN) = (32, 32) 恰好等于该 tiler
2428 // → partition_D: zipped_divide(sC, tiler=(32,32)) 后 M/N 维无 remainder
2429 // → 结果 shape 的 M/N remainder = _1, _1
2430 // 最后一维 pipe=4 来自 SmemLayoutC 的第三维 kSmemLayoutCBatch, 而不是
2431 // “thread-level tensor 自动只剩 CPY”这一条规则。_1 表示该逻辑 mode 的 extent
2432 // 是 1, 并非该 mode 被删除或 C tile 没有 M/N 覆盖。
2433 auto tCsC_r2s = r2s_thr_copy_c.partition_D(sC); // (CPY, _1, _1, pipe)
2434
2435 // S2G TiledCopy: shared memory → global memory (128-bit store)
2436 // S2GCopyAtomC(Copy_Atom<UniversalCopy<cute::uint128_t>, T>) -> S2GCopyC
2437 S2GCopyC s2g_tiled_copy_c;
2438 auto s2g_thr_copy_c = s2g_tiled_copy_c.get_slice(idx);
2439 // tCsC_s2g 与 tCsC_r2s 都从同一个 sC 产生, 因而保留相同的 pipe=4; 前者是
2440 // S2G source mapping, 后者是 R2S destination mapping。tCgC_s2g 则面向完整
2441 // gD=(BM,BN)=(128,256), 相对于 S2G copy tiler 仍有非平凡的 M/N repetition,
2442 // 所以它保留 CPY_M、CPY_N。这里的 CPY/CPY_M/CPY_N 都是编译期 copy-partition
2443 // 的逻辑 mode, 不应直接理解为固定的 lane 数、warp 数或物理连续维度。
2444 auto tCsC_s2g = s2g_thr_copy_c.partition_S(sC); // (CPY, _1, _1, pipe)
2445 auto tCgC_s2g = s2g_thr_copy_c.partition_D(gD); // (CPY, CPY_M, CPY_N)
2446
2447 // group_modes<B,E> 将 layout 的半开区间 [B,E) 组合成一个嵌套 mode:
2448 // group_modes<1,3>(Tensor(CPY, CPY_M, CPY_N))
2449 // → Tensor(CPY, (CPY_M, CPY_N))
2450 // 它只重组 Tensor 的 layout, 不分配内存、不移动 data(), 也不改变元素访问的物理地址。
2451 // 第二个 mode 仍然保存原来 CPY_M/CPY_N 的层次关系, 只是现在可以用一个坐标
2452 // i+j 访问这个组合 mode; 因此这里的 group_modes 更准确地说“合并 mode”,
2453 // 而不是把底层数据拷贝或无条件 flatten 成普通一维数组。
2454 // 从 type/shape 结构看, 结果确实是 Tensor(CPY, (CPY_M, CPY_N)); 但 grouped
2455 // mode 的 cardinality 是两个子 mode 的乘积, 因此 size<1>(…) 可作为一个标量
2456 // 范围遍历其全部 logical coordinate。于是 (_, i+j) 中的 i+j 是该 nested mode

```

```

2457 // 的线性 logical coordinate, 不是在 C++ 中对二维 tuple 做加法, 更不是 raw
2458 // pointer offset; 最终物理地址仍由各自 Tensor 的 grouped layout 计算。
2459 //
2460 // 这里为什么要对 tCrC_r2s 和 tCgC_s2g 同时 group?
2461 //   - tCrC_r2s: 每个线程持有的寄存器 C fragment, 原始形状为 (CPY, CPY_M, CPY_N);
2462 //   - tCgC_s2g: 该线程对应的 global-memory C 输出位置, 形状与源 Tensor 对齐;
2463 //   - 两者使用相同的 [1,3] 合并后, 源/目的 Tensor 仍保持相同的坐标空间,
2464 //     cute::copy 可以按相同的 (CPY, i+j) 坐标完成寄存器到 global 的对应搬运。
2465 //
2466 // tCsC_r2s/tCsC_s2g 没有 group 第 3 个 pipe mode, 因为它们还需要显式保留
2467 // shared-memory C scratchpad 的 pipeline 维度; step=size<3>(tCsC_r2s) 表示
2468 // 每一轮可以复用的 C shared-memory pipeline 深度。外层 i 在合并后的 C 元素空间中
2469 // 按 step 前进, 内层 j 选择当前 pipeline slot: 寄存器先写入 sC(_,0,0,j),
2470 // 再从同一个 slot 写回 global memory。
2471 // 当前 kSmemLayoutCBatch=4, 因此 step=4。代码未对 i+j 做尾部 predication,
2472 // 这个循环依赖当前静态 layout 中 grouped extent 能被 pipe depth 整除; 不应将
2473 // “每轮正好处理 4 个 fragment” 误认为任意 tile/copy 配置都自动成立的通用规则。
2474 auto tCgC_s2gx = group_modes<1, 3>(tCgC_s2g);
2475 auto tCrC_r2sx = group_modes<1, 3>(tCrC_r2s);
2476
2477 int step = size<3>(tCsC_r2s); // C scratchpad 的 pipe 数 (由 kSmemLayoutCBatch=4 指定)
2478 // 双层循环的语义 (注意 step 与 CPY_N 无关):
2479 //   group_modes<1,3> 之后, tCrC_r2sx 的 shape 为 (CPY, (CPY_M, CPY_N))。
2480 //   size<1>(tCrC_r2sx) = CPY_M * CPY_N, 即该线程需处理的 C fragment 总数。
2481 //   外循环以 step=4 为步长, 内循环 j=0..3 每次处理 1 个 fragment, 将其写入
2482 //   第 j 号 C scratchpad pipe slot, 再读出写回 global。
2483 //
2484 //   这里 step=4 是 scratchpad pipeline 深度, 不是 CPY_N。CPY_N 的值取决于
2485 //   TiledMMA 的 C-layout 如何将 (128,256) 的 gD tile 分配到 128 个线程上,
2486 //   通常 ≠ 4。循环之所以成立, 是因为 grouped mode 用线性坐标 i+j 遍历整个
2487 //   CPY_M * CPY_N 的扁平空间——它不再区分 M 和 N 方向, 也不要求 CPY_N == step。
2488 //   唯一前提是 CPY_M * CPY_N 能被 step 整除 (当前静态 layout 编译期保证)。
2489 //
2490 //   第 j 号 pipe slot 在 sC 上的物理地址由 R2S partition_D / S2G partition_S
2491 //   各自推导; 因为二者都从同一个 sC (SmemLayoutC) 出发, pipe mode 保持一致,
2492 //   写入和读取命中同一段 shared memory。
2493 #pragma unroll
2494 for (int i = 0; i < size<1>(tCrC_r2sx); i += step) {
2495     // 每轮处理 step 个 fragment, 内层 j 选 pipe slot。
2496 #pragma unroll
2497     for (int j = 0; j < step; ++j) {
2498         // 这两行是 R2R staging: make_tensor_like<T> 分配当前线程私有、拥有 storage
2499         // 的 register Tensor; 普通 cute::copy 将 accumulator fragment materialize
2500         // 成输出元素类型 T 的临时 payload。
2501         //
2502         // cute::copy (无 copy-atom 的普通版) 内部按元素做
2503         //   dst(i) = static_cast<T>(static_cast<SrcType>(src(i)))
2504         // 因此当 accumulator 与 T 类型不同时, 它自动完成 dtype 转换; 当二者相同
2505         // (如本 kernel 的 f16 accumulator + T=half), cast 退化为 no-op。
2506         //
2507         // 即使类型一致, t 还有一个不可省略的作用: layout 归一化。
2508         // tCrC_r2sx(_, i+j) 的 layout 是 retile_S → group_modes → slice 层层
2509         // 叠加的 composed layout, 而 make_tensor_like<T> + cute::copy 把它
2510         // 物化到一个拥有简单 strides 的 owning register Tensor 中。后续
2511         // r2s_tiled_copy_c 的 copy_unpack 需要按 copy-atom 的 val-layout
2512         // 拆分 source 元素, 简单 owning layout 比复杂的 composed layout 更容易
2513         // 被编译器推导内联。上游同源实现将其概括为"cope with accumulator and
2514         // output data type difference", 实际承担了类型适配和 layout 归一化两个职责。
2515         //
2516         // 这不是 retile_S 的一部分, 也不是 warp shuffle: 源/目的都在当前线程的
2517         // register 中, 代码没有显式 __shfl_sync。不能仅根据这段 C++ 断言最终 SASS
2518         // 绝不会含 shuffle; 但语义上的跨线程 regrouping 不由此 R2R copy 承担, 而是
2519         // 由随后的 R2S -> shared memory -> S2G 路径完成。若特定 accumulator/output

```

```

2520 // type 与 R2S copy-atom contract 已直接兼容, 可以设计直接 R2S 的变体; 本例
2521 // 保持同源实现的通用 staging 写法。
2522 auto t = make_tensor_like<T>(tCrC_r2sx(_, i + j));
2523 cute::copy(tCrC_r2sx(_, i + j), t);
2524 // R2S 的 copy atom 才在此处将 thread-local payload 写入 j 号 C scratchpad
2525 // slot; SmemLayoutC 定义写入后的跨线程地址重组。
2526 //
2527 // cute::copy(r2s_tiled_copy_c, src, dst) 的调用链路:
2528 //   r2s_tiled_copy_c 是 TiledCopy, 但继承自 Copy_Atom<UniversalCopy<int>, T>
2529 //   → 匹配 copy(Copy_Atom<...> const&, src, dst) 重载 [copy.hpp:~L190]
2530 //   → src(t) 与 dst(tCsC_r2s(_, 0, 0, j)) 都是 rank-1 → 直接 copy_atom.call
2531 //   → copy_atom.call 内部: 若 size(src)==NumValSrc (atom 编译期 val-count),
2532 //     走 copy_unpack; 否则递归剥 mode, 最终都落到 UniversalCopy<int>::copy
2533 //     → 硬件层面就是逐 uint32_t 的 register-to-smem store [arch/copy.hpp:~L46]
2534 //
2535 // 这里没有任何运行时"查找表": t 的第 i 个元素之所以对应 dst 的第 i 个
2536 // shared memory offset, 是因为 pre-partition 阶段已经把映射烧进 layout 了:
2537 //   - t 的 layout 来自 retile_S → group_modes → slice → make_tensor_like
2538 //     → 元素顺序已按 R2S atom 的 source-side value ordering 排好
2539 //   - tCsC_r2s 的 layout 来自 r2s_thr_copy_c.partition_D(sC)
2540 //     → TiledCopy::tidfrg_D 用 LayoutCopy_TV + Tiler_MN + ValLayoutDst
2541 //     计算了当前线程每个 val 在 smem 中的物理 offset
2542 //   两者共享同一个 copy atom 的 ValLayoutSrc/ValLayoutDst 定义,
2543 //   因此逐元素 copy 天然把正确的数据写到了正确的地址。
2544 cute::copy(r2s_tiled_copy_c, t, tCsC_r2s(_, 0, 0, j));
2545 }
2546 // 这是 CTA 范围的 rendezvous: 所有线程完成当前批 R2S 写入后, S2G 才能从
2547 // sC 的重组布局读取完整数据。它承担了手写版中显式跨 lane 拼接所需的协作边界。
2548 __syncthreads();
2549
2550 // S→G: 从同一个 pipe slot 读回 global memory, 保持源/目的坐标一一对应。
2551 #pragma unroll
2552 for (int j = 0; j < step; ++j) {
2553   // S2GCopyC 的 atom 为 UniversalCopy<uint128_t>: 与 R2S 的 UniversalCopy<int>
2554   // (32-bit) 不同, 它一次搬运 128-bit (8 个 half), 映射为一条 wide store
2555   // (如 st.global.v4.f32 或 st.global.b128)。
2556   //
2557   // cute::copy(s2g_tiled_copy_c, src, dst) 的调用链路:
2558   //   s2g_tiled_copy_c 是 TiledCopy, 继承自 Copy_Atom<UniversalCopy<uint128_t>, T>
2559   //   → 匹配 copy(Copy_Atom<...> const&, src, dst) 重载 [copy.hpp:~L190]
2560   //   → src=tCsC_s2g(_, 0, 0, j) 与 dst=tCgC_s2gx(_, i+j) 都是 rank-1
2561   //   → 直接 copy_atom.call(src, dst)
2562   //   → copy_atom.call 内部: size(src)==NumValSrc → copy_unpack
2563   //     → 逐 128-bit chunk 调用 UniversalCopy<uint128_t>::copy
2564   //     → 硬件层面就是 shared-memory-to-global wide store [arch/copy.hpp:~L46]
2565   //
2566   // S2G 的 pre-partition 与 R2S 对称, 但 source/destination 角色互换:
2567   //   - tCsC_s2g 来自 s2g_thr_copy_c.partition_S(sC): TiledCopy::tidfrg_S
2568   //     用 LayoutCopy_TV + Tiler_MN + ValLayoutSrc 计算了当前线程每个 val
2569   //     在 sC (shared memory) 中的物理 offset。
2570   //   S2GCopyC 的 tiler = product(ThrLayout{32,4}, ValLayout{1,8})
2571   //     = (32×1, 4×8) = (32, 32) — 恰好与 R2S tiler 相同,
2572   //   因此 tCsC_s2g 也是 (CPY, _1, _1, pipe), _1,_1 来自 sC=(32,32,4) 无
2573   //   M/N remainder。
2574   //   - tCgC_s2gx(_, i+j) 来自 s2g_thr_copy_c.partition_D(gD) → group_modes
2575   //     → slice。gD=(128,256) 相对于 tiler (32,32) 有 4×8 个 tile repetition,
2576   //     因此 tCgC_s2g 为 (CPY, CPY_M, CPY_N), group_modes<1,3> 合并 M/N
2577   //     repetition 后得到 (CPY, (CPY_M, CPY_N)), 再用线性坐标 i+j 切片。
2578   //   两者共享同一个 copy atom 的 ValLayoutSrc/ValLayoutDst 定义,
2579   //   因此逐 chunk copy 天然从正确的 smem 地址读到正确的 gmem 地址。
2580   cute::copy(s2g_tiled_copy_c, tCsC_s2g(_, 0, 0, j), tCgC_s2gx(_, i + j));
2581 }
2582 // 所有线程都完成当前 pipe slots 的 S2G 读取后, 下一轮 R2S 才能覆盖这 4 个

```

```

2583 // scratchpad slots, 避免一部分线程仍在读取旧数据时被其他线程提前重写。
2584 __syncthreads();
2585 }
2586
2587 // 与 hgemm_mma_stages_tn 的手写 epilogue 对照: 手写版必须显式处理 MMA fragment
2588 // 的物理分布, 使用 RC0/RC1 暂存, 再用 __shfl_sync 从同一 warp 的相邻 lane 收齐
2589 // 8 个 half, 并由 lane_id % 4 == 0 的线程做 float4 (128-bit) global store。
2590 // CuTe 版没有把这些 lane 映射写死在 kernel 源码中: retile_S 表达 accumulator
2591 // fragment 与 R2S copy 的对应, R2S/sC/S2G 通过 shared-memory scratchpad 完成
2592 // CTA 范围的数据重组, S2GCopyC 以 uint128_t 表达 wide store。两者的共同目标都是
2593 // 将分散在计算线程寄存器中的 C fragment 变为适合连续 global-memory 写回的布局;
2594 // 区别是手写版直接编排 shuffle/store, CuTe 版将映射交给 TiledMMA/TiledCopy 类型推导。
2595 }
2596
2597 // =====
2598 // Phase 7c-2: CuTe HGEMM Launch Wrapper — 类型实例化 + Grid/Block 配置
2599 // =====
2600 // CuTe 的核心设计理念: "类型即配置" (Type-level configuration)
2601 // 所有的 MMA atom、TiledCopy、SmemLayout、Swizzle 都是 **编译期类型**,
2602 // kernel 接收这些类型的实例 (通常为 struct), 在编译期完成全部映射推导。
2603 //
2604 // 以下每个类型定义后面都标注了它在 kernel body 中的对应变量/用法, 形成 1:1 对照。
2605 //
2606 // 面试常问: 「CuTe 为什么比手写 PTX 简洁?」
2607 // 答: 手写需要:
2608 //   1) 手动计算每线程的 smem 地址偏移
2609 //   2) 手动计算 ldmatrix 的 lane 映射
2610 //   3) 手动插入 swizzle XOR 到每个地址计算点
2611 //   4) 手动做 epilogue 的 warp shuffle 编排
2612 // CuTe 的 partition + copy + gemm 通过 TiledCopy/TiledMMA 的类型信息
2613 // 在编译期自动完成上述全部推导, 生成与手写等价的 PTX 指令序列。
2614 // =====
2615 template <typename T, const int Stages = 2, const int BlockSwizzle = 0>
2616 void launch_hgemm_mma_stages_tn_cute(T *a, T *b, T *c, int M, int N, int K) {
2617     using namespace cute;
2618
2619     // — Tile 尺寸 (kernel 模板参数 BM/BN/BK/kStage 的值) —
2620     //
2621     // kernel 用法对照:
2622     //   BM=128:   local_tile(A, make_tile(Int<BM>{}, Int<BK>{}), ...) → gA=(BM,BK,num_k_tiles)
2623     //             local_tile(D, make_tile(Int<BM>{}, Int<BN>{}), ...) → gD=(BM,BN)
2624     //   BN=256:   local_tile(B, make_tile(Int<BN>{}, Int<BK>{}), ...) → gB=(BN,BK,num_k_tiles)
2625     //             local_tile(D, ...) → gD 的列方向
2626     //   BK=32:    num_k_tiles = k / BK; 每个 BK tile 内 num_k_steps = BK/kMmaPK = 2 个 MMA_K slice
2627     //   kStage=2: sA/sB 的第三维 = (BM,BK,kStage); cp_async_wait<kStage-2>() 流水线同步
2628     //   kSmemLayoutCBatch=4: step = size<3>(tCsC_r2s) → C scratchpad pipe 深度 = 4
2629     auto BM = Int<128>{};
2630     auto BN = Int<256>{};
2631     auto BK = Int<32>{};
2632     auto KStage = Int<Stages>{};
2633     auto kSmemLayoutCBatch = Int<4>{};
2634
2635     // — SmemLayoutA / SmemLayoutB —
2636     // kernel 用法:
2637     // auto sA = make_tensor(make_smem_ptr(Ashm), SmemLayoutA{}); // (BM, BK, kStage)
2638     // auto sB = make_tensor(make_smem_ptr(Bshm), SmemLayoutB{}); // (BN, BK, kStage)
2639     // SmemLayout 由三部分组成: Swizzle<3,3,3> + atom layout(8×BK) + tile_to_shape 扩展到完整 tile+stage。
2640     //
2641     // 对当前 base layout shape=(8,32), stride=(32,1), T=half 的例子:
2642     //   - M=3: 连续 2^3=8 个 half 组成一个基本元素, 即 16 bytes;
2643     //   - S=3: 连续 2^3=8 个基本元素组成一行, 即 128 bytes;
2644     //   - B=3: 共有 2^3=8 行。
2645     // 于是一个 8×32 的逻辑 tile 正好对应 8 行 × 128B, 覆盖全部 32 个 shared-memory bank。

```



```

2646 // Swizzle 对每个基本元素的列坐标执行  $icol' = irow \wedge icol$ ,
2647 // 再将 (irow, icol') 和基本元素内部的 3 个低位编码回物理 offset。
2648 // 其位级形式为  $offset' = offset \wedge ((offset \& (0b111 \ll 6)) \gg 3)$ :
2649 // 地址位 [6:8] XOR 到 [3:5], 地址位 [0:2] 不参与置换。
2650 // composition(Swizzle, base_layout) 的语义是  $R(c) = Swizzle(base\_layout(c))$ :
2651 // 逻辑坐标仍由 base_layout 给出, 只有最终的 shared-memory offset 被重排。
2652 // tile_to_shape: 通过 blocked product 重复这个带 swizzle 的 block layout,
2653 // 使结果 shape 匹配  $BM \times BK \times kStage$ ; 默认按目标 shape 的 mode order 重复各维。
2654 using SmemLayoutAtom = decltype(composition(
2655     Swizzle<3, 3, 3>{},
2656     make_layout(make_shape(Int<8>{}, Int<BK>{}),
2657         make_stride(Int<BK>{}, Int<1>{}))));
2658 using SmemLayoutA = decltype(tile_to_shape(
2659     SmemLayoutAtom{}, make_shape(Int<BM>{}, Int<BK>{}, Int<kStage>{})));
2660 using SmemLayoutB = decltype(tile_to_shape(
2661     SmemLayoutAtom{}, make_shape(Int<BN>{}, Int<BK>{}, Int<kStage>{})));
2662
2663 // — MMA (TiledMMA) —
2664 // kernel 用法:
2665 // TiledMMA tiled_mma;
2666 // auto thr_mma = tiled_mma.get_slice(threadIdx.x); // 每线程的 MMA slice
2667 // auto tCrA = thr_mma.partition_fragment_A(gA(_,_,0)); // (MMA, MMA_M, MMA_K)
2668 // auto tCrB = thr_mma.partition_fragment_B(gB(_,_,0)); // (MMA, MMA_N, MMA_K)
2669 // auto tCrD = thr_mma.partition_fragment_C(gD); // (MMA, MMA_M, MMA_N)
2670 // cute::gemm(tiled_mma, tCrD, tCrA(_,_,k_step), tCrB(_,_,k_step), tCrD);
2671 // MMA 还决定 S2R/R2S copy 的 tiler: make_tiled_copy_A/B/C 都依赖 tiled_mma 的
2672 // tile_size<0/1>(mma) = (32,32) 来推导线程→数据映射。
2673 //
2674 // 推导链: SM80_16x8x16_F16F16F16F16_TN → MMA_Atom
2675 // → make_tiled_mma(atom, EURepeat{2,2,1}, ValTile{32,32,16})
2676 // → TiledMMA: 128 threads = 4 warps × (2×2 EU slices), 逻辑 MMA tile = 32×32×16
2677 // TN = A row-major, B col-major; 因此传给本 kernel 的 B 指针实际指向
2678 //  $B^T[N,K]$  的 row-major 存储 (等价于 GEMM 语义中的  $B[K,N]$  col-major)。
2679 using mma_op = SM80_16x8x16_F16F16F16F16_TN;
2680 using mma_traits = MMA_Traits<mma_op>;
2681 using mma_atom = MMA_Atom<mma_traits>;
2682 using mma_atom_shape = mma_traits::Shape_MNK; // (Int<16>, Int<8>, Int<16>)
2683
2684 static constexpr int kMmaEURepeatM = 2;
2685 static constexpr int kMmaEURepeatN = 2;
2686 static constexpr int kMmaEURepeatK = 1;
2687 static constexpr int kMmaPM = 1 * kMmaEURepeatM * get<0>(mma_atom_shape{}); // 32
2688 static constexpr int kMmaPN = 2 * kMmaEURepeatN * get<1>(mma_atom_shape{}); // 32
2689 static constexpr int kMmaPK = 1 * kMmaEURepeatK * get<2>(mma_atom_shape{}); // 16
2690 // kMmaPK=16 → kernel 中 num_k_steps = size<2>(tCrA) =  $BK/kMmaPK = 32/16 = 2$ 
2691
2692 using MMA_EU_RepeatT = decltype(make_layout(make_shape(
2693     Int<kMmaEURepeatM>{}, Int<kMmaEURepeatN>{}, Int<kMmaEURepeatK>{})));
2694 using MMA_P_T = Tile<Int<kMmaPM>, Int<kMmaPN>, Int<kMmaPK>>;
2695 using MMA = decltype(make_tiled_mma(mma_atom{}, MMA_EU_RepeatT{}, MMA_P_T{}));
2696
2697 // — G2SCopyA / G2SCopyB —
2698 // kernel 用法:
2699 // G2SCopyA g2s_tiled_copy_a; G2SCopyB g2s_tiled_copy_b;
2700 // cute::copy(g2s_tiled_copy_a, tAgA_copy(_,_,_,istage), tAsA_copy(_,_,_,istage));
2701 // cute::copy(g2s_tiled_copy_b, tBgB_copy(_,_,_,istage), tBsB_copy(_,_,_,istage));
2702 // G→S: 128-bit cp.async. ThrLayout{32,4} × ValLayout{1,8}:
2703 // 128 个线程, 每线程搬运  $1 \times 8 = 8$  个 half = 128 bits。
2704 using g2s_copy_op = SM80_CP_ASYNC_CACHEGLOBAL<cute::uint128_t>;
2705 using g2s_copy_traits = Copy_Traits<g2s_copy_op>;
2706 using g2s_copy_atom = Copy_Atom<g2s_copy_traits, T>;
2707 using G2SCopyA = decltype(make_tiled_copy(
2708     g2s_copy_atom{},

```

```

2709     make_layout(make_shape(Int<32>{}, Int<4>{}),
2710                 make_stride(Int<4>{}, Int<1>{})),
2711     make_layout(make_shape(Int<1>{}, Int<8>{}))));
2712 using G2SCopyB = G2SCopyA;
2713
2714 // — S2RCopyAtomA / S2RCopyAtomB —
2715 // kernel 用法:
2716 // auto s2r_tiled_copy_a = make_tiled_copy_A(S2RCopyAtomA{}, tiled_mma);
2717 // auto s2r_tiled_copy_b = make_tiled_copy_B(S2RCopyAtomB{}, tiled_mma);
2718 // cute::copy(s2r_tiled_copy_a, tAsA(_,_,k_step_next,ismem_read), tCrA_view(_,_,k_step_next));
2719 // cute::copy(s2r_tiled_copy_b, tBsB(_,_,k_step_next,ismem_read), tCrB_view(_,_,k_step_next));
2720 // S→R: ldmatrix (SM75_U32x4_LDSM_N)。make_tiled_copy_A/B 根据 TiledMMA 自动推导
2721 // 与 MMA fragment 对齐的 shared→register 映射, 无需手动指定 ThrLayout/ValLayout。
2722 using s2r_copy_op = SM75_U32x4_LDSM_N;
2723 using s2r_copy_traits = Copy_Traits<s2r_copy_op>;
2724 using s2r_copy_atom = Copy_Atom<s2r_copy_traits, T>;
2725 using S2RCopyAtomA = s2r_copy_atom;
2726 using S2RCopyAtomB = s2r_copy_atom;
2727
2728 // — SmemLayoutC —
2729 // kernel 用法:
2730 // auto sC = make_tensor(sA(_,_,ismem_read).data(), SmemLayoutC{});
2731 // 复用当前 A stage 的空间作为 C scratchpad
2732 // sC shape = (kMmaPM, kMmaPN, kSmemLayoutCBatch) = (32, 32, 4)
2733 // pipe 数 4 由 kSmemLayoutCBatch 指定 → step = size<3>(tCsC_r2s) = 4
2734 // 与 A/B 相同的 Swizzle<3,3,3> 规则, 但 base layout 是 32×32 (MMA tile 大小),
2735 // 再扩展为 4 个 epilogue scratchpad pipe slots。这 4 个 slots 不等同于主循环
2736 // 的 kStage K-tile pipeline。
2737 using SmemLayoutAtomC = decltype(composition(
2738     Swizzle<3, 3, 3>{},
2739     make_layout(make_shape(Int<kMmaPM>{}, Int<kMmaPN>{}),
2740                 make_stride(Int<kMmaPN>{}, Int<1>{}))));
2741 using SmemLayoutC = decltype(tile_to_shape(
2742     SmemLayoutAtomC{},
2743     make_shape(Int<kMmaPM>{}, Int<kMmaPN>{}, Int<kSmemLayoutCBatch>{})));
2744
2745 // — R2SCopyAtomC —
2746 // kernel 用法:
2747 // auto r2s_tiled_copy_c = make_tiled_copy_C(R2SCopyAtomC{}, tiled_mma);
2748 // cute::copy(r2s_tiled_copy_c, t, tCsC_r2s(_, 0, 0, j));
2749 // R→S: UniversalCopy<int> 以 32-bit 粒度将寄存器 payload 写入 C scratchpad。
2750 // make_tiled_copy_C 从 TiledMMA 的 tile_size<0/1>(mma)=(32,32) 推导 tiler,
2751 // 因此 R2S copy 的 tiler = (32,32), 与 SmemLayoutC 的 (32,32) 恰好匹配。
2752 using R2SCopyAtomC = Copy_Atom<UniversalCopy<int>, T>;
2753
2754 // — S2GCopyC —
2755 // kernel 用法:
2756 // S2GCopyC s2g_tiled_copy_c;
2757 // cute::copy(s2g_tiled_copy_c, tCsC_s2g(_, 0, 0, j), tCgC_s2gx(_, i+j));
2758 // S→G: UniversalCopy<uint128_t> 以 128-bit 粒度做 shared→global wide store。
2759 // ThrLayout{32,4} × ValLayout{1,8} → tiler = product((32,4),(1,8)) = (32,32),
2760 // 与 R2S tiler 相同, 因此 partition_S(sC) 得到的 pipe 维度一致。
2761 using S2GCopyAtomC = Copy_Atom<UniversalCopy<uint128_t>, T>;
2762 using S2GCopyC = decltype(make_tiled_copy(
2763     S2GCopyAtomC{},
2764     make_layout(make_shape(Int<32>{}, Int<4>{}),
2765                 make_stride(Int<4>{}, Int<1>{})),
2766     make_layout(make_shape(Int<1>{}, Int<8>{}))));
2767
2768 // — Grid/Block 配置 —
2769 // kernel 用法:
2770 // int idx = threadIdx.x;          // 0..127
2771 // int ix = ((int)BlockSwizzle) * blockIdx.z * gridDim.x + blockIdx.x;

```



```

2772 // int iy = blockIdx.y;          // M 方向 CTA 坐标
2773 // if (iy * BM >= m || ix * BN >= n) return; // CTA 级边界保护
2774 // 由于 kernel 没有 tile 内的逐元素 predication, 调用方需保证
2775 // M % BM == 0, N % BN == 0, K % BK == 0。
2776 // 当不启用 BlockSwizzle 时, BZ=1 退化为纯 2D grid, 行为不变。
2777 constexpr int kSwizzleStride = 2048;
2778 int BX = (N + BN - 1) / BN;
2779 int BY = (M + BM - 1) / BM;
2780 int BZ = BlockSwizzle ? (N + kSwizzleStride - 1) / kSwizzleStride : 1;
2781 BX = BlockSwizzle ? (BX + BZ - 1) / BZ : BX;
2782 dim3 block(size(MMA{})); // 128 threads (= size(TiledMMA))
2783 dim3 grid(BX, BY, BZ);
2784
2785 // — Dynamic Shared Memory 大小 —
2786 // kernel 用法:
2787 // extern __shared__ T shm_data[];
2788 // T *Ashm = shm_data;
2789 // T *Bshm = shm_data + cute::cosize(SmemLayoutA{});
2790 // cosize(SmemLayoutA) = BM*BK*kStage = 128*32*2 = 8192 个 T
2791 // cosize(SmemLayoutB) = BN*BK*kStage = 256*32*2 = 16384 个 T
2792 // 合计 A+B = 24576 个 T; C epilogue 复用 A stage (128*32 = 4096 个 T)。
2793 // static_assert 保证 C scratchpad (kMmaPM*kMmaPN*kSmemLayoutCBatch = 32*32*4 = 4096)
2794 // 能放进一个 A stage (128*32 = 4096)。
2795 static constexpr int shm_size_AB =
2796     cute::cosize(SmemLayoutA{}) + cute::cosize(SmemLayoutB{});
2797 static constexpr int shm_size_C = cute::cosize(SmemLayoutC{});
2798 static_assert(size<0>(SmemLayoutA{}) * size<1>(SmemLayoutA{}) >= size(SmemLayoutC{}),
2799     "C shared memory must fit within one A pipe");
2800 static constexpr int kShmSize =
2801     cute::max(shm_size_AB, shm_size_C) * sizeof(T);
2802
2803 cudaFuncSetAttribute(
2804     hgemm_mma_stages_tn_cute<T, BM, BN, BK, KStage, MMA, G2SCopyA, G2SCopyB,
2805         SmemLayoutA, SmemLayoutB, SmemLayoutC,
2806         S2RCopyAtomA, S2RCopyAtomB, R2SCopyAtomC,
2807         S2GCopyAtomC, S2GCopyC, BlockSwizzle>,
2808     cudaFuncAttributeMaxDynamicSharedMemorySize, kShmSize);
2809
2810 hgemm_mma_stages_tn_cute<T, BM, BN, BK, KStage, MMA, G2SCopyA, G2SCopyB,
2811     SmemLayoutA, SmemLayoutB, SmemLayoutC, S2RCopyAtomA,
2812     S2RCopyAtomB, R2SCopyAtomC, S2GCopyAtomC, S2GCopyC,
2813     BlockSwizzle>
2814     <<<grid, block, kShmSize>>>(a, b, c, M, N, K);
2815 }
2816
2817 #endif /* NOTES_V2_ENABLE_CUTE */
2818
2819 #if defined(NOTES_V2_ENABLE_WGMMMA) || defined(NOTES_V2_ENABLE_TMA_MMA_WS)
2820 // CUDA 13.2 promotes these TMA operations to cuda::ptx. Keep the older
2821 // experimental wrappers so the Hopper path remains buildable on prior toolkits.
2822 __device__ __forceinline__ void tma_fence_proxy_async_shared_cta() {
2823     #if CUDART_VERSION >= 13020
2824         cuda::ptx::fence_proxy_async(cuda::ptx::space_shared);
2825     #else
2826         cuda::device::experimental::fence_proxy_async_shared_cta();
2827     #endif
2828 }
2829
2830 __device__ __forceinline__ void tma_load_2d(
2831     void *dst, const CUTensorMap *tensor_map, int minor_coord, int major_coord,
2832     cuda::barrier<cuda::thread_scope_block> &barrier) {
2833     #if CUDART_VERSION >= 13020
2834         const int32_t coords[]{minor_coord, major_coord};

```

```

2835     auto *barrier_handle = cuda::device::barrier_native_handle(barrier);
2836     cuda::ptx::cp_async_bulk_tensor(cuda::ptx::space_cluster,
2837                                     cuda::ptx::space_global, dst, tensor_map,
2838                                     coords, barrier_handle);
2839 #else
2840     cuda::device::experimental::cp_async_bulk_tensor_2d_global_to_shared(
2841         dst, tensor_map, minor_coord, major_coord, barrier);
2842 #endif
2843 }
2844
2845 __device__ __forceinline__ void tma_arrive_expect_tx(
2846     cuda::barrier<cuda::thread_scope_block> &barrier, uint32_t bytes) {
2847 #if CUDART_VERSION >= 13020
2848     auto *barrier_handle = cuda::device::barrier_native_handle(barrier);
2849     [[maybe_unused]] auto token = cuda::ptx::mbarrier_arrive_expect_tx(
2850         cuda::ptx::sem_release, cuda::ptx::scope_cta, cuda::ptx::space_shared,
2851         barrier_handle, bytes);
2852 #else
2853     [[maybe_unused]] auto token =
2854         cuda::device::barrier_arrive_tx(barrier, 1, bytes);
2855 #endif
2856 }
2857 #endif
2858
2859 #if defined(NOTES_V2_ENABLE_WGMMMA)
2860 // =====
2861 // Phase 7d: HGEMM WGMMMA — m64n128k16 + TMA + Warp Specialization (Hopper)
2862 // =====
2863 // 面试要点 (WGMMMA vs MMA 对比) :
2864 //   - MMA: warp 级 (32 threads) , 同步执行
2865 //   - WGMMMA: warpgroup 级 (128 threads = 4 warps) , 异步执行 (fire-and-forget)
2866 //   - m64n128k16: M=64, N=128, K=16 → 一次处理 64×128×16=131K 个乘加 (MMA m16n8k16的 4x16=64倍)
2867 //   - TMA (Tensor Memory Accelerator): 硬件 DMA, 2D 寻址, 零寄存器开销
2868 //   - Warp Specialization: Producer 做 TMA, Consumer 做 WGMMMA, 通过 barrier 同步
2869 //   - 128B swizzle: shared memory 的 128B swizzle 模式, 避免 bank conflict
2870
2871 // ---- WGMMMA 辅助函数 ----
2872 // wgmma.commit_group / wait_group 语义 (PTX ISA §9.7.15.7) :
2873 //   - commit_group: 将此前所有未提交的 wgmma.mma_async 归入一个新的 wgmma-group
2874 //     (per-warpgroup, 故需 .sync.aligned 确保所有线程同步执行) 。
2875 //   - wait_group N: 阻塞直到最多 N 个 wgmma-group 尚未完成 (pending ≤ N) 。
2876 //     N=0 → 等所有 group 完成。★ 与 cp.async.wait_group 语义一致 (PTX ISA §9.7.9.25.3.3) ,
2877 //     都是 "wait until only N or fewer groups are pending"。
2878 #define WGMMMA_FENCE() asm volatile("wgmma.fence.sync.aligned;\n" ::: "memory")
2879 #define WGMMMA_COMMIT_GROUP() \
2880     asm volatile("wgmma.commit_group.sync.aligned;\n" ::: "memory")
2881 #define WGMMMA_WAIT_GROUP(n) \
2882     asm volatile("wgmma.wait_group.sync.aligned %0;\n" :::"n"(n) : "memory")
2883
2884 // SMEM_DESC_ENCODE: 将 byte 偏移编码为 WGMMMA descriptor 位域中的 14-bit 值。
2885 // 编码规则 (PTX ISA §9.7.15.5.1.2.2) :
2886 //   encoded = (x & 0x3FFFF) >> 4
2887 // 其中 0x3FFFF = 2^18 - 1 = 262143, 只保留低 18 bit。
2888 // >>4 是因为 shared memory 地址/偏移必须 16B 对齐 (2^4) , 低 4 bit 恒为 0,
2889 // 可省略以扩大可表示范围。
2890 // 解码: decoded = encoded << 4 (回到原始 byte 值) 。
2891 #define SMEM_DESC_ENCODE(x) (((uint64_t)(x)) & 0x3FFFF) >> 0x4
2892
2893 // make_smem_desc: 构造 WGMMMA (warpgroup MMA) 的 64-bit shared memory 矩阵描述符。
2894 // 硬件 WGMMMA (如 wgmma.mma_async.m64n128k16) 需要 descA/descB 两个 64-bit 描述符,
2895 // 来定位 shared memory 中的 A/B tile 并告知 swizzle 模式。
2896 //
2897 // 64-bit descriptor 位域布局 (参见 PTX ISA §9.7.15.5.1.2.2 & CUTLASS GmmaDescriptor) :

```

```

2898 // -----
2899 // | 位域          | 范围      | 大小 | 含义
2900 // |-----|-----|-----|-----
2901 // | start_address  | [0, 14)   | 14   | smem 基址编码
2902 // | (unused)       | [14, 16)  | 2    | -
2903 // | leading_byte_offset | [16, 30) | 14   | 主要维度 (K) 的字节步长; swizzle 下硬件未使用 (assumed=1)
2904 // | (unused)       | [30, 32)  | 2    | -
2905 // | stride_byte_offset | [32, 46) | 14   | 跨越维度 (M/N) 的字节步长: K-Major 下为 8-row stripe 间距
2906 // | (unused)       | [46, 49)  | 3    | -
2907 // | base_offset    | [49, 52)  | 3    | swizzle pattern 偏移
2908 // | (unused)       | [52, 62)  | 10   | -
2909 // | layout_type    | [62, 64)  | 2    | swizzle 模式
2910 // -----
2911 // layout_type: 0=None, 1=128B swizzle, 2=64B swizzle, 3=32B swizzle
2912 //
2913 // K-Major + 128B swizzle + half(16-bit) 的几何推导 (PTX ISA §9.7.15.5.1.2) :
2914 // - 128B swizzle atom 在 128-bit 单位下为 8×8
2915 // - 归一化到 half(2B) 后: atom 覆盖 8×(128/16)=64 个 K 方向元素 × 8 个 M 方向元素
2916 // - 即每个 atom = 64(K) × 8(M) 个 half = 64×8×2 = 1024 bytes
2917 // - 这是 stride_byte_offset=1024 的来源
2918 //
2919 // - leading_byte_offset 在 K-Major swizzle 下 unused (hardware assumes 1) ,
2920 //   此处填 16 仅为占位, 编码后为 1。
2921 //
2922 // - base_offset=0 (bits [49,52) 未显式置位) , 表示 smem ptr 已 1024B 对齐。
2923 //   若未对齐需计算 (pattern_start_addr >> 7) & 0x7。
2924 //
2925 // 参考: CUTLASS GmmaDescriptor (cute/arch/mma_sm90_desc.hpp)
2926 // 注: descA/descB 共用此函数; 对 B 而言物理存储为 [BN, BK], 详见 WgmmaSMem 注释。
2927 __device__ inline uint64_t make_smem_desc(half *ptr) {
2928     // __cvta_generic_to_shared: 将通用地址空间中的指针转换为 shared memory 地址
2929     // (一个 32-bit 的 smem byte 偏移量, 相对于当前 CTA 的 shared memory 基址)。
2930     uint32_t addr = static_cast<uint32_t>(__cvta_generic_to_shared(ptr));
2931
2932     // 从全零开始: 所有位域初始化为 0。
2933     uint64_t desc = 0x0000000000000000;
2934
2935     // — bits [0, 14): start_address —
2936     // SMEM_DESC_ENCODE(addr) 将 addr 右移 4 位 (丢弃低 4 个 0 bit) ,
2937     // 只保留 14 位。硬件解码时左移 4 位恢复完整的 16B-aligned 地址。
2938     // 可寻址范围: 2^(14+4) = 2^18 = 256K 个 half = 512 KB 共享内存。
2939     desc |= SMEM_DESC_ENCODE(addr);
2940
2941     // — bits [16, 30): leading_byte_offset —
2942     // 原始值 16 (字节), 编码后为 (16 & 0x3FFFF) >> 4 = 1。
2943     // << 16 将编码值放到 bits [16, 30)。
2944     // K-Major + 128B swizzle 下该字段硬件未使用 (assumed to be 1, 参见 PTX ISA §9.7.15.5.1.2.1.1) ,
2945     // 此处填 16 仅为占位, 使编码值为 1 满足硬件预期。
2946     // 若为 MN-Major 或 INTERLEAVE 布局, LBO 有实际含义:
2947     //   对 INTERLEAVE: 同一 8×2 brick 内第一列到第二列的字节偏移。
2948     //   对 MN-Major swizzle: 偏移从首 (swizzle-byte-size/16) 行到下一组行。
2949     desc |= SMEM_DESC_ENCODE((uint64_t)16) << 16;
2950
2951     // — bits [32, 46): stride_byte_offset —
2952     // 原始值 1024 (字节), 编码后为 (1024 & 0x3FFFF) >> 4 = 64。
2953     // << 32 将编码值放到 bits [32, 46)。
2954     // K-Major 下定义为"从首 8 行到次 8 行的字节偏移" (PTX ISA §9.7.15.5.1.2.1.2) 。
2955     // 1024 的来源 (K-Major + 128B swizzle + half) :
2956     //   一个 128B swizzle atom 覆盖 64(K) × 8(M) 个 half。
2957     //   从 1 个 8-row stripe 到下一个 stripe 需要跨越 8 × 64 × 2 = 1024 bytes。
2958     // 若为 MN-Major: SBO 是从首列到下一列的偏移 (8 列一组) 。
2959     desc |= SMEM_DESC_ENCODE((uint64_t)1024) << 32;
2960

```

```

2961 // — bits [62, 64): layout_type (swizzle mode) —
2962 // 1llu << 62 = 二进制 01 在 bits [62, 64) = layout_type = 1。
2963 // 1 = 128B swizzle mode。
2964 // 其他取值: 0=None, 2=64B swizzle, 3=32B swizzle。
2965 // 128B swizzle 将 smem 中连续的行按 128B 粒度进行 XOR 重映射,
2966 // 确保同一 warp 内各线程访问不同 bank 的同一偏移时不会产生 bank conflict。
2967 desc |= 1llu << 62;
2968
2969 return desc;
2970 }
2971
2972 // ---- WGMMA PTX 指令宏 ----
2973 // wgmma.mma_async.sync.aligned.m64n128k16.f16.f16.f16
2974 // m64n128k16: M=64, N=128, K=16。一次 WGMMA 计算  $D[64 \times 128] += A[64 \times 16] \times B[16 \times 128]$ 。
2975 // f16.f16.f16: A=f16, B=f16, D=f16 (累加器也是 f16 精度)。
2976 // 每线程 32 个 uint32 输出:  $D=64 \times 128=8192$  half, warpgroup 128 线程分担,
2977 // 每线程 64 half = 32 uint32。
2978 // descA/descB: shared memory 描述符 (由 make_smem_desc 生成, 64-bit 编码)。
2979 // ScaleD: 0=清零累加器再写入, 1=与累加器中的旧值累加 (K 维迭代必须用 1)。
2980 // ScaleA/ScaleB: 1=正常符号, -1=翻转符号 (此处始终用 1)。
2981 // TransA/TransB: 0=K-major (K 维连续), 1=MN-major (M/N 维连续)。本实现始终用 0。
2982 // 参考: PTX ISA §9.7.15.4 (wgmma.mma_async)
2983 #define WGMMA_M64N128K16_F16F16F16(d, sA, sB, ScaleD, ScaleA, ScaleB, TransA, \
2984                                     TransB) \
2985 { \
2986     uint64_t desc_a = make_smem_desc(&(sA)[0]); \
2987     uint64_t desc_b = make_smem_desc(&(sB)[0]); \
2988     asm volatile( \
2989         "{\n" \
2990         "wgmma.mma_async.sync.aligned.m64n128k16.f16.f16.f16 " \
2991         "{%0, %1, %2, %3, %4, %5, %6, %7, " \
2992         "%8, %9, %10, %11, %12, %13, %14, %15, " \
2993         "%16, %17, %18, %19, %20, %21, %22, %23, " \
2994         "%24, %25, %26, %27, %28, %29, %30, %31}, " \
2995         "%32, " \
2996         "%33, " \
2997         "%34, %35, %36, %37, %38;\n" \
2998         "}\n" \
2999         : "+r"((d)[0][0]), "+r"((d)[0][1]), "+r"((d)[0][2]), "+r"((d)[0][3]), \
3000         "+r"((d)[1][0]), "+r"((d)[1][1]), "+r"((d)[1][2]), "+r"((d)[1][3]), \
3001         "+r"((d)[2][0]), "+r"((d)[2][1]), "+r"((d)[2][2]), "+r"((d)[2][3]), \
3002         "+r"((d)[3][0]), "+r"((d)[3][1]), "+r"((d)[3][2]), "+r"((d)[3][3]), \
3003         "+r"((d)[4][0]), "+r"((d)[4][1]), "+r"((d)[4][2]), "+r"((d)[4][3]), \
3004         "+r"((d)[5][0]), "+r"((d)[5][1]), "+r"((d)[5][2]), "+r"((d)[5][3]), \
3005         "+r"((d)[6][0]), "+r"((d)[6][1]), "+r"((d)[6][2]), "+r"((d)[6][3]), \
3006         "+r"((d)[7][0]), "+r"((d)[7][1]), "+r"((d)[7][2]), "+r"((d)[7][3]) \
3007         : "l"(desc_a), "l"(desc_b), "n"(int32_t(ScaleD)), \
3008         "n"(int32_t(ScaleA)), "n"(int32_t(ScaleB)), "n"(int32_t(TransA)), \
3009         "n"(int32_t(TransB))); \
3010 }
3011
3012 // ---- TMA Shared Memory Layout ----
3013 // Multi-stage pipeline: kStages 个 stage, 每个 stage 存储  $A[BM \times BK] + B[BK \times BN]$ 
3014 //
3015 // 每个 stage 包含两块 smem:
3016 //   A tile:  $[BM, BK]$  row-major  $\rightarrow BK \times BM$  个 half, 地址连续
3017 //   B tile: TMA 按  $[BN, BK]$  物理写入 (详见下方 *),  $BN \times BK$  个 half
3018 //
3019 // * 关键区别 — WGMMA 的 B 物理存储 vs 逻辑视图:
3020 //
3021 //   上层数据: 源矩阵  $B^T [N, K]$  row-major (TN 布局, B 的列以行优先形式存储)。
3022 //   物理存储: TMA 将  $B^T$  的 tile 写入 smem, 物理布局为  $[BN, BK]$  row-major
3023 //   (BN=128 行, BK=64 列, 每行 64 个连续的 half)。

```

```

3024 //      TMA 的 smem_box_shape = (BK=64, BN=128) 定义了每个 tile 的 shape,
3025 //      TMA 按行写入, 行内 BK 个 half 连续 → 物理上就是 [BN][BK]。
3026 //      逻辑视图: WGMMMA 指令 (wgmma.mma_async.m64n128k16, PTX ISA §9.7.15.4)
3027 //      通过 imm-trans-b=0 指定 B 为 K-major, 即逻辑上按 [BK, BN] 寻址。
3028 //      实际 [BN, BK] → 逻辑 [BK, BN] 的"转置"是通过 descB 中的 128B
3029 //      swizzle (layout_type=1) 在硬件地址重映射层完成的, 无需软件转置。
3030 //
3031 //      stride_byte_offset=1024 的来源 (K-major + 128B swizzle + f16, PTX ISA §9.7.15.5.1.2.1.2) :
3032 //      128B swizzle atom = 8(N) × 8(K) × 128-bit = 8 rows × 64 个 half。
3033 //      stride_byte_offset = 从当前 8-row stripe 到下一个 8-row stripe 的字节偏移
3034 //      = 8 rows × (BK=64) halves × 2 bytes = 1024。
3035 //      这个值恰好等于物理 [BN, BK] 布局中连续 8 行的跨度 ← 进一步证实物理存储是 [BN, BK]。
3036 //
3037 //      与 MMA(TN) 的对比:
3038 //      MMA (TN): smem 存 B^T [N, K] row-major, ldmatrix 按列解出 col-major B fragment。
3039 //      WGMMMA:    smem 物理存 [BN, BK] (TMA 直写), WGMMMA 通过 swizzle 硬件以 K-major [BK, BN] 逻辑视图读取。
3040 //      简言之: swizzle 桥接了 "物理 row-major [BN, BK]" 和 "逻辑 K-major [BK, BN]" 的差异。
3041 template <int BM, int BN, int BK, int QSIZE> struct WgmmaSMem {
3042     alignas(128) half A[BM * BK * QSIZE]; // A tile: row-major [BM, BK]
3043     // B tile 笔记:
3044     //   - 物理存储: TMA 从 B^T[N,K] row-major 搬入, 物理上是 [BN, BK] row-major (BN 行, BK 列)
3045     //   - 逻辑视图: WGMMMA 通过 descB 的 128B swizzle (PTX ISA §9.7.15.5.1.2) 将地址重映射,
3046     //     以 K-major [BK, BN] 逻辑视图供 wgmma.mma_async (imm-trans-b=0, PTX ISA §9.7.15.4) 读取
3047     //   - B[BN * BK * QSIZE] 与 B[BK * BN * QSIZE] 数值等价 (BN×BK = BK×BN) ,
3048     //     但写 BN×BK 更能体现物理存储形态
3049     alignas(128) half B[BN * BK * QSIZE];
3050 };
3051
3052 // ---- WGMMMA Kernel: Warp Specialization + TMA ----
3053 // 面试重点 — Warp Specialization (Hopper 最核心的编程模型变化) :
3054 //
3055 // 背景: 传统 GEMM kernel 中, 所有线程同步地"加载→计算→存回",
3056 // 数据搬运和计算无法重叠。Hopper 的 TMA + WGMMMA 支持将工作拆分为两个
3057 // warpgroup, 让数据搬运和矩阵乘完全异步执行:
3058 //
3059 //   WG0 (128 threads, Producer): 仅 thread 0 提交 TMA 2D 拷贝指令,
3060 //   将 A/B tile 从 HBM 搬到 shared memory。
3061 //   WG1 (128 threads, Consumer): 所有 128 个线程参与 WGMMMA 矩阵乘。
3062 //
3063 // Producer 和 Consumer 通过 cuda::barrier (CTA 级别) 同步:
3064 //   - full[stage]: Producer 发信号表示 stage stage 的数据已就绪, 可被使用
3065 //   - empty[stage]: Consumer 发信号表示 stage stage 的使用完毕, 可以被覆盖
3066 //
3067 // 本节重点理解:
3068 //   1) 为什么 Producer 只需要 thread 0? TMA 是硬件 DMA 指令, 一次提交
3069 //     即可搬运整个 2D tile, 无需所有线程参与。
3070 //   2) barrier 的 arrive count = 129: 128 个 Consumer 线程 + 1 个 Producer 提交线程。
3071 //   3) 多 stage pipeline 使 Consumer 计算 stage stage 的同时,
3072 //     Producer 可以搬运 stage (stage+1), 隐藏 HBM→SMEM 延迟。
3073 //
3074 // Tile Hierarchy (与 MMA m16n8k16 kernel 对比) :
3075 //   WGMMMA Atom:    m64n128k16 (一次处理 64×128×16, 是 MMA 的 64 倍)
3076 //   K Tile:         BK=64, 每个 K tile 包含 BK/kWgmmaK=4 个 WGMMMA atom
3077 //   M Tile:         BM=128, 每个 M tile 包含 BM/kWgmmaM=2 个 WGMMMA atom
3078 //   N Tile:         BN=128, 单个 kWgmmaN=128 即可覆盖, 无需在 N 方向分块
3079 //   Block Tile:     C[128,128] = A[128,64] × B[64,128]
3080 //   Threads:        256 = 2 warpgroups × 128 threads/warpgroup
3081 //   Producer(WG0):  128 threads (仅 thread 0 做 TMA 提交)
3082 //   Consumer(WG1):  128 threads = 4 warps (全部参与 WGMMMA 和写回)
3083 //
3084 // kStages=3: 3 级流水线。Consumer 滞后 Producer 最多 2 步, 确保 HBM→SMEM
3085 // 的延迟被计算完全掩盖。
3086 //

```



```

3087 // Grid: ((N+127)/128/S, (M+127)/128, S), S=(N+2047)/2048, 3D block swizzle
3088 // - grid.z = S 个 swizzle 分区, 将连续 block 打散到不同 N 区域改善 L2 命中
3089 // Block: (256, 1, 1), 2 warpgroups
3090 // source: LeetCUDA/kernels/hgemm/wgmma/hgemm_wgmma_fp16acc_stages_tn.cu
3091 template <const int kWgmmaM = 64,           // WGMMMA atom M dim (m64n128k16)
3092           const int kWgmmaN = 128,         // WGMMMA atom N dim
3093           const int kWgmmaK = 16,          // WGMMMA atom K dim
3094           const int BM = 128,              // block tile M, 2 × kWgmmaM
3095           const int BN = 128,              // block tile N, 1 × kWgmmaN
3096           const int BK = 64,               // block tile K, 4 × kWgmmaK
3097           const int kNumThreads = 256,     // 2 warpgroups × 128 threads
3098           const int kStages = 3,           // TMA pipeline depth (full/empty barriers)
3099           const int kBlockSwizzle = 0>     // 1 enables 3D grid swizzle for L2 locality
3100 __global__ void __launch_bounds__(kNumThreads)
3101   hgemm_wgmma_stages_tn(
3102     int M, int N, int K, half *C,
3103     const CUtensorMap *__restrict__ tensorMapA,
3104     const CUtensorMap *__restrict__ tensorMapB) {
3105   static_assert(kBlockSwizzle == 0 || kBlockSwizzle == 1, "kBlockSwizzle must be 0 or 1");
3106   // 注意: tensorMapA/tensorMapB 需要由 host 侧按当前 tile 布局预先创建;
3107   // 对 row-major [M, K] 矩阵, TMA shape 参数写的是 (K, M) 而不是 (M, K),
3108   // 也就是TMA descriptor中把连续的维度写在最内层, 非连续的维度写在最外层。
3109   // 这是 TMA descriptor 最容易背错的地方之一。notes 这里只保留 kernel 主体,
3110   // 不展开宿主侧 create_tensor_map 细节。
3111
3112   // Block Swizzle: 在 grid x 维度做 swizzle, 改善 L2 cache 局部性
3113   // bx = blockIdx.z * gridDim.x + blockIdx.x, 将相邻 block 打散到不同 N 区域
3114   const int bx = ((int)kBlockSwizzle) * blockIdx.z * gridDim.x + blockIdx.x;
3115   const int by = blockIdx.y;
3116   constexpr int kConsumerThreads = kNumThreads / 2; // 128 threads = 1 warpgroup
3117   // kNumConsumers = (kNumThreads / 128) - 1 = 2 - 1 = 1 (1 个 consumer WG)
3118   // 这里的 -1 是因为 2 个 warpgroup 中 1 个是 producer, 剩余都是 consumer
3119   constexpr int kNumConsumers = (kNumThreads / kConsumerThreads) - 1; // 1 consumer WG
3120   // kWarpgroupM = BM / kNumConsumers: 每个 consumer warpgroup 负责的 M 行数
3121   // 当只有一个 consumer 时, 它负责全部 BM=128 行
3122   constexpr int kWarpgroupM = BM / kNumConsumers; // 128
3123
3124   // 边界检查: 确保当前 block 不超出 M/N 范围
3125   if (bx >= div_ceil(N, BN) || by >= div_ceil(M, BM))
3126     return;
3127
3128   // ---- Shared Memory 分配 ----
3129   // 动态 shared memory (由 host 侧通过 kernel launch 的 smem 参数指定大小)
3130   // TMA + WGMMMA 仅需 __align__(128), 而非 __align__(1024)。原因:
3131   //   make_smem_desc() 构造 WGMMMA descriptor 时, start_address 字段
3132   //   完整编码了 __cvta_generic_to_shared 的绝对 32-bit 偏移量,
3133   //   硬件根据该绝对地址自动推导并补偿 swizzle phase 偏移 (等价于
3134   //   descriptor 内置的 base_offset 机制)。因此即使 smem 基址未落到
3135   //   1024B 边界, WGMMMA 硬件也能正确读取 TMA 写入的数据。
3136   // 对比 hgemm_tma_mma_ws_tn: 消费者使用 ldmatrix + 手写 swizzle<64>()
3137   //   , 该函数无法感知 smem 绝对地址, 硬编码了 phase=0 的假设, 因此必须
3138   //   __align__(1024) 来保证 phase 确实为 0。从健壮性角度看本 kernel 也
3139   //   应该用 __align__(1024); 这里保留 128 是为了突出两种消费者的特点与对比。
3140   extern __shared__ __align__(128) uint8_t smem_tma_wgmma_ws[];
3141   WgmmaSMem<BM, BN, BK, kStages> &s =
3142     *reinterpret_cast<WgmmaSMem<BM, BN, BK, kStages>*>(smem_tma_wgmma_ws);
3143   half *s_a = s.A;
3144   half *s_b = s.B;
3145
3146   // ---- cuda::barrier 初始化 ----
3147   //
3148   // ★ Barrier 机制速查 (arrive / wait 核心语义):
3149   //

```



```

3150 // cuda::barrier 是一个 CTA 级同步原语, 基于 **phase (奇偶交替)** 工作:
3151 //
3152 //   init(bar, N): 设置 arrive_count = N。
3153 //   含义: 每 phase 需要 N 个线程各调用一次 arrive(), barrier 才翻转 phase。
3154 //
3155 //   arrive(): 线程声明"我已到达此 barrier 点"。
3156 //   - 非阻塞, 立即返回一个 token (包含当前 phase 值)。
3157 //   - 不关心"谁"到达, 只关心"到达次数"是否达到 arrive_count。
3158 //
3159 //   wait(token): 线程阻塞, 直到 barrier 的当前 phase ≠ token 中的 phase。
3160 //   - 即: 等待 phase 翻转。
3161 //   - Phase 翻转条件: (1) arrive 次数达到 arrive_count
3162 //                     (2) 若使用了 barrier_arrive_tx, 还需 TMA 字节全部写完
3163 //
3164 //   典型用法: b.wait(b.arrive())
3165 //   - arrive() 注册自己到达, 拿到 token (当前 phase = P)
3166 //   - wait(token) 阻塞直到 phase 翻转 (P → P+1)
3167 //   - 如果自己是第 N 个到达者 → phase 立即翻转 → wait 立即返回 (不阻塞)
3168 //   - 如果自己不是最后一个 → wait 阻塞, 直到第 N 个到达者触发翻转
3169 //
3170 // ★ 本 kernel 的 Pipeline 同步协议 (kStages=3, arrive_count=129) :
3171 //
3172 //   Producer (1 thread)           Consumer (128 threads)
3173 //   ────────────                 ────────────
3174 //                               C0: arrive(empty[*]) ×128 [init: 标记所有 stage 为空]
3175 //   ┌ for each k_tile: ─┐       ┌ for each k_tile: ─┐
3176 //   │ P1: arrive+wait(empty[q]) │ │ C1: arrive+wait(full[q]) │
3177 //   │   ↓ 等 stage q 被消费完   │ │   ↓ 等 TMA 数据就绪     │
3178 //   │ P2: TMA(A[q]) + TMA(B[q]) │ │ C2: WGMMMA 计算         │
3179 //   │   ↓ 异步拷贝             │ │ C3: wait WGMMMA 完成     │
3180 //   │ P3: arrive_tx(full[q])    │ │ C4: arrive(empty[q])    │
3181 //   │   ↑ 通知: stage q 数据就绪 │ │   ↑ 通知: stage q 已消费完 │
3182 //   └──────────────────┘       └──────────────────┘
3183 //
3184 //   关键不变式 (invariant) :
3185 //   - Producer 不会覆盖 Consumer 正在读的 stage
3186 //   - Consumer 不会读 Producer 还没写完的 stage
3187 //   - 通过 full/empty 两个 barrier 的 phase 交替来保证
3188 //
3189 //   每个 stage 有两个 barrier:
3190 //   full[stage]: TMA 数据就绪信号。Producer 发 (arrive_tx), Consumer 等 (wait)。
3191 //   empty[stage]: Stage 空闲信号。Consumer 发 (arrive), Producer 等 (wait)。
3192 //
3193 //   arrive_count = 128 (consumer) + 1 (producer) = 129:
3194 //   每 phase, 128 个 consumer 线程 + 1 个 producer 线程都要 arrive 一次。
3195 #pragma nv_diag_suppress static_var_with_dynamic_init
3196 __shared__ cuda::barrier<cuda::thread_scope_block> full[kStages];
3197 __shared__ cuda::barrier<cuda::thread_scope_block> empty[kStages];
3198 #pragma nv_diag_default static_var_with_dynamic_init
3199
3200 // K 方向总 tile 数。要求 K 能被 BK 整除, 否则尾 tile 被丢弃。
3201 const int NUM_K_TILES = div_ceil(K, BK);
3202 const int wg_idx = threadIdx.x / kConsumerThreads; // 0=Producer, 1=Consumer
3203 const int wg_tid = threadIdx.x % kConsumerThreads; // 0~127 within warpgroup
3204
3205 // 初始化 barriers (仅 thread 0 执行)
3206 if (threadIdx.x == 0) {
3207     for (int i = 0; i < kStages; ++i) {
3208         // arrive_count = 129: 128 consumer + 1 producer
3209         // init 是 CUDA barrier 的 hidden friend 函数 (定义在 cuda::barrier 类体内),
3210         // 只能通过 ADL (Argument-Dependent Lookup) 调用。因为第一个参数 &full[i] 的类型是
3211         // cuda::barrier<cuda::thread_scope_block>*, 编译器通过 ADL 在 cuda 命名空间中自动找到它。
3212         init(&full[i], kConsumerThreads + 1); // 128 consumer + 1 producer

```

```

3213     init(&empty[i], kConsumerThreads + 1); // same
3214 }
3215 // fence_proxy_async_shared_cta: 确保 barrier 在 smem 中的初始化
3216 // 对 async proxy (TMA/WGMMMA) 可见。
3217 tma_fence_proxy_async_shared_cta();
3218 }
3219 __syncthreads();
3220
3221 // =====
3222 // ★ Warp Specialization 并发模型 — 为什么只有 if/else 没有 while?
3223 // =====
3224 //
3225 // 256 个线程在同一个 SM 上**并发执行**。if/else 只是分工，不是先后顺序：
3226 //   - WG0 (threadIdx.x 0~127): 全部走 if 分支，做 Producer。
3227 //   - WG1 (threadIdx.x 128~255): 全部走 else 分支，做 Consumer。
3228 //
3229 // SM 的 warp scheduler 会交替调度来自 WG0 和 WG1 的 warp，两者**同时**推进：
3230 //   Producer: for (k_iter = 0 .. NUM_K_TILES) { P1→P2→P3 } ← 自己的循环
3231 //   Consumer: for (k_iter = 0 .. NUM_K_TILES) { C1→C2→C3→C4 } ← 自己的循环
3232 //
3233 // 两个 warpgroups 各自独立迭代 K-tiles，通过 full[] / empty[] barrier 同步：
3234 //   - Producer 领先 Consumer 太多 → wait(empty[q]) 阻塞，等 Consumer 消费完
3235 //   - Consumer 领先 Producer 太多 → wait(full[q]) 阻塞，等 TMA 搬运完
3236 //
3237 // 这是生产者-消费者模型的 GPU 实现：不需要共享循环变量，barrier 的 phase
3238 // 交替天然保证了流水线串行化 (at most kStages-1 steps ahead)。
3239 // =====
3240 // Producer Warpgroup (WG0, threadIdx.x 0~127)
3241 // 职责：提交 TMA 2D 拷贝，将 A/B tile 从 HBM 异步搬运到 SMEM。
3242 // 只有 wg_tid==0 执行实际拷贝提交，其余 127 个线程空闲。
3243 // =====
3244 if (wg_idx == 0) {
3245     if (wg_tid == 0) {
3246         // stage: 当前操作的 stage 索引 (round-robin 0 -> 1 -> 2 -> 0 -> ...)
3247         int stage = 0;
3248         for (int k = 0; k < NUM_K_TILES; ++k, ++stage) {
3249             if (stage == kStages)
3250                 stage = 0;
3251
3252             // Step P1: 等待 stage stage 变为"空" (可被覆盖写入)
3253             //
3254             // 模式 empty[stage].wait(empty[stage].arrive()):
3255             //   - arrive(): Producer (1 个线程) 在 empty barrier 上注册到达，
3256             //     获得当前 phase 的 token。如果 Consumer 已经在 C4 步骤积累了
3257             //     128 次 arrive (来自上一轮迭代或 C0 初始化)，则加上这次共 129 次
3258             //     → phase 立即翻转。
3259             //   - wait(token): 阻塞直到 barrier phase ≠ token 中的 phase。
3260             //     由于 arrive() 是第 129 次到达，phase 在 arrive 时已翻转，
3261             //     wait 看到 phase 已变 → 立即返回 (首轮不阻塞)。
3262             //
3263             // 语义：Producer 说"我准备好写 stage stage 了，Consumer 用完没有？"
3264             //     如果 Consumer 还没用完 → phase 未翻转 → wait 阻塞等待。
3265             //     如果 Consumer 已用完 → phase 已翻转 → wait 立即返回。
3266             empty[stage].wait(empty[stage].arrive());
3267
3268             // Step P2: 提交 TMA 2D 拷贝指令
3269             // cp_async_bulk_tensor_2d_global_to_shared:
3270             //   参数1(dst): smem 目标地址 (当前 stage 的 A/B tile 起始位置)
3271             //   参数2(tensorMap): Host 预创建的 TMA descriptor (描述源矩阵的 shape/stride/dtype)
3272             //   参数3/4(coords): (k_offset, m_offset) 或 (k_offset, n_offset) 的全局坐标
3273             //   参数5(barrier): 拷贝完成后自动 arrive 到此 barrier
3274             //
3275             // TMA 是硬件 DMA 引擎：一次指令提交即可搬运整个 2D tile,

```

```

3276 // 无需线程逐元素搬运，零寄存器开销。
3277 // TMA 2D 加载 A tile: coords = (k_offset, m_offset)
3278 tma_load_2d(&s_a[stage * BK * BM], tensorMapA, k * BK, by * BM,
3279             full[stage]);
3280
3281 // TMA 2D 加载 B tile: coords = (k_offset, n_offset)
3282 tma_load_2d(&s_b[stage * BK * BN], tensorMapB, k * BK, bx * BN,
3283             full[stage]);
3284
3285 // Step P3: 通知 Consumer: stage stage 的 TMA 数据已就绪
3286 //
3287 // barrier_arrive_tx(bar, arrive_count_update, byte_count):
3288 //   - 在 bar 上注册 1 次到达 (arrive_count_update=1)
3289 //   - 同时声明预期有 byte_count 字节将通过 async copy (TMA) 写入 smem
3290 //   - Phase 翻转条件:
3291 //       (a) 总 arrive 次数达到 129 (128 consumer + 1 producer)
3292 //       (b) 所有声明的 async 字节已写入 smem
3293 //   两个条件都满足后 phase 才翻转, Consumer 的 wait() 才返回。
3294 //   - 即: Consumer 不会在 TMA 数据完整到达前就开始读 smem。
3295 tma_arrive_expect_tx(full[stage], (BK * BN + BK * BM) * sizeof(half));
3296 }
3297 }
3298 }
3299 // =====
3300 // Consumer Warpgroup (WG1, threadIdx.x 128~255)
3301 // 职责: 等待 TMA 数据就绪 -> 发射 WGMMMA 做矩阵乘 -> 积累结果 -> 写回 C
3302 // 所有 128 个线程 (4 warps) 全部参与。
3303 // =====
3304 else {
3305     // Step C0: Consumer 初始化 — 标记所有 stage 为"空" (可被 Producer 写入)
3306     //
3307     // 所有 128 个 Consumer 线程对每个 stage 的 empty barrier 调用 arrive()。
3308     // 这是 Pipeline 的"预热"步骤——没有它, Producer 的 empty[stage].wait()
3309     // 在第一轮会永远阻塞 (因为 Producer 的 1 次 arrive 不足以凑够 129)。
3310     //
3311     // 注意: 此时每个 empty[i] 只有 128 次 arrive, 未达 129, phase 不翻转。
3312     // Producer 后续的 empty[stage].arrive() 作为第 129 次, 触发 phase 翻转。
3313     for (int i = 0; i < kStages; ++i) {
3314         [[maybe_unused]] auto token = empty[i].arrive();
3315     }
3316
3317     // 累加器寄存器声明
3318     // d[kWarpgroupM / kWmmaM][kWmmaN / 16][4]:
3319     //   - d[0][*][*]: M 方向第 1 个 WGMMMA atom (rows 0~63)
3320     //   - d[1][*][*]: M 方向第 2 个 WGMMMA atom (rows 64~127)
3321     //   - d[*][g][*]: N 方向第 g 组 16 列
3322     //   - d[*][*][0..3]: 4 条 uint32 寄存器, 共 8 个 half (覆盖 16x16 子块)
3323     // 每个线程总共 2 * 8 * 4 = 2 * 32 = 64 uint32 = 128 half (每次WGMMMA Atom 32 uint32 输出)
3324     // 128 线程 * 128 half = 16384 half = 128 * 128 = BM*BN (刚好覆盖整个 C tile)
3325     uint32_t d[kWarpgroupM / kWmmaM][kWmmaN / 16][4] = {};
3326
3327     int stage = 0;
3328     // K 维外循环: 沿 K tile 迭代 (BK=64, 每个 K tile 做 4 次 WGMMMA 累加)
3329     for (int k = 0; k < NUM_K_TILES; ++k, ++stage) {
3330         if (stage == kStages)
3331             stage = 0;
3332
3333         // Step C1: 等待 TMA 数据就绪 (full 信号)
3334         //
3335         // 模式 full[stage].wait(full[stage].arrive()):
3336         //   - 128 个 Consumer 线程各调用 arrive(), 共 128 次到达。
3337         //   - 当前 phase 累积: 128/129, phase 尚未翻转。
3338         //   - wait(token) 阻塞, 直到 Producer 的 barrier_arrive_tx (Step P3)

```

```

3339 // 贡献第 129 次到达 + TMA 字节全部写完 → phase 翻转 → wait 返回。
3340 //
3341 // 语义: Consumer 说"我准备好读 stage stage 了, 数据到了没有?"
3342 full[stage].wait(full[stage].arrive());
3343
3344 // Step C2: 发射 WGMMMA 指令序列
3345 //
3346 // WGMMMA 是异步指令 (fire-and-forget), 发射后立即返回, 不等待计算完成。
3347 // 标准流程: FENCE → 发射 WGMMMA → COMMIT → WAIT。
3348 //
3349 // WGMMMA_FENCE (wgmma.fence.sync.aligned) :
3350 // - 确保 TMA 写入 smem 的数据对 async proxy (WGMMMA) 可见
3351 // - 确保累加器寄存器 d[] 已准备好接收 WGMMMA 输出
3352 // - 本质是 proxy 间的内存序 (memory ordering), 不是传统 __syncthreads
3353 //
3354 // 每个 WGMMMA_M64N128K16_F16F16F16 指令:
3355 // D[64×128] += A[64×16] × B[16×128]
3356 // A/B 均为 K-major (imm-trans=0), 由 descA/descB 描述 smem 中的布局。
3357 // ScaleD=1: 累加。K 维有 BK/kWgmmaK=4 次迭代, 每次都用 ScaleD=1。
3358 WGMMMA_FENCE();
3359
3360 // M 维迭代: BM/kWgmmaM = 128/64 = 2 个 WGMMMA atom
3361 // 每个 atom 处理 64 行 M 方向数据。
3362 #pragma unroll
3363 for (int m = 0; m < kWarpgroupM / kWgmmaM; ++m) {
3364     // wgmma_sA 指向当前 stage 中 A tile 的第 m 个 64*BK 子块
3365     // s_a 布局: [kStages][BM][BK] -> stage * BK*BM + BK * (m*64)
3366     half *wgmma_sA = s_a + stage * BK * BM + BK * m * kWgmmaM;
3367
3368     // K 维迭代: BK/kWgmmaK = 64/16 = 4 次 WGMMMA (累加)
3369     // 每次处理 K=16 维的矩阵乘, 4 次累加后覆盖完整的 BK=64。
3370 #pragma unroll
3371     for (int k_step = 0; k_step < BK / kWgmmaK; ++k_step) {
3372         // 第 k_step 次 WGMMMA:
3373         // A: wgmma_sA + k_step * kWgmmaK (A 的 K 维起始位置)
3374         // B: s_b + stage * BK * BN + k_step * kWgmmaK (B 的 K 维起始位置)
3375         // 注意 B 的 smem 布局是 [BK, BN] row-major, K-major 读取时从
3376         // 第 k_it*16 行开始取 16 行, 每行 BN=128 列。
3377         // ScaleD=1: 累加 (K 维迭代需要累积结果)
3378         WGMMMA_M64N128K16_F16F16F16(d[m], wgmma_sA + k_step * kWgmmaK,
3379                                     s_b + stage * BK * BN + k_step * kWgmmaK,
3380                                     1, // ScaleD=1: accumulate (不清零)
3381                                     1, 1, 0, 0);
3382     }
3383 }
3384
3385 // Step C3: 提交并等待 WGMMMA 完成
3386 //
3387 // WGMMMA_COMMIT_GROUP (wgmma.commit_group.sync.aligned):
3388 // 将自上一次 COMMIT 以来发射的所有 WGMMMA 归为一组, 提交到 async proxy 执行。
3389 // 类比 cp.async.commit_group, 但 scope 是 warpgroup 而非 per-thread。
3390 //
3391 // WGMMMA_WAIT_GROUP(0) (wgmma.wait_group.sync.aligned 0):
3392 // 阻塞直到最多 N 个 group 尚未完成 (pending ≤ N)。N=0 即等待所有 group。
3393 // ★ 语义与 cp.async.wait_group 完全一致 (PTX ISA §9.7.15.7.3 vs §9.7.9.25.3.3) :
3394 // 两者都是 "wait until only N or fewer of the most recent groups are pending"。
3395 // 此处用 0 是因为 Consumer 在本次迭代中只 commit 了 1 个 group,
3396 // 必须等它全部完成才能进入下一步 (读下一 stage 的数据)。
3397 WGMMMA_COMMIT_GROUP();
3398 WGMMMA_WAIT_GROUP(0);
3399
3400 // Step C4: 释放 stage stage — 通知 Producer 可以覆盖写入
3401 //

```

```

3402 // 128 个 Consumer 线程在 empty[stage] 上 arrive(), 为 **下一 phase** 积累到达次数。
3403 // 这 128 次 arrive 不会立即翻转 phase (还需 Producer 在下一轮的 1 次 arrive)。
3404 //
3405 // 语义: Consumer 说"stage stage 我已经读完了, 你可以放心覆盖了。"
3406 [[maybe_unused]] auto token = empty[stage].arrive();
3407 }
3408
3409 // =====
3410 // Epilogue: 将寄存器中的累加结果写回 global memory C
3411 // =====
3412 //
3413 // WGMMMA m64n128k16.f16.f16.f16 的输出寄存器映射:
3414 //
3415 // 对 warpgroup 内每个线程, 输出 64 个 half = 32 个 uint32。
3416 // 这些 half 分布在 d[2][8][4] 数组中:
3417 // d[m_it][g][0]: (row, col) 和 (row, col+2) 位置的 2 个 half
3418 // d[m_it][g][1]: (row+8, col) 和 (row+8, col+2) 位置的 2 个 half
3419 // d[m_it][g][2]: (row, col+8) 和 (row, col+10) 位置的 2 个 half
3420 // d[m_it][g][3]: (row+8, col+8) 和 (row+8, col+10) 位置的 2 个 half
3421 //
3422 // 其中:
3423 // row = warp * 16 + lane / 4 (0~63, 4 warps * 16 rows)
3424 // col = g * 16 + 2 * (lane % 4) (0~126, step 2, 8 g's * 16 cols)
3425 //
3426 // 每个线程覆盖一个 16x16 子块中的 16 个 half:
3427 // +-----+-----+
3428 // | reg[0] | reg[2] | <- rows [row, row+8)
3429 // |(col,+2)|(col+8)|
3430 // +-----+-----+
3431 // | reg[1] | reg[3] | <- rows [row+8, row+16)
3432 // |(col,+2)|(col+8)|
3433 // +-----+-----+
3434 // 每个 reg 包含 2 个连续 half (col 和 col+2), 用 uint32 存储。
3435 //
3436 // 三维遍历:
3437 // m_it=0: rows 0~63, m_it=1: rows 64~127
3438 // g=0..7: 每个覆盖 16 列, 8*16=128 列 ok
3439 // 每个线程写 4 uint32 * 2 half/uint32 * 8 g * 2 m_it = 128 half。
3440 // 128 线程 * 128 half = 16384 half = 128 * 128 ok
3441
3442 const int lane = wg_tid % 32;
3443 const int warp = wg_tid / 32;
3444 // row: 当前线程在当前 WGMMMA atom 中负责的起始行。
3445 // warp 0~3 各负责 16 行: rows [warp*16, warp*16+15]。
3446 // lane/4: 每 4 个连续 lane 负责同一行 (因为 WGMMMA fragment 中每行有 4 个 uint32,
3447 // 分别覆盖列对 {c0,c1}、{c2,c3}、{c8,c9}、{c10,c11}, 由 lane%4 区分)。
3448 // 因此 lane/4 给出该行在 16-row block 内的行号 (0~7 对应 rows 0~7,
3449 // 同一 warp 中 lane/4==0 的有 lane 0,1,2,3, 都负责 row 0)。
3450 const int row = warp * 16 + lane / 4;
3451 // block_C: 当前 C tile 在 global memory 中的起始地址
3452 // by*BM 是 M 方向偏移, bx*BN 是 N 方向偏移
3453 half *block_C = C + by * BM * N + bx * BN;
3454
3455 // 2 * (8 * 4) = 2 * 32 = 64 uint32 = 128 half
3456 // NOTE: 这里可以考虑通过 R->S, S->G 的方式做一次 128 bits写回。
3457 #pragma unroll
3458 for (int m = 0; m < kWarpgroupM / kWgmmaM; ++m) {
3459     int yo = m * kWgmmaM; // M 方向行偏移 (0 或 64)
3460 #pragma unroll
3461     for (int g = 0; g < kWgmmaN / 16; ++g) {
3462         int col = g * 16 + 2 * (lane % 4); // N 方向列偏移 (0,2,4,...,14 then 16,18,...)
3463         // 一次 uint32 store 写入 2 个 half (连续列 col 和 col+2)
3464         // 左上象限: (row, col)

```

```

3465     *reinterpret_cast<uint32_t*>(&block_C[(row + yo) * N + col]) = d[m][g][0];
3466     // 左下象限: (row+8, col)
3467     *reinterpret_cast<uint32_t*>(&block_C[(row + yo + 8) * N + col]) = d[m][g][1];
3468     // 右上象限: (row, col+8)
3469     *reinterpret_cast<uint32_t*>(&block_C[(row + yo) * N + col + 8]) = d[m][g][2];
3470     // 右下象限: (row+8, col+8)
3471     *reinterpret_cast<uint32_t*>(&block_C[(row + yo + 8) * N + col + 8]) = d[m][g][3];
3472 }
3473 }
3474 }
3475 }
3476
3477 #endif /* NOTES_V2_ENABLE_WGMMA */
3478
3479 #if defined(NOTES_V2_ENABLE_TMA_MMA_WS)
3480 // SM120 不支持 WGMMA, 但支持相同的 TMA 生产者协议和 warp 级 mma.sync。
3481 // 保持 128B TMA swizzle 并显式声明物理布局供 ldmatrix 消费者使用。
3482 template <int BM, int BN, int BK, int QSIZE> struct TmaMmaWSSMem {
3483     static_assert(BK == 64, "The 128B swizzle helper below is specialized for BK=64");
3484     half A[BM * BK * QSIZE];
3485     half B[BN * BK * QSIZE];
3486 };
3487
3488 // 消费者 MMA 层级映射参考 hgemm_mma_stages_tn_swizzle。与那个
3489 // 八 warp kernel 不同, 本 warp-specialized 消费者仅拥有四个 warp。
3490 template <const int kMmaM = 16,           // MMA atom M dim (m16n8k16)
3491           const int kMmaN = 8,           // MMA atom N dim
3492           const int kMmaK = 16,          // MMA atom K dim
3493           const int kMmaTileM = 2,       // consumer warps along M, warp tile M = 32
3494           const int kMmaTileN = 2,       // consumer warps along N, warp tile N = 16
3495           const int kValTileM = 4,       // value-repeat along M, BM = 16*2*4 = 128
3496           const int kValTileN = 8,       // value-repeat along N, BN = 8*2*8 = 128
3497           const int kValTileK = 4,       // MMA_K slices per BK tile, BK = 16*4 = 64
3498           const int kStages = 2,         // TMA full/empty pipeline depth
3499           const int kNumThreads = 256,   // 128 producer + 128 consumer threads
3500           const int kBlockSwizzle = 0>   // 1 enables 3D grid swizzle for L2 locality
3501 __global__ void __launch_bounds__(kNumThreads)
3502     hgemm_tma_mma_ws_tn(
3503         int M, int N, int K, half *C,
3504         const CUtensorMap *__restrict__ tensorMapA,
3505         const CUtensorMap *__restrict__ tensorMapB) {
3506
3507     constexpr int BM = kMmaM * kMmaTileM * kValTileM;
3508     constexpr int BN = kMmaN * kMmaTileN * kValTileN;
3509     constexpr int BK = kMmaK * kValTileK;
3510     static_assert(kMmaM == 16 && kMmaN == 8 && kMmaK == 16, "This kernel uses mma.sync.m16n8k16");
3511     static_assert(kMmaTileM * kMmaTileN == 4, "The consumer warpgroup has exactly four warps");
3512     static_assert(BM == 128 && BN == 128 && BK == 64, "TMA desc and 128B swizzle require 128x128x64");
3513     static_assert(kNumThreads == 256, "Use one producer and one consumer warpgroup");
3514     static_assert(kBlockSwizzle == 0 || kBlockSwizzle == 1, "kBlockSwizzle must be 0 or 1");
3515
3516     const int bx = kBlockSwizzle * blockIdx.z * gridDim.x + blockIdx.x;
3517     const int by = blockIdx.y;
3518     constexpr int kConsumerThreads = kNumThreads / 2;
3519     static_assert(kConsumerThreads == kMmaTileM * kMmaTileN * kWarpSize,
3520         "Consumer threads must cover the MMA warp grid");
3521
3522     if (bx >= div_ceil(N, BN) || by >= div_ceil(M, BM))
3523         return;
3524
3525     // TMA CU_TENSOR_MAP_SWIZZLE_128B 要求 smem 基地址 1024 字节对齐,
3526     // 以确保硬件 swizzle phase 从零开始。消费者 swizzle<64>()
3527     // 假设零 phase; 其他对齐 (如 128) 会导致 phase 偏移, 产生不匹配的

```



```

3528 // 物理地址，破坏整个输出。
3529 //
3530 // 为什么 hgemm_wgmma_stages_tn 只需要 __align__(128)，而本 kernel
3531 // 必须 __align__(1024)? 核心区别在于消费者如何感知 swizzle phase:
3532 //
3533 // WGMMMA 消费者: 通过 make_smem_desc() 构造 64-bit WGMMMA descriptor,
3534 // 其中 bits [49,52) 的 base_offset 字段编码了 smem 基址相对于
3535 // 1024B 边界的偏移量 ((addr >> 7) & 0x7)。硬件在读取 smem 时会
3536 // 用这个 base_offset 自动补偿 swizzle phase, 因此 WGMMMA 路径不
3537 // 需要 1024 字节对齐也能得到正确数据。
3538 //
3539 // 本 kernel 消费者: 使用 ldmatrix + 手写 swizzle<64>() 来计算
3540 // smem 物理地址。这个函数是纯软件公式, 没有任何 base_offset
3541 // 补偿机制。它假设 smem 基址恰好落在 1024B 边界上 (phase=0)。
3542 // 如果基址只满足 128B 对齐, TMA 硬件会以非零 phase 写入数据,
3543 // 而 ldmatrix 以零 phase 读取 → 物理地址彻底错位 → 全输出错误。
3544 //
3545 // 简言之: WGMMMA 用硬件 descriptor base_offset 自动解决 phase 偏移;
3546 // ldmatrix 路径没有等价机制, 必须靠 1024B 对齐来保证 phase=0。
3547 extern __shared__ __align__(1024) uint8_t smem_tma_mma_ws[];
3548 TmaMmaWSSMem<BM, BN, BK, kStages> &s =
3549     *reinterpret_cast<TmaMmaWSSMem<BM, BN, BK, kStages> *>(smem_tma_mma_ws);
3550 half *s_a = s.A;
3551 half *s_b = s.B;
3552
3553 #pragma nv_diag_suppress static_var_with_dynamic_init
3554 __shared__ cuda::barrier<cuda::thread_scope_block> full[kStages];
3555 __shared__ cuda::barrier<cuda::thread_scope_block> empty[kStages];
3556 #pragma nv_diag_default static_var_with_dynamic_init
3557
3558 const int NUM_K_TILES = div_ceil(K, BK);
3559 const int wg_idx = threadIdx.x / kConsumerThreads;
3560 const int wg_tid = threadIdx.x % kConsumerThreads;
3561
3562 if (threadIdx.x == 0) {
3563     for (int stage = 0; stage < kStages; ++stage) {
3564         init(&full[stage], kConsumerThreads + 1);
3565         init(&empty[stage], kConsumerThreads + 1);
3566     }
3567     tma_fence_proxy_async_shared_cta();
3568 }
3569 __syncthreads();
3570
3571 if (wg_idx == 0) {
3572     // 生产者 Warpgroup (wg_idx==0)
3573     if (wg_tid == 0) {
3574         int stage = 0;
3575         for (int k = 0; k < NUM_K_TILES; ++k, ++stage) {
3576             if (stage == kStages)
3577                 stage = 0;
3578
3579             empty[stage].wait(empty[stage].arrive());
3580
3581             tma_load_2d(&s_a[stage * BM * BK], tensorMapA, k * BK, by * BM,
3582                 full[stage]);
3583             tma_load_2d(&s_b[stage * BN * BK], tensorMapB, k * BK, bx * BN,
3584                 full[stage]);
3585             tma_arrive_expect_tx(full[stage], (BM * BK + BN * BK) * sizeof(half));
3586         }
3587     }
3588 } else {
3589     // 消费者 Warpgroup (wg_idx==1)
3590     // 初始化: 标记所有 stage 为"空" (可被 Producer 写入)

```

```

3591 for (int stage = 0; stage < kStages; ++stage) {
3592     [[maybe_unused]] auto token = empty[stage].arrive();
3593 }
3594
3595 const int warp_id = wg_tid / kWarpSize;
3596 const int lane_id = wg_tid % kWarpSize;
3597 // WG1 内形成 2x2 warp grid, warp_m / warp_n 分别表示 warp 在 M/N 方向的索引。
3598 const int warp_m = warp_id % kMmaTileM; // kMmaTileM = 2, {0,1}
3599 const int warp_n = warp_id / kMmaTileM; // kMmaTileN = 2, {0,1}
3600 uint32_t RA[kValTileM][4];
3601 uint32_t RB[kValTileN][2];
3602 uint32_t RC[kValTileM][kValTileN][2] = {0};
3603
3604 const uint32_t smem_a_base_ptr = __cvta_generic_to_shared(s_a);
3605 const uint32_t smem_b_base_ptr = __cvta_generic_to_shared(s_b);
3606 int stage = 0;
3607 for (int k = 0; k < NUM_K_TILES; ++k, ++stage) {
3608     if (stage == kStages)
3609         stage = 0;
3610
3611     full[stage].wait(full[stage].arrive());
3612     tma_fence_proxy_async_shared_cta();
3613
3614     // 与 cp.async pipeline (hgemm_mma_stages_tn 等) 不同, 这里 ldmatrix +
3615     // mma.sync 之前/后 不需要 __syncthreads, 原因在于 TMA + async proxy 的
3616     // 同步机制与 warp specialization 的线程分工:
3617     //
3618     // 1. 生产者-消费者解耦: TMA 拷贝由 producer (wg_idx==0 的单个线程)
3619     //    通过 tma_load_2d + tma_arrive_expect_tx 提交, 而非所有 256 个线程
3620     //    各自做 cp.async。cp.async 需要 __syncthreads 确保所有线程的异步
3621     //    拷贝都完成后进入计算; 而 TMA 路径用 cuda::barrier 的 full/empty
3622     //    协议完成 CTA 级同步——full[stage].wait() 返回时, producer 已通过
3623     //    expect_tx 声明了完整的传输字节数, 且硬件保证这些字节已全部写入 smem。
3624     //
3625     // 2. async proxy fence 替代 __syncthreads: fence_proxy_async(space_shared)
3626     //    在 async proxy (TMA 写入通道) 和后续 smem load (ldmatrix 读取通道)
3627     //    之间建立内存序。它确保 fence 之前的所有 async proxy 写操作 (TMA)
3628     //    对 fence 之后的所有 smem 读操作 (ldmatrix) 可见。这是一个轻量级
3629     //    proxy 间 ordering, 不需要阻塞线程, 也不会让 warp 等待其他 warp。
3630     //
3631     // 3. 消费者内部 warp 间无依赖: WG1 内的 4 个 warp (2x2 grid) 各自独立
3632     //    读取 smem 中互不重叠的 A/B 区域 (warp_m 和 warp_n 分别索引不同的
3633     //    行/列)。它们之间不需要读写共享数据, 因此不需要 __syncthreads 来
3634     //    对齐 warp 间的执行进度。mma.sync 本身是 warp 级同步指令, 保证同一
3635     //    warp 内 32 个线程的 ldmatrix 和 mma 操作正确协作。
3636
3637     // 注意: 这里已经没有 NUM_K_TILES 循环了, 因为 K loop load 已经被 WG0 生产者处理了
3638 #pragma unroll
3639     for (int k_step = 0; k_step < kValTileK; ++k_step) {
3640
3641 #pragma unroll
3642         for (int i = 0; i < kValTileM; ++i) { // kValTileM = 4
3643             // kMmaM * kValTileM = 16*4 = 64, 每个消费者 warp 在 M 方向覆盖 64 行。
3644             const int warp_smem_a_m = warp_m * (kMmaM * kValTileM) + i * kMmaM;
3645             const int lane_smem_a_m = warp_smem_a_m + lane_id % 16;
3646             // 与 hgemm_mma_stages_tn_swizzle 不同: k_step * kMmaK 必须放在
3647             // lane_smem_a_k 中传给 swizzle<64>(), 而非像 swizzle kernel
3648             // 那样作为 smem_k_offset 加在 swizzle 外部。原因:
3649             //   swizzle<64> 作用于完整 BK=64, swizzle 周期覆盖全部 64 列,
3650             //   k_step=0 和 k_step=1 的 chunk 会被 XOR 交叉混合 → k_step
3651             //   偏移必须在 swizzle 内部参与 chunk 计算。
3652             //   swizzle<kMmaK> 作用于 kMmaK=16, 每个 kMmaK slice 独立
3653             //   swizzle, slice 之间互不跨越 → k_step * kMmaK 可以加在外部。

```

```

3654     const int lane_smem_a_k = (k_step * kMmaK) + (lane_id / 16) * 8;
3655     const uint32_t lane_smem_a_ptr = smem_a_base_ptr +
3656         (stage * BM * BK + lane_smem_a_m * BK +
3657         swizzle<64>(lane_smem_a_m, lane_smem_a_k)) *
3658         sizeof(half);
3659     LDMATRIX_X4(RA[i][0], RA[i][1], RA[i][2], RA[i][3], lane_smem_a_ptr);
3660 }
3661
3662 #pragma unroll
3663 for (int j = 0; j < kValTileN; ++j) { // kValTileN = 8
3664     // kMmaN * kValTileN = 8*8 = 64, 每个消费者 warp 在 B^T 的 N 方向覆盖 64 行。
3665     const int warp_smem_b_n = warp_n * (kMmaN * kValTileN) + j * kMmaN;
3666     const int lane_smem_b_n = warp_smem_b_n + lane_id % 8;
3667     const int lane_smem_b_k = (k_step * kMmaK) + ((lane_id / 8) % 2) * 8;
3668     const uint32_t lane_smem_b_ptr = smem_b_base_ptr +
3669         (stage * BN * BK + lane_smem_b_n * BK +
3670         swizzle<64>(lane_smem_b_n, lane_smem_b_k)) *
3671         sizeof(half);
3672     LDMATRIX_X2(RB[j][0], RB[j][1], lane_smem_b_ptr);
3673 }
3674
3675 #pragma unroll
3676 for (int i = 0; i < kValTileM; ++i) {
3677     #pragma unroll
3678     for (int j = 0; j < kValTileN; ++j) {
3679         HMMAT16816(RC[i][j][0], RC[i][j][1],
3680             RA[i][0], RA[i][1], RA[i][2], RA[i][3],
3681             RB[j][0], RB[j][1],
3682             RC[i][j][0], RC[i][j][1]);
3683     }
3684 }
3685 }
3686 [[maybe_unused]] auto token = empty[stage].arrive();
3687 }
3688
3689 // Epilogue: 复用 RA[0] 和 RA[1] (已在寄存器中) 作为 shuffle 缓冲,
3690 // 与 hgemm_mma_stages_tn_swizzle 的 epilogue 模式一致。四个相邻
3691 // lane 各持有两个 uint32 fragment; shuffle 汇聚为每行一个 float4,
3692 // 由 lane 0 发出对齐的 128-bit store。
3693 #pragma unroll
3694 for (int i = 0; i < kValTileM; ++i) {
3695     const int store_warp_smem_c_m = warp_m * (kMmaM * kValTileM) + i * kMmaM;
3696     const int store_lane_gmem_c_m = by * BM + store_warp_smem_c_m + lane_id / 4;
3697     #pragma unroll
3698     for (int j = 0; j < kValTileN; ++j) {
3699         const int store_warp_smem_c_n = warp_n * (kMmaN * kValTileN) + j * kMmaN;
3700         RA[0][0] = RC[i][j][0];
3701         RA[1][0] = RC[i][j][1];
3702         RA[0][1] = __shfl_sync(0xffffffff, RC[i][j][0], lane_id + 1);
3703         RA[0][2] = __shfl_sync(0xffffffff, RC[i][j][0], lane_id + 2);
3704         RA[0][3] = __shfl_sync(0xffffffff, RC[i][j][0], lane_id + 3);
3705         RA[1][1] = __shfl_sync(0xffffffff, RC[i][j][1], lane_id + 1);
3706         RA[1][2] = __shfl_sync(0xffffffff, RC[i][j][1], lane_id + 2);
3707         RA[1][3] = __shfl_sync(0xffffffff, RC[i][j][1], lane_id + 3);
3708         if (lane_id % 4 == 0) {
3709             const int store_lane_gmem_c_n = bx * BN + store_warp_smem_c_n;
3710             const int store_gmem_c_addr_0 =
3711                 store_lane_gmem_c_m * N + store_lane_gmem_c_n;
3712             const int store_gmem_c_addr_1 =
3713                 (store_lane_gmem_c_m + 8) * N + store_lane_gmem_c_n;
3714             *reinterpret_cast<float4*>(&C[store_gmem_c_addr_0]) =
3715                 *reinterpret_cast<float4*>(&RA[0][0]);
3716             *reinterpret_cast<float4*>(&C[store_gmem_c_addr_1]) =

```

```

3717         *reinterpret_cast<float4*>(&RA[1][0]);
3718     }
3719 }
3720 }
3721 }
3722 }
3723 #endif /* NOTES_V2_ENABLE_TMA_MMA_WS */
3724
3725 #if defined(NOTES_V2_ENABLE_WGMMMA) || defined(NOTES_V2_ENABLE_TMA_MMA_WS)
3726 // ---- Host-side TMA Tensor Map helpers (WGMMMA test) ----
3727 // 面试要点 (TMA descriptor 创建 — cuTensorMapEncodeTiled):
3728 //   - TMA descriptor 描述 global memory 中矩形的 shape/stride/dtype,
3729 //     以及硬件搬运的 box (tile) 大小和 smem swizzle 模式
3730 //   - 关键易错点: TMA shape 参数写的是 (W,H) 而不是 (H,W)!
3731 //     对 row-major [M,K] 矩阵, TMA shape = (K,M), minor=K, major=M
3732 //   - cuTensorMapEncodeTiled 是 CUDA Driver API, 需 #include <cuda.h> 并链接 -lcuda
3733 //   - smem_box 维度顺序也是 (minor, major) = (BK, BM) 或 (BK, BN)
3734 //   - Swizzle 模式通常选 CU_TENSOR_MAP_SWIZZLE_128B 配合 WGMMMA 的 128B swizzle
3735 //
3736 // * gmem_prob_stride + 1 的含义:
3737 // 语法层面 — 数组名退化 + 指针算术:
3738 //   gmem_prob_stride 类型为 uint64_t[5], 在表达式中退化为 uint64_t*
3739 //   (指向首元素 gmem_prob_stride[0])。+ 1 偏移 1 个 uint64_t, 指向
3740 //   gmem_prob_stride[1], 等价于 &gmem_prob_stride[1]。
3741 // 语义层面 — 跳过隐式的最内维 stride:
3742 //   cuTensorMapEncodeTiled 的 globalStrides 参数只接收 tensorRank-1 个
3743 //   stride 值。最内维 (fastest-changing dimension) 的 stride 是隐式的,
3744 //   固定等于 sizeof(element_type), 不需要显式传入。
3745 //   对于 tensorRank=2 的 FP16 矩阵:
3746 //       dim 0 (内维, K) : stride = sizeof(half)    ← 隐式, API 不需要
3747 //       dim 1 (外维, M) : stride = sizeof(half)*K  ← 需要传给 API
3748 //   gmem_prob_stride 数组的布局是内维在前:
3749 //       [0] = sizeof(half)                        ← 内维, 不传
3750 //       [1] = sizeof(half) * BlockMinorSize * blocks_width ← 外维, 传给 API
3751 //   所以 + 1 的作用是跳过 [0], 让 API 从 [1] 开始读取, 正好得到外维 stride。
3752 //
3753 // * smem_box_stride 为什么不需要 +1?
3754 //   与 globalStrides 不同, smemBoxStrides 参数接收完整的 tensorRank 个 stride,
3755 //   最内维 stride 也必须显式传入 (不隐式)。
3756 //   smem_box_stride[5] = {1, 1, 1, 1, 1} 表示 smem tile 中所有维度元素
3757 //   都是连续存放的, 维度间没有 padding/stride。
3758 //   对于 tensorRank=2: strides[0]=1 (内维 stride), strides[1]=1 (外维 stride),
3759 //   即 smem 中第 (i,j) 元素的偏移 = i*1 + j*1 = i+j (元素连续排列)。
3760 //   所以这里直接传 smem_box_stride (即 &smem_box_stride[0]), API 会读到全部
3761 //   两个 stride 值, 不需要跳过任何一个。
3762 //
3763 // 参考: CUDA Programming Guide §TMA, PTX ISA §9.7.15.5, CUTLASS GmmaDescriptor
3764
3765 template <int BlockMajorSize, int BlockMinorSize>
3766 __host__ static inline void create_tensor_map(CUtensorMap *tma_map,
3767                                               half *gmem_ptr,
3768                                               int blocks_height,
3769                                               int blocks_width) {
3770     void *gmem_address = (void *)gmem_ptr;
3771     uint64_t gmem_prob_shape[5] = {(uint64_t)BlockMinorSize * blocks_width,
3772                                     (uint64_t)BlockMajorSize * blocks_height,
3773                                     1, 1, 1};
3774     uint64_t gmem_prob_stride[5] = {
3775         sizeof(half), sizeof(half) * BlockMinorSize * blocks_width, 0, 0, 0};
3776     uint32_t smem_box_shape[5] = {uint32_t(BlockMinorSize),
3777                                    uint32_t(BlockMajorSize), 1, 1, 1};
3778     uint32_t smem_box_stride[5] = {1, 1, 1, 1, 1};
3779     CUresult result = cuTensorMapEncodeTiled(

```

```

3780     tma_map, CU_TENSOR_MAP_DATA_TYPE_FLOAT16, 2, gmem_address,
3781     gmem_prob_shape, gmem_prob_stride + 1, smem_box_shape, smem_box_stride,
3782     CU_TENSOR_MAP_INTERLEAVE_NONE, CU_TENSOR_MAP_SWIZZLE_128B,
3783     CU_TENSOR_MAP_L2_PROMOTION_NONE, CU_TENSOR_MAP_FLOAT_OOB_FILL_NONE);
3784     if (result != CUDA_SUCCESS)
3785         printf("cuTensorMapEncodeTiled failed: %d\n", (int)result);
3786 }
3787
3788 __host__ static inline CUTensorMap *allocate_and_create_tensor_map(
3789     half *src, int blocks_height, int blocks_width) {
3790     CUTensorMap *tma_map_d;
3791     cudaMalloc(&tma_map_d, sizeof(CUTensorMap));
3792     CUTensorMap tma_map_host;
3793     create_tensor_map<128, 64>(&tma_map_host, src, blocks_height, blocks_width);
3794     cudaMemcpy(tma_map_d, &tma_map_host, sizeof(CUTensorMap),
3795         cudaMemcpyHostToDevice);
3796     return tma_map_d;
3797 }
3798 #endif /* NOTES_V2_ENABLE_WGMMA || NOTES_V2_ENABLE_TMA_MMA_WS */
3799
3800 // =====
3801 // Phase 8: FlashAttention-2 + MMA m16n8k16)
3802 // =====
3803 // 面试题点 (FlashAttention-2算法) :
3804 // 1. 核心问题: 标准 Attention 的  $O(N^2)$  中间矩阵 ( $S=QK^T$ ) 必须写入 HBM,
3805 //    但 HBM 带宽是瓶颈 → FlashAttention-2用 tiling + online softmax 避免写回
3806 // 2. FA 三板斧:
3807 //    a) Tiling: Q 分块 [Br,d], K/V 沿 seqLen 分块 [Bc,d]
3808 //    b) Online Softmax: 迭代更新 m(行max) 和 l(行sum), 无需全局同步
3809 //    c) Recomputation(反向): 反向传播时重新计算 S/P, 而非存储中间矩阵
3810 // 3. Split-Q 设计: 所有 warp 共享同一块 K, 各 warp 处理 Q 的不同行片段
3811 //    - warp_KV=0 (所有 warp 共享 K), warp_QP=warp_id (各 warp 不同 Q 行)
3812 //    - 优点: 减少 warp 间通信和 shuffle
3813 // 4. Online rescaling 公式 (FA 核心, arXiv:2307.08691) :
3814 //    for each K,V tile:
3815 //        S_cur = Q @ K^T // 未缩放, 存入 R_S
3816 //        m_new = max(m_old, row_max(S_cur * scale))
3817 //        P_cur = exp(S_cur * scale - m_new) // ← 写回 R_S 寄存器!
3818 //        l_new = exp(m_old - m_new) * l_old + row_sum(P_cur)
3819 //        O_new = diag(exp(m_old - m_new)) * O_old + P_cur @ V
3820 //        O_final = O_new / l_final
3821 // 5. 为什么 R_S 可以直接用作 P@V 的 A 矩阵?
3822 //    - R_S 经过 softmax 后, 存储的是  $P = \exp(S - m)$ , 数据仍然是 half 精度
3823 //    - 当前实现依赖 m16n8k16 这一路径下约定好的 fragment 布局, 使 softmax
3824 //      后的 P 可以继续留在 R_S 中供后面的 P@V 直接消费
3825 //    - 这是此实现的寄存器布局复用技巧, 不要背成“所有 MMA A/C fragment
3826 //      都天然同构”的通用结论
3827 //
3828 // 本实现参考: FlashAttention-2 (Dao et al., arXiv:2307.08691)
3829 // 从 LeetCUDA flash-attn/mma/basic/flash_attn_mma_split_q.cu 提取
3830 // Grid: ((N + 63) / 64, B * H, 1), Br=64
3831 // Block: (128, 1, 1), kNumThreads=kWarpSize×kMmaTileSeqLenQ×kMmaTileSeqLenK=128
3832 // source: LeetCUDA/kernels/flash-attn/mma/basic/flash_attn_mma_split_q.cu
3833
3834 // ---- 寄存器填充辅助函数 ----
3835 template <typename T, int M, const int N, const int K = 2>
3836 __device__ inline void fill_3D_regs(T (&R)[M][N][K], T val) {
3837     #pragma unroll
3838     for (int i = 0; i < M; ++i)
3839         #pragma unroll
3840         for (int j = 0; j < N; ++j)
3841             #pragma unroll
3842             for (int k = 0; k < K; ++k)

```

```

3843     R[i][j][k] = val;
3844 }
3845
3846 template <typename T, int M, const int N = 2>
3847 __device__ inline void fill_2D_regs(T (&R)[M][N], T val) {
3848     #pragma unroll
3849     for (int i = 0; i < M; ++i)
3850     #pragma unroll
3851         for (int j = 0; j < N; ++j)
3852             R[i][j] = val;
3853 }
3854
3855 // =====
3856 // FlashAttention-2 Split-Q Kernel (完整实现)
3857 // =====
3858 // Q,K,V,0: [batch_size, num_heads, seq_len, head_dim], [B,H,N,d]
3859 //
3860 // Tile 设计 (以 kHeadDim=64 为例) :
3861 //   Br = kMmaAtomM * kMmaTileSeqLenQ * kValTileSeqLenQ = 16*4*1 = 64
3862 //   Bc = kMmaAtomN * kMmaTileSeqLenK * kValTileSeqLenK = 8*1*8 = 64
3863 //   Warp 布局: 4 warps, warp_QP 0~3 各处理 16 行, warp_KV=0 共享 K
3864 //
3865 // 执行流程:
3866 //   1) 预加载 Q[Br,d] 到 smem (只加载一次, split-Q 的核心优势)
3867 //   2) 外循环: 沿 K seqLen 分块迭代 (Tc = seqLen/Bc)
3868 //   3)   3a: cp.async 加载当前 V + 预加载下一个 K (多 stage pipeline)
3869 //   4)   3b: Q@K^T — 沿 head dim 内循环 → ldmatrix Q/K → HMMA16816
3870 //   5)   3c: Online Safe Softmax — warp reduce max/row → exp(S*scale - max)
3871 //       → 关键: 将 P 写回 R_S 寄存器 (替换 S), R_S 现在存储 P matrix
3872 //   6)   3d: P@V — 沿 V_Bc 内循环 → ldmatrix V (transposed) → 直接用 R_S 做 A
3873 //       → 当前实现依赖同一组寄存器布局约定, 避免为 P@V 额外重组 P fragment
3874 //   7)   3e: Online rescaling — O_new = exp(m_old-m_new)*O_old + P@V
3875 //   8) 最终 rescale: O_final = (1/l_final) * O_final
3876 //   9) Epilogue: warp shuffle + 128-bit collective store
3877 //
3878 // * Pad 与手动 XOR swizzle 的 profile 结论 (SM120, B=1,H=32,N=4096,D=64) :
3879 //   - XOR 能消除目标 ldmatrix lane pattern 的 bank conflict, 但收益是局部的;
3880 //     XOR 本身及 `% 2` 的计算不是主要瓶颈, nvcc 已将该式化简为 bit operations.
3881 //   - compact Q/K/V XOR 改变了 cp.async 的 shared destination pattern: LDGSTS
3882 //     wavefronts 从 pad 的 30.15M 增至 68.16M (2.26x), long-scoreboard、
3883 //     LG-throttle、MIO-throttle 分别约为 pad 的 3.25x、2.74x、1.60x。
3884 //   - compact XOR 虽节省约 5 KiB smem, 但没有提高此 kernel 的 occupancy; 最终
3885 //     约 120.0 TFLOPS, 显著低于 Q/K/V kPad=8 的 166.6 TFLOPS。
3886 //   - 所以当前 kernel 默认对 Q/K/V 都用 kPad=8。保留 swizzle 路径是为了学习和消融: 评价
3887 //     shared layout 必须同时观察 ldmatrix reads 与 cp.async/LDGSTS writes, 不能只看
3888 //     bank-conflict counter, 也不能只优化 XOR 地址算术。
3889
3890 template <
3891     const int kHeadDim,           // head dim: 32, 64, 128
3892     const int kMmaAtomM,          // 16 (MMA instruction M dimension)
3893     const int kMmaAtomN,          // 8 (MMA instruction N dimension)
3894     const int kMmaAtomK,          // 16 (MMA instruction K dimension)
3895     const int kMmaTileSeqLenQ,    // MMA tiles along Q's M dim, 4 → Br=16*4=64
3896     const int kMmaTileSeqLenK,    // MMA tiles along K's N dim, 1 → Bc basis=8
3897     const int kMmaTileSeqLenP,    // MMA tiles for P@V M dim, must equal kMmaTileSeqLenQ
3898     const int kMmaTileHeadDimV,   // MMA tiles for P@V N dim (head dim direction)
3899     const int kValTileSeqLenQ,    // value tiles along Q's M, 1 → Br per warp=16
3900     const int kValTileSeqLenK,    // value tiles along K's N, 8 → Bc_warp=8*8=64
3901     const int kValTileSeqLenP,    // value tiles for P@V M dim, 1
3902     const int kValTileHeadDimV,   // value tiles for P@V N dim, kHeadDim/(8*kMmaTileHeadDimV)
3903     const int kStagesK,           // pipeline stages for K: >= 1; NO stages required for Q/V
3904     const int kPadQ,              // Q row padding; 0 selects compact XOR swizzle
3905     const int kPadK,              // K row padding; 0 selects compact XOR swizzle

```



```

3906     const int kPadV>          // V row padding; 0 selects compact XOR swizzle
3907 __global__ void flash_attn_mma_stages_split_q(
3908     half *Q, half *K, half *V, half *O, int N, int H) {
3909     static_assert(kStagesK >= 1, "kStagesK must be >= 1");
3910     static_assert(kPadQ >= 0 && kPadK >= 0 && kPadV >= 0,
3911         "Q/K/V padding must be non-negative");
3912     constexpr bool kSwizzleQ = kPadQ == 0;
3913     constexpr bool kSwizzleK = kPadK == 0;
3914     constexpr bool kSwizzleV = kPadV == 0;
3915     constexpr int Br = kMmaAtomM * kMmaTileSeqLenQ * kValTileSeqLenQ; // 64
3916     constexpr int Bc = kMmaAtomN * kMmaTileSeqLenK * kValTileSeqLenK; // 64
3917     constexpr int kNumThreads = kWarpSize * kMmaTileSeqLenQ * kMmaTileSeqLenK; // 128
3918     const int Tc = (N + Bc - 1) / Bc;
3919     // 原始实现默认 seqLen 与 Bc 对齐; 最后一个不完整 tile 需要额外 pad/边界处理。
3920     // 这里保留 ceil 写法是为了说明 tile 划分方式, 不等于当前实现已经完整处理了尾 tile。
3921     const float scale = 1.0f / sqrtf((float)kHeadDim);
3922
3923     // Block indexing
3924     const int Nb_id = blockIdx.y / H; // batch id
3925     const int Nh_id = blockIdx.y % H; // head id
3926     const int Q_tile_id = blockIdx.x; // tile id along Q's M dim (Br)
3927     const int tid = threadIdx.x;
3928     const int warp_id = tid / kWarpSize;
3929     const int lane_id = tid % kWarpSize;
3930     const int warp_QP = warp_id; // Split-Q: 每个 warp 处理不同的 Q 行片段
3931     const int warp_KV = 0; // 所有 warp 共享 K (减少跨 warp 通信)
3932
3933     // Global memory base offsets for this (batch, head)
3934     // 这里默认 Q/K/V 共享同一 per-head 基址布局, 对应 self-attention 场景
3935     const int Q_gmem_offset = (Nb_id * H * N + Nh_id * N) * kHeadDim;
3936     const int K_gmem_offset = Q_gmem_offset;
3937     const int V_gmem_offset = Q_gmem_offset;
3938     const int O_gmem_offset = Q_gmem_offset;
3939
3940     // Thread-to-smem mapping for cooperative load
3941     int load_smem_Q_Br = tid / (kNumThreads / Br);
3942     int load_smem_Q_d = (tid % (kNumThreads / Br)) * (kHeadDim / (kNumThreads / Br));
3943     int load_smem_K_Bc = tid / (kNumThreads / Bc);
3944     int load_smem_K_d = (tid % (kNumThreads / Bc)) * (kHeadDim / (kNumThreads / Bc));
3945     int load_smem_V_Bc = tid / (kNumThreads / Bc);
3946     int load_smem_V_d = (tid % (kNumThreads / Bc)) * (kHeadDim / (kNumThreads / Bc));
3947
3948     int load_gmem_Q_Br = Q_tile_id * Br + load_smem_Q_Br;
3949     if (load_gmem_Q_Br >= N)
3950         return;
3951
3952     // Shared memory layout
3953     // Q/K/V independently use padded row-major when kPad* > 0 and compact XOR
3954     // swizzle when kPad* == 0. Padding and XOR are never combined per operand.
3955     // The swizzled physical layout is [col / 16][row][16]; swizzle<16>() selects
3956     // the 0/8 phase inside the final 16-half tile.
3957     extern __shared__ half smem[];
3958     constexpr int Q_tile_size = Br * (kHeadDim + kPadQ);
3959     constexpr int K_tile_size = Bc * (kHeadDim + kPadK);
3960     constexpr int kSmemStrideQ = kHeadDim + kPadQ;
3961     constexpr int kSmemStrideK = kHeadDim + kPadK;
3962     constexpr int kSmemStrideV = kHeadDim + kPadV;
3963     half *Q_tile_smem = smem;
3964     half *K_tile_smem = Q_tile_smem + Q_tile_size;
3965     half *V_tile_smem = K_tile_smem + kStagesK * K_tile_size;
3966
3967     uint32_t smem_Q_base_ptr = __cvta_generic_to_shared(Q_tile_smem);
3968     uint32_t smem_K_base_ptr = __cvta_generic_to_shared(K_tile_smem);

```

```

3969 uint32_t smem_V_base_ptr = __cvta_generic_to_shared(V_tile_smem);
3970
3971 // Online Softmax persistent state
3972 // lane_block_row_max_old[i][r]: running max for row r of warp tile i
3973 // lane_block_row_sum_old[i][r]: running denominator l for row r of warp tile i
3974 float lane_block_row_max_old[kValTileSeqLenQ][2];
3975 float lane_block_row_sum_old[kValTileSeqLenQ][2];
3976 fill_2D_regs<float, kValTileSeqLenQ, 2>(lane_block_row_max_old, -INFINITY);
3977 fill_2D_regs<float, kValTileSeqLenQ, 2>(lane_block_row_sum_old, 0.0f);
3978
3979 uint32_t R_Q[kValTileSeqLenQ][4]; // Q regs
3980 uint32_t R_K[kValTileSeqLenK][2]; // K regs
3981 uint32_t R_V[kValTileHeadDimV][2]; // V regs
3982 // R_S / R_O / R_D 都按 mma.sync.aligned.m16n8k16 的 fragment 约定存储。
3983 // 对单个 m16n8k16 tile 而言:
3984 //   - reg[0] 持有该 tile 前 8 行里的两个 half 值
3985 //   - reg[1] 持有该 tile 后 8 行里的两个 half 值
3986 // 后续 softmax、P@V、online rescale 都直接围绕这组 fragment 布局做寄存器内变换。
3987 uint32_t R_S[kValTileSeqLenQ][kValTileSeqLenK][2]; // S=Q@K^T / P=softmax(S)
3988 uint32_t R_O[kValTileSeqLenP][kValTileHeadDimV][2]; // O for current tile
3989 uint32_t R_D[kValTileSeqLenP][kValTileHeadDimV]
3990           [2]; // O accumulator (final output)
3991
3992 fill_3D_regs<uint32_t, kValTileSeqLenQ, kValTileSeqLenK, 2>(R_S, 0);
3993 fill_3D_regs<uint32_t, kValTileSeqLenP, kValTileHeadDimV, 2>(R_D, 0);
3994
3995 // =====
3996 // Step 1: 加载 Q[Br, d] 到 shared memory (整个外循环只加载一次)
3997 // =====
3998 {
3999     int load_gmem_Q_addr =
4000         Q_gmem_offset + load_gmem_Q_Br * kHeadDim + load_smem_Q_d;
4001     if constexpr (kSwizzleQ) {
4002 #pragma unroll
4003         for (int i = 0; i < (kHeadDim / (kNumThreads / Br)); i += 8) {
4004             int col = load_smem_Q_d + i;
4005             uint32_t ptr = smem_Q_base_ptr +
4006                 ((col / kMmaAtomK) * Br * kMmaAtomK +
4007                  load_smem_Q_Br * kMmaAtomK +
4008                  swizzle<kMmaAtomK>(load_smem_Q_Br, col % kMmaAtomK)) *
4009                 sizeof(half);
4010             CP_ASYNC_CG(ptr, &Q[load_gmem_Q_addr + i], 16);
4011         }
4012     } else {
4013         uint32_t load_smem_Q_ptr =
4014             smem_Q_base_ptr +
4015             (load_smem_Q_Br * kSmemStrideQ + load_smem_Q_d) * sizeof(half);
4016 #pragma unroll
4017         for (int i = 0; i < (kHeadDim / (kNumThreads / Br)); i += 8) {
4018             CP_ASYNC_CG(load_smem_Q_ptr + i * 2, &Q[load_gmem_Q_addr + i], 16);
4019         }
4020     }
4021     CP_ASYNC_COMMIT_GROUP();
4022 }
4023
4024 // =====
4025 // Step 2: 预加载前 (kStagesK-1) 个 K tile (多 stage pipeline 预热)
4026 // 注意: Q 由 blockIdx.x 固定到当前 Q tile; 而 K/V 的 seqLen 遍历始终从 tile 0 开始,
4027 // 后续在外循环里不断递增到 tile 1/2/3/.../Tc-1。
4028 // =====
4029 if constexpr (kStagesK > 1) {
4030 #pragma unroll
4031     for (int stage = 0; stage < (kStagesK - 1); ++stage) {

```

```

4032     int load_gmem_K_Bc = stage * Bc + load_smem_K_Bc;
4033     int load_gmem_K_addr =
4034         K_gmem_offset + load_gmem_K_Bc * kHeadDim + load_smem_K_d;
4035     if constexpr (kSwizzleK) {
4036 #pragma unroll
4037         for (int i = 0; i < (kHeadDim / (kNumThreads / Bc)); i += 8) {
4038             int col = load_smem_K_d + i;
4039             uint32_t ptr = smem_K_base_ptr +
4040                 (stage * K_tile_size +
4041                  (col / kMmaAtomK) * Bc * kMmaAtomK +
4042                  load_smem_K_Bc * kMmaAtomK +
4043                  swizzle<kMmaAtomK>(load_smem_K_Bc, col % kMmaAtomK)) *
4044                 sizeof(half);
4045             CP_ASYNC_CG(ptr, &K[load_gmem_K_addr + i], 16);
4046         }
4047     } else {
4048         uint32_t load_smem_K_ptr =
4049             smem_K_base_ptr +
4050             (stage * K_tile_size + load_smem_K_Bc * kSmemStrideK +
4051              load_smem_K_d) *
4052             sizeof(half);
4053 #pragma unroll
4054         for (int i = 0; i < (kHeadDim / (kNumThreads / Bc)); i += 8) {
4055             CP_ASYNC_CG(load_smem_K_ptr + i * 2, &K[load_gmem_K_addr + i], 16);
4056         }
4057     }
4058     CP_ASYNC_COMMIT_GROUP();
4059 }
4060 CP_ASYNC_WAIT_GROUP(kStagesK - 2);
4061 __syncthreads();
4062 }
4063
4064 // =====
4065 // Step 3: 外循环 — 沿 K seqlen 迭代 (Tc = ceil(seqlen/Bc))
4066 // 每次迭代处理一个 K[Bc,d] + V[Bc,d] tile
4067 // =====
4068 #pragma unroll 1
4069 for (int tile_K_seqlen = 0; tile_K_seqlen < Tc; ++tile_K_seqlen) {
4070     int smem_sel = tile_K_seqlen % kStagesK;
4071     int smem_sel_next = (tile_K_seqlen + (kStagesK - 1)) % kStagesK;
4072
4073     // ---- 3a: 异步加载 K/V tile (多 stage pipeline) ----
4074     if constexpr (kStagesK > 1) {
4075         // 只有 kStagesK>1 才能真正做 K 的 pipeline:
4076         // smem_sel 负责 “当前正在计算” 的 K tile, smem_sel_next 负责 “下一轮预取” 的 K tile。
4077         // 若 kStagesK=1, 这两个槽位永远都等于 0, 当前 K 还没算完就无法安全覆盖同一块 smem。
4078         // Load current V tile (no pipeline for V — one stage is enough)
4079         {
4080             int load_gmem_V_Bc = tile_K_seqlen * Bc + load_smem_V_Bc;
4081             int load_gmem_V_addr =
4082                 V_gmem_offset + load_gmem_V_Bc * kHeadDim + load_smem_V_d;
4083             if constexpr (kSwizzleV) {
4084 #pragma unroll
4085                 for (int i = 0; i < (kHeadDim / (kNumThreads / Bc)); i += 8) {
4086                     int col = load_smem_V_d + i;
4087                     uint32_t ptr = smem_V_base_ptr +
4088                         ((col / kMmaAtomK) * Bc * kMmaAtomK +
4089                          load_smem_V_Bc * kMmaAtomK +
4090                          swizzle<kMmaAtomK>(load_smem_V_Bc, col % kMmaAtomK)) *
4091                         sizeof(half);
4092                     CP_ASYNC_CG(ptr, &V[load_gmem_V_addr + i], 16);
4093                 }
4094             } else {

```

```

4095     uint32_t load_smem_V_ptr =
4096         smem_V_base_ptr +
4097         (load_smem_V_Bc * kSmemStrideV + load_smem_V_d) * sizeof(half);
4098 #pragma unroll
4099     for (int i = 0; i < (kHeadDim / (kNumThreads / Bc)); i += 8) {
4100         CP_ASYNC_CG(load_smem_V_ptr + i * 2, &V[load_gmem_V_addr + i], 16);
4101     }
4102 }
4103 CP_ASYNC_COMMIT_GROUP();
4104 }
4105
4106 // Prefetch next K tile (pipelined)
4107 if ((tile_K_seqlen + 1) < Tc) {
4108     int load_gmem_K_Bc = (tile_K_seqlen + 1) * Bc + load_smem_K_Bc;
4109     int load_gmem_K_addr =
4110         K_gmem_offset + load_gmem_K_Bc * kHeadDim + load_smem_K_d;
4111     if constexpr (kSwizzleK) {
4112 #pragma unroll
4113         for (int i = 0; i < (kHeadDim / (kNumThreads / Bc)); i += 8) {
4114             int col = load_smem_K_d + i;
4115             uint32_t ptr = smem_K_base_ptr +
4116                 (smem_sel_next * K_tile_size +
4117                  (col / kMmaAtomK) * Bc * kMmaAtomK +
4118                  load_smem_K_Bc * kMmaAtomK +
4119                  swizzle<kMmaAtomK>(load_smem_K_Bc, col % kMmaAtomK)) *
4120                 sizeof(half);
4121             CP_ASYNC_CG(ptr, &K[load_gmem_K_addr + i], 16);
4122         }
4123     } else {
4124         uint32_t load_smem_K_ptr =
4125             smem_K_base_ptr +
4126             (smem_sel_next * K_tile_size +
4127              load_smem_K_Bc * kSmemStrideK +
4128              load_smem_K_d) *
4129             sizeof(half);
4130 #pragma unroll
4131         for (int i = 0; i < (kHeadDim / (kNumThreads / Bc)); i += 8) {
4132             CP_ASYNC_CG(load_smem_K_ptr + i * 2, &K[load_gmem_K_addr + i], 16);
4133         }
4134     }
4135     CP_ASYNC_COMMIT_GROUP();
4136 }
4137 }
4138
4139 // 3b: Q@K^T = S[Br, Bc] — 沿 head dim (d/kMmaAtomK=16) 内循环
4140 fill_3D_regs<uint32_t, kValTileSeqLenQ, kValTileSeqLenK, 2>(R_S, 0);
4141 #pragma unroll
4142 for (int tile_K_d = 0; tile_K_d < (kHeadDim / kMmaAtomK); ++tile_K_d) {
4143     // ldmatrix.x4: 加载 Q 的 m16k16 片段到 R_Q
4144 #pragma unroll
4145     for (int i = 0; i < kValTileSeqLenQ; ++i) {
4146         int warp_smem_Q_Br =
4147             warp_QP * (kMmaAtomM * kValTileSeqLenQ) + i * kMmaAtomM;
4148         int lane_smem_Q_Br =
4149             warp_smem_Q_Br + lane_id % 16; // ldmatrix uses 16 lanes
4150         int lane_smem_Q_d = tile_K_d * kMmaAtomK + (lane_id / 16) * 8; // 0, 8
4151         uint32_t lane_smem_Q_ptr;
4152         if constexpr (kSwizzleQ) {
4153             lane_smem_Q_ptr = smem_Q_base_ptr +
4154                 ((lane_smem_Q_d / kMmaAtomK) * Br * kMmaAtomK +
4155                  lane_smem_Q_Br * kMmaAtomK +
4156                  swizzle<kMmaAtomK>(lane_smem_Q_Br,
4157                                     lane_smem_Q_d % kMmaAtomK)) *

```

```

4158         sizeof(half);
4159     } else {
4160         lane_smem_Q_ptr = smem_Q_base_ptr +
4161             (lane_smem_Q_Br * kSmemStrideQ + lane_smem_Q_d) * sizeof(half);
4162     }
4163     LDMATRIX_X4(R_Q[i][0], R_Q[i][1], R_Q[i][2], R_Q[i][3],
4164         lane_smem_Q_ptr);
4165 }
4166
4167 // ldmatrix.x2: 加载 K 的 k16n8 片段到 R_K
4168 // K[Bc,d] row-major = K^T[d,Bc] col-major (NT 布局的 B 矩阵)
4169 #pragma unroll
4170 for (int j = 0; j < kValTileSeqLenK; ++j) {
4171     int warp_smem_K_Bc =
4172         warp_KV * (kMmaAtomN * kValTileSeqLenK) + j * kMmaAtomN;
4173     int lane_smem_K_Bc =
4174         warp_smem_K_Bc + lane_id % 8; // ldmatrix B uses 8 lanes
4175     int lane_smem_K_d =
4176         tile_K_d * kMmaAtomK + ((lane_id / 8) % 2) * 8; // 0, 8
4177     uint32_t lane_smem_K_ptr;
4178     if constexpr (kSwizzleK) {
4179         lane_smem_K_ptr = smem_K_base_ptr +
4180             (smem_sel * K_tile_size +
4181             (lane_smem_K_d / kMmaAtomK) * Bc * kMmaAtomK +
4182             lane_smem_K_Bc * kMmaAtomK +
4183             swizzle<kMmaAtomK>(lane_smem_K_Bc,
4184                 lane_smem_K_d % kMmaAtomK)) *
4185                 sizeof(half);
4186     } else {
4187         lane_smem_K_ptr = smem_K_base_ptr +
4188             (smem_sel * K_tile_size + lane_smem_K_Bc * kSmemStrideK +
4189             lane_smem_K_d) *
4190             sizeof(half);
4191     }
4192     LDMATRIX_X2(R_K[j][0], R_K[j][1], lane_smem_K_ptr);
4193 }
4194
4195 // MMA: S[tile] += Q[tile] @ K^T[tile]
4196 #pragma unroll
4197 for (int i = 0; i < kValTileSeqLenQ; ++i) {
4198     #pragma unroll
4199     for (int j = 0; j < kValTileSeqLenK; ++j) {
4200         HMMMA16816(R_S[i][j][0], R_S[i][j][1], R_Q[i][0], R_Q[i][1], R_Q[i][2],
4201             R_Q[i][3], R_K[j][0], R_K[j][1], R_S[i][j][0],
4202             R_S[i][j][1]);
4203     }
4204 }
4205 } // end loop over d
4206 __syncthreads();
4207
4208 // =====
4209 // 3c: Online Safe Softmax — row-wise max + exp + sum, then store P back to R_S
4210 // =====
4211 // MMA C fragment layout for m16n8k16 (PTX ISA 对应的线程寄存器分布):
4212 // - R_S[i][j][0] 对应当前 16x8 tile 的 rows 0~7 里的两个 half 值 {c0,c1}
4213 // - R_S[i][j][1] 对应 rows 8~15 里的两个 half 值 {c2,c3}
4214 // - lane 0~3 持有 row 0 的片段, lane 4~7 持有 row 1, ..., lane 28~31 持有 row 7
4215 // - 对于 rows 8~15 也是同样的 lane 分组, 只是读取的是 reg[1]
4216 // 这就是为什么后面做 row max / row sum 时, warp 内真正参与同一行归约的是
4217 // {0,4,8,...,28} 这一类 4-lane 子组, 而不是整 warp 32 个线程。
4218 // Each (i, j) pair = one 16x8 MMA tile; there are kValTileSeqLenQ x
4219 // kValTileSeqLenK tiles.
4220

```

```

4221 float lane_row_max_new[kValTileSeqLenQ][2];
4222 float lane_row_sum_new[kValTileSeqLenQ][2];
4223 fill_2D_regs<float, kValTileSeqLenQ, 2>(lane_row_max_new, -INFINITY);
4224 fill_2D_regs<float, kValTileSeqLenQ, 2>(lane_row_sum_new, 0.0f);
4225
4226 // Pass 1: Thread-level reduce max across all columns (kValTileSeqLenK tiles)
4227 #pragma unroll
4228 for (int i = 0; i < kValTileSeqLenQ; ++i) {
4229 #pragma unroll
4230 for (int j = 0; j < kValTileSeqLenK; ++j) {
4231     // Extract half values from R_S registers (C matrix fragment layout)
4232     float2 t_reg_S_0 =
4233         __half22float2(HALF2(R_S[i][j][0])); // rows 0~7: {c0, c1}
4234     float2 t_reg_S_1 =
4235         __half22float2(HALF2(R_S[i][j][1])); // rows 8~15: {c2, c3}
4236     // S = (Q@K^T) * scale
4237     float tmp_max_0 = max(t_reg_S_0.x, t_reg_S_0.y) * scale;
4238     float tmp_max_1 = max(t_reg_S_1.x, t_reg_S_1.y) * scale;
4239     lane_row_max_new[i][0] = max(lane_row_max_new[i][0], tmp_max_0);
4240     lane_row_max_new[i][1] = max(lane_row_max_new[i][1], tmp_max_1);
4241 }
4242 // Warp-level reduce max (kWarpWidth = 4 for Q@K^T — only lanes
4243 // {0,4,8,...,28} hold valid data)
4244 lane_row_max_new[i][0] =
4245     warp_reduce_max<4, float>(lane_row_max_new[i][0]);
4246 lane_row_max_new[i][1] =
4247     warp_reduce_max<4, float>(lane_row_max_new[i][1]);
4248 }
4249
4250 // Pass 2: Compute P = exp(S*scale - m_new), store back to R_S
4251 // 面试关键点: 这里将 P 写回 R_S 寄存器!
4252 // 为什么可以? 当前实现依赖 m16n8k16 这一路径下约定好的 fragment 布局,
4253 // 使 softmax 后的 P 能继续留在 R_S 中供后面的 P@V 直接消费, 无需额外重组。
4254 #pragma unroll
4255 for (int i = 0; i < kValTileSeqLenQ; ++i) {
4256     // m_new = max(m_old, m_cur)
4257     float block_row_max_new_0 =
4258         max(lane_block_row_max_old[i][0], lane_row_max_new[i][0]);
4259     float block_row_max_new_1 =
4260         max(lane_block_row_max_old[i][1], lane_row_max_new[i][1]);
4261
4262 #pragma unroll
4263 for (int j = 0; j < kValTileSeqLenK; ++j) {
4264     float2 t_reg_S_0 = __half22float2(HALF2(R_S[i][j][0]));
4265     float2 t_reg_S_1 = __half22float2(HALF2(R_S[i][j][1]));
4266
4267     // P = exp(S * scale - m_new), 用 fma 保证精度
4268     t_reg_S_0.x =
4269         __expf(__fmaf_rn(t_reg_S_0.x, scale, -block_row_max_new_0));
4270     t_reg_S_0.y =
4271         __expf(__fmaf_rn(t_reg_S_0.y, scale, -block_row_max_new_0));
4272     t_reg_S_1.x =
4273         __expf(__fmaf_rn(t_reg_S_1.x, scale, -block_row_max_new_1));
4274     t_reg_S_1.y =
4275         __expf(__fmaf_rn(t_reg_S_1.y, scale, -block_row_max_new_1));
4276
4277     // Accumulate row sums
4278     lane_row_sum_new[i][0] += (t_reg_S_0.x + t_reg_S_0.y);
4279     lane_row_sum_new[i][1] += (t_reg_S_1.x + t_reg_S_1.y);
4280
4281     // 关键: 将 P 写回 R_S! R_S 现在存储的是 P = softmax(S), 不是 S
4282     HALF2(R_S[i][j][0]) = __float22half2_rn(t_reg_S_0);
4283     HALF2(R_S[i][j][1]) = __float22half2_rn(t_reg_S_1);

```



```

4284     }
4285
4286     // Warp-level reduce sum (kWarpWidth = 4, same as max)
4287     lane_row_sum_new[i][0] =
4288         warp_reduce_sum<4, float>(lane_row_sum_new[i][0]);
4289     lane_row_sum_new[i][1] =
4290         warp_reduce_sum<4, float>(lane_row_sum_new[i][1]);
4291 }
4292 __syncthreads();
4293
4294 // =====
4295 // 3d: P@V — P[Br,Bc] @ V[Bc,d] = O[Br,d]
4296 // =====
4297 // Wait for V to be ready before computing P@V
4298 if constexpr (kStagesK > 1) {
4299     if ((tile_K_seqlen + 1) < Tc) {
4300         CP_ASYNC_WAIT_GROUP(1);
4301     } else {
4302         CP_ASYNC_WAIT_GROUP(0);
4303     }
4304 } else {
4305     CP_ASYNC_WAIT_GROUP(0);
4306 }
4307 __syncthreads();
4308
4309 fill_3D_regs<uint32_t, kValTileSeqLenP, kValTileHeadDimV, 2>(R_0, 0);
4310
4311 // tile_V_Bc: iterate over chunks of Bc/K=16 columns in P matrix
4312 // Bc=kMmaAtomK=16 → 1 iteration for kHeadDim≤64 configurations
4313 #pragma unroll
4314 for (int tile_V_Bc = 0; tile_V_Bc < (Bc / kMmaAtomK); ++tile_V_Bc) {
4315     // ldmatrix.x2.trans: load V[Bc,d] with transposition
4316     // V is row-major [Bc,d], but NN matmul needs B matrix in col-major → use
4317     // transposed ldmatrix
4318 #pragma unroll
4319 for (int j = 0; j < kValTileHeadDimV; ++j) {
4320     int warp_smem_V_d =
4321         warp_KV * (kMmaAtomN * kValTileHeadDimV) + j * kMmaAtomN;
4322     int lane_smem_V_Bc = tile_V_Bc * kMmaAtomK + lane_id % 16;
4323     int lane_smem_V_d = warp_smem_V_d;
4324     uint32_t lane_smem_V_ptr;
4325     if constexpr (kSwizzleV) {
4326         lane_smem_V_ptr = smem_V_base_ptr +
4327             ((lane_smem_V_d / kMmaAtomK) * Bc * kMmaAtomK +
4328              lane_smem_V_Bc * kMmaAtomK +
4329              swizzle<kMmaAtomK>(lane_smem_V_Bc,
4330                               lane_smem_V_d % kMmaAtomK)) *
4331             sizeof(half);
4332     } else {
4333         lane_smem_V_ptr = smem_V_base_ptr +
4334             (lane_smem_V_Bc * kSmemStrideV + lane_smem_V_d) * sizeof(half);
4335     }
4336     LDMATRIX_X2_T(R_V[j][0], R_V[j][1], lane_smem_V_ptr);
4337 }
4338
4339 // P matrix layout in R_S[i][j][2]:
4340 //   MMA = m16n8k16, Br=16x4=64, Bc=8x8=64, layout: 4 warps
4341 //   | 64x64 | warp_KV 0 |
4342 //   | warp_QP 0 | MMA 0 ... MMA 0 (x8) |
4343 //   | warp_QP 1 | MMA 1 ... MMA 1 (x8) |
4344 //   | warp_QP 2 | MMA 2 ... MMA 2 (x8) |
4345 //   | warp_QP 3 | MMA 3 ... MMA 3 (x8) |
4346 // tile_V_Bc selects which 16-column slice of P to use:

```

```

4347 // tile_V_Bc=0 → cols 0:16 → MMA indices 0,1 → w=0
4348 // tile_V_Bc=1 → cols 16:32 → MMA indices 2,3 → w=2
4349 // tile_V_Bc=2 → cols 32:48 → MMA indices 4,5 → w=4
4350 // tile_V_Bc=3 → cols 48:64 → MMA indices 6,7 → w=6
4351 // 对应的 MMA A fragment 布局可以这样记:
4352 // - rows 0~7: lane 0~3 → {a0,a1} 与 {a4,a5}, lane 4~7 → 下一行, 依次类推
4353 // - rows 8~15: lane 0~3 → {a2,a3} 与 {a6,a7}, lane 4~7 → 下一行, 依次类推
4354 // 当前实现正是利用这一路径下 A fragment 与前面生成的 P fragment 可以直接对接,
4355 // 才能把 R_S 中的 P 直接喂给 HMMA16816 做 P@V; 复习时不要把它背成对所有 MMA
4356 // fragment 都无条件成立的通用结论。layout转换逻辑:
4357 // C layout: 8 x (16 x 8) layout → A layout: 4 x (16 x 16) layout
4358 int w = tile_V_Bc * 2;
4359 #pragma unroll
4360 for (int i = 0; i < kValTileSeqLenP; ++i) {
4361 #pragma unroll
4362 for (int j = 0; j < kValTileHeadDimV; ++j) {
4363     HMMA16816(R_0[i][j][0], R_0[i][j][1], // C fragment output
4364              R_S[i][w][0], R_S[i][w][1], R_S[i][w+1][0], R_S[i][w+1][1], // A fragment = P
4365              R_V[j][0], R_V[j][1], // B fragment = V
4366              R_0[i][j][0], R_0[i][j][1]); // C fragment output
4367 }
4368 }
4369 } // end for tile_V_Bc
4370 __syncthreads();
4371
4372 // =====
4373 // 3e: Online rescaling — 0_new = exp(m_old - m_new) * 0_old + P@V
4374 // =====
4375 // 公式来源: FA2 paper Eq.(7-8), 使用 exp(m_old - m_new) 做 0 与 l 的 rescale
4376 #pragma unroll
4377 for (int i = 0; i < kValTileSeqLenP; ++i) {
4378     float block_row_max_new_0 = lane_row_max_new[i][0];
4379     float block_row_max_new_1 = lane_row_max_new[i][1];
4380     float block_row_sum_new_0 = lane_row_sum_new[i][0];
4381     float block_row_sum_new_1 = lane_row_sum_new[i][1];
4382
4383     float block_row_max_old_0 = lane_block_row_max_old[i][0];
4384     float block_row_max_old_1 = lane_block_row_max_old[i][1];
4385
4386     block_row_max_new_0 = max(block_row_max_old_0, block_row_max_new_0);
4387     block_row_max_new_1 = max(block_row_max_old_1, block_row_max_new_1);
4388
4389     // Handle first iteration: m_old = -inf, need to use m_new directly
4390     block_row_max_old_0 =
4391         (tile_K_seqlen > 0 ? block_row_max_old_0 : block_row_max_new_0);
4392     block_row_max_old_1 =
4393         (tile_K_seqlen > 0 ? block_row_max_old_1 : block_row_max_new_1);
4394
4395     float rescale_o_factor_0 =
4396         __expf(block_row_max_old_0 - block_row_max_new_0);
4397     float rescale_o_factor_1 =
4398         __expf(block_row_max_old_1 - block_row_max_new_1);
4399
4400     // Rescale 0_old + Add P@V in one fused step
4401 #pragma unroll
4402 for (int j = 0; j < kValTileHeadDimV; ++j) {
4403     // R_0 / R_D 与前面的 R_S 一样, 都按 MMA C fragment 布局解释:
4404     // reg[0] → rows 0~7 的 {c0,c1}
4405     // reg[1] → rows 8~15 的 {c2,c3}
4406     float2 t_reg_0_0 = __half22float2(HALF2(R_0[i][j][0]));
4407     float2 t_reg_0_1 = __half22float2(HALF2(R_0[i][j][1]));
4408     float2 t_reg_D_0 = __half22float2(HALF2(R_D[i][j][0]));
4409     float2 t_reg_D_1 = __half22float2(HALF2(R_D[i][j][1]));

```

```

4410 // 0_new = exp(m_old - m_new) * 0_old + P@V (fused multiply-add)
4411 t_reg_D_0.x = __fmaf_rn(rescale_o_factor_0, t_reg_D_0.x, t_reg_0_0.x);
4412 t_reg_D_0.y = __fmaf_rn(rescale_o_factor_0, t_reg_D_0.y, t_reg_0_0.y);
4413 t_reg_D_1.x = __fmaf_rn(rescale_o_factor_1, t_reg_D_1.x, t_reg_0_1.x);
4414 t_reg_D_1.y = __fmaf_rn(rescale_o_factor_1, t_reg_D_1.y, t_reg_0_1.y);
4415
4416 HALF2(R_D[i][j][0]) = __float22half2_rn(t_reg_D_0);
4417 HALF2(R_D[i][j][1]) = __float22half2_rn(t_reg_D_1);
4418 }
4419
4420 // Update l: l_new = exp(m_old - m_new) * l_old + row_sum(P)
4421 float block_row_sum_old_0 = lane_block_row_sum_old[i][0];
4422 float block_row_sum_old_1 = lane_block_row_sum_old[i][1];
4423 lane_block_row_sum_old[i][0] = __fmaf_rn(
4424     rescale_o_factor_0, block_row_sum_old_0, block_row_sum_new_0);
4425 lane_block_row_sum_old[i][1] = __fmaf_rn(
4426     rescale_o_factor_1, block_row_sum_old_1, block_row_sum_new_1);
4427
4428 // Update m
4429 lane_block_row_max_old[i][0] = block_row_max_new_0;
4430 lane_block_row_max_old[i][1] = block_row_max_new_1;
4431 }
4432
4433 // Wait for next K tile to be ready in smem before next iteration
4434 if constexpr(kStagesK > 1) {
4435     if ((tile_K_seqlen + 1) < Tc) {
4436         CP_ASYNC_WAIT_GROUP(0);
4437     }
4438     __syncthreads();
4439 }
4440 } // end outer loop over K seqlen
4441 __syncthreads();
4442
4443 // =====
4444 // Step 4: 最终 rescale — 0_final = (1/l_final) * 0_final
4445 // =====
4446 #pragma unroll
4447 for (int i = 0; i < kValTileSeqLenP; ++i) {
4448     float rescale_factor_0 = __frcp_rn(lane_block_row_sum_old[i][0]);
4449     float rescale_factor_1 = __frcp_rn(lane_block_row_sum_old[i][1]);
4450 #pragma unroll
4451 for (int j = 0; j < kValTileHeadDimV; ++j) {
4452     float2 t_reg_D_0 = __half22float2(HALF2(R_D[i][j][0]));
4453     float2 t_reg_D_1 = __half22float2(HALF2(R_D[i][j][1]));
4454     t_reg_D_0.x = rescale_factor_0 * t_reg_D_0.x;
4455     t_reg_D_0.y = rescale_factor_0 * t_reg_D_0.y;
4456     t_reg_D_1.x = rescale_factor_1 * t_reg_D_1.x;
4457     t_reg_D_1.y = rescale_factor_1 * t_reg_D_1.y;
4458     HALF2(R_D[i][j][0]) = __float22half2_rn(t_reg_D_0);
4459     HALF2(R_D[i][j][1]) = __float22half2_rn(t_reg_D_1);
4460 }
4461 }
4462
4463 // =====
4464 // Step 5: Epilogue — Collective store via warp shuffle + 128-bit store
4465 // =====
4466 // 利用 warp shuffle 将分散在各 lane 的寄存器数据收集到 lane 0~3,
4467 // 然后用 LDST128BITS (st.global.v4.f32) 一次性写入 16 bytes
4468 #pragma unroll
4469 for (int i = 0; i < kValTileSeqLenP; ++i) {
4470 #pragma unroll
4471 for (int j = 0; j < kValTileHeadDimV; ++j) {

```

```

4473     uint32_t R_Z[2][4];
4474     R_Z[0][0] = R_D[i][j][0];
4475     R_Z[1][0] = R_D[i][j][1];
4476     R_Z[0][1] = __shfl_sync(0xffffffff, R_D[i][j][0], lane_id + 1, 4);
4477     R_Z[0][2] = __shfl_sync(0xffffffff, R_D[i][j][0], lane_id + 2, 4);
4478     R_Z[0][3] = __shfl_sync(0xffffffff, R_D[i][j][0], lane_id + 3, 4);
4479     R_Z[1][1] = __shfl_sync(0xffffffff, R_D[i][j][1], lane_id + 1, 4);
4480     R_Z[1][2] = __shfl_sync(0xffffffff, R_D[i][j][1], lane_id + 2, 4);
4481     R_Z[1][3] = __shfl_sync(0xffffffff, R_D[i][j][1], lane_id + 3, 4);
4482
4483     // st.global.v4.f32: 128-bit store, 4 lanes × 32-bit
4484     if (lane_id % 4 == 0) {
4485         int store_warp_regs_0_Br =
4486             warp_QP * (kMmaAtomM * kValTileSeqLenP) + i * kMmaAtomM;
4487         int store_lane_gmem_0_Br =
4488             Q_tile_id * Br + store_warp_regs_0_Br + lane_id / 4;
4489         int store_warp_regs_0_d =
4490             warp_KV * (kMmaAtomN * kValTileHeadDimV) + j * kMmaAtomN;
4491         int store_lane_gmem_0_d = store_warp_regs_0_d;
4492         int store_gmem_0_addr_0 = 0_gmem_offset +
4493             (store_lane_gmem_0_Br + 0) * kHeadDim +
4494             store_lane_gmem_0_d;
4495         int store_gmem_0_addr_1 = 0_gmem_offset +
4496             (store_lane_gmem_0_Br + 8) * kHeadDim +
4497             store_lane_gmem_0_d;
4498         // LDST128BITS = reinterpret_cast<float4*>
4499         *reinterpret_cast<float4*>(&0[store_gmem_0_addr_0]) =
4500             *reinterpret_cast<float4*>(&R_Z[0][0]);
4501         *reinterpret_cast<float4*>(&0[store_gmem_0_addr_1]) =
4502             *reinterpret_cast<float4*>(&R_Z[1][0]);
4503     }
4504 }
4505 }
4506 }
4507
4508 // =====
4509 // 以下是测试代码, 验证 Phase 1 - Phase 8 的kernel的正确性, 不评估性能。
4510 // =====
4511
4512 static inline void check(cudaError_t err, const char *msg) {
4513     if (err != cudaSuccess) {
4514         fprintf(stderr, "[ERROR] %s: %s\n", msg, cudaGetErrorString(err));
4515         exit(EXIT_FAILURE);
4516     }
4517 }
4518
4519
4520 static void test_block_reduce(int N) {
4521
4522     srand(42);
4523     float *h_a = (float *)malloc((size_t)N * sizeof(float));
4524     for (int i = 0; i < N; i++)
4525         h_a[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
4526
4527     // CPU reference: sum of all elements
4528     double ref = 0.0;
4529     for (int i = 0; i < N; i++) ref += (double)h_a[i];
4530
4531     float *d_a, *d_y;
4532     check(cudaMalloc(&d_a, (size_t)N * sizeof(float)), "blockreduce alloc A");
4533     check(cudaMalloc(&d_y, sizeof(float)), "blockreduce alloc Y");
4534
4535     check(cudaMemcpy(d_a, h_a, (size_t)N * sizeof(float), cudaMemcpyHostToDevice), "blockreduce H2D A");

```

```

4536 check(cudaMemset(d_y, 0, sizeof(float)), "blockreduce zero Y");
4537
4538 dim3 block(128);
4539 dim3 grid((N + 127) / 128);
4540 block_reduce_all<128><<<grid, block>>>(d_a, d_y, N);
4541 check(cudaGetLastError(), "blockreduce launch");
4542 check(cudaDeviceSynchronize(), "blockreduce sync");
4543
4544 float result;
4545 check(cudaMemcpy(&result, d_y, sizeof(float), cudaMemcpyDeviceToHost), "blockreduce D2H");
4546
4547 float err = fabsf(result - (float)ref);
4548 printf("| %-42s | %.6e | %-4s | %-8s |\n", "BlockReduce", err,
4549     err < 1e-2f ? "PASS" : "FAIL", "None");
4550
4551 free(h_a);
4552 cudaFree(d_a); cudaFree(d_y);
4553 }
4554
4555
4556 static void test_dot(int N) {
4557
4558     srand(42);
4559     float *h_a = (float *)malloc((size_t)N * sizeof(float));
4560     float *h_b = (float *)malloc((size_t)N * sizeof(float));
4561     for (int i = 0; i < N; i++) {
4562         h_a[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
4563         h_b[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
4564     }
4565
4566     // CPU reference
4567     double ref = 0.0;
4568     for (int i = 0; i < N; i++) ref += (double)h_a[i] * (double)h_b[i];
4569
4570     float *d_a, *d_b, *d_y;
4571     check(cudaMalloc(&d_a, (size_t)N * sizeof(float)), "dot alloc A");
4572     check(cudaMalloc(&d_b, (size_t)N * sizeof(float)), "dot alloc B");
4573     check(cudaMalloc(&d_y, sizeof(float)), "dot alloc Y");
4574
4575     check(cudaMemcpy(d_a, h_a, (size_t)N * sizeof(float), cudaMemcpyHostToDevice), "dot H2D A");
4576     check(cudaMemcpy(d_b, h_b, (size_t)N * sizeof(float), cudaMemcpyHostToDevice), "dot H2D B");
4577     check(cudaMemset(d_y, 0, sizeof(float)), "dot zero Y");
4578
4579     dim3 block(128);
4580     dim3 grid((N + 127) / 128);
4581     dot<128><<<grid, block>>>(d_a, d_b, d_y, N);
4582     check(cudaGetLastError(), "dot launch");
4583     check(cudaDeviceSynchronize(), "dot sync");
4584
4585     float result;
4586     check(cudaMemcpy(&result, d_y, sizeof(float), cudaMemcpyDeviceToHost), "dot D2H");
4587
4588     float err = fabsf(result - (float)ref);
4589     printf("| %-42s | %.6e | %-4s | %-8s |\n", "Dot", err,
4590         err < 1e-2f ? "PASS" : "FAIL", "None");
4591
4592     // ---- Dot Vec4 ----
4593     check(cudaMemset(d_y, 0, sizeof(float)), "dot_vec4 zero Y");
4594     dim3 block_v4(32);
4595     dot_vec4<32><<<grid, block_v4>>>(d_a, d_b, d_y, N);
4596     check(cudaGetLastError(), "dot_vec4 launch");
4597     check(cudaDeviceSynchronize(), "dot_vec4 sync");
4598

```

```

4599 check(cudaMemcpy(&result, d_y, sizeof(float), cudaMemcpyDeviceToHost), "dot_vec4 D2H");
4600 float err_v4 = fabsf(result - (float)ref);
4601 printf("| %-42s | %.6e | %-4s | %-8s |\n", "Dot-Vec4", err_v4, err_v4 < 1e-2f ? "PASS" : "FAIL", "None");
4602
4603 free(h_a); free(h_b);
4604 cudaFree(d_a); cudaFree(d_b); cudaFree(d_y);
4605 }
4606
4607
4608 static void test_relu(int N) {
4609     srand(42);
4610     float *h_x = (float *)malloc((size_t)N * sizeof(float));
4611     float *h_y = (float *)malloc((size_t)N * sizeof(float));
4612     for (int i = 0; i < N; i++)
4613         h_x[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
4614
4615     float *d_x, *d_y;
4616     check(cudaMalloc(&d_x, (size_t)N * sizeof(float)), "relu alloc X");
4617     check(cudaMalloc(&d_y, (size_t)N * sizeof(float)), "relu alloc Y");
4618     check(cudaMemcpy(d_x, h_x, (size_t)N * sizeof(float), cudaMemcpyHostToDevice), "relu H2D");
4619
4620     for (int i = 0; i < N; i++) h_y[i] = fmaxf(0.0f, h_x[i]);
4621     dim3 block256(256);
4622     dim3 grid256((N + 255) / 256);
4623     relu<<<grid256, block256>>>(d_x, d_y, N);
4624     check(cudaGetLastError(), "relu launch");
4625     check(cudaDeviceSynchronize(), "relu sync");
4626     check(cudaMemcpy(h_y, d_y, (size_t)N * sizeof(float), cudaMemcpyDeviceToHost), "relu D2H");
4627     float max_err = 0.0f;
4628     for (int i = 0; i < N; i++) {
4629         float expected = fmaxf(0.0f, h_x[i]);
4630         float err = fabsf(h_y[i] - expected);
4631         if (err > max_err) max_err = err;
4632     }
4633     printf("| %-42s | %.6e | %-4s | %-8s |\n", "ReLU", max_err, max_err < 1e-4f ? "PASS" : "FAIL", "None");
4634
4635     dim3 block64(64);
4636     relu_vec4<<<grid256, block64>>>(d_x, d_y, N);
4637     check(cudaGetLastError(), "relu_vec4 launch");
4638     check(cudaDeviceSynchronize(), "relu_vec4 sync");
4639     check(cudaMemcpy(h_y, d_y, (size_t)N * sizeof(float), cudaMemcpyDeviceToHost), "relu_vec4 D2H");
4640     max_err = 0.0f;
4641     for (int i = 0; i < N; i++) {
4642         float expected = fmaxf(0.0f, h_x[i]);
4643         float err = fabsf(h_y[i] - expected);
4644         if (err > max_err) max_err = err;
4645     }
4646     printf("| %-42s | %.6e | %-4s | %-8s |\n", "ReLU-Vec4", max_err, max_err < 1e-4f ? "PASS" : "FAIL", "None");
4647
4648     free(h_x); free(h_y);
4649     cudaFree(d_x); cudaFree(d_y);
4650 }
4651
4652
4653 static void test_elementwise(int N) {
4654     srand(42);
4655     float *h_a = (float *)malloc((size_t)N * sizeof(float));
4656     float *h_b = (float *)malloc((size_t)N * sizeof(float));
4657     for (int i = 0; i < N; i++) {
4658         h_a[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
4659         h_b[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
4660     }

```



```

4661 float *d_a, *d_b, *d_c;
4662 check(cudaMalloc(&d_a, (size_t)N * sizeof(float)), "eadd alloc A");
4663 check(cudaMalloc(&d_b, (size_t)N * sizeof(float)), "eadd alloc B");
4664 check(cudaMalloc(&d_c, (size_t)N * sizeof(float)), "eadd alloc C");
4665 check(cudaMemcpy(d_a, h_a, (size_t)N * sizeof(float), cudaMemcpyHostToDevice), "eadd H2D A");
4666 check(cudaMemcpy(d_b, h_b, (size_t)N * sizeof(float), cudaMemcpyHostToDevice), "eadd H2D B");
4667
4668
4669 dim3 block256(256);
4670 dim3 grid256((N + 255) / 256);
4671 elementwise_add<<<grid256, block256>>>(d_a, d_b, d_c, N);
4672 check(cudaGetLastError(), "eadd launch");
4673 check(cudaDeviceSynchronize(), "eadd sync");
4674 float *h_c = (float *)malloc((size_t)N * sizeof(float));
4675 check(cudaMemcpy(h_c, d_c, (size_t)N * sizeof(float), cudaMemcpyDeviceToHost), "eadd D2H");
4676 float max_err = 0.0f;
4677 for (int i = 0; i < N; i++) {
4678     float err = fabsf(h_c[i] - (h_a[i] + h_b[i]));
4679     if (err > max_err) max_err = err;
4680 }
4681 printf("| %-42s | %.6e | %-4s | %-8s |\n", "ElemwiseAdd", max_err, max_err < 1e-4f ? "PASS" : "FAIL", "
    None");
4682
4683 dim3 block64(64);
4684 check(cudaMemset(d_c, 0, (size_t)N * sizeof(float)), "eadd_vec4 zero C");
4685 elementwise_add_vec4<<<grid256, block64>>>(d_a, d_b, d_c, N);
4686 check(cudaGetLastError(), "eadd_vec4 launch");
4687 check(cudaDeviceSynchronize(), "eadd_vec4 sync");
4688 check(cudaMemcpy(h_c, d_c, (size_t)N * sizeof(float), cudaMemcpyDeviceToHost), "eadd_vec4 D2H");
4689 max_err = 0.0f;
4690 for (int i = 0; i < N; i++) {
4691     float err = fabsf(h_c[i] - (h_a[i] + h_b[i]));
4692     if (err > max_err) max_err = err;
4693 }
4694 printf("| %-42s | %.6e | %-4s | %-8s |\n", "ElemwiseAdd-Vec4", max_err, max_err < 1e-4f ? "PASS" : "FAIL", "
    None");
4695
4696 free(h_a); free(h_b); free(h_c);
4697 cudaFree(d_a); cudaFree(d_b); cudaFree(d_c);
4698 }
4699
4700
4701 static void test_histogram(int N) {
4702     srand(42);
4703     int BINS = 16;
4704     int *h_hist = (int *)calloc(BINS, sizeof(int));
4705     int *h_hist_ref = (int *)calloc(BINS, sizeof(int));
4706     int *h_idx = (int *)malloc((size_t)N * sizeof(int));
4707     for (int i = 0; i < N; i++) h_idx[i] = rand() % BINS;
4708     for (int i = 0; i < N; i++) h_hist_ref[h_idx[i]]++;
4709
4710     int *d_idx, *d_hist;
4711     check(cudaMalloc(&d_idx, (size_t)N * sizeof(int)), "hist alloc idx");
4712     check(cudaMalloc(&d_hist, BINS * sizeof(int)), "hist alloc hist");
4713     check(cudaMemcpy(d_idx, h_idx, (size_t)N * sizeof(int), cudaMemcpyHostToDevice), "hist H2D idx");
4714     check(cudaMemset(d_hist, 0, BINS * sizeof(int)), "hist zero");
4715
4716     dim3 block256(256);
4717     dim3 grid256((N + 255) / 256);
4718     histogram<<<grid256, block256>>>(d_idx, d_hist, N);
4719     check(cudaGetLastError(), "histogram launch");
4720     check(cudaDeviceSynchronize(), "histogram sync");
4721     check(cudaMemcpy(h_hist, d_hist, BINS * sizeof(int), cudaMemcpyDeviceToHost), "hist D2H");

```

```

4722 float max_err = 0.0f;
4723 for (int i = 0; i < BINS; i++) {
4724     float err = fabsf((float)(h_hist[i] - h_hist_ref[i]));
4725     if (err > max_err) max_err = err;
4726 }
4727 printf("| %-42s | %.6e | %-4s | %-8s |\n", "Histogram", max_err, max_err < 1e-4f ? "PASS" : "FAIL", "None"
4728 );
4729
4730 free(h_hist); free(h_hist_ref); free(h_idx);
4731 cudaFree(d_idx); cudaFree(d_hist);
4732 }
4733
4734
4735 static void test_merge_attn_states(int num_tokens, int num_heads,
4736                                   int head_size) {
4737
4738     size_t out_size = (size_t)num_tokens * num_heads * head_size * sizeof(float);
4739     size_t lse_size = (size_t)num_heads * num_tokens * sizeof(float);
4740
4741     float *h_prefix_out = (float *)malloc(out_size);
4742     float *h_suffix_out = (float *)malloc(out_size);
4743     float *h_prefix_lse = (float *)malloc(lse_size);
4744     float *h_suffix_lse = (float *)malloc(lse_size);
4745     float *h_output_ref = (float *)malloc(out_size);
4746
4747     srand(42);
4748     for (int i = 0; i < num_tokens * num_heads * head_size; i++) {
4749         h_prefix_out[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
4750         h_suffix_out[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
4751     }
4752     for (int i = 0; i < num_heads * num_tokens; i++) {
4753         h_prefix_lse[i] = ((float)rand() / RAND_MAX) * 20.0f - 10.0f;
4754         h_suffix_lse[i] = ((float)rand() / RAND_MAX) * 20.0f - 10.0f;
4755     }
4756
4757     // CPU reference: 与 kernel 完全相同的浮点计算
4758     for (int t = 0; t < num_tokens; t++) {
4759         for (int h = 0; h < num_heads; h++) {
4760             float p_lse = h_prefix_lse[h * num_tokens + t];
4761             float s_lse = h_suffix_lse[h * num_tokens + t];
4762             p_lse = isinf(p_lse) ? -INFINITY : p_lse;
4763             s_lse = isinf(s_lse) ? -INFINITY : s_lse;
4764
4765             float max_lse = fmaxf(p_lse, s_lse);
4766             p_lse -= max_lse;
4767             s_lse -= max_lse;
4768             float p_se = expf(p_lse);
4769             float s_se = expf(s_lse);
4770             float p_scale = p_se / (p_se + s_se);
4771             float s_scale = s_se / (p_se + s_se);
4772
4773             int head_off = t * num_heads * head_size + h * head_size;
4774             for (int d = 0; d < head_size; d++) {
4775                 h_output_ref[head_off + d] =
4776                     h_prefix_out[head_off + d] * p_scale +
4777                     h_suffix_out[head_off + d] * s_scale;
4778             }
4779         }
4780     }
4781
4782     float *d_prefix_out, *d_suffix_out, *d_prefix_lse, *d_suffix_lse, *d_output;
4783     check(cudaMalloc(&d_prefix_out, out_size), "merge_attn alloc p_out");

```

```

4784 check(cudaMalloc(&d_suffix_out, out_size), "merge_attn alloc s_out");
4785 check(cudaMalloc(&d_prefix_lse, lse_size), "merge_attn alloc p_lse");
4786 check(cudaMalloc(&d_suffix_lse, lse_size), "merge_attn alloc s_lse");
4787 check(cudaMalloc(&d_output, out_size), "merge_attn alloc output");
4788
4789 check(cudaMemcpy(d_prefix_out, h_prefix_out, out_size,
4790                 cudaMemcpyHostToDevice),
4791         "merge_attn H2D p_out");
4792 check(cudaMemcpy(d_suffix_out, h_suffix_out, out_size,
4793                 cudaMemcpyHostToDevice),
4794         "merge_attn H2D s_out");
4795 check(cudaMemcpy(d_prefix_lse, h_prefix_lse, lse_size,
4796                 cudaMemcpyHostToDevice),
4797         "merge_attn H2D p_lse");
4798 check(cudaMemcpy(d_suffix_lse, h_suffix_lse, lse_size,
4799                 cudaMemcpyHostToDevice),
4800         "merge_attn H2D s_lse");
4801
4802 int threads_per_head = head_size / 4;
4803 int total_threads = num_tokens * num_heads * threads_per_head;
4804 dim3 block(128);
4805 dim3 grid((total_threads + 127) / 128);
4806 merge_attn_states<<<grid, block>>>(
4807     d_output, d_prefix_out, d_prefix_lse, d_suffix_out, d_suffix_lse,
4808     num_tokens, num_heads, head_size);
4809 check(cudaGetLastError(), "merge_attn launch");
4810 check(cudaDeviceSynchronize(), "merge_attn sync");
4811
4812 float *h_output = (float *)malloc(out_size);
4813 check(cudaMemcpy(h_output, d_output, out_size, cudaMemcpyDeviceToHost),
4814         "merge_attn D2H");
4815
4816 float max_err = 0.0f;
4817 for (int i = 0; i < num_tokens * num_heads * head_size; i++) {
4818     float err = fabsf(h_output[i] - h_output_ref[i]);
4819     if (err > max_err) max_err = err;
4820 }
4821 printf("| %-42s | %.6e | %-4s | %-8s |\n", "MergeAttnStates", max_err,
4822        max_err < 1e-4f ? "PASS" : "FAIL", "None");
4823
4824 // 边界测试: +inf LSE → 权重退化为 0 (空 attention 段)
4825 h_prefix_lse[0] = INFINITY; // head 0, token 0 的 LSE = +inf
4826 h_suffix_lse[0] = 0.0f;
4827 check(cudaMemcpy(d_prefix_lse, h_prefix_lse, lse_size,
4828                 cudaMemcpyHostToDevice),
4829         "merge_attn H2D inf lse");
4830 merge_attn_states<<<grid, block>>>(
4831     d_output, d_prefix_out, d_prefix_lse, d_suffix_out, d_suffix_lse,
4832     num_tokens, num_heads, head_size);
4833 check(cudaGetLastError(), "merge_attn inf launch");
4834 check(cudaDeviceSynchronize(), "merge_attn inf sync");
4835 check(cudaMemcpy(h_output, d_output, out_size, cudaMemcpyDeviceToHost),
4836         "merge_attn inf D2H");
4837
4838 // token 0, head 0 的所有元素应等于 suffix_output (prefix 权重  $\alpha=0$ )
4839 float inf_err = 0.0f;
4840 for (int d = 0; d < head_size; d++) {
4841     float err = fabsf(h_output[d] - h_suffix_out[d]);
4842     if (err > inf_err) inf_err = err;
4843 }
4844 printf("| %-42s | %.6e | %-4s | %-8s |\n", "MergeAttnStates-inf", inf_err,
4845        inf_err < 1e-4f ? "PASS" : "FAIL", "None");
4846

```

```

4847 free(h_prefix_out);
4848 free(h_suffix_out);
4849 free(h_prefix_lse);
4850 free(h_suffix_lse);
4851 free(h_output_ref);
4852 free(h_output);
4853 cudaFree(d_prefix_out);
4854 cudaFree(d_suffix_out);
4855 cudaFree(d_prefix_lse);
4856 cudaFree(d_suffix_lse);
4857 cudaFree(d_output);
4858 }
4859
4860
4861 static void test_softmax(int N) {
4862     // Use N = blockDim.x so one block processes all elements as one token.
4863     constexpr int kNumThreads = 256;
4864     if (N != kNumThreads) {
4865         printf(" OnlineSafeSoftmax: N must be %d (got %d), skipping.\n", kNumThreads, N);
4866         return;
4867     }
4868
4869     srand(42);
4870     float *h_x = (float *)malloc((size_t)N * sizeof(float));
4871     float *h_y_ref = (float *)malloc((size_t)N * sizeof(float));
4872     for (int i = 0; i < N; i++)
4873         h_x[i] = ((float)rand() / RAND_MAX) * 10.0f - 5.0f; // [-5, 5]
4874
4875     // CPU reference: softmax(x_i) = exp(x_i - max) / sum(exp(x_j - max))
4876     float max_val = -FLT_MAX;
4877     for (int i = 0; i < N; i++) if (h_x[i] > max_val) max_val = h_x[i];
4878     double sum_exp = 0.0;
4879     for (int i = 0; i < N; i++) sum_exp += (double)expf(h_x[i] - max_val);
4880     for (int i = 0; i < N; i++)
4881         h_y_ref[i] = expf(h_x[i] - max_val) / (float)sum_exp;
4882
4883     float *d_x, *d_y;
4884     check(cudaMalloc(&d_x, (size_t)N * sizeof(float)), "softmax alloc X");
4885     check(cudaMalloc(&d_y, (size_t)N * sizeof(float)), "softmax alloc Y");
4886
4887     check(cudaMemcpy(d_x, h_x, (size_t)N * sizeof(float), cudaMemcpyHostToDevice), "softmax H2D X");
4888
4889     dim3 block(256);
4890     dim3 grid(1); // one token covering all N elements
4891     online_safe_softmax_per_token<<<grid, block>>>(d_x, d_y, N);
4892     check(cudaGetLastError(), "softmax launch");
4893     check(cudaDeviceSynchronize(), "softmax sync");
4894
4895     float *h_y = (float *)malloc((size_t)N * sizeof(float));
4896     check(cudaMemcpy(h_y, d_y, (size_t)N * sizeof(float), cudaMemcpyDeviceToHost), "softmax D2H");
4897
4898     float max_err = 0.0f;
4899     for (int i = 0; i < N; i++) {
4900         float err = fabsf(h_y[i] - h_y_ref[i]);
4901         if (err > max_err) max_err = err;
4902     }
4903     printf("| %-42s | %.6e | %-4s | %-8s |\n", "OnlineSafeSoftmax", max_err, max_err < 1e-4f ? "PASS" : "FAIL",
4904           , "None");
4905
4906     // ---- Safe Softmax ----
4907     safe_softmax_per_token<<<grid, block>>>(d_x, d_y, N);
4908     check(cudaGetLastError(), "safe_softmax launch");
4909     check(cudaDeviceSynchronize(), "safe_softmax sync");

```

```

4909 check(cudaMemcpy(h_y, d_y, (size_t)N * sizeof(float), cudaMemcpyDeviceToHost), "safe_softmax D2H");
4910 max_err = 0.0f;
4911 for (int i = 0; i < N; i++) {
4912     float err = fabsf(h_y[i] - h_y_ref[i]);
4913     if (err > max_err) max_err = err;
4914 }
4915 printf("| %-42s | %.6e | %-4s | %-8s |\n", "SafeSoftmax", max_err, max_err < 1e-4f ? "PASS" : "FAIL", "
    None");
4916
4917 // ---- Naive Softmax ----
4918 softmax_per_token<<<grid, block>>>(d_x, d_y, N);
4919 check(cudaGetLastError(), "naive_softmax launch");
4920 check(cudaDeviceSynchronize(), "naive_softmax sync");
4921 check(cudaMemcpy(h_y, d_y, (size_t)N * sizeof(float), cudaMemcpyDeviceToHost), "naive_softmax D2H");
4922 max_err = 0.0f;
4923 for (int i = 0; i < N; i++) {
4924     float err = fabsf(h_y[i] - h_y_ref[i]);
4925     if (err > max_err) max_err = err;
4926 }
4927 printf("| %-42s | %.6e | %-4s | %-8s |\n", "NaiveSoftmax", max_err, max_err < 1e-4f ? "PASS" : "FAIL", "
    None");
4928
4929 free(h_x); free(h_y); free(h_y_ref);
4930 cudaFree(d_x); cudaFree(d_y);
4931 }
4932
4933
4934 static void test_rms_norm(int N, int K) {
4935
4936     srand(42);
4937     float *h_x = (float *)malloc((size_t)N * K * sizeof(float));
4938     float *h_y_ref = (float *)malloc((size_t)N * K * sizeof(float));
4939     for (int i = 0; i < N * K; i++)
4940         h_x[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
4941     float g = 1.5f; // gain
4942
4943     // CPU reference: y = (x / rms(x)) * g
4944     float epsilon = 1e-5f;
4945     for (int n = 0; n < N; n++) {
4946         double sum_sq = 0.0;
4947         for (int k = 0; k < K; k++) sum_sq += (double)h_x[n * K + k] * (double)h_x[n * K + k];
4948         float rms = sqrtf((float)sum_sq / (float)K + epsilon);
4949         for (int k = 0; k < K; k++)
4950             h_y_ref[n * K + k] = (h_x[n * K + k] / rms) * g;
4951     }
4952
4953     float *d_x, *d_y;
4954     check(cudaMalloc(&d_x, (size_t)N * K * sizeof(float)), "rmsnorm alloc X");
4955     check(cudaMalloc(&d_y, (size_t)N * K * sizeof(float)), "rmsnorm alloc Y");
4956     check(cudaMemcpy(d_x, h_x, (size_t)N * K * sizeof(float), cudaMemcpyHostToDevice), "rmsnorm H2D X");
4957
4958     dim3 block(128);
4959     dim3 grid(N);
4960     rms_norm<<<grid, block>>>(d_x, d_y, g, N, K);
4961     check(cudaGetLastError(), "rmsnorm launch");
4962     check(cudaDeviceSynchronize(), "rmsnorm sync");
4963
4964     float *h_y = (float *)malloc((size_t)N * K * sizeof(float));
4965     check(cudaMemcpy(h_y, d_y, (size_t)N * K * sizeof(float), cudaMemcpyDeviceToHost), "rmsnorm D2H");
4966
4967     float max_err = 0.0f;
4968     for (int i = 0; i < N * K; i++) {
4969         float err = fabsf(h_y[i] - h_y_ref[i]);

```

```

4970     if (err > max_err) max_err = err;
4971 }
4972 printf("| %-42s | %.6e | %-4s | %-8s |\n", "RMSNorm", max_err, max_err < 1e-4f ? "PASS" : "FAIL", "None");
4973
4974 // ---- RMS Norm Vec4 ----
4975 dim3 block_rv4(32);
4976 rms_norm_vec4<<<grid, block_rv4>>>(d_x, d_y, g, N, K);
4977 check(cudaGetLastError(), "rmsnorm_vec4 launch");
4978 check(cudaDeviceSynchronize(), "rmsnorm_vec4 sync");
4979 check(cudaMemcpy(h_y, d_y, (size_t)N * K * sizeof(float), cudaMemcpyDeviceToHost), "rmsnorm_vec4 D2H");
4980 max_err = 0.0f;
4981 for (int i = 0; i < N * K; i++) {
4982     float err = fabsf(h_y[i] - h_y_ref[i]);
4983     if (err > max_err) max_err = err;
4984 }
4985 printf("| %-42s | %.6e | %-4s | %-8s |\n", "RMSNorm-Vec4", max_err, max_err < 1e-4f ? "PASS" : "FAIL", "None");
4986
4987 free(h_x); free(h_y); free(h_y_ref);
4988 cudaFree(d_x); cudaFree(d_y);
4989 }
4990
4991
4992 static void test_layer_norm(int N, int K) {
4993
4994     srand(42);
4995     float *h_x = (float *)malloc((size_t)N * K * sizeof(float));
4996     float *h_y_ref = (float *)malloc((size_t)N * K * sizeof(float));
4997     for (int i = 0; i < N * K; i++)
4998         h_x[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
4999     float g = 1.5f, b = 0.3f; // gain and bias
5000
5001     // CPU reference: y = ((x - mean) / std) * g + b
5002     float epsilon = 1e-5f;
5003     for (int n = 0; n < N; n++) {
5004         double sum = 0.0;
5005         for (int k = 0; k < K; k++) sum += (double)h_x[n * K + k];
5006         float mean = (float)sum / (float)K;
5007         double sum_sq = 0.0;
5008         for (int k = 0; k < K; k++) {
5009             float diff = h_x[n * K + k] - mean;
5010             sum_sq += (double)diff * (double)diff;
5011         }
5012         float std = sqrtf((float)sum_sq / (float)K + epsilon);
5013         for (int k = 0; k < K; k++)
5014             h_y_ref[n * K + k] = ((h_x[n * K + k] - mean) / std) * g + b;
5015     }
5016
5017     float *d_x, *d_y;
5018     check(cudaMalloc(&d_x, (size_t)N * K * sizeof(float)), "layernorm alloc X");
5019     check(cudaMalloc(&d_y, (size_t)N * K * sizeof(float)), "layernorm alloc Y");
5020     check(cudaMemcpy(d_x, h_x, (size_t)N * K * sizeof(float), cudaMemcpyHostToDevice), "layernorm H2D X");
5021
5022     dim3 block(128);
5023     dim3 grid(N);
5024     layer_norm<<<grid, block>>>(d_x, d_y, g, b, N, K);
5025     check(cudaGetLastError(), "layernorm launch");
5026     check(cudaDeviceSynchronize(), "layernorm sync");
5027
5028     float *h_y = (float *)malloc((size_t)N * K * sizeof(float));
5029     check(cudaMemcpy(h_y, d_y, (size_t)N * K * sizeof(float), cudaMemcpyDeviceToHost), "layernorm D2H");
5030
5031     float max_err = 0.0f;

```



```

5032 for (int i = 0; i < N * K; i++) {
5033     float err = fabsf(h_y[i] - h_y_ref[i]);
5034     if (err > max_err) max_err = err;
5035 }
5036 printf("| %-42s | %.6e | %-4s | %-8s |\n", "LayerNorm", max_err, max_err < 1e-4f ? "PASS" : "FAIL", "None"
    );
5037
5038 // ---- Layer Norm Vec4 ----
5039 dim3 block_lv4(32);
5040 layer_norm_vec4<<<grid, block_lv4>>>(d_x, d_y, g, b, N, K);
5041 check(cudaGetLastError(), "layernorm_vec4 launch");
5042 check(cudaDeviceSynchronize(), "layernorm_vec4 sync");
5043 check(cudaMemcpy(h_y, d_y, (size_t)N * K * sizeof(float), cudaMemcpyDeviceToHost), "layernorm_vec4 D2H");
5044 max_err = 0.0f;
5045 for (int i = 0; i < N * K; i++) {
5046     float err = fabsf(h_y[i] - h_y_ref[i]);
5047     if (err > max_err) max_err = err;
5048 }
5049 printf("| %-42s | %.6e | %-4s | %-8s |\n", "LayerNorm-Vec4", max_err, max_err < 1e-4f ? "PASS" : "FAIL", "
    None");
5050
5051 free(h_x); free(h_y); free(h_y_ref);
5052 cudaFree(d_x); cudaFree(d_y);
5053 }
5054
5055
5056 static void test_rope(int seq_len, int N) {
5057
5058     int total_pairs = seq_len * N;
5059     int total_elems = total_pairs * 2;
5060     size_t size = (size_t)total_elems * sizeof(float);
5061
5062     float *h_x = (float *)malloc(size);
5063     float *h_y_ref = (float *)malloc(size);
5064
5065     srand(42);
5066     for (int i = 0; i < total_elems; i++)
5067         h_x[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
5068
5069     // CPU reference: 2D rotation for each pair
5070     for (int idx = 0; idx < total_pairs; idx++) {
5071         int token_pos = idx / N;
5072         int token_idx = idx % N;
5073         float x1 = h_x[idx * 2];
5074         float x2 = h_x[idx * 2 + 1];
5075         float theta = 1.0f / powf(10000.0f, 2.0f * token_idx / (N * 2.0f));
5076         float angle = (float)token_pos * theta;
5077         float cos_v = cosf(angle);
5078         float sin_v = sinf(angle);
5079         h_y_ref[idx * 2] = x1 * cos_v - x2 * sin_v;
5080         h_y_ref[idx * 2 + 1] = x1 * sin_v + x2 * cos_v;
5081     }
5082
5083     float *d_x, *d_y;
5084     check(cudaMalloc(&d_x, size), "rope alloc X");
5085     check(cudaMalloc(&d_y, size), "rope alloc Y");
5086     check(cudaMemcpy(d_x, h_x, size, cudaMemcpyHostToDevice), "rope H2D");
5087
5088     dim3 block(256);
5089     dim3 grid((total_pairs + 255) / 256);
5090     rope<<<grid, block>>>(d_x, d_y, seq_len, N);
5091     check(cudaGetLastError(), "rope launch");
5092     check(cudaDeviceSynchronize(), "rope sync");

```

```

5093 float *h_y = (float *)malloc(size);
5094 check(cudaMemcpy(h_y, d_y, size, cudaMemcpyDeviceToHost), "rope D2H");
5095
5096 float max_err = 0.0f;
5097 for (int i = 0; i < total_elems; i++) {
5098     float err = fabsf(h_y[i] - h_y_ref[i]);
5099     if (err > max_err) max_err = err;
5100 }
5101 printf("| %-42s | %.6e | %-4s | %-8s |\n", "RoPE", max_err, max_err < 1e-4f ? "PASS" : "FAIL", "None");
5102
5103 free(h_x);
5104 free(h_y);
5105 free(h_y_ref);
5106 cudaFree(d_x);
5107 cudaFree(d_y);
5108 }
5109
5110
5111 static void test_mat_transpose(int row, int col) {
5112
5113     size_t size_in = (size_t)row * col * sizeof(float);
5114     size_t size_out = (size_t)col * row * sizeof(float);
5115
5116     float *h_x = (float *)malloc(size_in);
5117     float *h_y_ref = (float *)malloc(size_out);
5118
5119     srand(42);
5120     for (int i = 0; i < row * col; i++)
5121         h_x[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
5122
5123     // CPU reference: y[j][i] = x[i][j] (row-major)
5124     for (int i = 0; i < row; i++)
5125         for (int j = 0; j < col; j++)
5126             h_y_ref[j * row + i] = h_x[i * col + j];
5127
5128     float *d_x, *d_y;
5129     check(cudaMalloc(&d_x, size_in), "mattrans alloc X");
5130     check(cudaMalloc(&d_y, size_out), "mattrans alloc Y");
5131     check(cudaMemcpy(d_x, h_x, size_in, cudaMemcpyHostToDevice), "mattrans H2D");
5132
5133     dim3 block(16, 16);
5134     dim3 grid((col + 15) / 16, (row + 15) / 16);
5135     mat_transpose<<<grid, block>>>(d_x, d_y, row, col);
5136     check(cudaGetLastError(), "mattrans launch");
5137     check(cudaDeviceSynchronize(), "mattrans sync");
5138
5139     float *h_y = (float *)malloc(size_out);
5140     check(cudaMemcpy(h_y, d_y, size_out, cudaMemcpyDeviceToHost), "mattrans D2H");
5141
5142     float max_err = 0.0f;
5143     for (int i = 0; i < col * row; i++) {
5144         float err = fabsf(h_y[i] - h_y_ref[i]);
5145         if (err > max_err) max_err = err;
5146     }
5147     printf("| %-42s | %.6e | %-4s | %-8s |\n", "MatTranspose", max_err, max_err < 1e-6f ? "PASS" : "FAIL", "None");
5148
5149     free(h_x);
5150     free(h_y);
5151     free(h_y_ref);
5152     cudaFree(d_x);
5153     cudaFree(d_y);

```

```

5155 }
5156
5157
5158 static void test_mat_transpose_padded(int row, int col) {
5159
5160     size_t size_in = (size_t)row * col * sizeof(float);
5161     size_t size_out = (size_t)col * row * sizeof(float);
5162
5163     float *h_x = (float *)malloc(size_in);
5164     float *h_y_ref = (float *)malloc(size_out);
5165
5166     srand(42);
5167     for (int i = 0; i < row * col; i++)
5168         h_x[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
5169
5170     // CPU reference: y[j][i] = x[i][j] (row-major)
5171     for (int i = 0; i < row; i++)
5172         for (int j = 0; j < col; j++)
5173             h_y_ref[j * row + i] = h_x[i * col + j];
5174
5175     float *d_x, *d_y;
5176     check(cudaMalloc(&d_x, size_in), "mattrans_padded alloc X");
5177     check(cudaMalloc(&d_y, size_out), "mattrans_padded alloc Y");
5178     check(cudaMemcpy(d_x, h_x, size_in, cudaMemcpyHostToDevice), "mattrans_padded H2D");
5179
5180     dim3 block(16, 16);
5181     dim3 grid((col + 15) / 16, (row + 63) / 64);
5182     mat_transpose_padded<<<grid, block>>>(d_x, d_y, row, col);
5183     check(cudaGetLastError(), "mattrans_padded launch");
5184     check(cudaDeviceSynchronize(), "mattrans_padded sync");
5185
5186     float *h_y = (float *)malloc(size_out);
5187     check(cudaMemcpy(h_y, d_y, size_out, cudaMemcpyDeviceToHost), "mattrans_padded D2H");
5188
5189     float max_err = 0.0f;
5190     for (int i = 0; i < col * row; i++) {
5191         float err = fabsf(h_y[i] - h_y_ref[i]);
5192         if (err > max_err) max_err = err;
5193     }
5194     printf("| %-42s | %.6e | %-4s | %-8s |\n", "MatTransposePadded", max_err, max_err < 1e-6f ? "PASS" : "FAIL", "None");
5195
5196     free(h_x);
5197     free(h_y);
5198     free(h_y_ref);
5199     cudaFree(d_x);
5200     cudaFree(d_y);
5201 }
5202
5203
5204 static void test_sgemv(int M, int K) {
5205
5206     srand(42);
5207     float *h_a = (float *)malloc((size_t)M * K * sizeof(float));
5208     float *h_x = (float *)malloc((size_t)K * sizeof(float));
5209     float *h_y_ref = (float *)malloc((size_t)M * sizeof(float));
5210     for (int i = 0; i < M * K; i++) h_a[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
5211     for (int i = 0; i < K; i++) h_x[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
5212
5213     // CPU reference: y[m] = sum_k A[m,k] * x[k]
5214     for (int m = 0; m < M; m++) {
5215         double sum = 0.0;
5216         for (int k = 0; k < K; k++) sum += (double)h_a[m * K + k] * (double)h_x[k];

```

```

5217     h_y_ref[m] = (float)sum;
5218 }
5219
5220 float *d_a, *d_x, *d_y;
5221 check(cudaMalloc(&d_a, (size_t)M * K * sizeof(float)), "sgemv alloc A");
5222 check(cudaMalloc(&d_x, (size_t)K * sizeof(float)), "sgemv alloc X");
5223 check(cudaMalloc(&d_y, (size_t)M * sizeof(float)), "sgemv alloc Y");
5224
5225 check(cudaMemcpy(d_a, h_a, (size_t)M * K * sizeof(float), cudaMemcpyHostToDevice), "sgemv H2D A");
5226 check(cudaMemcpy(d_x, h_x, (size_t)K * sizeof(float), cudaMemcpyHostToDevice), "sgemv H2D X");
5227
5228 dim3 block(32, 4);
5229 dim3 grid((M + 3) / 4);
5230 sgemv_k128<<<grid, block>>>(d_a, d_x, d_y, M, K);
5231 check(cudaGetLastError(), "sgemv launch");
5232 check(cudaDeviceSynchronize(), "sgemv sync");
5233
5234 float *h_y = (float *)malloc((size_t)M * sizeof(float));
5235 check(cudaMemcpy(h_y, d_y, (size_t)M * sizeof(float), cudaMemcpyDeviceToHost), "sgemv D2H");
5236
5237 float max_err = 0.0f;
5238 for (int m = 0; m < M; m++) {
5239     float err = fabsf(h_y[m] - h_y_ref[m]);
5240     if (err > max_err) max_err = err;
5241 }
5242 printf("| %-42s | %.6e | %-4s | %-8s |\n", "SGEMV-K128", max_err, max_err < 1e-2f ? "PASS" : "FAIL", "None");
5243
5244 // ---- SGEMV K32 ----
5245 check(cudaMemset(d_y, 0, M * sizeof(float)), "sgemv_k32 zero Y");
5246 sgemv_k32<<<grid, block>>>(d_a, d_x, d_y, M, K);
5247 check(cudaGetLastError(), "sgemv_k32 launch");
5248 check(cudaDeviceSynchronize(), "sgemv_k32 sync");
5249
5250 check(cudaMemcpy(h_y, d_y, (size_t)M * sizeof(float), cudaMemcpyDeviceToHost), "sgemv_k32 D2H");
5251
5252 max_err = 0.0f;
5253 for (int m = 0; m < M; m++) {
5254     float err = fabsf(h_y[m] - h_y_ref[m]);
5255     if (err > max_err) max_err = err;
5256 }
5257 printf("| %-42s | %.6e | %-4s | %-8s |\n", "SGEMV-K32", max_err, max_err < 1e-2f ? "PASS" : "FAIL", "None");
5258
5259 // ---- SGEMV K16 ----
5260 free(h_a); free(h_x); free(h_y); free(h_y_ref);
5261 cudaFree(d_a); cudaFree(d_x); cudaFree(d_y);
5262
5263 int K16 = 16;
5264 h_a = (float *)malloc((size_t)M * K16 * sizeof(float));
5265 h_x = (float *)malloc((size_t)K16 * sizeof(float));
5266 h_y_ref = (float *)malloc((size_t)M * sizeof(float));
5267 for (int i = 0; i < M * K16; i++) h_a[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
5268 for (int i = 0; i < K16; i++) h_x[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
5269
5270 for (int m = 0; m < M; m++) {
5271     double sum = 0.0;
5272     for (int k = 0; k < K16; k++) sum += (double)h_a[m * K16 + k] * (double)h_x[k];
5273     h_y_ref[m] = (float)sum;
5274 }
5275
5276 check(cudaMalloc(&d_a, (size_t)M * K16 * sizeof(float)), "sgemv_k16 alloc A");
5277 check(cudaMalloc(&d_x, (size_t)K16 * sizeof(float)), "sgemv_k16 alloc X");

```

```

5278 check(cudaMalloc(&d_y, (size_t)M * sizeof(float)), "sgemv_k16 alloc Y");
5279
5280 check(cudaMemcpy(d_a, h_a, (size_t)M * K16 * sizeof(float), cudaMemcpyHostToDevice), "sgemv_k16 H2D A");
5281 check(cudaMemcpy(d_x, h_x, (size_t)K16 * sizeof(float), cudaMemcpyHostToDevice), "sgemv_k16 H2D X");
5282
5283 dim3 grid_k16((M + 7) / 8);
5284 sgemv_k16<2><<<grid_k16, block>>>(d_a, d_x, d_y, M, K16);
5285 check(cudaGetLastError(), "sgemv_k16 launch");
5286 check(cudaDeviceSynchronize(), "sgemv_k16 sync");
5287
5288 h_y = (float *)malloc((size_t)M * sizeof(float));
5289 check(cudaMemcpy(h_y, d_y, (size_t)M * sizeof(float), cudaMemcpyDeviceToHost), "sgemv_k16 D2H");
5290
5291 max_err = 0.0f;
5292 for (int m = 0; m < M; m++) {
5293     float err = fabsf(h_y[m] - h_y_ref[m]);
5294     if (err > max_err) max_err = err;
5295 }
5296 printf("| %-42s | %.6e | %-4s | %-8s |\n", "SGEMV-K16", max_err, max_err < 1e-2f ? "PASS" : "FAIL", "None"
5297 );
5298 free(h_a); free(h_x); free(h_y); free(h_y_ref);
5299 cudaFree(d_a); cudaFree(d_x); cudaFree(d_y);
5300 }
5301
5302
5303 static void test_sgemm(int M, int N, int K) {
5304
5305     size_t size_a = (size_t)M * K * sizeof(float);
5306     size_t size_b = (size_t)K * N * sizeof(float);
5307     size_t size_c = (size_t)M * N * sizeof(float);
5308
5309     float *h_a = (float *)malloc(size_a);
5310     float *h_b = (float *)malloc(size_b);
5311     float *h_c_ref = (float *)malloc(size_c);
5312
5313     srand(42);
5314     for (int i = 0; i < M * K; i++) h_a[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
5315     for (int i = 0; i < K * N; i++) h_b[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
5316
5317     float *d_a, *d_b, *d_c;
5318     check(cudaMalloc(&d_a, size_a), "sgemm alloc A");
5319     check(cudaMalloc(&d_b, size_b), "sgemm alloc B");
5320     check(cudaMalloc(&d_c, size_c), "sgemm alloc C");
5321
5322     check(cudaMemcpy(d_a, h_a, size_a, cudaMemcpyHostToDevice), "sgemm H2D A");
5323     check(cudaMemcpy(d_b, h_b, size_b, cudaMemcpyHostToDevice), "sgemm H2D B");
5324
5325     // cuBLAS reference (row-major idiom: swap M/N, swap A/B)
5326     cublasHandle_t handle;
5327     cublasCreate(&handle);
5328     float alpha = 1.0f, beta = 0.0f;
5329     cublasSgemm(handle, CUBLAS_OP_N, CUBLAS_OP_N, N, M, K,
5330                 &alpha, d_b, N, d_a, K, &beta, d_c, N);
5331     check(cudaMemcpy(h_c_ref, d_c, size_c, cudaMemcpyDeviceToHost), "sgemm D2H ref");
5332
5333     // Kernel
5334     cudaEvent_t start, stop;
5335     cudaEventCreate(&start);
5336     cudaEventCreate(&stop);
5337
5338     dim3 block(32, 32);
5339     dim3 grid((N + 31) / 32, (M + 31) / 32);

```

```

5340     sgemm<<<grid, block>>>(d_a, d_b, d_c, M, N, K);
5341     check(cudaGetLastError(), "sgemm launch");
5342     check(cudaDeviceSynchronize(), "sgemm sync");
5343
5344     float *h_c = (float *)malloc(size_c);
5345     check(cudaMemcpy(h_c, d_c, size_c, cudaMemcpyDeviceToHost), "sgemm D2H");
5346
5347     // Verify
5348     float max_err = 0.0f;
5349     for (int i = 0; i < M * N; i++) {
5350         float err = fabsf(h_c[i] - h_c_ref[i]);
5351         if (err > max_err) max_err = err;
5352     }
5353     printf("| %-42s | %.6e | %-4s | %-8s |\n", "SGEMM", max_err, max_err < 1e-2f ? "PASS" : "FAIL", "None");
5354
5355     // ---- SGEMM Vec4 (128x128 tile, 4x4 thread tile) ----
5356
5357     dim3 grid_vec4((N + 127) / 128, (M + 127) / 128);
5358     check(cudaMemset(d_c, 0, size_c), "sgemm_vec4 zero C");
5359     sgemm_vec4<<<grid_vec4, block>>>(d_a, d_b, d_c, M, N, K);
5360     check(cudaGetLastError(), "sgemm_vec4 launch");
5361     check(cudaDeviceSynchronize(), "sgemm_vec4 sync");
5362
5363     check(cudaMemcpy(h_c, d_c, size_c, cudaMemcpyDeviceToHost), "sgemm_vec4 D2H");
5364
5365     max_err = 0.0f;
5366     for (int i = 0; i < M * N; i++) {
5367         float err = fabsf(h_c[i] - h_c_ref[i]);
5368         if (err > max_err) max_err = err;
5369     }
5370     printf("| %-42s | %.6e | %-4s | %-8s |\n", "SGEMM-Vec4", max_err, max_err < 1e-2f ? "PASS" : "FAIL", "None");
5371
5372     free(h_a); free(h_b); free(h_c); free(h_c_ref);
5373     cudaFree(d_a); cudaFree(d_b); cudaFree(d_c);
5374     cublasDestroy(handle);
5375     cudaEventDestroy(start);
5376     cudaEventDestroy(stop);
5377 }
5378
5379
5380 static void test_hgemmm_mma(int M, int N, int K) {
5381
5382     size_t size_a = (size_t)M * K * sizeof(half);
5383     size_t size_b = (size_t)K * N * sizeof(half);
5384     size_t size_c = (size_t)M * N * sizeof(half);
5385
5386     half *h_a = (half *)malloc(size_a);
5387     half *h_b = (half *)malloc(size_b);
5388     half *h_c_ref = (half *)malloc(size_c);
5389
5390     srand(42);
5391     for (int i = 0; i < M * K; i++) h_a[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
5392     for (int i = 0; i < K * N; i++) h_b[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
5393
5394     // Kernel expects B^T [NxK] row-major layout (TN layout convention).
5395     // We store h_b as B [KxN] row-major for cuBLAS, then create B^T for kernel.
5396     size_t size_b_t = (size_t)N * K * sizeof(half);
5397     half *h_b_t = (half *)malloc(size_b_t);
5398     for (int n = 0; n < N; n++)
5399         for (int k = 0; k < K; k++)
5400             h_b_t[n * K + k] = h_b[k * N + n];
5401

```



```

5402     half *d_a, *d_b, *d_b_t, *d_c;
5403     check(cudaMalloc(&d_a, size_a), "hgemm alloc A");
5404     check(cudaMalloc(&d_b, size_b), "hgemm alloc B (cuBLAS)");
5405     check(cudaMalloc(&d_b_t, size_b_t), "hgemm alloc B_t (kernel)");
5406     check(cudaMalloc(&d_c, size_c), "hgemm alloc C");
5407
5408     check(cudaMemcpy(d_a, h_a, size_a, cudaMemcpyHostToDevice), "hgemm H2D A");
5409     check(cudaMemcpy(d_b, h_b, size_b, cudaMemcpyHostToDevice), "hgemm H2D B (cuBLAS)");
5410     check(cudaMemcpy(d_b_t, h_b_t, size_b_t, cudaMemcpyHostToDevice), "hgemm H2D B_t (kernel)");
5411
5412     // cuBLAS FP16 reference (row-major idiom: swap M/N, swap A/B)
5413     // Note: use CUBLAS_COMPUTE_16F to match the kernel's f16.f16.f16.f16 accumulation.
5414     cublasHandle_t handle;
5415     cublasCreate(&handle);
5416     half alpha_h = __float2half(1.0f), beta_h = __float2half(0.0f);
5417     cublasGemmEx(handle, CUBLAS_OP_N, CUBLAS_OP_N, N, M, K,
5418                 &alpha_h, d_b, CUDA_R_16F, N, d_a, CUDA_R_16F, K,
5419                 &beta_h, d_c, CUDA_R_16F, N,
5420                 CUBLAS_COMPUTE_16F, CUBLAS_GEMM_DEFAULT);
5421     check(cudaMemcpy(h_c_ref, d_c, size_c, cudaMemcpyDeviceToHost), "hgemm D2H ref");
5422
5423     // MMA kernel (TN layout: A row-major, B^T row-major = B col-major)
5424     cudaEvent_t start, stop;
5425     cudaEventCreate(&start);
5426     cudaEventCreate(&stop);
5427
5428     constexpr int BM = 128, BN = 128, BK = 16, kStages = 3;
5429     size_t smem_bytes = kStages * (BM * BK + BN * BK) * sizeof(half); // 24576
5430     dim3 block(256);
5431     dim3 grid((N + BN - 1) / BN, (M + BM - 1) / BM);
5432     hgemm_mma_stages_tn<<<grid, block, smem_bytes>>>(d_a, d_b_t, d_c, M, N, K);
5433     check(cudaGetLastError(), "hgemm launch");
5434     check(cudaDeviceSynchronize(), "hgemm sync");
5435
5436     half *h_c = (half *)malloc(size_c);
5437     check(cudaMemcpy(h_c, d_c, size_c, cudaMemcpyDeviceToHost), "hgemm D2H");
5438
5439     // Verify
5440     float max_err = 0.0f;
5441     for (int i = 0; i < M * N; i++) {
5442         float err = fabsf(__half2float(h_c[i]) - __half2float(h_c_ref[i]));
5443         if (err > max_err) max_err = err;
5444     }
5445     printf("| %-42s | %.6e | %-4s | %-8s |\n", "HGEMM MMA", max_err, max_err < 1.0f ? "PASS" : "FAIL", "None");
5446     ;
5447
5448     free(h_a); free(h_b); free(h_b_t); free(h_c); free(h_c_ref);
5449     cudaFree(d_a); cudaFree(d_b); cudaFree(d_b_t); cudaFree(d_c);
5450     cublasDestroy(handle);
5451     cudaEventDestroy(start);
5452     cudaEventDestroy(stop);
5453 }
5454
5455 static void test_hgemm_swizzle(int M, int N, int K) {
5456     // HGEMM MMA Swizzle — m16n8k16 + multistage pipeline + TN 布局 + XOR swizzle
5457     // + Register Double Buffering (kValTileK=4, BK=64)
5458     // TN layout: C[M×N] = A[M×K] × B^T[N×K]
5459     // Kernel: hgemm_mma_stages_tn_swizzle with default template params
5460     // (kValTileK=4, kStages=2, BK=64)
5461     // smem: kStages × (BM×BK + BN×BK) halves
5462
5463     size_t size_a = (size_t)M * K * sizeof(half);

```

```

5464 size_t size_b = (size_t)K * N * sizeof(half);
5465 size_t size_c = (size_t)M * N * sizeof(half);
5466
5467 half *h_a = (half *)malloc(size_a);
5468 half *h_b = (half *)malloc(size_b);
5469 half *h_c_ref = (half *)malloc(size_c);
5470
5471 srand(42);
5472 for (int i = 0; i < M * K; i++) h_a[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
5473 for (int i = 0; i < K * N; i++) h_b[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
5474
5475 // Kernel expects B^T [N×K] row-major layout (TN layout convention).
5476 size_t size_b_t = (size_t)N * K * sizeof(half);
5477 half *h_b_t = (half *)malloc(size_b_t);
5478 for (int n = 0; n < N; n++)
5479     for (int k = 0; k < K; k++)
5480         h_b_t[n * K + k] = h_b[k * N + n];
5481
5482 half *d_a, *d_b, *d_b_t, *d_c;
5483 check(cudaMalloc(&d_a, size_a), "hgemm_swizzle alloc A");
5484 check(cudaMalloc(&d_b, size_b), "hgemm_swizzle alloc B (cuBLAS)");
5485 check(cudaMalloc(&d_b_t, size_b_t), "hgemm_swizzle alloc B_t (kernel)");
5486 check(cudaMalloc(&d_c, size_c), "hgemm_swizzle alloc C");
5487
5488 check(cudaMemcpy(d_a, h_a, size_a, cudaMemcpyHostToDevice), "hgemm_swizzle H2D A");
5489 check(cudaMemcpy(d_b, h_b, size_b, cudaMemcpyHostToDevice), "hgemm_swizzle H2D B (cuBLAS)");
5490 check(cudaMemcpy(d_b_t, h_b_t, size_b_t, cudaMemcpyHostToDevice), "hgemm_swizzle H2D B_t (kernel)");
5491
5492 // cuBLAS FP16 reference
5493 cublasHandle_t handle;
5494 cublasCreate(&handle);
5495 half alpha_h = __float2half(1.0f), beta_h = __float2half(0.0f);
5496 cublasGemmEx(handle, CUBLAS_OP_N, CUBLAS_OP_N, N, M, K,
5497             &alpha_h, d_b, CUDA_R_16F, N, d_a, CUDA_R_16F, K,
5498             &beta_h, d_c, CUDA_R_16F, N,
5499             CUBLAS_COMPUTE_16F, CUBLAS_GEMM_DEFAULT);
5500 check(cudaMemcpy(h_c_ref, d_c, size_c, cudaMemcpyDeviceToHost), "hgemm_swizzle D2H ref");
5501
5502 // MMA swizzle kernel (default params: kStages=2, kValTileK=4, BK=64)
5503 constexpr int BM = 128, BN = 128, BK = 64, K_STAGE_S = 2;
5504 size_t smem_bytes = K_STAGE_S * (BM * BK + BN * BK) * sizeof(half);
5505 cudaFuncSetAttribute(
5506     (const void *)hgemm_mma_stages_tn_swizzle<16, 8, 16, 2, 4, 4, 4, 4, K_STAGE_S, 0>,
5507     cudaFuncAttributeMaxDynamicSharedMemorySize, smem_bytes);
5508 dim3 block(256);
5509 dim3 grid((N + BN - 1) / BN, (M + BM - 1) / BM);
5510 hgemm_mma_stages_tn_swizzle<<<grid, block, smem_bytes>>>(d_a, d_b_t, d_c, M, N, K);
5511 check(cudaGetLastError(), "hgemm_swizzle launch");
5512 check(cudaDeviceSynchronize(), "hgemm_swizzle sync");
5513
5514 half *h_c = (half *)malloc(size_c);
5515 check(cudaMemcpy(h_c, d_c, size_c, cudaMemcpyDeviceToHost), "hgemm_swizzle D2H");
5516
5517 // Verify
5518 float max_err = 0.0f;
5519 for (int i = 0; i < M * N; i++) {
5520     float err = fabsf(__half2float(h_c[i]) - __half2float(h_c_ref[i]));
5521     if (err > max_err) max_err = err;
5522 }
5523 printf("| %-4s | %.6e | %-4s | %-8s |\n", "HGEMM Swizzle + Reg2x", max_err, max_err < 1.0f ? "PASS" : "
5524     FAIL", "None");
5525
5526 free(h_a); free(h_b); free(h_b_t); free(h_c); free(h_c_ref);

```

```

5526     cudaFree(d_a); cudaFree(d_b); cudaFree(d_b_t); cudaFree(d_c);
5527     cublasDestroy(handle);
5528 }
5529
5530
5531 #if defined(NOTES_V2_ENABLE_CUTE)
5532 static void test_hgemmm_cute(int M, int N, int K) {
5533     // HGEMM CuTe — SM80_16x8x16_F16F16F16F16_TN + Swizzle<3,3,3> + kStage=2
5534     // TN layout: C[M×N] = A[M×K] × BT[N×K]
5535     // Kernel: hgemm_mma_stages_tn_cute via launch_hgemm_mma_stages_tn_cute
5536     // Tile: BM=128, BN=256, BK=32, 128 threads/block
5537
5538     size_t size_a = (size_t)M * K * sizeof(half);
5539     size_t size_b = (size_t)K * N * sizeof(half);
5540     size_t size_c = (size_t)M * N * sizeof(half);
5541
5542     half *h_a = (half *)malloc(size_a);
5543     half *h_b = (half *)malloc(size_b);
5544     half *h_c_ref = (half *)malloc(size_c);
5545
5546     srand(42);
5547     for (int i = 0; i < M * K; i++)
5548         h_a[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
5549     for (int i = 0; i < K * N; i++)
5550         h_b[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
5551
5552     // CuTe kernel expects BT [N×K] row-major (TN layout).
5553     size_t size_b_t = (size_t)N * K * sizeof(half);
5554     half *h_b_t = (half *)malloc(size_b_t);
5555     for (int n = 0; n < N; n++)
5556         for (int k = 0; k < K; k++)
5557             h_b_t[n * K + k] = h_b[k * N + n];
5558
5559     half *d_a, *d_b, *d_b_t, *d_c;
5560     check(cudaMalloc(&d_a, size_a), "hgemm_cute alloc A");
5561     check(cudaMalloc(&d_b, size_b), "hgemm_cute alloc B (cuBLAS)");
5562     check(cudaMalloc(&d_b_t, size_b_t), "hgemm_cute alloc B_t (kernel)");
5563     check(cudaMalloc(&d_c, size_c), "hgemm_cute alloc C");
5564
5565     check(cudaMemcpy(d_a, h_a, size_a, cudaMemcpyHostToDevice), "hgemm_cute H2D A");
5566     check(cudaMemcpy(d_b, h_b, size_b, cudaMemcpyHostToDevice), "hgemm_cute H2D B (cuBLAS)");
5567     check(cudaMemcpy(d_b_t, h_b_t, size_b_t, cudaMemcpyHostToDevice), "hgemm_cute H2D B_t (kernel)");
5568
5569     // cuBLAS FP16 reference (row-major idiom: swap M/N, swap A/B)
5570     cublasHandle_t handle;
5571     cublasCreate(&handle);
5572     half alpha_h = __float2half(1.0f), beta_h = __float2half(0.0f);
5573     cublasGemmEx(handle, CUBLAS_OP_N, CUBLAS_OP_N, N, M, K, &alpha_h, d_b,
5574                 CUDA_R_16F, N, d_a, CUDA_R_16F, K, &beta_h, d_c, CUDA_R_16F, N,
5575                 CUBLAS_COMPUTE_16F, CUBLAS_GEMM_DEFAULT);
5576     check(cudaMemcpy(h_c_ref, d_c, size_c, cudaMemcpyDeviceToHost), "hgemm_cute D2H ref");
5577
5578     // CuTe kernel (Stages=2, TN layout)
5579     launch_hgemm_mma_stages_tn_cute<half, 2, 0>(d_a, d_b_t, d_c, M, N, K);
5580     check(cudaGetLastError(), "hgemm_cute launch");
5581     check(cudaDeviceSynchronize(), "hgemm_cute sync");
5582
5583     half *h_c = (half *)malloc(size_c);
5584     check(cudaMemcpy(h_c, d_c, size_c, cudaMemcpyDeviceToHost), "hgemm_cute D2H");
5585
5586     // Verify
5587     float max_err = 0.0f;
5588     for (int i = 0; i < M * N; i++) {

```

```

5589     float err = fabsf(__half2float(h_c[i]) - __half2float(h_c_ref[i]));
5590     if (err > max_err) max_err = err;
5591 }
5592 printf("| %-42s | %.6e | %-4s | %-8s |\n", "HGEMM CuTe Swizzle + Reg2x",
5593        max_err, max_err < 1.0f ? "PASS" : "FAIL", "None");
5594
5595 free(h_a); free(h_b); free(h_b_t); free(h_c); free(h_c_ref);
5596 cudaFree(d_a); cudaFree(d_b); cudaFree(d_b_t); cudaFree(d_c);
5597 cublasDestroy(handle);
5598 }
5599 #endif /* NOTES_V2_ENABLE_CUTE */
5600
5601
5602 #if defined(NOTES_V2_ENABLE_WGMMMA)
5603 static void test_hgemm_wgmma(int M, int N, int K) {
5604     // HGEMM WGMMMA — m64n128k16 + TMA + Warp Specialization (Hopper SM90+)
5605     // TN layout: C[M×N] = A[M×K] × BT[N×K]
5606     // Kernel: hgemm_wgmma_stages_tn with default template params
5607
5608     constexpr int BM = 128, BN = 128, BK = 64, kStages = 3, kNumThreads = 256;
5609
5610     // M, K must be divisible by tile dims
5611     if (M % BM != 0 || N % BN != 0 || K % BK != 0) {
5612         printf("| %-42s | %-12s | %-4s | %-8s |\n",
5613                "HGEMM WGMMMA", "SKIP", "SKIP", "None");
5614         return;
5615     }
5616
5617     size_t size_a = (size_t)M * K * sizeof(half);
5618     size_t size_b = (size_t)K * N * sizeof(half);
5619     size_t size_c = (size_t)M * N * sizeof(half);
5620
5621     half *h_a = (half *)malloc(size_a);
5622     half *h_b = (half *)malloc(size_b);
5623     half *h_c_ref = (half *)malloc(size_c);
5624
5625     srand(42);
5626     for (int i = 0; i < M * K; i++)
5627         h_a[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
5628     for (int i = 0; i < K * N; i++)
5629         h_b[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
5630
5631     // BT [N×K] row-major for TN layout (same as hgemm_mma kernel)
5632     size_t size_b_t = (size_t)N * K * sizeof(half);
5633     half *h_b_t = (half *)malloc(size_b_t);
5634     for (int n = 0; n < N; n++)
5635         for (int k = 0; k < K; k++)
5636             h_b_t[n * K + k] = h_b[k * N + n];
5637
5638     half *d_a, *d_b, *d_b_t, *d_c;
5639     check(cudaMalloc(&d_a, size_a), "wgmma alloc A");
5640     check(cudaMalloc(&d_b, size_b), "wgmma alloc B (cuBLAS)");
5641     check(cudaMalloc(&d_b_t, size_b_t), "wgmma alloc B_t (kernel)");
5642     check(cudaMalloc(&d_c, size_c), "wgmma alloc C");
5643
5644     check(cudaMemcpy(d_a, h_a, size_a, cudaMemcpyHostToDevice), "wgmma H2D A");
5645     check(cudaMemcpy(d_b, h_b, size_b, cudaMemcpyHostToDevice),
5646            "wgmma H2D B (cuBLAS)");
5647     check(cudaMemcpy(d_b_t, h_b_t, size_b_t, cudaMemcpyHostToDevice),
5648            "wgmma H2D B_t (kernel)");
5649
5650     // cuBLAS FP16 reference (row-major idiom, same as hgemm_mma)
5651     cublasHandle_t handle;

```

```

5652 cublasCreate(&handle);
5653 half alpha_h = __float2half(1.0f), beta_h = __float2half(0.0f);
5654 cublasGemmEx(handle, CUBLAS_OP_N, CUBLAS_OP_N, N, M, K, &alpha_h, d_b,
5655             CUDA_R_16F, N, d_a, CUDA_R_16F, K, &beta_h, d_c, CUDA_R_16F, N,
5656             CUBLAS_COMPUTE_16F, CUBLAS_GEMM_DEFAULT);
5657 check(cudaMemcpy(h_c_ref, d_c, size_c, cudaMemcpyDeviceToHost),
5658       "wgmma D2H ref");
5659
5660 // Create TMA tensor maps for A and B^T
5661 // A[MxK] row-major: TMA box=(BK=64, BM=128), global shape=(K, M)
5662 // B^T[NxK] row-major: TMA box=(BK=64, BN=128), global shape=(K, N)
5663 CUTensorMap *tma_a =
5664     allocate_and_create_tensor_map(d_a, M / BM, K / BK);
5665 CUTensorMap *tma_b =
5666     allocate_and_create_tensor_map(d_b_t, N / BN, K / BK);
5667
5668 // Launch WGMMMA kernel
5669 // kBlockSwizzle=false → 2D grid, no swizzle
5670 size_t smem_bytes =
5671     kStages * (BM * BK + BN * BK) * sizeof(half); // 3*16384*2 = 96KB
5672 cudaFuncSetAttribute(
5673     hgemm_wgmma_stages_tn<64, 128, 16, BM, BN, BK, kNumThreads, kStages,
5674     false>,
5675     cudaFuncAttributeMaxDynamicSharedMemorySize, smem_bytes);
5676
5677 dim3 block(kNumThreads);
5678 dim3 grid((N + BN - 1) / BN, (M + BM - 1) / BM);
5679 hgemm_wgmma_stages_tn<64, 128, 16, BM, BN, BK, kNumThreads, kStages, false>
5680     <<<grid, block, smem_bytes>>>(M, N, K, d_c, tma_a, tma_b);
5681 check(cudaGetLastError(), "wgmma launch");
5682 check(cudaDeviceSynchronize(), "wgmma sync");
5683
5684 half *h_c = (half *)malloc(size_c);
5685 check(cudaMemcpy(h_c, d_c, size_c, cudaMemcpyDeviceToHost), "wgmma D2H");
5686
5687 // Verify
5688 float max_err = 0.0f;
5689 for (int i = 0; i < M * N; i++) {
5690     float err = fabsf(__half2float(h_c[i]) - __half2float(h_c_ref[i]));
5691     if (err > max_err)
5692         max_err = err;
5693 }
5694 printf("| %-42s | %.6e | %-4s | %-8s |\n", "HGEMM TMA WGMMMA WS (3-stage)", max_err,
5695       max_err < 1.0f ? "PASS" : "FAIL", "None");
5696
5697 free(h_a);
5698 free(h_b);
5699 free(h_b_t);
5700 free(h_c);
5701 free(h_c_ref);
5702 cudaFree(d_a);
5703 cudaFree(d_b);
5704 cudaFree(d_b_t);
5705 cudaFree(d_c);
5706 cudaFree(tma_a);
5707 cudaFree(tma_b);
5708 cublasDestroy(handle);
5709 }
5710 #endif /* NOTES_V2_ENABLE_WGMMMA */
5711
5712 #if defined(NOTES_V2_ENABLE_TMA_MMA_WS)
5713 template <int kStages, int kBlockSwizzle = 0>
5714 static bool launch_hgemm_tma_mma_ws(int M, int N, int K, half *d_a,

```

```

5715         half *d_b_t, half *d_c,
5716         CUTensorMap *tma_a, CUTensorMap *tma_b) {
5717     constexpr int kMmaM = 16, kMmaN = 8, kMmaK = 16;
5718     constexpr int kMmaTileM = 2, kMmaTileN = 2;
5719     constexpr int kValTileM = 4, kValTileN = 8, kValTileK = 4;
5720     constexpr int BM = kMmaM * kMmaTileM * kValTileM;
5721     constexpr int BN = kMmaN * kMmaTileN * kValTileN;
5722     constexpr int BK = kMmaK * kValTileK;
5723     constexpr int kNumThreads = 256;
5724     constexpr size_t payload_bytes =
5725         kStages * (BM * BK + BN * BK) * sizeof(half);
5726     constexpr size_t smem_bytes = payload_bytes;
5727     using Kernel = void (*)(int, int, int, half *, const CUTensorMap *,
5728                             const CUTensorMap *);
5729     Kernel kernel = hgemm_tma_mma_ws_tn<
5730         kMmaM, kMmaN, kMmaK, kMmaTileM, kMmaTileN, kValTileM, kValTileN,
5731         kValTileK, kStages, kNumThreads, kBlockSwizzle>;
5732
5733     int device = 0;
5734     int max_smem = 0;
5735     cudaFuncAttributes attributes{};
5736     check(cudaGetDevice(&device), "tma_mma_ws get device");
5737     check(cudaDeviceGetAttribute(&max_smem,
5738                                 cudaDevAttrMaxSharedMemoryPerBlockOptin,
5739                                 device),
5740           "tma_mma_ws max shared memory");
5741     check(cudaFuncGetAttributes(&attributes, kernel),
5742           "tma_mma_ws function attributes");
5743     if (smem_bytes + attributes.sharedSizeBytes > size_t(max_smem)) {
5744         printf("| %-42s | %-12s | %-4s | %-8s |\n",
5745             kStages == 2 ? "HGEMM TMA MMA WS (S=2, SW=0)"
5746             : "HGEMM TMA MMA WS (S=3, SW=0)",
5747             "SMEM SKIP", "SKIP", "None");
5748         return false;
5749     }
5750
5751     check(cudaFuncSetAttribute(kernel, cudaFuncAttributeMaxDynamicSharedMemorySize,
5752                               smem_bytes),
5753           "tma_mma_ws set dynamic shared memory");
5754     dim3 block(kNumThreads);
5755     constexpr int kSwizzleN = 16;
5756     const int n_tiles = N / BN;
5757     const int grid_x = kBlockSwizzle ? div_ceil(n_tiles, kSwizzleN) : n_tiles;
5758     const int grid_z = kBlockSwizzle ? kSwizzleN : 1;
5759     dim3 grid(grid_x, M / BM, grid_z);
5760     kernel<<<grid, block, smem_bytes>>>(M, N, K, d_c, tma_a, tma_b);
5761     check(cudaGetLastError(), "tma_mma_ws launch");
5762     check(cudaDeviceSynchronize(), "tma_mma_ws sync");
5763     return true;
5764 }
5765
5766 static void test_hgemm_tma_mma_ws(int M, int N, int K) {
5767     constexpr int BM = 128, BN = 128, BK = 64;
5768     if (M % BM != 0 || N % BN != 0 || K % BK != 0) {
5769         printf("| %-42s | %-12s | %-4s | %-8s |\n", "HGEMM TMA MMA WS", "SKIP",
5770             "SKIP", "None");
5771         return;
5772     }
5773
5774     const size_t size_a = size_t(M) * K * sizeof(half);
5775     const size_t size_b = size_t(K) * N * sizeof(half);
5776     const size_t size_b_t = size_t(N) * K * sizeof(half);
5777     const size_t size_c = size_t(M) * N * sizeof(half);

```



```

5778 half *h_a = static_cast<half *>(malloc(size_a));
5779 half *h_b = static_cast<half *>(malloc(size_b));
5780 half *h_b_t = static_cast<half *>(malloc(size_b_t));
5781 half *h_c = static_cast<half *>(malloc(size_c));
5782 half *h_c_ref = static_cast<half *>(malloc(size_c));
5783 srand(42);
5784 for (int i = 0; i < M * K; ++i)
5785     h_a[i] = __float2half((float(rand()) / RAND_MAX) * 2.0f - 1.0f);
5786 for (int i = 0; i < K * N; ++i)
5787     h_b[i] = __float2half((float(rand()) / RAND_MAX) * 2.0f - 1.0f);
5788 for (int n = 0; n < N; ++n)
5789     for (int k = 0; k < K; ++k)
5790         h_b_t[n * K + k] = h_b[k * N + n];
5791
5792 half *d_a, *d_b, *d_b_t, *d_c;
5793 check(cudaMalloc(&d_a, size_a), "tma_mma_ws alloc A");
5794 check(cudaMalloc(&d_b, size_b), "tma_mma_ws alloc B");
5795 check(cudaMalloc(&d_b_t, size_b_t), "tma_mma_ws alloc B_t");
5796 check(cudaMalloc(&d_c, size_c), "tma_mma_ws alloc C");
5797 check(cudaMemcpy(d_a, h_a, size_a, cudaMemcpyHostToDevice), "tma_mma_ws H2D A");
5798 check(cudaMemcpy(d_b, h_b, size_b, cudaMemcpyHostToDevice), "tma_mma_ws H2D B");
5799 check(cudaMemcpy(d_b_t, h_b_t, size_b_t, cudaMemcpyHostToDevice),
5800         "tma_mma_ws H2D B_t");
5801
5802 cublasHandle_t handle;
5803 cublasCreate(&handle);
5804 half alpha = __float2half(1.0f), beta = __float2half(0.0f);
5805 cublasGemmEx(handle, CUBLAS_OP_N, CUBLAS_OP_N, N, M, K, &alpha, d_b,
5806              CUDA_R_16F, N, d_a, CUDA_R_16F, K, &beta, d_c, CUDA_R_16F, N,
5807              CUBLAS_COMPUTE_16F, CUBLAS_GEMM_DEFAULT);
5808 check(cudaMemcpy(h_c_ref, d_c, size_c, cudaMemcpyDeviceToHost),
5809         "tma_mma_ws D2H reference");
5810
5811 CUTensorMap *tma_a = allocate_and_create_tensor_map(d_a, M / BM, K / BK);
5812 CUTensorMap *tma_b = allocate_and_create_tensor_map(d_b_t, N / BN, K / BK);
5813 for (int block_swizzle : {0, 1}) {
5814     for (int stages : {2, 3}) {
5815         const bool launched = stages == 2
5816             ? (block_swizzle
5817                ? launch_hgemm_tma_mma_ws<2, 1>(M, N, K, d_a, d_b_t, d_c,
5818                                                  tma_a, tma_b)
5819                : launch_hgemm_tma_mma_ws<2>(M, N, K, d_a, d_b_t, d_c,
5820                                              tma_a, tma_b))
5821             : (block_swizzle
5822                ? launch_hgemm_tma_mma_ws<3, 1>(M, N, K, d_a, d_b_t, d_c,
5823                                                  tma_a, tma_b)
5824                : launch_hgemm_tma_mma_ws<3>(M, N, K, d_a, d_b_t, d_c,
5825                                              tma_a, tma_b));
5826         if (!launched)
5827             continue;
5828         check(cudaMemcpy(h_c, d_c, size_c, cudaMemcpyDeviceToHost),
5829               "tma_mma_ws D2H");
5830         float max_err = 0.0f;
5831         for (int i = 0; i < M * N; ++i)
5832             max_err = fmaxf(max_err, fabsf(__half2float(h_c[i]) -
5833                                              __half2float(h_c_ref[i])));
5834         printf("| %-42s | %.6e | %-4s | %-8s |\n",
5835               block_swizzle
5836                 ? (stages == 2 ? "HGEMM TMA MMA WS (S=2, SW=1)"
5837                               : "HGEMM TMA MMA WS (S=3, SW=1)")
5838                 : (stages == 2 ? "HGEMM TMA MMA WS (S=2, SW=0)"
5839                               : "HGEMM TMA MMA WS (S=3, SW=0)"),
5840               max_err, max_err < 1.0f ? "PASS" : "FAIL", "None");

```

```

5841     }
5842 }
5843
5844 free(h_a); free(h_b); free(h_b_t); free(h_c); free(h_c_ref);
5845 cudaFree(d_a); cudaFree(d_b); cudaFree(d_b_t); cudaFree(d_c);
5846 cudaFree(tma_a); cudaFree(tma_b);
5847 cublasDestroy(handle);
5848 }
5849 #endif /* NOTES_V2_ENABLE_TMA_MMA_WS */
5850
5851
5852 static void test_flash_attn(int seqlen, int head_dim) {
5853     // FlashAttention-2 with split-Q, MMA m16n8k16
5854     int B = 1, H = 8;
5855
5856     size_t sz = (size_t)B * H * seqlen * head_dim * sizeof(half);
5857
5858     srand(42);
5859     half *h_q = (half *)malloc(sz);
5860     half *h_k = (half *)malloc(sz);
5861     half *h_v = (half *)malloc(sz);
5862     for (int i = 0; i < B * H * seqlen * head_dim; i++) {
5863         h_q[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
5864         h_k[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
5865         h_v[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
5866     }
5867
5868     // CPU reference in FP32: 0 = softmax(Q @ K^T / sqrt(d)) @ V
5869     float *ref_q = (float *)malloc(sz * 4 / sizeof(half)); // 4x for float
5870     float *ref_k = (float *)malloc(sz * 4 / sizeof(half));
5871     float *ref_v = (float *)malloc(sz * 4 / sizeof(half));
5872     float *ref_o = (float *)malloc(sz * 4 / sizeof(half));
5873     int count = B * H * seqlen * head_dim;
5874     for (int i = 0; i < count; i++) {
5875         ref_q[i] = __half2float(h_q[i]);
5876         ref_k[i] = __half2float(h_k[i]);
5877         ref_v[i] = __half2float(h_v[i]);
5878     }
5879
5880     float scale = 1.0f / sqrtf((float)head_dim);
5881     for (int bi = 0; bi < B * H; bi++) {
5882         for (int qi = 0; qi < seqlen; qi++) {
5883             // S[qi, kj] = Q[qi,:] @ K[kj,:]^T * scale
5884             float smax = -INFINITY;
5885             float *S = (float *)malloc((size_t)seqlen * sizeof(float));
5886             for (int kj = 0; kj < seqlen; kj++) {
5887                 float s = 0.0f;
5888                 for (int d = 0; d < head_dim; d++)
5889                     s += ref_q[bi * seqlen * head_dim + qi * head_dim + d] *
5890                         ref_k[bi * seqlen * head_dim + kj * head_dim + d];
5891                 S[kj] = s * scale;
5892                 if (S[kj] > smax) smax = S[kj];
5893             }
5894             // softmax
5895             double sum_exp = 0.0;
5896             for (int kj = 0; kj < seqlen; kj++) sum_exp += (double)expf(S[kj] - smax);
5897             float inv_sum = 1.0f / (float)sum_exp;
5898             // O[qi, :] = sum_kj P[qi, kj] * V[kj, :]
5899             for (int d = 0; d < head_dim; d++) {
5900                 double o_acc = 0.0;
5901                 for (int kj = 0; kj < seqlen; kj++)
5902                     o_acc += (double)(expf(S[kj] - smax) * inv_sum) *
5903                         ref_v[bi * seqlen * head_dim + kj * head_dim + d];

```

```

5904         ref_o[bi * seqlen * head_dim + qi * head_dim + d] = (float)o_acc;
5905     }
5906     free(S);
5907 }
5908 }
5909
5910 half *d_q, *d_k, *d_v, *d_o;
5911 check(cudaMalloc(&d_q, sz), "fa alloc Q");
5912 check(cudaMalloc(&d_k, sz), "fa alloc K");
5913 check(cudaMalloc(&d_v, sz), "fa alloc V");
5914 check(cudaMalloc(&d_o, sz), "fa alloc O");
5915 check(cudaMemcpy(d_q, h_q, sz, cudaMemcpyHostToDevice), "fa H2D Q");
5916 check(cudaMemcpy(d_k, h_k, sz, cudaMemcpyHostToDevice), "fa H2D K");
5917 check(cudaMemcpy(d_v, h_v, sz, cudaMemcpyHostToDevice), "fa H2D V");
5918
5919 // Template params for kHeadDim=64, kStagesK=2
5920 constexpr int kHeadDim = 64;
5921 constexpr int kStagesK = 2;
5922 constexpr int kPadQ = 8;
5923 constexpr int kPadK = 8;
5924 constexpr int kPadV = 8;
5925 constexpr int kMmaAtomM = 16;
5926 constexpr int kMmaAtomN = 8;
5927 constexpr int kMmaAtomK = 16;
5928 constexpr int kMmaTileSeqLenQ = 8;
5929 constexpr int kMmaTileSeqLenK = 1;
5930 constexpr int kMmaTileSeqLenP = 8;
5931 constexpr int kMmaTileHeadDimV = 1;
5932 constexpr int kValTileSeqLenQ = 1;
5933 constexpr int kValTileSeqLenK = 8;
5934 constexpr int kValTileSeqLenP = 1;
5935 constexpr int kValTileHeadDimV = kHeadDim / (8 * kMmaTileHeadDimV);
5936
5937 constexpr int Br = kMmaAtomM * kMmaTileSeqLenQ * kValTileSeqLenQ;
5938 constexpr int Bc = kMmaAtomN * kMmaTileSeqLenK * kValTileSeqLenK;
5939 if (seqlen < Br) return; // kernel requires seqlen >= tile size
5940 size_t smem_bytes =
5941     (Br * (kHeadDim + kPadQ) +
5942      kStagesK * Bc * (kHeadDim + kPadK) +
5943      Bc * (kHeadDim + kPadV)) * sizeof(half);
5944
5945 dim3 block(256);
5946 dim3 grid((seqlen + Br - 1) / Br, B * H);
5947
5948 using FAKernel = void (*)(half *, half *, half *, half *, int, int);
5949 FAKernel fa_k = flash_attn_mma_stages_split_q<kHeadDim, kMmaAtomM, kMmaAtomN,
5950     kMmaAtomK, kMmaTileSeqLenQ, kMmaTileSeqLenK, kMmaTileSeqLenP,
5951     kMmaTileHeadDimV, kValTileSeqLenQ, kValTileSeqLenK, kValTileSeqLenP,
5952     kValTileHeadDimV, kStagesK, kPadQ, kPadK, kPadV>;
5953 cudaFuncSetAttribute(fa_k, cudaFuncAttributeMaxDynamicSharedMemorySize, smem_bytes);
5954 fa_k<<<grid, block, smem_bytes>>>(d_q, d_k, d_v, d_o, seqlen, H);
5955 check(cudaGetLastError(), "fa launch");
5956 check(cudaDeviceSynchronize(), "fa sync");
5957
5958 half *h_o = (half *)malloc(sz);
5959 check(cudaMemcpy(h_o, d_o, sz, cudaMemcpyDeviceToHost), "fa D2H");
5960
5961 float max_err = 0.0f;
5962 for (int i = 0; i < count; i++) {
5963     float err = fabsf(__half2float(h_o[i]) - ref_o[i]);
5964     if (err > max_err) max_err = err;
5965 }
5966 printf("| %-42s | %.6e | %-4s | %-8s |\n", "FlashAttention-2 (kStagesK=2, Pad)",

```

```

5967         max_err, max_err < 5e-1f ? "PASS" : "FAIL", "None");
5968
5969     free(h_q); free(h_k); free(h_v); free(h_o);
5970     free(ref_q); free(ref_k); free(ref_v); free(ref_o);
5971     cudaFree(d_q); cudaFree(d_k); cudaFree(d_v); cudaFree(d_o);
5972 }
5973
5974
5975 // Bench mode globals and helpers
5976 static bool g_bench_mode = false;
5977 static bool g_bench_fa = false;
5978 static bool g_bench_only_fa = false;
5979 static bool g_fa_skip_check = false;
5980 enum class FALayout {All, Pad, SwizzleQ, SwizzleK, SwizzleV, SwizzleQK, SwizzleQV, SwizzleKV, Swizzle};
5981 static FALayout g_fa_layout = FALayout::Pad;
5982 static int g_bench_M = 4096, g_bench_N = 4096, g_bench_K = 4096;
5983 static int g_bench_B = 1, g_bench_H = 32, g_bench_Nfa = 4096, g_bench_D = 64;
5984 static int g_warmup = 5, g_repeat = 10;
5985
5986 static float bench_hgemm_tflops(int M, int N, int K, float time_ms) {
5987     double flops = 2.0 * M * N * K;
5988     return (float)(flops / (double)time_ms / 1e9);
5989 }
5990
5991 static float bench_fa_tflops(int B, int H, int N, int D, float time_ms) {
5992     double flops = 4.0 * B * H * N * N * D;
5993     return (float)(flops / (double)time_ms / 1e9);
5994 }
5995
5996 static bool check_smem_feasible(const void *kernel_func, size_t dyn_smem_bytes) {
5997     int device = 0;
5998     int max_smem = 0;
5999     cudaFuncAttributes attrs{};
6000     cudaGetDevice(&device);
6001     cudaDeviceGetAttribute(&max_smem, cudaDevAttrMaxSharedMemoryPerBlockOptin, device);
6002     cudaFuncGetAttributes(&attrs, kernel_func);
6003     return dyn_smem_bytes + attrs.sharedSizeBytes <= (size_t)max_smem;
6004 }
6005
6006 // =====
6007 // Bench: HGEMM MMA (basic m16n8k16 + multistage pipeline)
6008 // =====
6009 template <int kStages, int kBlockSwizzle>
6010 static bool launch_timed_hgemm_mma(
6011     half *d_a, half *d_b_t, half *d_c, half *h_c,
6012     half *h_c_ref, int M, int N, int K, size_t size_c,
6013     cudaEvent_t start, cudaEvent_t stop, float &max_err,
6014     float &time_ms
6015 ) {
6016     constexpr int BM = 128, BN = 128, BK = 16;
6017     using Kernel = void (*)(half *, half *, half *, int, int, int);
6018     Kernel k = hgemm_mma_stages_tn<16, 8, 16, 2, 4, 4, 4, kStages, kBlockSwizzle>;
6019     size_t smem = kStages * (BM * BK + BN * BK) * sizeof(half);
6020     if (!check_smem_feasible((const void *)k, smem)) return false;
6021     dim3 block(256);
6022     const int tiles_n = (N + BN - 1) / BN;
6023     constexpr int kSwizzleN = 16;
6024     int gx = kBlockSwizzle ? div_ceil(tiles_n, kSwizzleN) : tiles_n;
6025     int gz = kBlockSwizzle ? kSwizzleN : 1;
6026     dim3 grid(gx, (M + BM - 1) / BM, gz);
6027     for (int w = 0; w < g_warmup; w++)
6028         k<<<grid, block, smem>>>(d_a, d_b_t, d_c, M, N, K);
6029     cudaDeviceSynchronize();

```

```

6030 cudaEventRecord(start);
6031 for (int r = 0; r < g_repeat; r++)
6032     k<<<grid, block, smem>>>(d_a, d_b_t, d_c, M, N, K);
6033 cudaEventRecord(stop);
6034 cudaEventSynchronize(stop);
6035 cudaEventElapsedTime(&time_ms, start, stop);
6036 time_ms /= g_repeat;
6037 check(cudaMemcpy(h_c, d_c, size_c, cudaMemcpyDeviceToHost), "bench mma D2H");
6038 max_err = 0.0f;
6039 for (int i = 0; i < M * N; i++) {
6040     float err = fabsf(__half2float(h_c[i]) - __half2float(h_c_ref[i]));
6041     if (err > max_err) max_err = err;
6042 }
6043 return true;
6044 }
6045
6046 static void bench_hgemm_mma(int M, int N, int K) {
6047     size_t size_a = (size_t)M * K * sizeof(half);
6048     size_t size_b = (size_t)K * N * sizeof(half);
6049     size_t size_c = (size_t)M * N * sizeof(half);
6050     half *h_a = (half *)malloc(size_a);
6051     half *h_b = (half *)malloc(size_b);
6052     half *h_c_ref = (half *)malloc(size_c);
6053     srand(42);
6054     for (int i = 0; i < M * K; i++)
6055         h_a[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
6056     for (int i = 0; i < K * N; i++)
6057         h_b[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
6058     size_t size_b_t = (size_t)N * K * sizeof(half);
6059     half *h_b_t = (half *)malloc(size_b_t);
6060     for (int n = 0; n < N; n++)
6061         for (int k = 0; k < K; k++)
6062             h_b_t[n * K + k] = h_b[k * N + n];
6063     half *d_a, *d_b, *d_b_t, *d_c;
6064     check(cudaMalloc(&d_a, size_a), "bench mma alloc A");
6065     check(cudaMalloc(&d_b, size_b), "bench mma alloc B");
6066     check(cudaMalloc(&d_b_t, size_b_t), "bench mma alloc B_t");
6067     check(cudaMalloc(&d_c, size_c), "bench mma alloc C");
6068     check(cudaMemcpy(d_a, h_a, size_a, cudaMemcpyHostToDevice), "bench mma H2D A");
6069     check(cudaMemcpy(d_b, h_b, size_b, cudaMemcpyHostToDevice), "bench mma H2D B");
6070     check(cudaMemcpy(d_b_t, h_b_t, size_b_t, cudaMemcpyHostToDevice), "bench mma H2D B_t");
6071     cublasHandle_t handle;
6072     cublasCreate(&handle);
6073     half alpha_h = __float2half(1.0f), beta_h = __float2half(0.0f);
6074     cublasGemmEx(handle, CUBLAS_OP_N, CUBLAS_OP_N, N, M, K, &alpha_h, d_b, CUDA_R_16F, N,
6075         d_a, CUDA_R_16F, K, &beta_h, d_c, CUDA_R_16F, N, CUBLAS_COMPUTE_16F,
6076         CUBLAS_GEMM_DEFAULT);
6077     check(cudaMemcpy(h_c_ref, d_c, size_c, cudaMemcpyDeviceToHost), "bench mma D2H ref");
6078     half *h_c = (half *)malloc(size_c);
6079     cudaEvent_t start, stop;
6080     cudaEventCreate(&start);
6081     cudaEventCreate(&stop);
6082     for (int stages : {2, 3}) {
6083         for (int swizzle : {0, 1}) {
6084             float max_err = 0, time_ms = 0;
6085             bool ok = false;
6086             if (stages == 2)
6087                 ok = swizzle ? launch_timed_hgemm_mma<2, 1>(d_a, d_b_t, d_c, h_c, h_c_ref, M, N,
6088                     K, size_c, start, stop, max_err, time_ms)
6089                     : launch_timed_hgemm_mma<2, 0>(d_a, d_b_t, d_c, h_c, h_c_ref, M, N,
6090                     K, size_c, start, stop, max_err, time_ms);
6091             else
6092                 ok = swizzle ? launch_timed_hgemm_mma<3, 1>(d_a, d_b_t, d_c, h_c, h_c_ref, M, N,

```

```

6093         K, size_c, start, stop, max_err, time_ms)
6094     : launch_timed_hgemm_mma<3, 0>(d_a, d_b_t, d_c, h_c, h_c_ref, M, N,
6095         K, size_c, start, stop, max_err, time_ms);
6096     char label[64];
6097     snprintf(label, sizeof(label), "HGEMM MMA (S=%d, SW=%d)", stages, swizzle);
6098     if (!ok) {
6099         printf("| %-42s | %-12s | %-4s | %-8s |\n", label, "SMEM SKIP", "SKIP", "None");
6100     } else {
6101         float tflops = bench_hgemm_tflops(M, N, K, time_ms);
6102         printf("| %-42s | %.6e | %-4s | %-8.1f |\n", label, max_err,
6103             max_err < 1.0f ? "PASS" : "FAIL", tflops);
6104     }
6105 }
6106 }
6107 cudaEventDestroy(start);
6108 cudaEventDestroy(stop);
6109 free(h_a);
6110 free(h_b);
6111 free(h_b_t);
6112 free(h_c);
6113 free(h_c_ref);
6114 cudaFree(d_a);
6115 cudaFree(d_b);
6116 cudaFree(d_b_t);
6117 cudaFree(d_c);
6118 cublasDestroy(handle);
6119 }
6120
6121 // =====
6122 // Bench: HGEMM MMA Swizzle + Register Double Buffering (kValTileK=4, BK=64)
6123 // =====
6124 template <int kStages, int kBlockSwizzle>
6125 static bool launch_timed_hgemm_swizzle(
6126     half *d_a, half *d_b_t, half *d_c, half *h_c,
6127     half *h_c_ref, int M, int N, int K, size_t size_c,
6128     cudaEvent_t start, cudaEvent_t stop, float &max_err,
6129     float &time_ms
6130 ) {
6131     constexpr int BM = 128, BN = 128, BK = 64;
6132     using Kernel = void (*)(half *, half *, half *, int, int, int);
6133     Kernel k = hgemm_mma_stages_tn_swizzle<16, 8, 16, 2, 4, 4, 4, 4, kStages, kBlockSwizzle>;
6134     size_t smem = kStages * (BM * BK + BN * BK) * sizeof(half);
6135     if (!check_smem_feasible((const void *)k, smem)) return false;
6136     cudaFuncSetAttribute((const void *)k, cudaFuncAttributeMaxDynamicSharedMemorySize, smem);
6137     dim3 block(256);
6138     const int tiles_n = (N + BN - 1) / BN;
6139     constexpr int kSwizzleN = 16;
6140     int gx = kBlockSwizzle ? div_ceil(tiles_n, kSwizzleN) : tiles_n;
6141     int gz = kBlockSwizzle ? kSwizzleN : 1;
6142     dim3 grid(gx, (M + BM - 1) / BM, gz);
6143     for (int w = 0; w < g_warmup; w++)
6144         k<<<grid, block, smem>>>(d_a, d_b_t, d_c, M, N, K);
6145     cudaDeviceSynchronize();
6146     cudaEventRecord(start);
6147     for (int r = 0; r < g_repeat; r++)
6148         k<<<grid, block, smem>>>(d_a, d_b_t, d_c, M, N, K);
6149     cudaEventRecord(stop);
6150     cudaEventSynchronize(stop);
6151     cudaEventElapsedTime(&time_ms, start, stop);
6152     time_ms /= g_repeat;
6153     check(cudaMemcpy(h_c, d_c, size_c, cudaMemcpyDeviceToHost), "bench swizzle D2H");
6154     max_err = 0.0f;
6155     for (int i = 0; i < M * N; i++) {

```



```

6156     float err = fabsf(__half2float(h_c[i]) - __half2float(h_c_ref[i]));
6157     if (err > max_err) max_err = err;
6158 }
6159 return true;
6160 }
6161
6162 static void bench_hgemm_swizzle(int M, int N, int K) {
6163     size_t size_a = (size_t)M * K * sizeof(half);
6164     size_t size_b = (size_t)K * N * sizeof(half);
6165     size_t size_c = (size_t)M * N * sizeof(half);
6166     half *h_a = (half *)malloc(size_a);
6167     half *h_b = (half *)malloc(size_b);
6168     half *h_c_ref = (half *)malloc(size_c);
6169     srand(42);
6170     for (int i = 0; i < M * K; i++)
6171         h_a[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
6172     for (int i = 0; i < K * N; i++)
6173         h_b[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
6174     size_t size_b_t = (size_t)N * K * sizeof(half);
6175     half *h_b_t = (half *)malloc(size_b_t);
6176     for (int n = 0; n < N; n++)
6177         for (int k = 0; k < K; k++)
6178             h_b_t[n * K + k] = h_b[k * N + n];
6179     half *d_a, *d_b, *d_b_t, *d_c;
6180     check(cudaMalloc(&d_a, size_a), "bench swizzle alloc A");
6181     check(cudaMalloc(&d_b, size_b), "bench swizzle alloc B");
6182     check(cudaMalloc(&d_b_t, size_b_t), "bench swizzle alloc B_t");
6183     check(cudaMalloc(&d_c, size_c), "bench swizzle alloc C");
6184     check(cudaMemcpy(d_a, h_a, size_a, cudaMemcpyHostToDevice), "bench swizzle H2D A");
6185     check(cudaMemcpy(d_b, h_b, size_b, cudaMemcpyHostToDevice), "bench swizzle H2D B");
6186     check(cudaMemcpy(d_b_t, h_b_t, size_b_t, cudaMemcpyHostToDevice), "bench swizzle H2D B_t");
6187     cublasHandle_t handle;
6188     cublasCreate(&handle);
6189     half alpha_h = __float2half(1.0f), beta_h = __float2half(0.0f);
6190     cublasGemmEx(handle, CUBLAS_OP_N, CUBLAS_OP_N, N, M, K, &alpha_h, d_b, CUDA_R_16F, N,
6191         d_a, CUDA_R_16F, K, &beta_h, d_c, CUDA_R_16F, N, CUBLAS_COMPUTE_16F,
6192         CUBLAS_GEMM_DEFAULT);
6193     check(cudaMemcpy(h_c_ref, d_c, size_c, cudaMemcpyDeviceToHost), "bench swizzle D2H ref");
6194     half *h_c = (half *)malloc(size_c);
6195     cudaEvent_t start, stop;
6196     cudaEventCreate(&start);
6197     cudaEventCreate(&stop);
6198     for (int stages : {2, 3}) {
6199         for (int swizzle : {0, 1}) {
6200             float max_err = 0, time_ms = 0;
6201             bool ok = false;
6202             if (stages == 2)
6203                 ok = swizzle ? launch_timed_hgemm_swizzle<2, 1>(d_a, d_b_t, d_c, h_c, h_c_ref, M,
6204                     N, K, size_c, start, stop, max_err, time_ms)
6205                     : launch_timed_hgemm_swizzle<2, 0>(d_a, d_b_t, d_c, h_c, h_c_ref, M,
6206                     N, K, size_c, start, stop, max_err, time_ms);
6207             else
6208                 ok = swizzle ? launch_timed_hgemm_swizzle<3, 1>(d_a, d_b_t, d_c, h_c, h_c_ref, M,
6209                     N, K, size_c, start, stop, max_err, time_ms)
6210                     : launch_timed_hgemm_swizzle<3, 0>(d_a, d_b_t, d_c, h_c, h_c_ref, M,
6211                     N, K, size_c, start, stop, max_err, time_ms);
6212             char label[64];
6213             snprintf(label, sizeof(label), "HGEMM Swizzle+Reg2x (S=%d, SW=%d)", stages, swizzle);
6214             if (!ok) {
6215                 printf("| %-42s | %-12s | %-4s | %-8s |\n", label, "SMEM SKIP", "SKIP", "None");
6216             } else {
6217                 float tflops = bench_hgemm_tflops(M, N, K, time_ms);
6218                 printf("| %-42s | %-6e | %-4s | %-8.1f |\n", label, max_err,

```

```

6219         max_err < 1.0f ? "PASS" : "FAIL", tflops);
6220     }
6221 }
6222 }
6223 cudaEventDestroy(start);
6224 cudaEventDestroy(stop);
6225 free(h_a);
6226 free(h_b);
6227 free(h_b_t);
6228 free(h_c);
6229 free(h_c_ref);
6230 cudaFree(d_a);
6231 cudaFree(d_b);
6232 cudaFree(d_b_t);
6233 cudaFree(d_c);
6234 cublasDestroy(handle);
6235 }
6236
6237 #if defined(NOTES_V2_ENABLE_CUTE)
6238 // =====
6239 // Bench: HGEMM CuTe
6240 // =====
6241 static void bench_hgemm_cute(int M, int N, int K) {
6242     size_t size_a = (size_t)M * K * sizeof(half);
6243     size_t size_b = (size_t)K * N * sizeof(half);
6244     size_t size_c = (size_t)M * N * sizeof(half);
6245     half *h_a = (half *)malloc(size_a);
6246     half *h_b = (half *)malloc(size_b);
6247     half *h_c_ref = (half *)malloc(size_c);
6248     srand(42);
6249     for (int i = 0; i < M * K; i++)
6250         h_a[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
6251     for (int i = 0; i < K * N; i++)
6252         h_b[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
6253     size_t size_b_t = (size_t)N * K * sizeof(half);
6254     half *h_b_t = (half *)malloc(size_b_t);
6255     for (int n = 0; n < N; n++)
6256         for (int k = 0; k < K; k++)
6257             h_b_t[n * K + k] = h_b[k * N + n];
6258     half *d_a, *d_b, *d_b_t, *d_c;
6259     check(cudaMalloc(&d_a, size_a), "bench cute alloc A");
6260     check(cudaMalloc(&d_b, size_b), "bench cute alloc B");
6261     check(cudaMalloc(&d_b_t, size_b_t), "bench cute alloc B_t");
6262     check(cudaMalloc(&d_c, size_c), "bench cute alloc C");
6263     check(cudaMemcpy(d_a, h_a, size_a, cudaMemcpyHostToDevice), "bench cute H2D A");
6264     check(cudaMemcpy(d_b, h_b, size_b, cudaMemcpyHostToDevice), "bench cute H2D B");
6265     check(cudaMemcpy(d_b_t, h_b_t, size_b_t, cudaMemcpyHostToDevice), "bench cute H2D B_t");
6266     cublasHandle_t handle;
6267     cublasCreate(&handle);
6268     half alpha_h = __float2half(1.0f), beta_h = __float2half(0.0f);
6269     cublasGemmEx(handle, CUBLAS_OP_N, CUBLAS_OP_N, N, M, K, &alpha_h, d_b, CUDA_R_16F, N,
6270         d_a, CUDA_R_16F, K, &beta_h, d_c, CUDA_R_16F, N, CUBLAS_COMPUTE_16F,
6271         CUBLAS_GEMM_DEFAULT);
6272     check(cudaMemcpy(h_c_ref, d_c, size_c, cudaMemcpyDeviceToHost), "bench cute D2H ref");
6273     half *h_c = (half *)malloc(size_c);
6274     cudaEvent_t start, stop;
6275     cudaEventCreate(&start);
6276     cudaEventCreate(&stop);
6277     for (int stages : {2, 3}) {
6278         for (int swizzle : {0, 1}) {
6279             char label[64];
6280             snprintf(label, sizeof(label), "HGEMM CuTe Swizzle (S=%d, SW=%d)", stages, swizzle);
6281             bool ok = false;

```

```

6282 float time_ms = 0, max_err = 0;
6283 if (stages == 2) {
6284     auto k = swizzle ? launch_hgemm_mma_stages_tn_cute<half, 2, 1>
6285                       : launch_hgemm_mma_stages_tn_cute<half, 2, 0>;
6286     for (int w = 0; w < g_warmup; w++) k(d_a, d_b_t, d_c, M, N, K);
6287     cudaDeviceSynchronize();
6288     cudaEventRecord(start);
6289     for (int r = 0; r < g_repeat; r++) k(d_a, d_b_t, d_c, M, N, K);
6290     cudaEventRecord(stop);
6291     cudaEventSynchronize(stop);
6292     cudaEventElapsedTime(&time_ms, start, stop);
6293     time_ms /= g_repeat;
6294     ok = true;
6295 } else {
6296     auto k = swizzle ? launch_hgemm_mma_stages_tn_cute<half, 3, 1>
6297                       : launch_hgemm_mma_stages_tn_cute<half, 3, 0>;
6298     for (int w = 0; w < g_warmup; w++) k(d_a, d_b_t, d_c, M, N, K);
6299     cudaDeviceSynchronize();
6300     cudaEventRecord(start);
6301     for (int r = 0; r < g_repeat; r++) k(d_a, d_b_t, d_c, M, N, K);
6302     cudaEventRecord(stop);
6303     cudaEventSynchronize(stop);
6304     cudaEventElapsedTime(&time_ms, start, stop);
6305     time_ms /= g_repeat;
6306     ok = true;
6307 }
6308 if (ok) {
6309     check(cudaMemcpy(h_c, d_c, size_c, cudaMemcpyDeviceToHost), "bench cute D2H");
6310     max_err = 0.0f;
6311     for (int i = 0; i < M * N; i++) {
6312         float err = fabsf(__half2float(h_c[i]) - __half2float(h_c_ref[i]));
6313         if (err > max_err) max_err = err;
6314     }
6315     float tflops = bench_hgemm_tflops(M, N, K, time_ms);
6316     printf("| %-42s | %.6e | %-4s | %-8.1f |\n", label, max_err,
6317           max_err < 1.0f ? "PASS" : "FAIL", tflops);
6318 } else {
6319     printf("| %-42s | %-12s | %-4s | %-8s |\n", label, "LAUNCH ERR", "FAIL", "None");
6320 }
6321 }
6322 }
6323 cudaEventDestroy(start);
6324 cudaEventDestroy(stop);
6325 free(h_a);
6326 free(h_b);
6327 free(h_b_t);
6328 free(h_c);
6329 free(h_c_ref);
6330 cudaFree(d_a);
6331 cudaFree(d_b);
6332 cudaFree(d_b_t);
6333 cudaFree(d_c);
6334 cublasDestroy(handle);
6335 }
6336 #endif
6337
6338 #if defined(NOTES_V2_ENABLE_WGMMA)
6339 // =====
6340 // Bench: HGEMM WGMMA (m64n128k16 + TMA + Warp Specialization, SM90+)
6341 // =====
6342 template <int kStages, int kBlockSwizzle>
6343 static bool launch_timed_hgemm_wgmma(half *d_c, half *h_c, half *h_c_ref,
6344                                     CUTensorMap *tma_a, CUTensorMap *tma_b,

```

```

6345         int M, int N, int K, size_t size_c,
6346         cudaEvent_t start, cudaEvent_t stop,
6347         float &max_err, float &time_ms) {
6348     constexpr int BM = 128, BN = 128, BK = 64, kNumThreads = 256;
6349     using Kernel = void (*)(int, int, int, half *, const CUTensorMap *,
6350         const CUTensorMap *);
6351     Kernel k = hgemm_wgmma_stages_tn<64, 128, 16, BM, BN, BK, kNumThreads, kStages,
6352         kBlockSwizzle>;
6353     size_t smem = kStages * (BM * BK + BN * BK) * sizeof(half);
6354     if (!check_smem_feasible((const void *)k, smem)) return false;
6355     cudaFuncSetAttribute(k, cudaFuncAttributeMaxDynamicSharedMemorySize, smem);
6356     dim3 block(kNumThreads);
6357     const int tiles_n = (N + BN - 1) / BN;
6358     constexpr int kSwizzleN = 16;
6359     int gx = kBlockSwizzle ? div_ceil(tiles_n, kSwizzleN) : tiles_n;
6360     int gz = kBlockSwizzle ? kSwizzleN : 1;
6361     dim3 grid(gx, (M + BM - 1) / BM, gz);
6362     for (int w = 0; w < g_warmup; w++)
6363         k<<<grid, block, smem>>>(M, N, K, d_c, tma_a, tma_b);
6364     cudaDeviceSynchronize();
6365     cudaEventRecord(start);
6366     for (int r = 0; r < g_repeat; r++)
6367         k<<<grid, block, smem>>>(M, N, K, d_c, tma_a, tma_b);
6368     cudaEventRecord(stop);
6369     cudaEventSynchronize(stop);
6370     cudaEventElapsedTime(&time_ms, start, stop);
6371     time_ms /= g_repeat;
6372     check(cudaMemcpy(h_c, d_c, size_c, cudaMemcpyDeviceToHost), "bench wgmma D2H");
6373     max_err = 0.0f;
6374     for (int i = 0; i < M * N; i++) {
6375         float err = fabsf(__half2float(h_c[i]) - __half2float(h_c_ref[i]));
6376         if (err > max_err) max_err = err;
6377     }
6378     return true;
6379 }
6380
6381 static void bench_hgemm_wgmma(int M, int N, int K) {
6382     constexpr int BM = 128, BN = 128, BK = 64;
6383     if (M % BM != 0 || N % BN != 0 || K % BK != 0) {
6384         printf("| %-42s | %-12s | %-4s | %-8s |\n", "HGEMM WGMMMA (unaligned)", "SKIP", "SKIP",
6385             "None");
6386         return;
6387     }
6388     size_t size_a = (size_t)M * K * sizeof(half);
6389     size_t size_b = (size_t)K * N * sizeof(half);
6390     size_t size_c = (size_t)M * N * sizeof(half);
6391     half *h_a = (half *)malloc(size_a);
6392     half *h_b = (half *)malloc(size_b);
6393     half *h_c_ref = (half *)malloc(size_c);
6394     srand(42);
6395     for (int i = 0; i < M * K; i++)
6396         h_a[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
6397     for (int i = 0; i < K * N; i++)
6398         h_b[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
6399     size_t size_b_t = (size_t)N * K * sizeof(half);
6400     half *h_b_t = (half *)malloc(size_b_t);
6401     for (int n = 0; n < N; n++)
6402         for (int k = 0; k < K; k++)
6403             h_b_t[n * K + k] = h_b[k * N + n];
6404     half *d_a, *d_b, *d_b_t, *d_c;
6405     check(cudaMalloc(&d_a, size_a), "bench wgmma alloc A");
6406     check(cudaMalloc(&d_b, size_b), "bench wgmma alloc B");
6407     check(cudaMalloc(&d_b_t, size_b_t), "bench wgmma alloc B_t");

```

```

6408 check(cudaMalloc(&d_c, size_c), "bench wgmma alloc C");
6409 check(cudaMemcpy(d_a, h_a, size_a, cudaMemcpyHostToDevice), "bench wgmma H2D A");
6410 check(cudaMemcpy(d_b, h_b, size_b, cudaMemcpyHostToDevice), "bench wgmma H2D B");
6411 check(cudaMemcpy(d_b_t, h_b_t, size_b_t, cudaMemcpyHostToDevice), "bench wgmma H2D B_t");
6412 cublasHandle_t handle;
6413 cublasCreate(&handle);
6414 half alpha_h = __float2half(1.0f), beta_h = __float2half(0.0f);
6415 cublasGemmEx(handle, CUBLAS_OP_N, CUBLAS_OP_N, N, M, K, &alpha_h, d_b, CUDA_R_16F, N,
6416             d_a, CUDA_R_16F, K, &beta_h, d_c, CUDA_R_16F, N, CUBLAS_COMPUTE_16F,
6417             CUBLAS_GEMM_DEFAULT);
6418 check(cudaMemcpy(h_c_ref, d_c, size_c, cudaMemcpyDeviceToHost), "bench wgmma D2H ref");
6419 CUTensorMap *tma_a = allocate_and_create_tensor_map(d_a, M / BM, K / BK);
6420 CUTensorMap *tma_b = allocate_and_create_tensor_map(d_b_t, N / BN, K / BK);
6421 half *h_c = (half *)malloc(size_c);
6422 cudaEvent_t start, stop;
6423 cudaEventCreate(&start);
6424 cudaEventCreate(&stop);
6425 for (int stages : {1, 2, 3}) {
6426     for (int swizzle : {0, 1}) {
6427         float max_err = 0, time_ms = 0;
6428         bool ok = false;
6429         if (stages == 2)
6430             ok = swizzle
6431                 ? launch_timed_hgemm_wgmma<2, 1>(d_c, h_c, h_c_ref, tma_a, tma_b, M, N,
6432                 K, size_c, start, stop, max_err, time_ms)
6433                 : launch_timed_hgemm_wgmma<2, 0>(d_c, h_c, h_c_ref, tma_a, tma_b, M, N, K,
6434                 size_c, start, stop, max_err, time_ms);
6435         else
6436             ok = swizzle
6437                 ? launch_timed_hgemm_wgmma<3, 1>(d_c, h_c, h_c_ref, tma_a, tma_b, M, N,
6438                 K, size_c, start, stop, max_err, time_ms)
6439                 : launch_timed_hgemm_wgmma<3, 0>(d_c, h_c, h_c_ref, tma_a, tma_b, M, N, K,
6440                 size_c, start, stop, max_err, time_ms);
6441         char label[64];
6442         snprintf(label, sizeof(label), "HGEMM TMA WGMMA WS (S=%d, SW=%d)", stages, swizzle);
6443         if (!ok) {
6444             printf("| %-42s | %-12s | %-4s | %-8s |\n", label, "SMEM SKIP", "SKIP", "None");
6445         } else {
6446             float tflops = bench_hgemm_tflops(M, N, K, time_ms);
6447             printf("| %-42s | %.6e | %-4s | %-8.1f |\n", label, max_err,
6448                 max_err < 1.0f ? "PASS" : "FAIL", tflops);
6449         }
6450     }
6451 }
6452 cudaEventDestroy(start);
6453 cudaEventDestroy(stop);
6454 free(h_a);
6455 free(h_b);
6456 free(h_b_t);
6457 free(h_c);
6458 free(h_c_ref);
6459 cudaFree(d_a);
6460 cudaFree(d_b);
6461 cudaFree(d_b_t);
6462 cudaFree(d_c);
6463 cudaFree(tma_a);
6464 cudaFree(tma_b);
6465 cublasDestroy(handle);
6466 }
6467 #endif
6468
6469 #if defined(NOTES_V2_ENABLE_TMA_MMA_WS)
6470 // =====

```

```

6471 // Bench: HGEMM TMA MMA WS (mma.sync + TMA + Warp Specialization, SM120)
6472 // =====
6473 template <int kStages, int kBlockSwizzle>
6474 static bool bench_launch_tma_mma_ws(
6475     int M, int N, int K, half *d_a, half *d_b_t,
6476     half *d_c, half *h_c, half *h_c_ref, size_t size_c,
6477     CUTensorMap *tma_a, CUTensorMap *tma_b,
6478     cudaEvent_t start, cudaEvent_t stop,
6479     float &max_err, float &time_ms) {
6480     constexpr int BM = 128, BN = 128, BK = 64, kNumThreads = 256;
6481     constexpr size_t payload_bytes = kStages * (BM * BK + BN * BK) * sizeof(half);
6482     using Kernel = void (*)(int, int, int, half *, const CUTensorMap *,
6483         const CUTensorMap *);
6484     Kernel kernel = hgemm_tma_mma_ws_tn<16, 8, 16, 2, 2, 4, 8, 4, kStages,
6485         kNumThreads, kBlockSwizzle>;
6486     if (!check_smem_feasible((const void *)kernel, payload_bytes)) return false;
6487     cudaFuncSetAttribute(kernel, cudaFuncAttributeMaxDynamicSharedMemorySize,
6488         payload_bytes);
6489     dim3 block(kNumThreads);
6490     constexpr int kSwizzleN = 16;
6491     const int n_tiles = N / BN;
6492     const int grid_x = kBlockSwizzle ? div_ceil(n_tiles, kSwizzleN) : n_tiles;
6493     const int grid_z = kBlockSwizzle ? kSwizzleN : 1;
6494     dim3 grid(grid_x, M / BM, grid_z);
6495     for (int w = 0; w < g_warmup; w++)
6496         kernel<<<grid, block, payload_bytes>>>(M, N, K, d_c, tma_a, tma_b);
6497     cudaDeviceSynchronize();
6498     cudaEventRecord(start);
6499     for (int r = 0; r < g_repeat; r++)
6500         kernel<<<grid, block, payload_bytes>>>(M, N, K, d_c, tma_a, tma_b);
6501     cudaEventRecord(stop);
6502     cudaEventSynchronize(stop);
6503     cudaEventElapsedTime(&time_ms, start, stop);
6504     time_ms /= g_repeat;
6505     check(cudaMemcpy(h_c, d_c, size_c, cudaMemcpyDeviceToHost), "bench tma mma ws D2H");
6506     max_err = 0.0f;
6507     for (int i = 0; i < M * N; ++i)
6508         max_err = fmaxf(max_err, fabsf(__half2float(h_c[i]) - __half2float(h_c_ref[i])));
6509     return true;
6510 }
6511
6512 static void bench_hgemm_tma_mma_ws(int M, int N, int K) {
6513     constexpr int BM = 128, BN = 128, BK = 64;
6514     if (M % BM != 0 || N % BN != 0 || K % BK != 0) {
6515         printf("| %-42s | %-12s | %-4s | %-8s |\n", "HGEMM TMA MMA WS (unaligned)", "SKIP",
6516             "SKIP", "None");
6517         return;
6518     }
6519     const size_t size_a = size_t(M) * K * sizeof(half);
6520     const size_t size_b = size_t(K) * N * sizeof(half);
6521     const size_t size_b_t = size_t(N) * K * sizeof(half);
6522     const size_t size_c = size_t(M) * N * sizeof(half);
6523     half *h_a = static_cast<half *>(malloc(size_a));
6524     half *h_b = static_cast<half *>(malloc(size_b));
6525     half *h_b_t = static_cast<half *>(malloc(size_b_t));
6526     half *h_c = static_cast<half *>(malloc(size_c));
6527     half *h_c_ref = static_cast<half *>(malloc(size_c));
6528     srand(42);
6529     for (int i = 0; i < M * K; ++i)
6530         h_a[i] = __float2half((float(rand()) / RAND_MAX) * 2.0f - 1.0f);
6531     for (int i = 0; i < K * N; ++i)
6532         h_b[i] = __float2half((float(rand()) / RAND_MAX) * 2.0f - 1.0f);
6533     for (int n = 0; n < N; ++n)

```



```

6534     for (int k = 0; k < K; ++k)
6535         h_b_t[n * K + k] = h_b[k * N + n];
6536     half *d_a, *d_b, *d_b_t, *d_c;
6537     check(cudaMalloc(&d_a, size_a), "bench tma mma ws alloc A");
6538     check(cudaMalloc(&d_b, size_b), "bench tma mma ws alloc B");
6539     check(cudaMalloc(&d_b_t, size_b_t), "bench tma mma ws alloc B_t");
6540     check(cudaMalloc(&d_c, size_c), "bench tma mma ws alloc C");
6541     check(cudaMemcpy(d_a, h_a, size_a, cudaMemcpyHostToDevice), "bench tma mma ws H2D A");
6542     check(cudaMemcpy(d_b, h_b, size_b, cudaMemcpyHostToDevice), "bench tma mma ws H2D B");
6543     check(cudaMemcpy(d_b_t, h_b_t, size_b_t, cudaMemcpyHostToDevice),
6544         "bench tma mma ws H2D B_t");
6545     cublasHandle_t handle;
6546     cublasCreate(&handle);
6547     half alpha = __float2half(1.0f), beta = __float2half(0.0f);
6548     cublasGemmEx(handle, CUBLAS_OP_N, CUBLAS_OP_N, N, M, K, &alpha, d_b, CUDA_R_16F, N,
6549         d_a, CUDA_R_16F, K, &beta, d_c, CUDA_R_16F, N, CUBLAS_COMPUTE_16F,
6550         CUBLAS_GEMM_DEFAULT);
6551     check(cudaMemcpy(h_c_ref, d_c, size_c, cudaMemcpyDeviceToHost),
6552         "bench tma mma ws D2H reference");
6553     CUTensorMap *tma_a = allocate_and_create_tensor_map(d_a, M / BM, K / BK);
6554     CUTensorMap *tma_b = allocate_and_create_tensor_map(d_b_t, N / BN, K / BK);
6555     cudaEvent_t start, stop;
6556     cudaEventCreate(&start);
6557     cudaEventCreate(&stop);
6558     for (int stages : {1, 2, 3}) {
6559         for (int swizzle : {0, 1}) {
6560             float max_err = 0, time_ms = 0;
6561             bool ok = false;
6562             if (stages == 2)
6563                 ok = swizzle ? bench_launch_tma_mma_ws<2, 1>(M, N, K, d_a, d_b_t, d_c, h_c,
6564                     h_c_ref, size_c, tma_a, tma_b,
6565                     start, stop, max_err, time_ms)
6566                     : bench_launch_tma_mma_ws<2, 0>(M, N, K, d_a, d_b_t, d_c, h_c,
6567                     h_c_ref, size_c, tma_a, tma_b,
6568                     start, stop, max_err, time_ms);
6569             else
6570                 ok = swizzle ? bench_launch_tma_mma_ws<3, 1>(M, N, K, d_a, d_b_t, d_c, h_c,
6571                     h_c_ref, size_c, tma_a, tma_b,
6572                     start, stop, max_err, time_ms)
6573                     : bench_launch_tma_mma_ws<3, 0>(M, N, K, d_a, d_b_t, d_c, h_c,
6574                     h_c_ref, size_c, tma_a, tma_b,
6575                     start, stop, max_err, time_ms);
6576             char label[64];
6577             snprintf(label, sizeof(label), "HGEMM TMA MMA WS (S=%d, SW=%d)", stages, swizzle);
6578             if (!ok) {
6579                 printf("| %-42s | %-12s | %-4s | %-8s |\n", label, "SMEM SKIP", "SKIP", "None");
6580             } else {
6581                 float tflops = bench_hgemm_tflops(M, N, K, time_ms);
6582                 printf("| %-42s | %.6e | %-4s | %-8.1f |\n", label, max_err,
6583                     max_err < 1.0f ? "PASS" : "FAIL", tflops);
6584             }
6585         }
6586     }
6587     cudaEventDestroy(start);
6588     cudaEventDestroy(stop);
6589     free(h_a);
6590     free(h_b);
6591     free(h_b_t);
6592     free(h_c);
6593     free(h_c_ref);
6594     cudaFree(d_a);
6595     cudaFree(d_b);
6596     cudaFree(d_b_t);

```

```

6597     cudaFree(d_c);
6598     cudaFree(tma_a);
6599     cudaFree(tma_b);
6600     cublasDestroy(handle);
6601 }
6602 #endif
6603
6604 // =====
6605 // Bench: FlashAttention-2Split-Q (template on kHeadDim for dispatch)
6606 // =====
6607 template <int kHeadDim, int kPadQ = 8, int kPadK = 8, int kPadV = 8>
6608 static void bench_fa_launch(int B, int H, int seqlen, int head_dim,
6609     half *h_o_ref, float *ref_o, half *d_q, half *d_k, half *d_v, half *d_o) {
6610     static_assert(kHeadDim == 64 || kHeadDim == 128, "Only D=64 and D=128 are supported");
6611     constexpr int kStagesK = 2;
6612     constexpr bool kSwizzleQ = kPadQ == 0;
6613     constexpr bool kSwizzleK = kPadK == 0;
6614     constexpr bool kSwizzleV = kPadV == 0;
6615     constexpr int kMmaAtomM = 16;
6616     constexpr int kMmaAtomN = 8;
6617     constexpr int kMmaAtomK = 16;
6618     constexpr int kMmaTileSeqLenQ = 8;
6619     constexpr int kMmaTileSeqLenK = 1;
6620     constexpr int kMmaTileSeqLenP = 8;
6621     constexpr int kMmaTileHeadDimV = 1;
6622     constexpr int kValTileSeqLenQ = 1;
6623     constexpr int kValTileSeqLenK = 8;
6624     constexpr int kValTileSeqLenP = 1;
6625     constexpr int kValTileHeadDimV = kHeadDim / (8 * kMmaTileHeadDimV);
6626     constexpr int Br = kMmaAtomM * kMmaTileSeqLenQ * kValTileSeqLenQ;
6627     constexpr int Bc = kMmaAtomN * kMmaTileSeqLenK * kValTileSeqLenK;
6628     constexpr const char *layout_name =
6629         kSwizzleQ && kSwizzleK && kSwizzleV ? "Swizzle" :
6630         kSwizzleQ && kSwizzleK ? "SwizzleQK" :
6631         kSwizzleQ && kSwizzleV ? "SwizzleQV" :
6632         kSwizzleK && kSwizzleV ? "SwizzleKV" :
6633         kSwizzleQ ? "SwizzleQ" :
6634         kSwizzleK ? "SwizzleK" :
6635         kSwizzleV ? "SwizzleV" : "Pad";
6636
6637     // Kernel requires seqlen >= Br (tile size); skip gracefully for short seqlen
6638     if (seqlen < Br) {
6639         char label[64];
6640         snprintf(label, sizeof(label), "FlashAttention-2 (D=%d, %s)", kHeadDim,
6641             layout_name);
6642         printf("| %-42s | %-12s | %-4s | %-8s |\n", label,
6643             "seqlen<Br", "SKIP", "None");
6644         return;
6645     }
6646     size_t smem_bytes =
6647         (Br * (kHeadDim + kPadQ) +
6648         kStagesK * Bc * (kHeadDim + kPadK) +
6649         Bc * (kHeadDim + kPadV)) * sizeof(half);
6650
6651     dim3 block(256);
6652     dim3 grid((seqlen + Br - 1) / Br, B * H);
6653
6654     cudaStream_t timing_stream;
6655     cudaStreamCreate(&timing_stream);
6656     cudaEvent_t start, stop;
6657     cudaEventCreate(&start);
6658     cudaEventCreate(&stop);
6659

```

```

6660 using FAKernel = void (*)(half *, half *, half *, half *, int, int);
6661 FAKernel fa_k = flash_attn_mma_stages_split_q<
6662     kHeadDim, kMmaAtomM, kMmaAtomN, kMmaAtomK, kMmaTileSeqLenQ, kMmaTileSeqLenK,
6663     kMmaTileSeqLenP, kMmaTileHeadDimV, kValTileSeqLenQ, kValTileSeqLenK,
6664     kValTileSeqLenP, kValTileHeadDimV, kStagesK, kPadQ, kPadK, kPadV>;
6665 bool smem_ok = check_smem_feasible((const void *)fa_k, smem_bytes);
6666 if (smem_ok) {
6667     cudaFuncSetAttribute(fa_k, cudaFuncAttributeMaxDynamicSharedMemorySize, smem_bytes);
6668 }
6669
6670 // Warmup
6671 if (smem_ok) {
6672     for (int w = 0; w < g_warmup; w++)
6673         fa_k<<<grid, block, smem_bytes, timing_stream>>>(d_q, d_k, d_v, d_o, seqlen, H);
6674     check(cudaStreamSynchronize(timing_stream), "fa warmup sync");
6675 }
6676
6677 // Timed repeat
6678 cudaEventRecord(start, timing_stream);
6679 if (smem_ok) {
6680     for (int r = 0; r < g_repeat; r++)
6681         fa_k<<<grid, block, smem_bytes, timing_stream>>>(d_q, d_k, d_v, d_o, seqlen, H);
6682 }
6683 cudaEventRecord(stop, timing_stream);
6684 cudaEventSynchronize(stop);
6685
6686 float time_ms = 0;
6687 cudaEventElapsedTime(&time_ms, start, stop);
6688 time_ms /= g_repeat;
6689
6690 size_t sz = (size_t)B * H * seqlen * head_dim * sizeof(half);
6691 half *h_o = (half *)malloc(sz);
6692 check(cudaMemcpy(h_o, d_o, sz, cudaMemcpyDeviceToHost), "bench fa D2H");
6693
6694 int count = B * H * seqlen * head_dim;
6695 char label[64];
6696 snprintf(label, sizeof(label), "FlashAttention-2 (S=2, D=%d, %s)", kHeadDim,
6697     layout_name);
6698 float max_err = 0.0f;
6699 bool checked = h_o_ref || ref_o;
6700 if (smem_ok && checked) {
6701     for (int i = 0; i < count; i++) {
6702         float ref_val = h_o_ref ? __half2float(h_o_ref[i]) : ref_o[i];
6703         float err = fabsf(__half2float(h_o[i]) - ref_val);
6704         if (err > max_err) max_err = err;
6705     }
6706 }
6707 if (smem_ok && checked) {
6708     float tflops = bench_fa_tflops(B, H, seqlen, head_dim, time_ms);
6709     printf("| %-42s | %.6e | %-4s | %-8.1f |\n", label, max_err,
6710         max_err < 5e-1f ? "PASS" : "FAIL", tflops);
6711 } else if (smem_ok) {
6712     float tflops = bench_fa_tflops(B, H, seqlen, head_dim, time_ms);
6713     printf("| %-42s | %-12s | %-4s | %-8.1f |\n", label, "unchecked", "SKIP", tflops);
6714 } else {
6715     printf("| %-42s | %-12s | %-4s | %-8s |\n", label, "SMEM too large", "SKIP", "None");
6716 }
6717
6718 cudaEventDestroy(start);
6719 cudaEventDestroy(stop);
6720 cudaStreamDestroy(timing_stream);
6721 free(h_o);
6722 }

```

```

6723
6724 static void bench_flash_attn(int B, int H, int N, int D) {
6725     int seqlen = N, head_dim = D;
6726
6727     if (head_dim != 64 && head_dim != 128) {
6728         printf("| %-42s | %-12s | %-4s | %-8s |\n", "FlashAttention-2", "unsupported D", "SKIP", "None");
6729         return;
6730     }
6731
6732     size_t sz = (size_t)B * H * seqlen * head_dim * sizeof(half);
6733
6734     srand(42);
6735     half *h_q = (half *)malloc(sz);
6736     half *h_k = (half *)malloc(sz);
6737     half *h_v = (half *)malloc(sz);
6738     for (int i = 0; i < B * H * seqlen * head_dim; i++) {
6739         h_q[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
6740         h_k[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
6741         h_v[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
6742     }
6743
6744     // Device allocations — shared by kernel and reference
6745     half *d_q, *d_k, *d_v, *d_o;
6746     check(cudaMalloc(&d_q, sz), "bench fa alloc Q");
6747     check(cudaMalloc(&d_k, sz), "bench fa alloc K");
6748     check(cudaMalloc(&d_v, sz), "bench fa alloc V");
6749     check(cudaMalloc(&d_o, sz), "bench fa alloc O");
6750     check(cudaMemcpy(d_q, h_q, sz, cudaMemcpyHostToDevice), "bench fa H2D Q");
6751     check(cudaMemcpy(d_k, h_k, sz, cudaMemcpyHostToDevice), "bench fa H2D K");
6752     check(cudaMemcpy(d_v, h_v, sz, cudaMemcpyHostToDevice), "bench fa H2D V");
6753
6754     // Reference output: either cuDNN (half) or CPU (float)
6755     half *h_o_ref = nullptr;
6756     float *ref_o = nullptr;
6757     int ref_count = B * H * seqlen * head_dim;
6758
6759     #if defined(NOTES_V2_ENABLE_CUDNN)
6760     if (!g_fa_skip_check) {
6761         // Try cuDNN SDPA first; fall back to CPU if unsupported on this SM
6762         bool cudnn_ok = false;
6763         half *d_o_ref;
6764         check(cudaMalloc(&d_o_ref, sz), "bench fa alloc O_ref");
6765         {
6766             cudnnHandle_t cudnn_handle;
6767             cudnnCreate(&cudnn_handle);
6768             auto graph = std::make_shared<fe::graph::Graph>();
6769             graph->set_io_data_type(fe::DataType_t::HALF)
6770                 .set_intermediate_data_type(fe::DataType_t::FLOAT)
6771                 .set_compute_data_type(fe::DataType_t::FLOAT);
6772
6773             auto Q = graph->tensor(fe::graph::Tensor_attributes()
6774                 .set_uid(1).set_dim({B, H, seqlen, head_dim})
6775                 .set_stride({H * seqlen * head_dim, seqlen * head_dim, head_dim, 1}));
6776             auto K = graph->tensor(fe::graph::Tensor_attributes()
6777                 .set_uid(2).set_dim({B, H, seqlen, head_dim})
6778                 .set_stride({H * seqlen * head_dim, seqlen * head_dim, head_dim, 1}));
6779             auto V = graph->tensor(fe::graph::Tensor_attributes()
6780                 .set_uid(3).set_dim({B, H, seqlen, head_dim})
6781                 .set_stride({H * seqlen * head_dim, seqlen * head_dim, head_dim, 1}));
6782
6783             auto [O_sdpa, Stats] = graph->sdpa(Q, K, V,
6784                 fe::graph::SDPA_attributes()
6785                 .set_name("sdpa_ref")

```

```

6786         .set_attn_scale(1.0f / sqrtf((float)head_dim)));
6787
6788     O_sdpa->set_output(true).set_uid(4)
6789         .set_dim({B, H, seqlen, head_dim})
6790         .set_stride({H * seqlen * head_dim, seqlen * head_dim, head_dim, 1});
6791
6792     // Try A first, then fallback (matching Python example: [cudnn.heur_mode.A, cudnn.heur_mode.FALLBACK])
6793     auto build_status = graph->build(cudnn_handle, {fe::HeurMode_t::A, fe::HeurMode_t::FALLBACK});
6794     if (build_status.is_good()) {
6795         std::unordered_map<fe::graph::Tensor_attributes::uid_t, void*> vp = {
6796             {1, d_q}, {2, d_k}, {3, d_v}, {4, d_o_ref}};
6797         int64_t ws_size = 0;
6798         if (graph->get_workspace_size(ws_size).is_good()) {
6799             int8_t *d_ws = nullptr;
6800             if (ws_size > 0) check(cudaMalloc(&d_ws, ws_size), "bench fa workspace");
6801             if (graph->execute(cudnn_handle, vp, d_ws).is_good()) {
6802                 check(cudaDeviceSynchronize(), "bench fa cudnn sync");
6803                 cudnn_ok = true;
6804             }
6805             if (d_ws) cudaFree(d_ws);
6806         }
6807     }
6808     cudnnDestroy(cudnn_handle);
6809 }
6810
6811 if (cudnn_ok) {
6812     h_o_ref = (half *)malloc(sz);
6813     check(cudaMemcpy(h_o_ref, d_o_ref, sz, cudaMemcpyDeviceToHost), "bench fa D2H ref");
6814 } else {
6815     fprintf(stderr, "cudnn SDPA unsupported on this SM, using CPU ref\n");
6816 }
6817 cudaFree(d_o_ref);
6818 }
6819 #endif
6820
6821 // CPU reference fallback
6822 if (!g_fa_skip_check && !h_o_ref) {
6823     float *ref_q = (float *)malloc(sz * 4 / sizeof(half));
6824     float *ref_k = (float *)malloc(sz * 4 / sizeof(half));
6825     float *ref_v = (float *)malloc(sz * 4 / sizeof(half));
6826     ref_o = (float *)malloc(sz * 4 / sizeof(half));
6827     for (int i = 0; i < ref_count; i++) {
6828         ref_q[i] = __half2float(h_q[i]);
6829         ref_k[i] = __half2float(h_k[i]);
6830         ref_v[i] = __half2float(h_v[i]);
6831     }
6832
6833     float scale = 1.0f / sqrtf((float)head_dim);
6834     for (int bi = 0; bi < B * H; bi++) {
6835         for (int qi = 0; qi < seqlen; qi++) {
6836             float smax = -INFINITY;
6837             float *S = (float *)malloc((size_t)seqlen * sizeof(float));
6838             for (int kj = 0; kj < seqlen; kj++) {
6839                 float s = 0.0f;
6840                 for (int d = 0; d < head_dim; d++)
6841                     s += ref_q[bi * seqlen * head_dim + qi * head_dim + d] *
6842                         ref_k[bi * seqlen * head_dim + kj * head_dim + d];
6843                 S[kj] = s * scale;
6844                 if (S[kj] > smax) smax = S[kj];
6845             }
6846             double sum_exp = 0.0;
6847             for (int kj = 0; kj < seqlen; kj++)
6848                 sum_exp += (double)expf(S[kj] - smax);

```

```

6849     float inv_sum = 1.0f / (float)sum_exp;
6850     for (int d = 0; d < head_dim; d++) {
6851         double o_acc = 0.0;
6852         for (int kj = 0; kj < seqlen; kj++)
6853             o_acc += (double)(expf(S[kj] - smax) * inv_sum) *
6854                 ref_v[bi * seqlen * head_dim + kj * head_dim + d];
6855         ref_o[bi * seqlen * head_dim + qi * head_dim + d] = (float)o_acc;
6856     }
6857     free(S);
6858 }
6859 }
6860 free(ref_q);
6861 free(ref_k);
6862 free(ref_v);
6863 }
6864
6865 if (head_dim == 64) {
6866     if (g_fa_layout == FALayout::All || g_fa_layout == FALayout::Pad) {
6867         cudaDeviceSynchronize();
6868         bench_fa_launch<64, 8, 8, 8>(B, H, seqlen, head_dim, h_o_ref, ref_o, d_q, d_k, d_v, d_o);
6869     }
6870     if (g_fa_layout == FALayout::All || g_fa_layout == FALayout::SwizzleQ) {
6871         cudaDeviceSynchronize();
6872         bench_fa_launch<64, 0, 8, 8>(B, H, seqlen, head_dim, h_o_ref, ref_o, d_q, d_k, d_v, d_o);
6873     }
6874     if (g_fa_layout == FALayout::All || g_fa_layout == FALayout::SwizzleK) {
6875         cudaDeviceSynchronize();
6876         bench_fa_launch<64, 8, 0, 8>(B, H, seqlen, head_dim, h_o_ref, ref_o, d_q, d_k, d_v, d_o);
6877     }
6878     if (g_fa_layout == FALayout::All || g_fa_layout == FALayout::SwizzleV) {
6879         cudaDeviceSynchronize();
6880         bench_fa_launch<64, 8, 8, 0>(B, H, seqlen, head_dim, h_o_ref, ref_o, d_q, d_k, d_v, d_o);
6881     }
6882     if (g_fa_layout == FALayout::All || g_fa_layout == FALayout::SwizzleQK) {
6883         cudaDeviceSynchronize();
6884         bench_fa_launch<64, 0, 0, 8>(B, H, seqlen, head_dim, h_o_ref, ref_o, d_q, d_k, d_v, d_o);
6885     }
6886     if (g_fa_layout == FALayout::All || g_fa_layout == FALayout::SwizzleQV) {
6887         cudaDeviceSynchronize();
6888         bench_fa_launch<64, 0, 8, 0>(B, H, seqlen, head_dim, h_o_ref, ref_o, d_q, d_k, d_v, d_o);
6889     }
6890     if (g_fa_layout == FALayout::All || g_fa_layout == FALayout::SwizzleKV) {
6891         cudaDeviceSynchronize();
6892         bench_fa_launch<64, 8, 0, 0>(B, H, seqlen, head_dim, h_o_ref, ref_o, d_q, d_k, d_v, d_o);
6893     }
6894     if (g_fa_layout == FALayout::All || g_fa_layout == FALayout::Swizzle) {
6895         cudaDeviceSynchronize();
6896         bench_fa_launch<64, 0, 0, 0>(B, H, seqlen, head_dim, h_o_ref, ref_o, d_q, d_k, d_v, d_o);
6897     }
6898 } else {
6899     if (g_fa_layout == FALayout::All || g_fa_layout == FALayout::Pad) {
6900         cudaDeviceSynchronize();
6901         bench_fa_launch<128, 8, 8, 8>(B, H, seqlen, head_dim, h_o_ref, ref_o, d_q, d_k, d_v, d_o);
6902     }
6903     if (g_fa_layout == FALayout::All || g_fa_layout == FALayout::SwizzleQ) {
6904         cudaDeviceSynchronize();
6905         bench_fa_launch<128, 0, 8, 8>(B, H, seqlen, head_dim, h_o_ref, ref_o, d_q, d_k, d_v, d_o);
6906     }
6907     if (g_fa_layout == FALayout::All || g_fa_layout == FALayout::SwizzleK) {
6908         cudaDeviceSynchronize();
6909         bench_fa_launch<128, 8, 0, 8>(B, H, seqlen, head_dim, h_o_ref, ref_o, d_q, d_k, d_v, d_o);
6910     }
6911     if (g_fa_layout == FALayout::All || g_fa_layout == FALayout::SwizzleV) {

```



```

6912     cudaDeviceSynchronize();
6913     bench_fa_launch<128, 8, 8, 0>(B, H, seqlen, head_dim, h_o_ref, ref_o, d_q, d_k, d_v, d_o);
6914 }
6915 if (g_fa_layout == FALayout::All || g_fa_layout == FALayout::SwizzleQK) {
6916     cudaDeviceSynchronize();
6917     bench_fa_launch<128, 0, 0, 8>(B, H, seqlen, head_dim, h_o_ref, ref_o, d_q, d_k, d_v, d_o);
6918 }
6919 if (g_fa_layout == FALayout::All || g_fa_layout == FALayout::SwizzleQV) {
6920     cudaDeviceSynchronize();
6921     bench_fa_launch<128, 0, 8, 0>(B, H, seqlen, head_dim, h_o_ref, ref_o, d_q, d_k, d_v, d_o);
6922 }
6923 if (g_fa_layout == FALayout::All || g_fa_layout == FALayout::SwizzleKV) {
6924     cudaDeviceSynchronize();
6925     bench_fa_launch<128, 8, 0, 0>(B, H, seqlen, head_dim, h_o_ref, ref_o, d_q, d_k, d_v, d_o);
6926 }
6927 if (g_fa_layout == FALayout::All || g_fa_layout == FALayout::Swizzle) {
6928     cudaDeviceSynchronize();
6929     bench_fa_launch<128, 0, 0, 0>(B, H, seqlen, head_dim, h_o_ref, ref_o, d_q, d_k, d_v, d_o);
6930 }
6931 }
6932 cudaDeviceSynchronize();
6933
6934 free(h_q);
6935 free(h_k);
6936 free(h_v);
6937 free(h_o_ref);
6938 free(ref_o);
6939 cudaFree(d_q);
6940 cudaFree(d_k);
6941 cudaFree(d_v);
6942 cudaFree(d_o);
6943 }
6944
6945
6946 int main(int argc, char *argv[]) {
6947     #if defined(NOTES_V2_ENABLE_WGMMA) || defined(NOTES_V2_ENABLE_TMA_MMA_WS)
6948         cuInit(0);
6949     #endif
6950
6951     for (int i = 1; i < argc; i++) {
6952         if (strcmp(argv[i], "--bench") == 0) {
6953             g_bench_mode = true;
6954         } else if (strcmp(argv[i], "--bench-fa") == 0) {
6955             g_bench_fa = true;
6956         } else if (strcmp(argv[i], "--bench-fa-all") == 0) {
6957             g_bench_fa = true;
6958             g_fa_layout = FALayout::All;
6959         } else if (strcmp(argv[i], "--only-fa") == 0) {
6960             g_bench_mode = true;
6961             g_bench_only_fa = true;
6962             g_bench_fa = true;
6963         } else if (strcmp(argv[i], "--mnk") == 0 && i + 1 < argc) {
6964             sscanf(argv[++i], "%d,%d,%d", &g_bench_M, &g_bench_N, &g_bench_K);
6965         } else if (strcmp(argv[i], "--bhnd") == 0 && i + 1 < argc) {
6966             sscanf(argv[++i], "%d,%d,%d,%d", &g_bench_B, &g_bench_H, &g_bench_Nfa, &g_bench_D);
6967         } else if (strcmp(argv[i], "--fa-layout") == 0 && i + 1 < argc) {
6968             const char *layout = argv[++i];
6969             if (strcmp(layout, "all") == 0)
6970                 g_fa_layout = FALayout::All;
6971             else if (strcmp(layout, "pad") == 0)
6972                 g_fa_layout = FALayout::Pad;
6973             else if (strcmp(layout, "swizzle-q") == 0)
6974                 g_fa_layout = FALayout::SwizzleQ;

```

```

6975     else if (strcmp(layout, "swizzle-k") == 0)
6976         g_fa_layout = FALayout::SwizzleK;
6977     else if (strcmp(layout, "swizzle-v") == 0)
6978         g_fa_layout = FALayout::SwizzleV;
6979     else if (strcmp(layout, "swizzle-qk") == 0)
6980         g_fa_layout = FALayout::SwizzleQK;
6981     else if (strcmp(layout, "swizzle-qv") == 0)
6982         g_fa_layout = FALayout::SwizzleQV;
6983     else if (strcmp(layout, "swizzle-kv") == 0)
6984         g_fa_layout = FALayout::SwizzleKV;
6985     else if (strcmp(layout, "swizzle") == 0)
6986         g_fa_layout = FALayout::Swizzle;
6987     else {
6988         fprintf(stderr, "Unsupported FA layout: %s\n", layout);
6989         return EXIT_FAILURE;
6990     }
6991 } else if (strcmp(argv[i], "--fa-skip-check") == 0) {
6992     g_fa_skip_check = true;
6993 } else if (strcmp(argv[i], "--warmup") == 0 && i + 1 < argc) {
6994     g_warmup = atoi(argv[++i]);
6995 } else if (strcmp(argv[i], "--repeat") == 0 && i + 1 < argc) {
6996     g_repeat = atoi(argv[++i]);
6997 }
6998 }
6999
7000 if (g_bench_mode) {
7001     printf("=== notes-v2.cu bench mode ===\n");
7002     printf("HGEMM: M=%d N=%d K=%d   FA: B=%d H=%d N=%d D=%d\n",
7003           g_bench_M, g_bench_N, g_bench_K,
7004           g_bench_B, g_bench_H, g_bench_Nfa, g_bench_D);
7005     printf("| %-42s | %-12s | %-4s | %-8s |\n", "Kernel", "Max Err", "Pass", "TFLOPS");
7006     printf("|-----|-----|-----|-----|\n");
7007
7008     if (!g_bench_only_fa) {
7009         test_block_reduce(g_bench_N);
7010         test_dot(g_bench_N);
7011         test_relu(1024);
7012         test_elementwise(1024);
7013         test_histogram(1024);
7014         test_merge_attn_states(512, 16, 128);
7015         test_softmax(256);
7016         test_rms_norm(8, 128);
7017         test_layer_norm(8, 128);
7018         test_rope(8, 128);
7019         test_mat_transpose(256, 256);
7020         test_mat_transpose_padded(256, 256);
7021         test_sgemv(256, 128);
7022         test_sgemm(g_bench_M, g_bench_N, g_bench_K);
7023         bench_hgemm_mma(g_bench_M, g_bench_N, g_bench_K);
7024         bench_hgemm_swizzle(g_bench_M, g_bench_N, g_bench_K);
7025 #if defined(NOTES_V2_ENABLE_CUTE)
7026         bench_hgemm_cute(g_bench_M, g_bench_N, g_bench_K);
7027 #endif
7028 #if defined(NOTES_V2_ENABLE_WGMMA)
7029         bench_hgemm_wgmma(g_bench_M, g_bench_N, g_bench_K);
7030 #endif
7031 #if defined(NOTES_V2_ENABLE_TMA_MMA_WS)
7032         bench_hgemm_tma_mma_ws(g_bench_M, g_bench_N, g_bench_K);
7033 #endif
7034     }
7035     if (g_bench_fa)
7036         bench_flash_attn(g_bench_B, g_bench_H, g_bench_Nfa, g_bench_D);
7037

```

```

7038     printf("=== Bench done ===\n");
7039     return 0;
7040 }
7041
7042 #if defined(NOTES_V2_ENABLE_TMA_MMA_WS)
7043     if (argc >= 2 && strcmp(argv[1], "--tma-mma-ws") == 0) {
7044         int M = 128, N = 128, K = 64;
7045         if (argc > 4) {
7046             M = atoi(argv[2]);
7047             N = atoi(argv[3]);
7048             K = atoi(argv[4]);
7049         }
7050         printf("=== SM120 TMA MMA WS validation ===\n");
7051         printf("| %-42s | %-12s | %-4s | %-8s |\n", "Kernel", "Max Err", "Pass", "TFLOPS");
7052         printf("|-----|-----|-----|-----|\n");
7053         test_hgemm_tma_mma_ws(M, N, K);
7054         return 0;
7055     }
7056 #endif
7057     int M = 1024, N = 1024, K = 1024;
7058     if (argc > 3) { M = atoi(argv[1]); N = atoi(argv[2]); K = atoi(argv[3]); }
7059
7060     printf("=== notes-v2.cu verification harness ===\n");
7061     printf("| %-42s | %-12s | %-4s | %-8s |\n", "Kernel", "Max Err", "Pass", "TFLOPS");
7062     printf("|-----|-----|-----|-----|\n");
7063
7064     test_block_reduce(N);
7065     test_dot(N);
7066     test_relu(1024);
7067     test_elementwise(1024);
7068     test_histogram(1024);
7069     test_merge_attn_states(512, 16, 128);
7070     test_softmax(256);
7071     test_rms_norm(8, 128);
7072     test_layer_norm(8, 128);
7073     test_rope(8, 128);
7074     test_mat_transpose(256, 256);
7075     test_mat_transpose_padded(256, 256);
7076     test_sgemv(256, 128);
7077     test_sgemm(M, N, K);
7078     test_hgemm_mma(M, N, K);
7079     test_hgemm_swizzle(M, N, K);
7080     #if (defined(NOTES_V2_ENABLE_CUTE))
7081         test_hgemm_cute(M, N, K);
7082     #endif
7083     #if (defined(NOTES_V2_ENABLE_WGMMMA))
7084         test_hgemm_wgmma(M, N, K);
7085     #endif
7086     #if defined(NOTES_V2_ENABLE_TMA_MMA_WS)
7087         test_hgemm_tma_mma_ws(M, N, K);
7088     #endif
7089     test_flash_attn(1024, 64);
7090
7091     printf("=== All tests done ===\n");
7092     return 0;
7093 }
7094
7095 // =====
7096 // Quick build & run reference
7097 // =====
7098 // # sm_86 + CuTe (Ampere, RTX 30/40 系列, 无 WGMMMA) :
7099 // nvcc -std=c++20 -O2 -arch=sm_86 -DNOTES_V2_ENABLE_CUTE \
7100 // -I ../../third-party/cutlass/include \

```

```

7101 // -lcublas -lcuda notes-v2.cu -o notes_v2_cute_sm86.bin
7102 //
7103 // # sm_89 单独编译 + 运行:
7104 // nvcc -std=c++20 -O2 -arch=sm_89 -lcublas -lcuda notes-v2.cu -o notes_v2_sm89.bin
7105 //
7106 // # sm_89 + CuTe (需要 CUTLASS include 路径, 编译 CUDA_HOME 指向的 CUTLASS 或
7107 // 项目内置 third-party/cutlass):
7108 // nvcc -std=c++20 -O2 -arch=sm_89 -DNOTES_V2_ENABLE_CUTE \
7109 // -I ../../third-party/cutlass/include \
7110 // -lcublas -lcuda notes-v2.cu -o notes_v2_cute_sm89.bin
7111 //
7112 // # sm_90a 单独编译 + 运行 (需要 Hopper GPU, H100/H200 均可):
7113 // nvcc -std=c++20 -O2 -gencode arch=compute_90a,code=sm_90a \
7114 // -DNOTES_V2_ENABLE_WGMMA -lcublas -lcuda notes-v2.cu -o notes_v2_sm90.bin
7115 //
7116 // # sm_90a + CuTe + WGMMA:
7117 // nvcc -std=c++20 -O2 -gencode arch=compute_90a,code=sm_90a \
7118 // -DNOTES_V2_ENABLE_CUTE -DNOTES_V2_ENABLE_WGMMA \
7119 // -I ../../third-party/cutlass/include \
7120 // -lcublas -lcuda notes-v2.cu -o notes_v2_cute_sm90.bin
7121 //
7122 // # sm_120a TMA + warp-specialized mma.sync (CUDA Toolkit >= 13.2;
7123 // RTX PRO 5000 / RTX 5090):
7124 // nvcc -std=c++20 -O2 -arch=sm_120a -DNOTES_V2_ENABLE_TMA_MMA_WS \
7125 // -lcublas -lcuda notes-v2.cu -o notes_v2_tma_mma_ws_sm120.bin
7126 // ./notes_v2_tma_mma_ws_sm120.bin --tma-mma-ws 1024 1024 1024
7127 //
7128 // # sm_120a + CUTE + TMA_MMA_WS + cuDNN SDPA (CUDA Toolkit >= 13.2;
7129 // requires cudnn-frontend submodule: git submodule update --init):
7130 // nvcc -std=c++20 -O2 -arch=sm_120a \
7131 // -DNOTES_V2_ENABLE_CUTE -DNOTES_V2_ENABLE_TMA_MMA_WS -DNOTES_V2_ENABLE_CUDNN \
7132 // -I ../../third-party/cutlass/include -I ../../third-party/cudnn-frontend/include \
7133 // -L/usr/local/cuda-13.2/targets/x86_64-linux/lib/stubs \
7134 // -lcublas -lcudnn -lnvrtc -lcuda notes-v2.cu -o notes_v2_cute_ws_sm120a.bin
7135 // # Default FA bench (kPadQ=kPadK=kPadV=8 only):
7136 // ./notes_v2_cute_ws_sm120a.bin --bench --bench-fa --bhnd 1,48,4096,64
7137 // # All FA layout variants:
7138 // ./notes_v2_cute_ws_sm120a.bin --bench --bench-fa-all --bhnd 1,48,4096,64

```